ODOO PAYROLL R&D · DAFL

Payroll, from Theory to Odoo Localization

Gross-to-net in any country, the Odoo salary engine, and how to design, build and review a payroll localization. One Belgian company is the case study from the first page to the last.

Built against Odoo **master** (community + enterprise) · October 2026
Every legal figure cites its official source. Every case-study payslip is computed twice: by Odoo and by an independent calculator.

Contents

Before you start

Part I — Payroll theory

1. What payroll is, and why it is hard
2. Gross to net
3. Contributions: employee and employer
4. Withholding tax
5. Time, proration and overtime
6. Irregular pay, corrections and leavers
7. Declarations and the compliance calendar
8. Accounting for payroll

Part II — Localization theory

9. What varies between countries, and what doesn't
10. Researching a new country
11. Laws that change over time

Part III — Odoo payroll, hands-on

12. Tour and setup
13. Employees and versions
14. Time: schedules, time off, worked days
15. Pay runs and payslips
16. Accounting and payment
17. Reports, declarations and a leaver

Part IV — How the payroll engine works

18. The engine in one picture
19. Structures and rules
20. Categories: how rules talk to each other
21. Parameters, updates and safe extensions

Part V — Building a localization

22. A template for any localization, and how to review one

Before you start

Who you will meet, and how the missions work

AFTER THIS CHAPTER YOU CAN

- Recognise the five people of MyVidangeCompany and what each one teaches
- Open a master database on runbot and play a mission

You work on Odoo payroll and may never have run a payroll yourself. This book goes from **theory** (what a payslip is) to **localization** (what changes between countries), then **Odoo** (hands-on, then the engine), and ends with **building and reviewing a localization**.

One company runs through the book: **MyVidangeCompany**, in Belgium, Odoo's most complete payroll localization. You need no Belgian law: terms are explained where they appear, and the *In Belgium* boxes can be skipped.

Meet MyVidangeCompany

MyVidangeCompany is a software company in Namur. It pays monthly and its office employees work a 38-hour week. Five of them carry the book:

Person	Role, monthly gross	Teaches	Net, Oct 2026: the law	Net, Oct 2026: Odoo
Mélanie	Developer, 4,000.00 €, single, no children	the standard case	2,614.26	2,614.27
Jurgen	Support agent, 2,300.00 €	a low wage: contribution relief that fades out as pay rises	2,143.05	2,143.06
Marie	Engineering manager, 7,500.00 €, married, two children, company car, meal vouchers	a high earner, benefits in kind, family situation in the tax	4,663.91	4,684.68
Yannick	Designer, 3,600.00 €, joins on **12 March 2026**	proration, the hiring declaration, a first year without holiday rights	2,444.37	2,444.37
Emma	Ops lead, 5,000.00 €, hired 1 Oct 2019, dismissed on **31 October 2026**	the leaver: final pay, notice indemnity, exit holiday pay	3,051.72	3,159.73

Every payslip was computed twice.[1,2] An independent calculator written from the law gives the numbers of Part I; the payslips Odoo master produced give those of Parts II to V. Where they differ, the chapter says why (Chapter 10).

> 🔑 **Key idea**
>
> Two independent computations of the same payslip are the cheapest test a localization can have. Where they agree, the payslip is right. Where they differ, someone learns something about the law or about the code.

The missions: on the master runbot

Part III is played, not read. Each chapter ends with **missions**: a short story, a goal, a number to check and a hint. They follow MyVidangeCompany through 2026 on one database you build from an empty Odoo, with only a browser and Odoo's runbot.

Take the **base** database of an *enterprise* build: it holds only the base apps, like a new customer. Runbot's "today" is the real date, so mission dates are typed explicitly.

Chapter	Missions	What you do
12 Tour and setup	12.1–12.4	install Payroll, create the company, the Belgian localization with its schedule and holidays, the settings, where the law lives
13 Employees and versions	13.1–13.4	hire the first four, Marie's car, the dashboard, Yannick joins and Jurgen's raise
14 Time	14.1–14.4	2026 leave, Jurgen's absences, preview April, Emma's holiday
15 Pay runs	15.1–15.4	January, a late correction, February to April, overtime in May
16 Accounting and payment	16.1–16.4	post January, pay February, what is still owed, June's adjustment line
17 Declarations	17.1–17.3	February's withholding return, summer, Emma's last month

🎯 Mission 0.1 · Open a master build ★

Situation. You have just joined MyVidangeCompany's payroll team. There is no payroll database yet.

Your goal. Open a fresh master database on runbot and log in as the administrator.

Check. You are logged in as *Mitchell Admin*; the only company is *YourCompany*, and the *Payroll* app is not installed yet.

▼ **Hint**

Take the base database of an enterprise build, not the one with all modules.

---

1. Independent calculator, `research/numbers_be_calc.md`, section 2 (October 2026), built from the official sources cited in each chapter of Part I. ↩
2. Payslips computed by Odoo master on the demo database `payroll_be`, `research/numbers_be.md`, October 2026 summary. Emma's Odoo October payslip includes a pro-rata sectoral bonus and eco-vouchers paid at her exit. ↩

PART I

Payroll theory

The ideas every country's payroll shares: parties and money flows, gross to net, contributions, withholding tax, time, irregular pay and leavers, declarations and accounting. Each idea comes first in general terms, then as a worked case with MyVidangeCompany, a small Belgian company, then compared with how Odoo's other localizations handle it around the world.

What payroll is, and why it is hard

Several parties, one monthly cycle, and rules that change during the year

AFTER THIS CHAPTER YOU CAN

- Name the parties of any payroll and the money that flows between them
- Walk through the six steps of the payroll cycle and the event, monthly, quarterly and yearly rhythm of the authorities
- Explain why a payroll error is costly, and who finds it
- Place the cycle in the Odoo models before you meet them in Part III

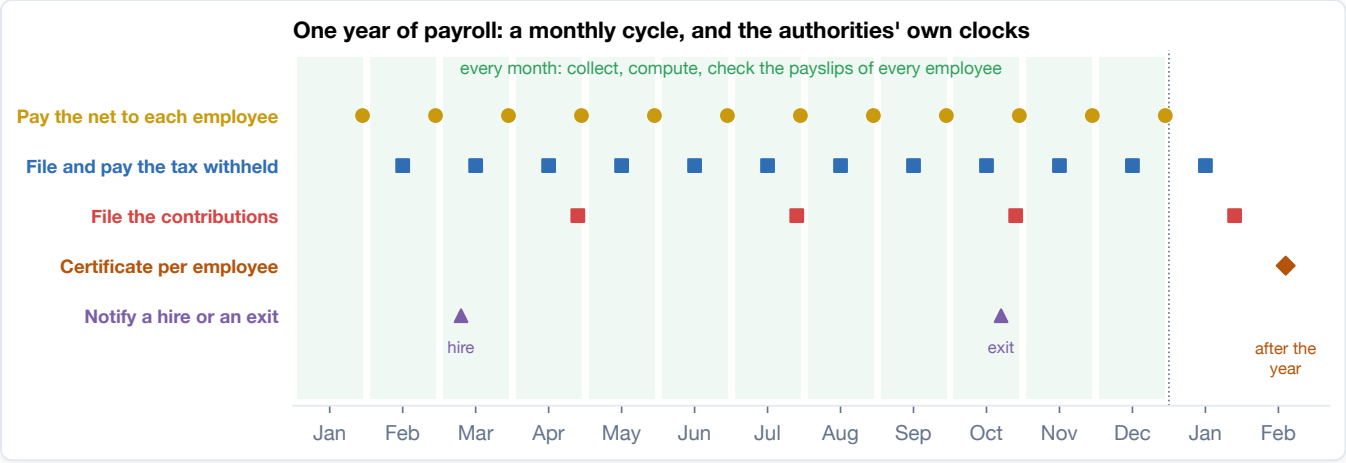
Early in November, MyVidangeCompany transfers 2,614.26 € to Mélanie for October. Her contract says 4,000.00 €, and the company spends 5,004.80 € on her. Nothing is lost: 1,385.74 € is owed to the social security office and the tax administration, and 1,004.80 € is the employer's own contribution. Payroll is getting every split right, on time, for every employee, in a way an inspector can reproduce.[1]

What payroll is

Payroll is the work an employer does every pay period to pay its employees: compute each one's pay from the contract and the time worked, pay the net to the employee, and pass the amounts it kept back on to the authorities, with the reports that prove them. The word also names the list of employees and amounts for one period (*the October payroll*). Other names: *payroll processing*, *salary administration*, *pay run*.

Three things follow from that definition:

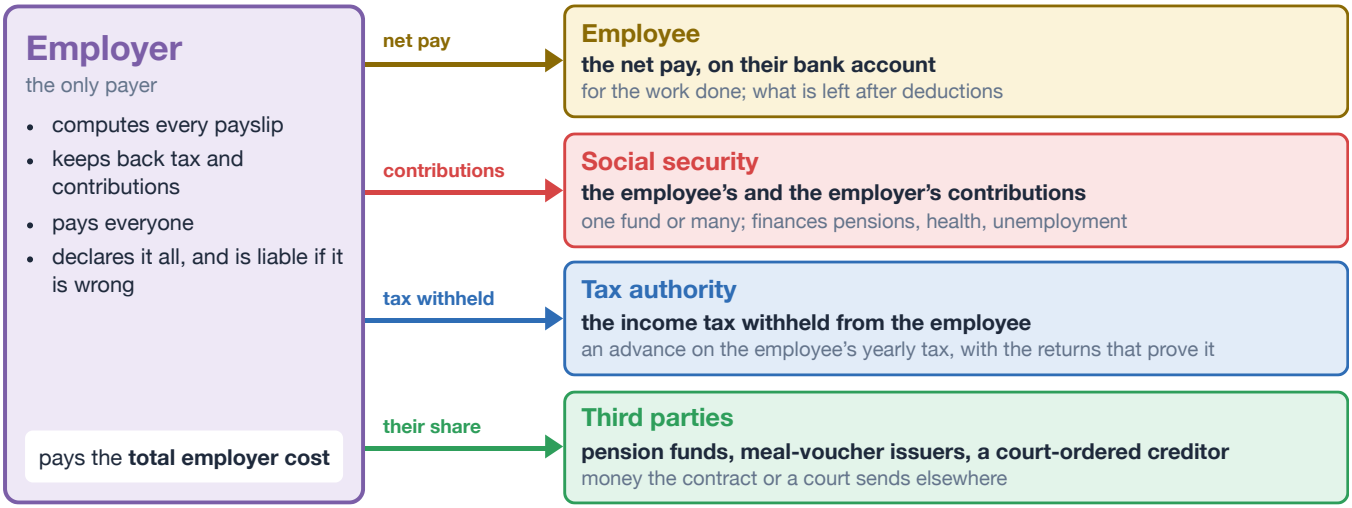
- **It never stops.** The same computation runs every period for every employee, with new inputs each time: a hire, a sick day, a rate that changes in July.
- **Much of the money is not the employee's to keep.** Part of the salary goes to social security and to the tax office; the employer collects it and is liable for it (next section, and Chapter 2).
- **Someone checks it.** The authorities receive the money and the reports on their own calendar, and can recompute any payslip.



Illustrative calendar. Every month the payroll is computed and the net is paid. The tax withheld is filed and paid the next month, the contributions after each quarter, a certificate per employee after the year, and a hire or an exit is notified when it happens. Dates differ by country; the four clocks are almost universal.

The parties and the money

In every country, payroll moves money from one payer to several receivers. The names change; the shape does not.

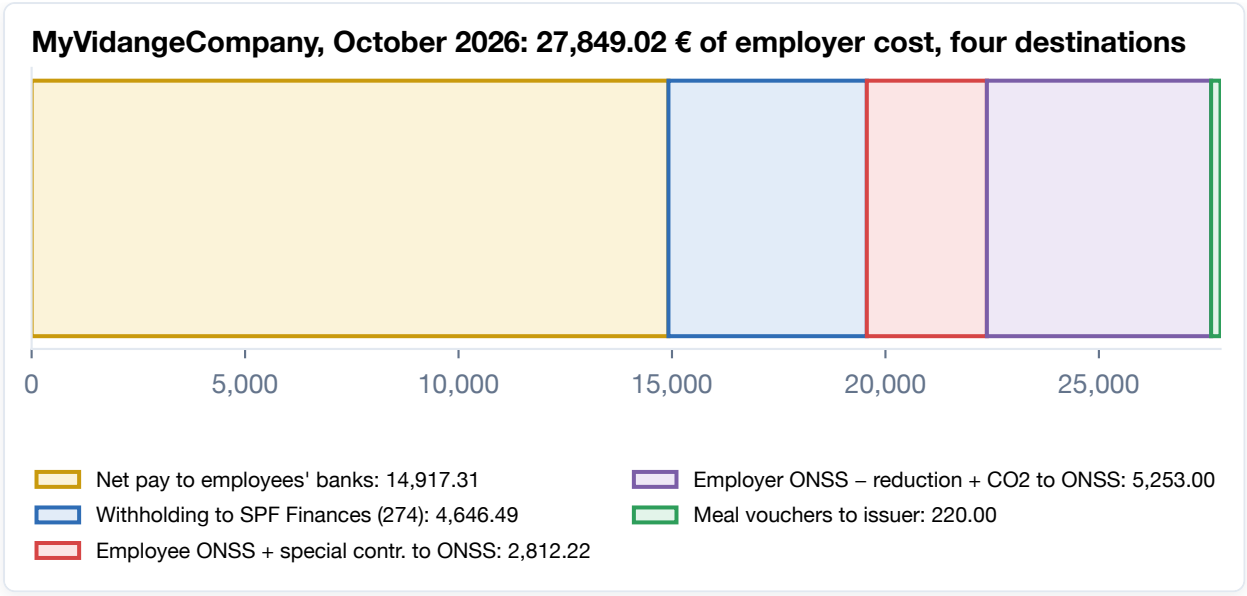


The **bank** only carries the transfers. A **payroll provider**, if the company uses one, computes and files on its behalf, but the employer stays responsible.

> 🔑 **Key idea**
>
> The employer is an **unpaid collection agent** for the state: it deducts, pays and reports for the employee, and is liable if it does not. In many countries, Belgium included, a contribution the employer forgot to deduct cannot be claimed back from the worker later.[3]

Case study: MyVidangeCompany, one month

MyVidangeCompany pays five people. In Belgium the social security office is the **ONSS** (one office collects every contribution) and the tax administration is **SPF Finances**. Marie gets meal vouchers, so a voucher issuer is paid too.



MyVidangeCompany, October 2026, five employees. Barely half of the employer cost reaches the employees' bank accounts. Numbers from the independent calculator.

Worked example: MyVidangeCompany, October 2026

The five regular payslips:[1]

Destination	Amount (€)	Share of cost
NET Employees' bank accounts	**14,917.31**	**53.56 %**
ONSS social security: employee and employer contributions	8,065.22	28.96 %
P.P Tax administration: withholding tax	4,646.49	16.68 %
MEAL Meal-voucher issuer	220.00	0.79 %
Total = employer cost	**27,849.02**	100.00 %

Gross salaries are 22,400.00 €; the employer adds its own contributions and voucher share.

In Odoo. Part I uses an independent calculator that follows the law; later parts use the payslips Odoo computed on the demo database. Odoo's October pay run shows 28,192.34 € of employer cost, a little more, because it includes Emma's pro-rata sector bonus (Chapter 6) and some small employer funds (Chapter 3).[2] Chapter 10 explains how to read such differences.

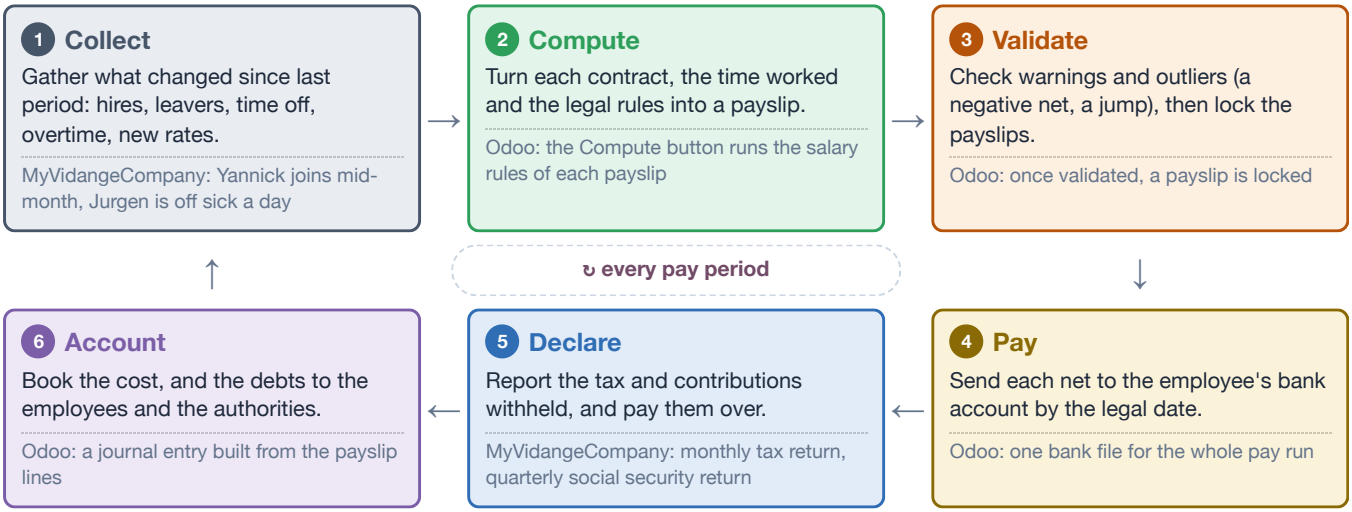
Around the world

The same diagram, with different boxes filled in:

- **United States**: federal taxes, plus a second layer per state and sometimes per city.
- **Switzerland**: many separate insurers (old age, unemployment, accident, pension…) and a cantonal source tax.
- **United Arab Emirates**: no income tax; a pension fund for nationals only; salaries paid through the monitored Wage Protection System.
- **Hong Kong**: no withholding; the employer reports salaries and each employee pays his own tax.
- **Australia**: no employee contribution; the employer pays superannuation into each employee's chosen fund.

The payroll cycle

Every pay period, the same six steps, in a loop: the next period starts as soon as the last one is booked.



Software automates step 2; teams spend their time on steps 1, 3 and 5.

In Odoo. The cycle is the life of a payslip: `hr.payslip` goes `draft` → `validated` → `paid` (or `cancel`). A pay run (`hr.payslip.run`) groups the payslips of one structure and one period. Chapter 15 runs it by hand.

The rhythm: event, month, quarter, year

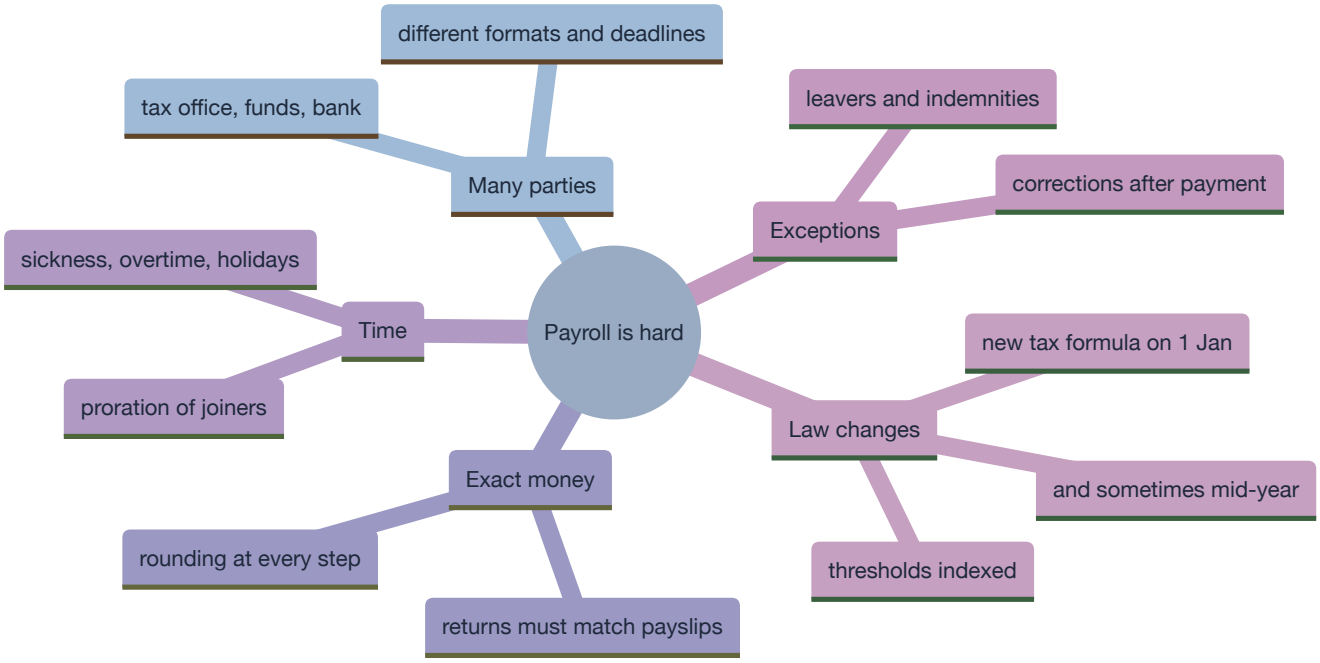
The four clocks of the opening figure, with MyVidangeCompany's dates:

Rhythm	Typical obligation	MyVidangeCompany (Belgium)
Event	notify a hire before work starts, and an exit	"Dimona" hire and exit notice to the social security office[6]
Every pay period	pay the net by a legal date	by the 4th working day after the month[5]
Monthly or quarterly	file and pay the tax withheld; file and pay contributions	monthly withholding return; quarterly social security return[4,7]

Rhythm	Typical obligation	MyVidangeCompany (Belgium)
Yearly	give each employee and the tax authority an annual certificate	annual tax slip per employee, by 28 February[8]

Chapter 7 puts MyVidangeCompany's filings on one calendar.

Why payroll is hard



Three properties make payroll unforgiving, in every country.

1. **It is law, not preference.** A forgotten hire notice is a criminal offence in Belgium.[6]
2. **It cannot be undone once paid.** You correct on a later payslip (Chapter 6).
3. **It is checked from outside.** The authorities cross-check returns, payments, yearly certificates and the employee's own tax return.

Rules that move during the year

A payroll rule is a value with a start date. Most countries change rates and tax tables on 1 January; many also change something mid-year. Belgium changed its withholding formula on 1 January and again on 1 November 2026.[9]

With the same gross, the November change lowers Mélanie's, Jurgen's and Yannick's withholding by 8.47 € a month but **raises** Marie's by 28.86 €: the new formula gives less relief to a married couple with one income.[1] Chapter 4 explains why; Chapter 11 shows how Odoo stores both formulas.

> **Worked example: what a forgotten update costs**
>
> Keep the January formula in November: Mélanie, Jurgen and Yannick are over-withheld 8.47 € a month and get it back a year later; Marie is under-withheld 28.86 € a month and pays at her assessment. Neither error shows on the day, and every monthly return is wrong until the fix.

> ⚠️ **Common pitfall**
>
> Treating payroll as "salary minus a few percentages". The percentages come with caps, low-wage reductions, family relief, special bonus rates and effective dates. Each is a bug waiting in a naive implementation; the next seven chapters take them one at a time.

> 💡 **In practice**
>
> Odoo customers often still use an accountant or payroll bureau for some filings. When you scope a project, ask early: **who files the social security return, who files the tax return, who pays the net?**

Summary

- The parties are always the same kinds: employer, employee, social security, tax authority, third parties, bank. Countries differ in how many funds and layers there are.
- The employer is a collection agent, liable for what it fails to deduct.
- MyVidangeCompany spends 27,849.02 € in October 2026 for 22,400.00 € of gross; 14,917.31 € is net pay.
- Six steps a period: collect, compute, validate, pay, declare, account. The authorities add event, monthly/quarterly and yearly deadlines.
- Rules change during the year, sometimes mid-year, so every rule needs an effective date.
- In Odoo the cycle is the life of **hr.payslip** (`draft`, `validated`, `paid`), batched in **hr.payslip.run**.

---

1. All cast numbers in Part I come from the independent calculator (`scripts/be_calc.py`, written to `research/numbers_be_calc.md`), built from the official sources cited in each chapter. ↩ ↩ ↩
2. Payslips computed by Odoo master on the demo database, `research/numbers_be.md`, section 6 (October 2026 monthly pay run); differences listed in `research/odoo_vs_calc.md`. ↩
3. ONSS, administrative instructions 2026/3, general rule on the payment of contributions: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/contributionpayment_obligations_periodicity/generalrule.html. ↩
4. SPF Finances, withholding tax calendar 2026 (including the dates for social secretariats): https://finances.belgium.be/fr/entreprises/personnel_et_remuneration/precompte_professionnel/calendrier. ↩
5. SPF Emploi, time and date of payment of the salary: https://emploi.belgique.be/fr/themes/remuneration/protection-de-la-remuneration/epoque-et-date-de-paiement. ↩
6. ONSS, Dimona, general and declarations: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/dimona/general.html and https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/dimona/declarations_modifications_cancellations.html. ↩ ↩
7. ONSS, DmfA general: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/dmfa/general.html. ↩

8. SPF Finances, Belcotax-on-web deadlines: https://finances.belgium.be/fr/E-services/Belcotaxonweb/delais-de-rentree-des-fiches. ↩
9. Withholding: Annex III AR/CIR 92, version history: https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/439228c6-67bb-4e75-a9e1-4cabe4dc62ec. Work bonus: ONSS instructions 2026/1, 2026/2 and latest: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/deductions/workers_reductions/workbonus.html. ↩

Gross to net

Why three different numbers are all called 'salary'

AFTER THIS CHAPTER YOU CAN

- Split any payslip into earnings, deductions, employer cost and tax
- Ask the two questions every earning needs: subject to contributions? taxable?
- Value a benefit in kind and treat an allowance that is exempt within limits
- Tell deductions that lower the taxable base from those that do not
- Compute the three bases (gross, contribution base, taxable base) and the employer cost

Mélanie's contract says 4,000.00 €. Social security computes on 4,000.00 €, the withholding tax starts from 3,477.20 €, and her bank receives 2,614.26 €. HR, social security and the tax office all say "salary" and mean different amounts. Confusing them is the most common design bug in payroll software.

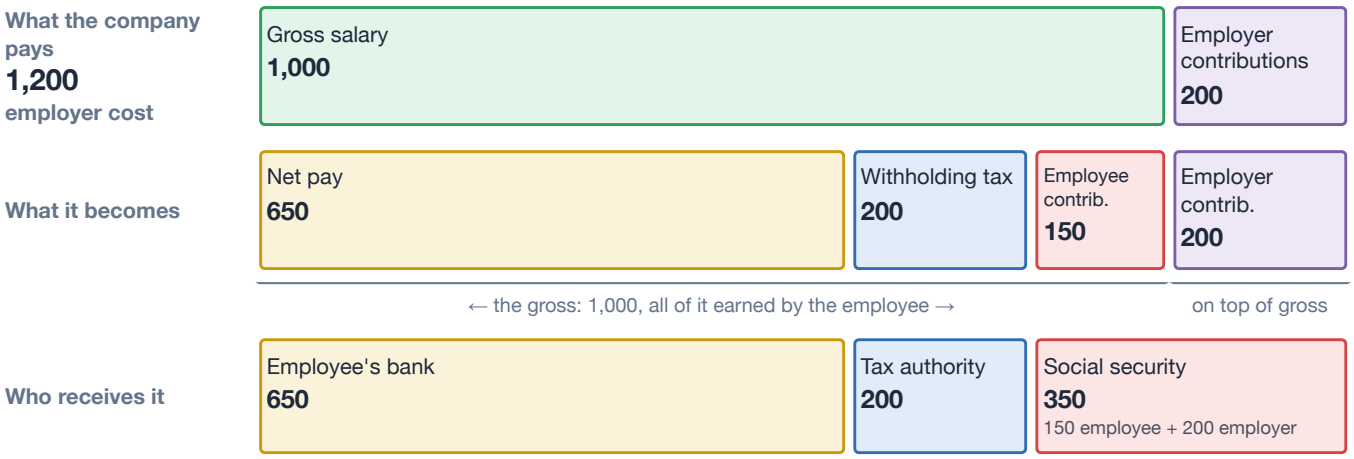
What gross and net pay are

Gross pay (*gross salary*, *gross wage*) is what the employee earns for the period before anything is taken off: the contract salary plus any other earnings. **Net pay** (*take-home pay*) is what reaches the employee's bank account, after the employee's contributions, the withholding tax and any other deductions. The calculation from one to the other is called **gross to net**, and the payslip is its written record.

Three things follow from that definition:

- **Gross is not what the company pays.** The employer adds its own contributions on top of the gross. They never touch the net: they are a cost *on top of* the salary, and the two together are the **employer cost**.
- **The gap between gross and net is the employee's money.** The employee contributions and the withholding tax are kept back by the employer and passed on to the authorities: the employer is only a collector.
- **"Salary" is ambiguous.** Gross, net, and the bases on which contributions and tax are computed are different numbers. This chapter computes each one.

Whatever the country, one month of salary is one pot of money, paid by the employer and split three ways. Illustrative numbers, to scale:



Top bar: the employer cost. Middle bar: the gross splits into the net and the employee's deductions, with the employer contributions beside it. Bottom bar: the authorities receive the tax and both halves of the contributions.

The shape is the same in every country; what changes is the number of recipients and whether the tax segment exists. Hong Kong is the clearest contrast: the employer withholds **no** income tax. It reports each employee's yearly pay to the tax office, which assesses the tax and collects it from the employee directly (the one exception: an employee leaving Hong Kong for good, whose final payment the employer must hold back). The only mandatory contribution is the pension fund, 5 % from each side. Illustrative numbers again:



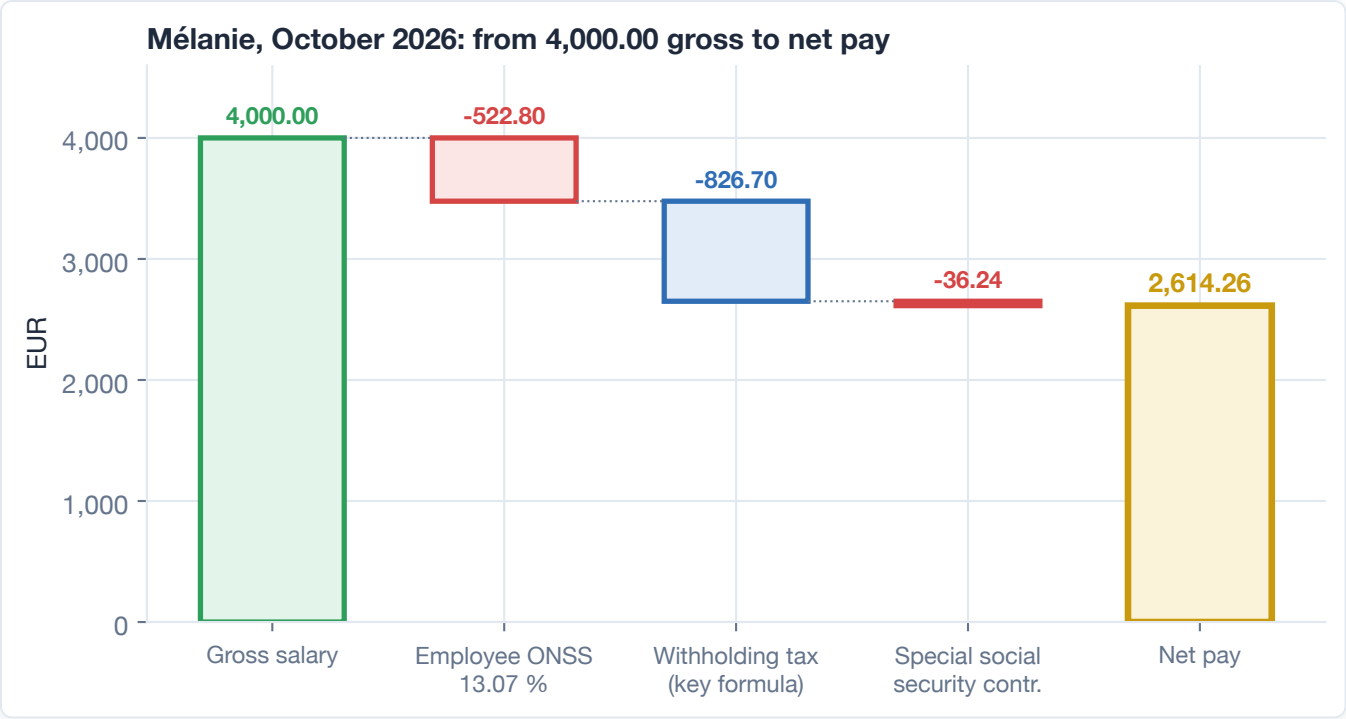
Not to scale: the pension boxes are enlarged so their numbers fit. No tax segment: nothing goes to the tax authority.

> 🌍 **Around the world**
>
> - **Australia**: no employee contribution at all; the employer pays superannuation into the employee's pension fund.
> - **United States, India**: several recipients instead of one social security office (federal and state tax; separate pension and health funds).
> - **United Arab Emirates**: no income tax; a pension contribution for nationals only.

$$\text{Employer cost} = \text{Gross} + \text{employer contributions}$$
$$\text{Taxable base} = \text{Gross} - \text{employee contributions} + \text{benefits in kind}$$
$$\text{Net} = \text{Gross} - \text{employee contributions} - \text{withholding tax} - \text{other deductions}$$

Benefits in kind are not paid in cash: they only raise the taxable base, and so the tax.

Case study: Mélanie's October



Mélanie, October 2026: 4,000.00 € gross, three deductions, 2,614.26 € net.

Worked example: Mélanie, October 2026

Line	Amount (€)
SALARY Gross salary	4,000.00
ONSS Employee social security 13.07 %	–522.80
GROSS Taxable base	3,477.20
P.P Withholding tax	–826.70
M.ONSS Special social security contribution	–36.24
NET **Net pay**	**2,614.26**

The codes are the Belgian salary rules in Odoo. The "special contribution" is a second, smaller employee contribution that does *not* lower the taxable base (Chapter 3).

Earnings: two questions for each

Every earning on a payslip has to answer two questions, because two different laws look at it: social-security law decides what contributions are taken from, tax law decides what is taxed. They often agree, but not always.

1. **Does social security take its share?** Is it in the *contribution base*?
2. **Is it taxed?** Is it in the *taxable base*, and at which rate?

Two yes-or-no answers give four boxes, and every earning falls into one of them:

	Taxed	Not taxed
Social security takes its share	**The usual case** • base salary, overtime, allowances • bonuses and 13th month, often at a special tax rate MyVidangeCompany: everyone's salary	**Rare** • pay set aside in a US 401(k) retirement plan: social security still applies, income tax is deferred
No social security	**Taxed only** • benefits in kind, valued by a legal formula, where the law exempts them from contributions: a company car in Belgium (other countries also take contributions on them) MyVidangeCompany: Marie's car	**Neither** • meal vouchers, within the legal limits • expense reimbursements that repay a real cost MyVidangeCompany: Marie's meal vouchers

The same earning can sit in a different box in another country, which is why a localization answers both questions for every earning it ships.

🔑 **Key idea**

Design each earning with **both** answers, stored on the rule. A Belgian company car is taxable but not subject to contributions. An "is it taxable?" flag alone is never enough.

🌍 **Around the world**

- **Saudi Arabia, UAE**: salary = basic + housing + transport, each fixed on the contract.
- **Switzerland**: a catalogue of numbered wage types, each a salary rule.
- **India**: the salary is split into components (basic, house rent, travel…) on the contract.
- **Kenya, Philippines, Mexico**: some allowances and vouchers are exempt up to a cap; the excess is taxable.
- **United States**: pre-tax benefits (retirement, medical) lower the taxable base.

Benefits in kind: Marie's company car

A benefit in kind is pay that never touches a bank account. The law values it with a formula, and payroll adds the value to the taxable base. In Belgium, a car's yearly benefit is catalogue value × age coefficient × 6/7 × a CO2 percentage (4 % for an electric car), with a minimum of 1,690.00 € in 2026.[4]

🧱 **Worked example: Marie's electric SUV**

Catalogue value 52,000.00 €, first registered on 5 January 2026 (age coefficient 100 %), 0 g CO2/km.

Step	Amount (€)
52,000.00 × 6/7 × 4 % × 100 %	1,782.86 per year
Minimum for 2026	1,690.00 per year
ATN.CAR Benefit retained, per month	148.57

The benefit goes into Marie's taxable base, **not** her contribution base; instead the employer pays a monthly CO2 contribution of 42.34 €.[5]

In Odoo. Odoo spreads the same yearly benefit by days instead of twelfths: 151.42 € in October (31/365), not 148.57 €. The year adds up the same.[1]

Exempt within limits: meal vouchers

Many countries exempt a benefit **only within limits**: a cap per day, a minimum employee share, a written policy. Break one condition and the whole amount becomes ordinary pay.

Worked example: Marie's meal vouchers, October 2026

Marie gets one 10.00 € voucher per day worked. October 2026 has 22 working days and she works all of them.

Line	Amount (€)
Vouchers credited: 22 × 10.00	220.00
Employer part 22 × 8.91	196.02
MEAL_V_EMP Employee part 22 × 1.09, deducted from net	−23.98

No voucher is due for a day of sickness or leave, so the count changes every month. The exemption holds only with a written agreement, one voucher per day worked, an employer part of at most 8.91 € and an employee part of at least 1.09 €.[3]

Deductions, and the order they come in

The second design question is about deductions: **does it come before tax or after?**

Kind of deduction	Lowers the taxable base?	Example at MyVidangeCompany
Compulsory employee contributions	yes, in most countries	employee social security, 13.07 %
Pre-tax benefits (pension, medical)	yes, where the law allows	—
Other contributions after tax	no	special social security contribution
Employee share of a benefit, garnishment	no	meal-voucher employee part

Order matters because each rule reads totals computed before it: the tax needs the contributions; a garnishment ceiling needs the net.

Common pitfall

Computing withholding on "gross minus all deductions". At MyVidangeCompany the special contribution and the meal-voucher share come **after** the tax. Tag each deduction with whether it lowers the taxable base.

Three bases, three numbers

Every contribution and every tax is a percentage **of something**. That something is its **base**. A payslip has three of them, and they are three different sums of the same lines:

- **Gross**: what the contract pays. It is the number on the job offer and on the top of the payslip.
- **Contribution base**: the part of the pay that social security takes its percentage of. Often the gross; sometimes less (capped, or without exempt items).
- **Taxable base**: the part of the pay the withholding tax is computed on. Usually the gross, **minus** the employee contributions (most countries do not tax money you never received), **plus** benefits in kind (pay you received, but not in cash).

The two questions of the previous sections decide, line by line, which base each payslip line enters. Marie's October payslip shows all three cases:

Payslip line	Amount (€)	Gross	Contribution base	Taxable base
Salary	7,500.00	✓	✓	✓
Company car (benefit in kind, never paid in cash)	148.57	–	–	✓
Employee contributions 13.07 %	–980.25	–	–	✓ lowers it
The base		**7,500.00**	**7,500.00**	**6,668.32**
What is computed on it		the contract	contributions: 7,500.00 × 13.07 % = 980.25	withholding tax: 1,770.92

Read the last two rows: the contributions are computed **first**, on the contribution base; only then is the taxable base known, because it needs the contributions. That is why a payslip has an order, and why "the salary" means three different numbers to HR, to social security and to the tax office.

Across the cast:

Base	Mélanie	Jurgen	Marie
Gross = contribution base	4,000.00	2,300.00	7,500.00
Employee contributions after low-wage reduction	522.80	1.08	980.25
Benefit in kind (car)	0.00	0.00	148.57
Taxable base	3,477.20	2,298.92	6,668.32

A low-wage reduction cancels nearly all Jurgen's contributions (Chapter 3). Marie's taxable base includes the car; her contribution base does not.

Worked example: Marie, October 2026

Line	Amount (€)
SALARY Gross salary	7,500.00
ONSS Employee social security 13.07 %	–980.25
ATN.CAR Company car (taxable, not paid)	148.57
GROSS Taxable base	6,668.32
P.P Withholding tax (married, spouse without income, 2 children)	–1,770.92
M.ONSS Special social security contribution	–60.94
MEAL_V_EMP Meal vouchers, employee part	–23.98
NET **Net pay**	**4,663.91**

The car is in the taxable base but not in the subtraction: it was never paid. (Odoo's net is 4,684.68 €; Chapter 4 explains the gap.)

Employer cost

Gross is not what an employee costs the company.

$$\text{Employer cost} = \text{gross} + \text{employer contributions} - \text{employer reductions} + \text{employer parts of benefits}$$

Mélanie costs 5,004.80 €, Jurgen 2,502.97 € and Marie 9,622.36 €.[2] Chapter 3 builds each of these line by line.

How Odoo names the bases

The generic engine has salary-rule categories `BASIC`, `GROSS` and `NET`. A localization adds its own totals and may give the generic codes a local meaning: Belgium adds **ONSS_BASE** and **WITHHOLDING_BASE**, and uses `GROSS` for the **taxable** salary.

Concept	Belgian rule code	Name in Odoo	Mélanie
Gross before contributions	`SALARY`	Gross Salary	4,000.00
Employee social security	`ONSS`	Social contribution	–522.80
Taxable base	`GROSS`	**Taxable Salary**	3,477.20
Net pay	**NET**	**Net Salary**	**2,614.26**

In Odoo. A salary rule can belong to several categories (**category_ids** on **hr.salary.rule**), so one line can feed both bases: the "two questions" become data. The car is a rule `ATN.CAR` that feeds the withholding base, followed by `ATN_DED` that takes the same amount out of the net.

> ⚠️ **Common pitfall**
>
> Assuming `GROSS` means the same in every localization. In `l10n_be_hr_payroll` it is the taxable salary, so a cross-country "gross salaries" report on `GROSS` is wrong by 13.07 % for Belgium.

> 💡 **In practice**
>
> When an accountant says "salary", ask which one: gross, contribution base or taxable. Put the answer in the category or rule name.

Summary

- Every payslip has earnings, employee deductions, tax, net and employer cost.
- Each earning needs two answers: is it subject to contributions, and is it taxable? They are independent.
- Benefits in kind raise the taxable base but are never paid: Marie's car adds 148.57 € to her taxable base and nothing to her net.
- Some benefits are exempt only within limits. Only some deductions come before tax; tag each one.
- Gross, contribution base and taxable base are three numbers: for Mélanie 4,000.00 €, 4,000.00 € and 3,477.20 €.
- Employer cost = gross + employer contributions − reductions + employer benefits: 5,004.80 € for Mélanie, 9,622.36 € for Marie.
- Localizations name their bases differently: in Belgium, `GROSS` is the taxable salary.

---

1. Payslips computed by Odoo master on the demo database, `research/numbers_be.md` (Marie, February 2026, and the October 2026 summary). ↩
2. Independent calculator, `research/numbers_be_calc.md`, sections 2 and 10.3. ↩
3. ONSS, meal vouchers, conditions (employer part 8.91 € from 1 January 2026): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/salary/particularcases/lunchcheques/salaryfeatures.html. ↩
4. SPF Finances, FAQ company cars 2026 (formula, reference emissions, minimum 1,690 €): https://finances.belgium.be/sites/default/files/downloads/121-faq-voitures-de-societe-2026.pdf. ↩
5. ONSS, solidarity contribution on company cars: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/special_contributions/companycar.html. ↩

Contributions: employee and employer

A rate, a base, a floor, a cap, and the reductions that make low wages cheaper

AFTER THIS CHAPTER YOU CAN

- Describe any social contribution with five parameters: base, rate, floor, cap and who pays
- Explain why a capped contribution is regressive and an uncapped one is not
- Explain how low-wage reductions work, and why they make a raise expensive
- Handle contributions computed over a quarter or a year, not a month
- Compare the employer cost of three employees, and say why rates must be stored with dates

Mélanie pays 522.80 € of social security on 4,000.00 €. Jurgen pays 1.08 € on 2,300.00 €. Same law, same 13.07 % rate. Some countries shape contributions with a **cap**; Belgium has none and uses **reductions** instead. Both are variations on one small model.

What a social contribution is

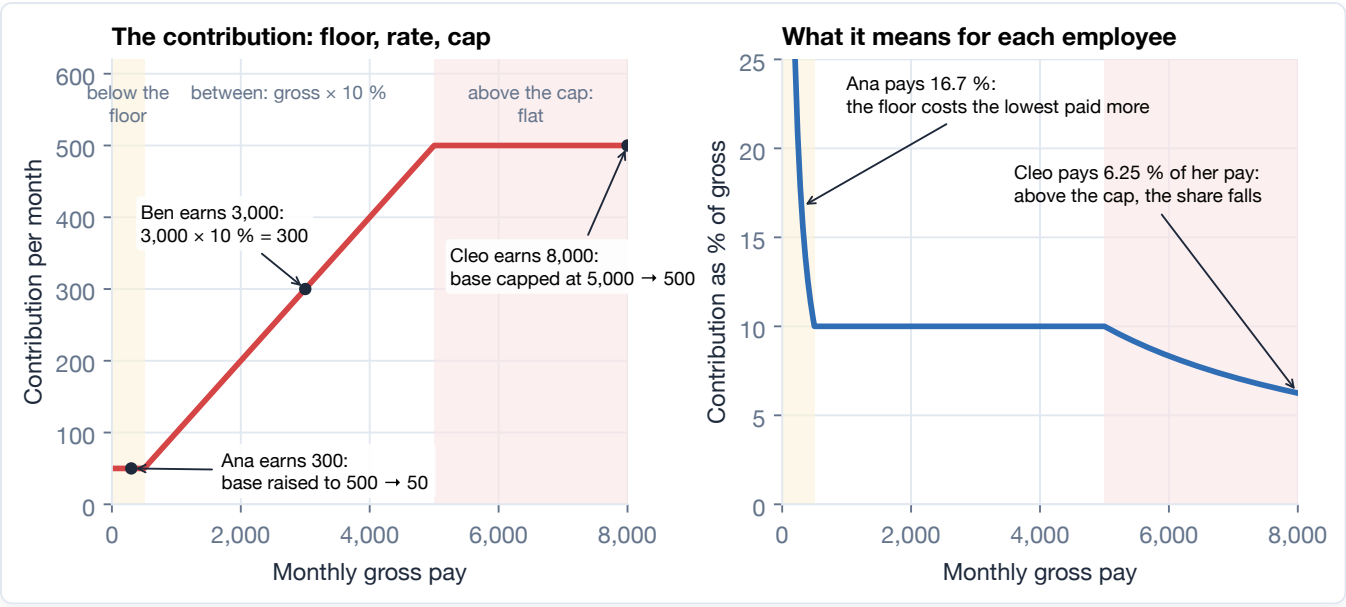
A **social contribution** (*social security contribution*, *social charge*, *national insurance*; some countries say *payroll tax*) is a compulsory payment, computed on pay, that finances social insurance: pensions, health care, unemployment, family benefits. Unlike income tax, it is tied to the job and usually paid by both sides: an **employee share**, deducted from the gross, and an **employer share**, paid on top of it.

Three things follow from that definition:

- **It is computed per period, on the pay of that period.** No yearly estimate, no settlement afterwards, unlike withholding (Chapter 4).
- **It buys rights.** Pensions and benefits are often computed from the pay declared for contributions, so a wrong base can cost the employee later, not only the fund now.
- **Each fund is a small formula.** A base, a rate, and sometimes a floor, a cap and a reduction: the next section takes it apart.

Anatomy of a contribution

A contribution is a percentage of the pay, with two limits. Take one imaginary fund with round numbers: **10 % of gross pay, with a minimum base of 500 and a maximum base of 5,000** a month, and three employees.



Illustrative fund: 10 % of gross, base at least 500 and at most 5,000. Left: what each employee pays. Right: the same amount as a share of their pay.

- **The rate** applies in the middle band. Ben earns 3,000 and pays 3,000 × 10 % = **300**.
- **The floor** is a minimum base: below it, the fund charges as if the pay were 500. Ana earns 300 and still pays **50**, that is 16.7 % of her pay. Some funds use a minimum contribution instead.
- **The cap** is a maximum base: above it, nothing more is due. Cleo earns 8,000 and pays **500**, only 6.25 % of her pay. A cap makes a contribution **regressive**: the higher the pay, the smaller the share.
- **Reductions.** Some laws then subtract a reduction, for low wages or for target groups; Jurgen's case below.

$$\text{Contribution} = \min(\max(\text{base}, \text{floor}), \text{cap}) \times \text{rate} - \text{reductions}$$

Every country's contributions are a combination of these knobs:

Knob	Question	Variants
Base	which earnings count?	gross; basic only; basic + housing
Rate	how much, for whom?	one per fund; by side, nationality or sector
Floor	minimum base or amount?	none; a minimum contribution
Cap	a ceiling?	none; per month; per year to date; tiers
Who pays	employee, employer, both?	employer only for accident funds
Reductions	who pays less?	low wages; first hires; target groups

Around the world

- **United States**: Social Security has a yearly cap tracked year to date; Medicare has none.
- **India**: provident fund on basic salary; state insurance only below a wage threshold.
- **Saudi Arabia**: on basic + housing; the employee share only for nationals.
- **Mexico**: on an integrated daily wage, capped at 25 × a legal reference unit.
- **Kenya**: pension in two tiers between a lower and an upper limit.

Case study: MyVidangeCompany's contributions

Belgium sits at one end of the scale: one office (ONSS) collects everything, the employee pays one rate on an uncapped base, and the employer pays a sum of rates on the same base.[1]

	Mélanie	Jurgen	Marie
Contribution base	4,000.00	2,300.00	7,500.00
Employee 13.07 %	522.80	300.61	980.25
Low-wage reduction	0.00	−299.53	0.00
Employee contributions withheld	522.80	1.08	980.25
Employer 25.12 %	1,004.80	577.76	1,884.00

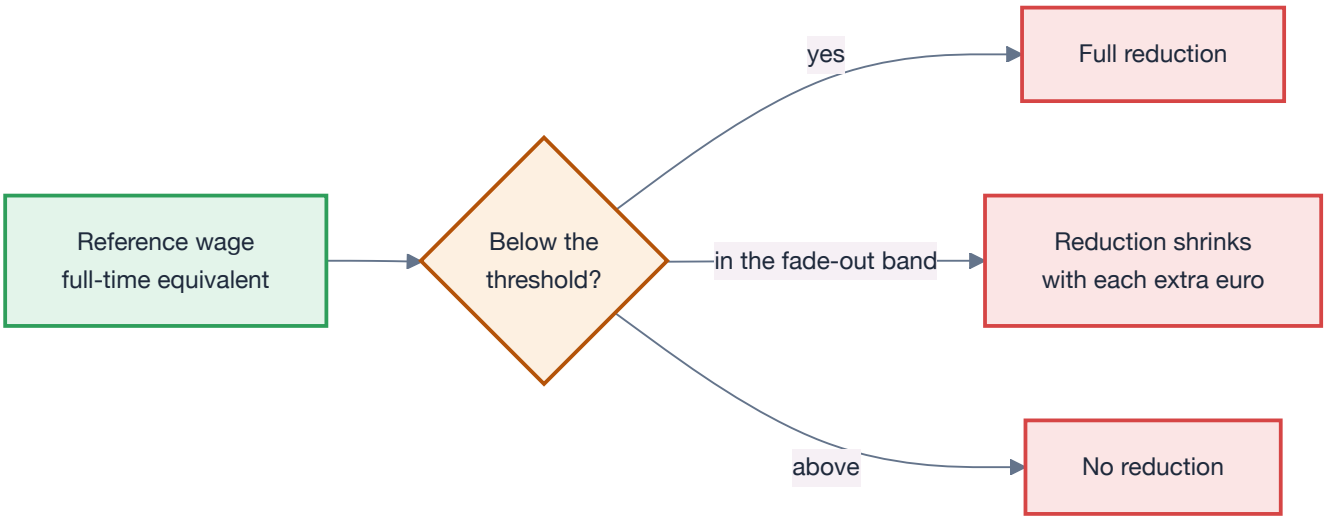
With no cap, Marie pays the same 13.07 % as Mélanie. The employer rate is a sum of components, each with its own dates: 25.13 % in the first three quarters of 2026, 25.12 % in the fourth, because a 0.01 % asbestos fund is collected only until September.[3]

In Odoo. Odoo reads the employer rates from the social security office's official rate file, which includes small funds (closure fund, sector fund) the calculator leaves out: Mélanie's October employer contribution is 1,030.49 € in Odoo against 1,004.80 €.[7]

> 💡 **In practice**
>
> An employer rate is never "a number". Store it as components, each with its own dates. One combined constant would silently be wrong for Q4 2026.

Reductions for low wages

Many countries want low-paid jobs to be cheap and a low wage's raise to reach the pocket. They use **reductions**: an amount subtracted from the contribution, large for low wages, fading to zero as the wage rises.

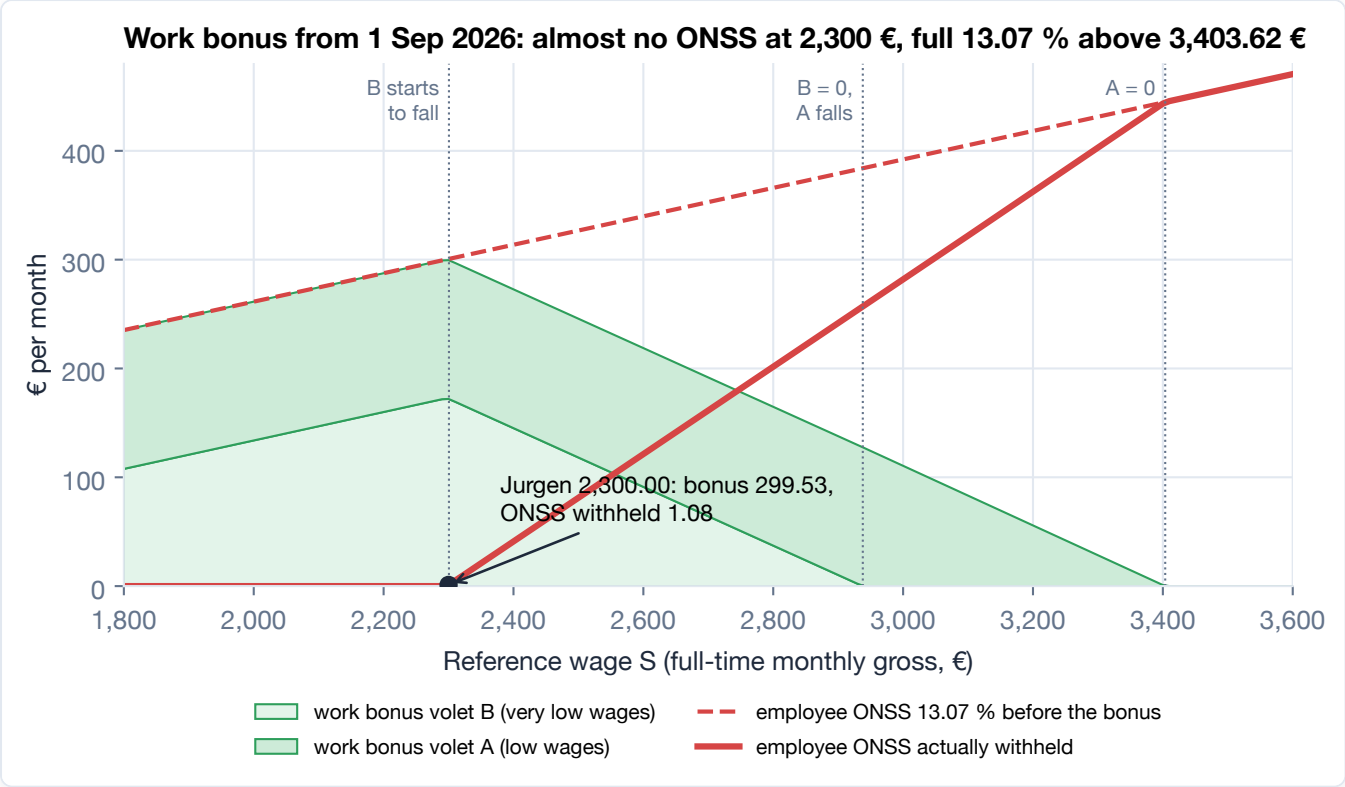


Two design points follow:

- It is tested on a **reference wage**, usually the full-time monthly equivalent, so a part-month joiner does not look like a low earner (Chapter 5).
- In the fade-out band, a raise also shrinks the reduction, so its **marginal** cost is much higher than the rate suggests.

Case study: Jurgen

Belgium has a reduction on each side: the employee "work bonus" and the employer "structural reduction".



Belgium, employee contributions before and after the low-wage reduction, by reference wage. At 2,300.00 € Jurgen gets the full reduction; above 3,403.62 € nobody gets any.

Worked example: Jurgen, October 2026

Line	Amount (€)
ONSS 13.07 % × 2,300.00	−300.61
EmpBonus.A + **EmpBonus.B** Employee low-wage reduction	+299.53
Employee contributions withheld	−1.08
ONSSEMPLOYER Employer 25.12 %	577.76
ONSS_STRUCTURAL Employer low-wage reduction	−374.79
Employer contributions after reduction	202.97

Jurgen's taxable base goes up by the same 299.53 €: a contribution not deducted is taxed like salary. Mélanie gets no reduction.[6]

In Belgium

The work bonus has two parts, each a flat amount that fades to zero across a wage band (fully gone at 3,403.62 €). Its table changed five times in 2026.[2] The employer reduction is a quarterly formula with parameters published each quarter (Q4 2026 not yet published, `UNVERIFIED`).[4]

In Odoo. Odoo computes the same employee reduction for Jurgen. Its employer reduction is 395.91 €, because the calculator reuses the Q3 parameters for Q4.[7]

Common pitfall

Computing the reference wage from "all remuneration of the month". In Belgium it leaves out the year-end premium, exit holiday pay and notice indemnities;[2] including them takes Jurgen's reduction away in December. Every reduction defines the wage it tests.

Contributions over a quarter or a year

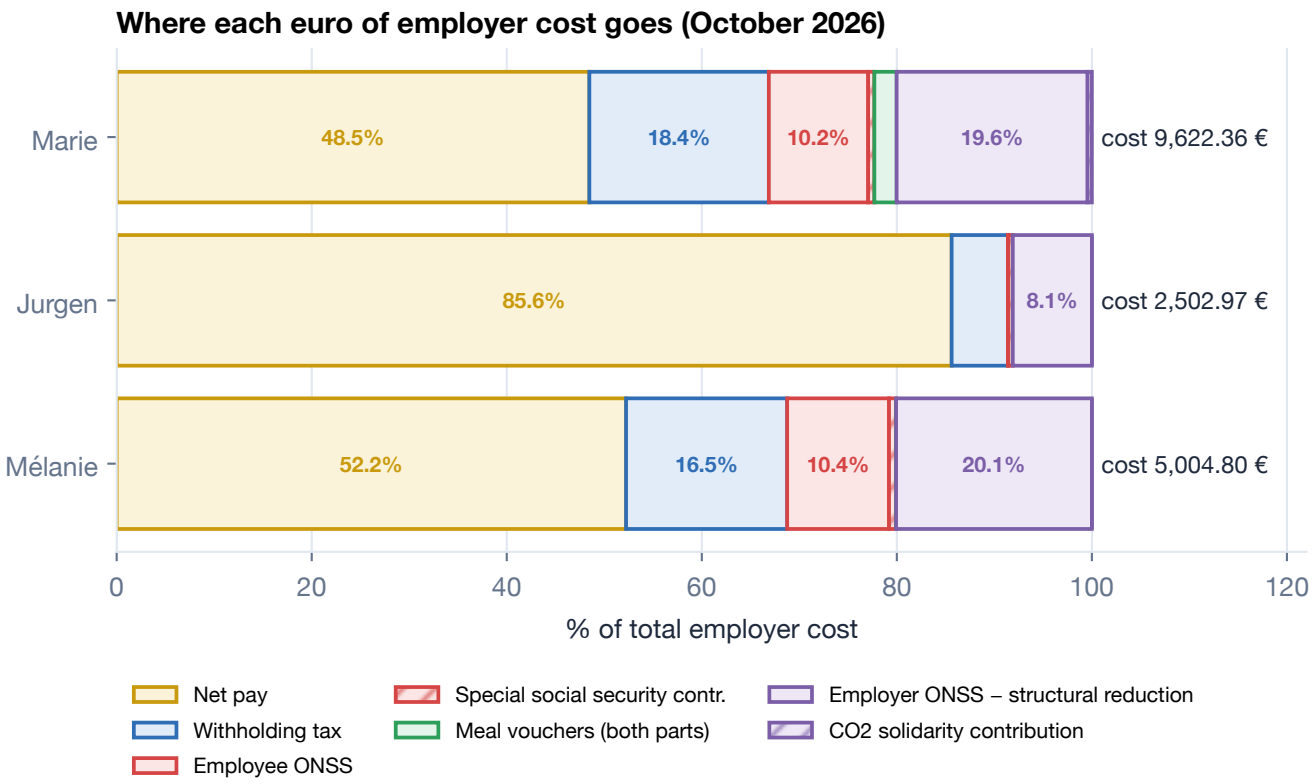
A cap or scale is often defined over a **longer period** than the payslip: a yearly cap (US Social Security) or a quarterly scale (Belgium). Each payslip withholds an advance and a later one **regularises**.

$$\text{This payslip} = \text{contribution on the period's base so far} - \text{what earlier payslips of the period already took}$$

Belgium's example is the special social security contribution: a scale on the **quarterly** base, withheld monthly and fixed on the quarter's last payslip. It does not lower the taxable base.[5] Mélanie pays 36.24 € a month; Marie, above the top of the scale, pays the maximum, 60.94 €. In December, Mélanie's year-end premium raises her quarterly base and her December payslip catches up (Chapter 6).

In Odoo. Year-to-date and quarter-to-date rules read the employee's earlier payslips of the period.

Employer cost: Mélanie, Jurgen, Marie



What each employee costs MyVidangeCompany in October 2026, split by destination. Net pay is 85.62 % of the cost for Jurgen, 52.24 % for Mélanie and 48.47 % for Marie.

October 2026	Mélanie	Jurgen	Marie
Gross	4,000.00	2,300.00	7,500.00
Employer cost	5,004.80	2,502.97	9,622.36
Net pay	**2,614.26**	**2,143.05**	**4,663.91**
Employer cost per 1 € of net	1.91	1.17	2.06

Reductions make a low wage cheap, but they fade out: Jurgen's 100 € raise costs MyVidangeCompany 155.12 €, Mélanie's 125.12 €.[6] With a cap the picture flips at the top: above it, a raise costs only the gross.

In Odoo. Each contribution is a salary rule (`ONSS` , `EmpBonus.A` , `M.ONSS` , then employer rules after the net such as `ONSSEMPLOYER`). Rates and thresholds are dated **hr.rule.parameter** values (Chapter 21).

Summary

- Any contribution is a base, a rate, a floor, a cap, a payer and possible reductions. Learn the knobs once.
- A cap makes a contribution regressive above it; an uncapped contribution stays proportional.
- Belgium: one uncapped employee rate of 13.07 %; an employer rate that is a sum of dated components.
- Low-wage reductions fade out with a full-time reference wage: Jurgen pays 1.08 € instead of 300.61 €, and his 100 € raise costs 155.12 €.

- Caps and scales defined over a quarter or a year are withheld as advances and regularised later.
- Employer cost per euro of net: 1.17 for Jurgen, 1.91 for Mélanie, 2.06 for Marie.

---

1. ONSS, contributions (employee 13.07 %, branches, employer basic rate 19.88 %, wage moderation, 1.60 %, 108 %): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/socialsecuritycontributions/contributions.html. ↩
2. ONSS, work bonus (2026/3 instructions; earlier quarters under `/fr/2026-1/` and `/fr/2026-2/`): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/deductions/workers_reductions/workbonus.html. ↩ ↩
3. ONSS, asbestos fund (collected in Q1, Q2 and Q3 2026): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/special_contributions/other_specialcontributions/asbestosfunds.html. ↩
4. ONSS, structural reduction (Q3 2026; Q1 and Q2 under `/fr/2026-1/` and `/fr/2026-2/`): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/deductions/structuralreduction_targetgroupreductions/structuralreduction.html. ↩
5. ONSS, special social security contribution: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/special_contributions/other_specialcontributions/specialsocialsecuritycontribution.html. ↩
6. Independent calculator, `research/numbers_be_calc.md`, sections 2, 10.3 and 10.4. ↩ ↩
7. Payslips computed by Odoo master on the demo database, `research/numbers_be.md` (Marie, February 2026, for the rate lines; October 2026 summary). ↩ ↩

Withholding tax

A yearly tax, collected in instalments: five ways to do it

AFTER THIS CHAPTER YOU CAN

- Explain withholding as an advance on a yearly income tax, and name the main withholding methods
- Compute a progressive tax slice by slice
- Run an annualisation formula by hand, for a single employee and for a married one
- Tell the marginal rate from the effective rate, and find the low-wage trap
- Explain why bonuses and other irregular pay need their own tax method

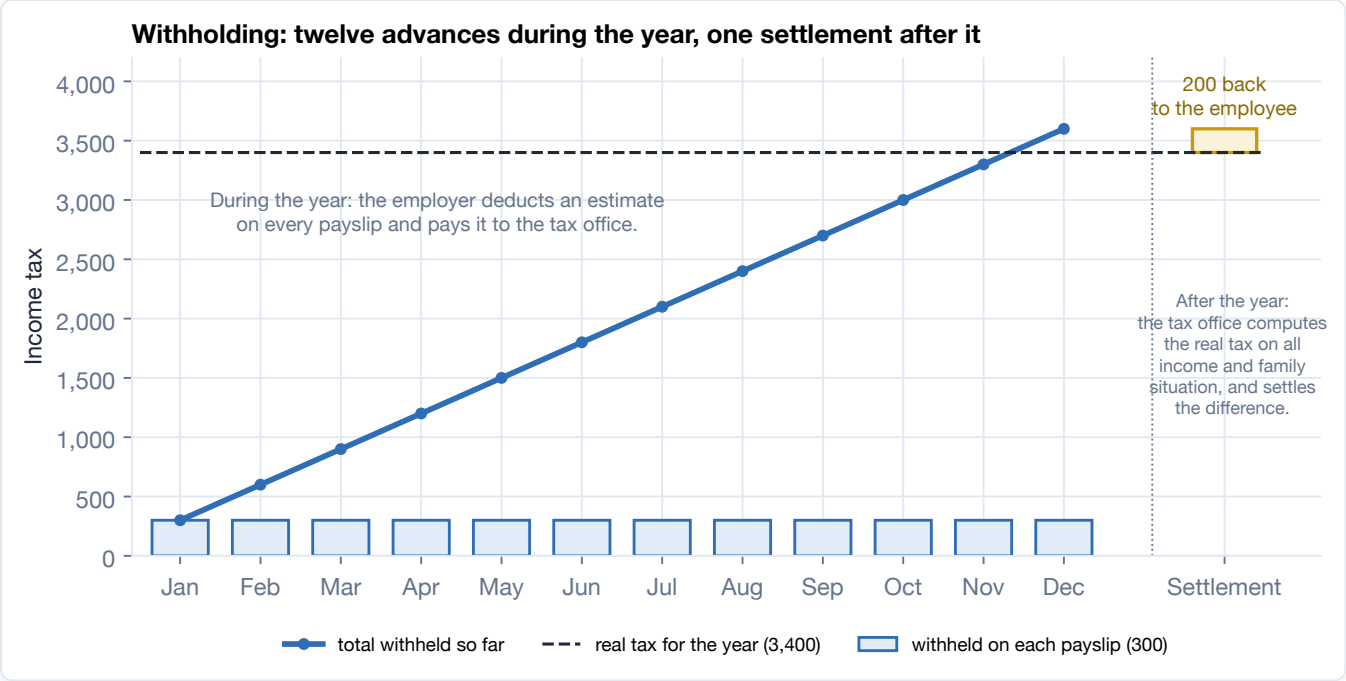
Mélanie's withholding tax for October 2026 is 826.70 €. It is not "the tax on 4,000 €": it is one twelfth of a yearly tax, computed as if she earned this salary every month. Another country would look at what she earned since January, read a table, or not withhold at all. Knowing which road a country takes explains almost every rule in its tax module.

What a withholding tax is

Income tax is a tax on a person's income for a whole year. In most countries the employee does not pay it in one go at the end of the year. Instead, the employer deducts a part of it from **every payslip** and pays that part to the tax office on the employee's behalf. That deduction is the **withholding tax** (also called *pay-as-you-earn*, *PAYE*, or *tax at source*).

Three things follow from that definition:

- **It is the employee's tax, not the employer's.** It comes out of the gross, like the employee contributions. The employer is only a collector: it deducts, declares and pays.
- **It is an estimate.** On the payslip, the employer does not know the employee's whole year: other income, a second job, a partner's income, deductible expenses. So each instalment is a best guess.
- **The real tax comes later.** After the year, the tax office computes the real tax on everything and settles the difference: a refund, or a bill to the employee.



Illustrative numbers. The employer withholds 300 a month, 3,600 over the year. The real tax, computed after the year, is 3,400: the tax office refunds 200 to the employee. Had the real tax been higher, the employee would pay the difference.

Withholding is an advance

So every localization answers one question: *how do you turn a yearly tax into a monthly instalment?* Countries pick one of five methods.

1 · Annualise one period

HOW	Tax this month's pay as if it came every month of the year, then divide the yearly tax by the number of periods.
WHERE	Belgium, US federal, Egypt, Pakistan
CORRECTS?	**No**: the final assessment settles the year.

2 · Cumulative

HOW	Tax the income since the start of the year (or projected for the year), minus the tax already withheld.
WHERE	India, Turkey, Poland
CORRECTS?	**Yes**, every period.

3 · Period tables

HOW	Read the tax from an official table made for the pay frequency (month, week…).
WHERE	Mexico, the Netherlands
CORRECTS?	Only as far as the table is designed to.

4 · None

HOW	No income tax on salaries, so nothing to withhold.
WHERE	UAE, Saudi Arabia
CORRECTS?	Nothing to correct.

5 · Employee files himself

HOW	The employer withholds nothing and only reports the salaries; the tax office collects from the employee.
WHERE	Hong Kong
CORRECTS?	Nothing to correct on the payslip.

The method shapes the code. An annualising country needs only this month's lines; a cumulative one needs every earlier payslip of the tax year, including a previous employer's.

🌍 **Around the world**

- **India**: cumulative; actual months plus projected months give the annual tax, spread over the months left.
- **Mexico**: tax tables per pay frequency, plus an employment subsidy.
- **United States**: annualise per period with the employee's W-4, then a second layer of state tax.
- **Australia**: period pay converted to weekly, put through coefficients (a × x − b), converted back.
- **Hong Kong**: no withholding; each employee pays his own tax.

Progressive scales: slice by slice

Most income taxes are **progressive**: each slice of income is taxed at its own band's rate. Belgium's 2026 scale:[3]

Annual net taxable (€)	Tax
0.01 – 16,710.00	26.75 %
16,710.01 – 29,500.00	4,469.93 + 42.80 % above 16,710
29,500.01 – 51,050.00	9,944.05 + 48.15 % above 29,500
above 51,050.00	20,320.38 + 53.50 % above 51,050

The fixed amounts are the tax on all slices below each band, precomputed so a rule looks up one row.

🌍 **Around the world**

- **Netherlands**: monthly brackets, the annual table divided by 12, with separate tables for residents and non-residents.
- **Kenya**: monthly bands, then fixed reliefs subtracted (personal relief, insurance, housing levy).
- **Morocco**: one monthly table where each bracket has a rate and a fixed deduction, the same shortcut as Belgium's fixed amounts.
- **Turkey, Poland**: the bands apply to the income since 1 January, so an employee climbs into a higher band during the year.
- **Lithuania**: one flat rate, but the tax-free amount shrinks as income rises, which makes the tax progressive anyway.

⚠️ **Common pitfall**

"Mélanie is in the 48.15 % band, so her tax is 48.15 %." Only the slice above 29,500 € is. Her withholding is 20.67 % of her gross.[7]

Case study: Belgium annualises

Belgium's official "key formula" turns one month into a year, applies the scale and family relief, and divides by twelve.[3]

Step	Rule (2026)
A. Annual gross	monthly taxable base × 12
B. Annual net	minus a flat allowance for professional expenses: 30 %, at most 6,070 €
C. Annual tax	scale, minus the tax on a tax-free sum, minus family reductions; never below 0
D. Monthly	annual tax ÷ 12
E. Reductions	low-wage tax credit, overtime, part of pension contributions; never below 0

🧱 **Worked example: Mélanie, October 2026**

Step	Amount (€)
GROSS Monthly taxable base	3,477.20
A. × 12	41,726.40
B. – professional expenses (capped)	–6,070.00
Annual net taxable	35,656.40
C. Scale: 9,944.05 + 48.15 % × (35,656.40 – 29,500)	12,908.36
– tax on the tax-free sum of 11,170 €	–2,987.98
Annual tax	9,920.38
P.P D. ÷ 12	826.70

The calculator reproduces the official simulator's examples to the cent.[5]

In Odoo. Mélanie's payslip shows 826.69 €: Odoo rounds the monthly amount down, where the official formula rounds half up. Chapter 10 lists this as a question for an expert.[8]

🔑 **Key idea**

An annualising method aims at the tax of someone who earns **this** month's pay twelve times. Anything else (other income, a partial year, a raise) is settled by the final assessment. A cumulative method makes the opposite choice: each month corrects the past.

Family situation

Almost every income tax adjusts for the household, with few levers: a personal allowance (Belgium, Kenya), a reduction per child (Belgium, Morocco), splitting with a spouse (Belgium), a tax class by family status (Luxembourg, Indonesia), or the employee's own form (US W-4).

Marie is married, her spouse has no income, and they have two children. Belgium gives her a **marital quotient**, which moves up to 30 % of her income (capped) to her spouse, taxed in the lowest band with a second tax-free sum, and a **reduction for two children**.[3]

Worked example: Marie, October 2026

Taxable base 6,668.32 € (with the car, Chapter 2).

Step	Amount (€)
Annual gross 6,668.32 × 12	80,019.84
– professional expenses	–6,070.00
Annual net taxable	73,949.84
Marital quotient: 30 % is above the cap, so	13,790.00
Scale on the quotient	3,688.83
Scale on the rest (73,949.84 – 13,790.00)	25,194.14
– tax-free sum twice (2 × 2,987.98)	–5,975.96
Base tax	22,907.01
– reduction for 2 children	–1,656.00
P.P ÷ 12	1,770.92

With the same taxable base, a single person would have 2,465.32 € withheld.[7]

? Open question

Odoo withholds 1,750.15 € from Marie, 20.77 € less, because it deducts a yearly 500 € transport exemption. Whether an employee with a company car is entitled to it is not settled by the sources read for this book (Chapter 10).[8]

A formula that changes in November

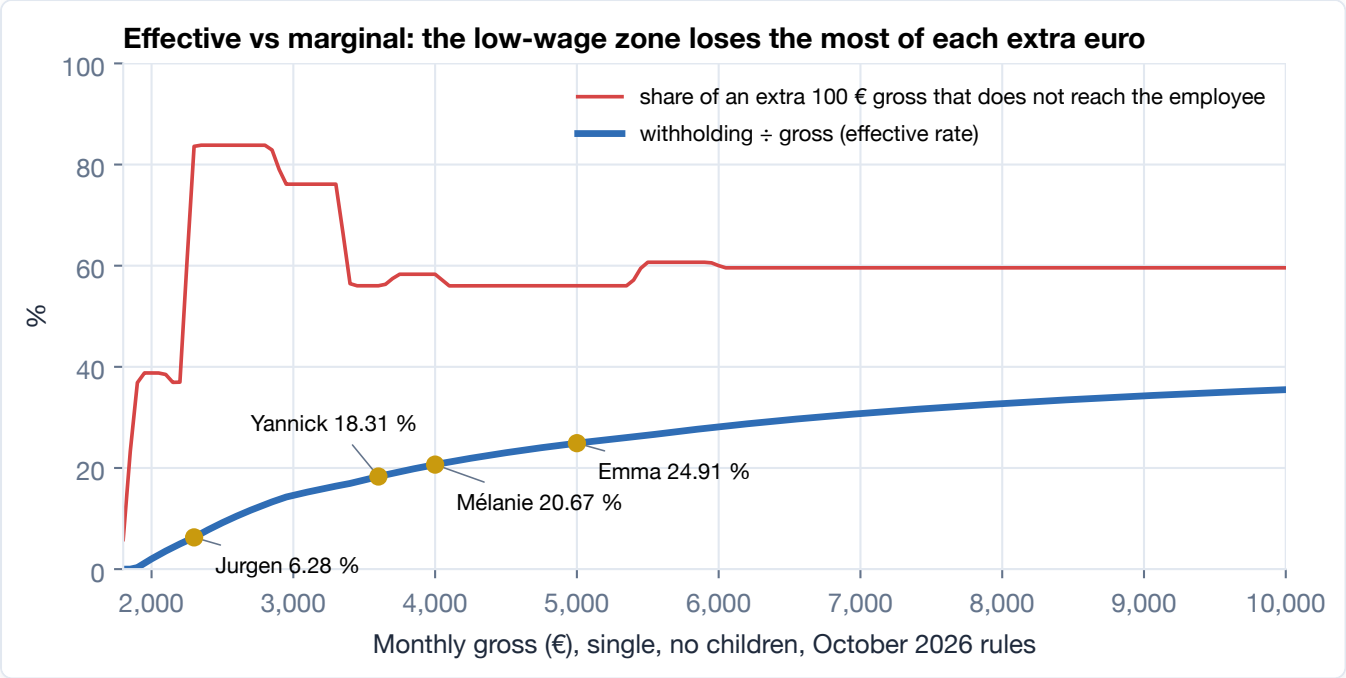
Tax parameters are dated. Belgium published a new formula on 2 October 2026 for pay from 1 November: a higher tax-free sum, but a lower cap on the marital quotient and smaller child reductions.[2]

	Mélanie	Jurgen	Marie
Withholding October (Jan-2026 formula)	826.70	144.33	1,770.92
Withholding November (Nov-2026 formula)	818.23	135.86	1,799.78
Difference	–8.47	–8.47	**+28.86**

The singles gain only the higher tax-free sum. Marie gets it twice, but her capped quotient keeps more of her income in her own high band.[4] Chapter 11 shows how Odoo keeps both formulas, each with its start date.

Marginal and effective rates

The **effective rate** is tax ÷ gross. The **marginal rate** is what happens to one more euro, and it is not the band rate when contributions and reductions move too.



Single employee, Belgian October 2026 rules. The effective withholding rate climbs smoothly. The share of an extra 100 € lost to contributions and tax jumps where the low-wage reduction fades out.

+100.00 € gross per month	Mélanie	Jurgen	Marie
Extra employee contributions	13.07	39.89	13.07
Extra withholding	41.86	42.61	46.51
Extra special contribution	3.38	1.10	0.00
Extra net	**41.69**	**16.40**	**40.42**
Share of the 100 € lost	58.31 %	83.60 %	59.58 %

Jurgen keeps 16.40 € of a 100 € raise: his contributions rise by 39.89 €, not 13.07 €, because his low-wage reduction shrinks. His effective rate is only 6.28 %, but his marginal loss is the cast's highest.[7] This "low-wage trap" exists wherever a reduction fades out.

> 💡 **In practice**
>
> When HR asks "what does a 100 € raise give net?", run the payslip twice and subtract. Never answer with the band rate.

Irregular pay: bonuses, holiday pay, indemnities

An annualising formula applied to a once-a-year bonus would tax it as if paid every month, far too high. So countries use a separate method:

Method	How	Examples in Odoo
Flat rate by bracket	a fixed rate chosen from normal annual pay	Belgium
Difference method	tax(year + bonus) − tax(year)	Mexico
Separate rule	its own table or formula	Luxembourg

Method	How	Examples in Odoo
None needed	a cumulative method absorbs the bonus	India, Turkey

In Belgium, holiday pay and bonuses get a flat rate chosen from annual normal pay; a notice indemnity gets a third scale.[6]

Mélanie, annual normal gross 41,726.40 €	Taxable	Rate	Withholding
Holiday bonus, June 2026	3,235.62	42.39 %	1,371.58
Year-end premium, December 2026	3,477.20	46.44 %	1,614.81

Odoo's June payslip gives Mélanie the same 1,371.58 €.[8] The calculator takes "annual normal gross" as (monthly gross − employee contributions) × 12, common practice that the rules retrieved do not define (`UNVERIFIED`).

Withholding is not the final tax

	Withholding	Final tax
Computed by	the employer, every period	the tax office, once a year
Basis	the payslip (one month, or the year so far)	the real year's income, all sources
Who settles the difference	—	the employee, with the tax office

Yannick joins in March. His March withholding of 163.37 € assumes he earns 1,991.49 € taxable every month of the year; the assessment settles the difference with his previous employer's income. This book computes no final tax (`UNVERIFIED`).[1]

In Odoo. Belgium's formula is a chain of salary rules, from `GROSS.Y` (× 12) through `Y.P.P` (scale) to `P.P`, with dated rule parameters for the scale and family table. A cumulative localization looks different: India's `TDS` rule calls a Python method that reads the year's payslips.

Summary

- Withholding is an advance on a yearly tax. Countries turn the year into periods by annualising, by cumulating year to date, with period tables, or not at all.
- Progressive scales are computed slice by slice; the band rate is not the tax rate.
- Belgium annualises: × 12, − professional expenses, scale, − tax-free sum, − family reductions, ÷ 12. Mélanie: 826.70 €.
- Family relief uses a few levers: allowances, dependants, spouse splitting, tax classes. Marie: 1,770.92 € instead of 2,465.32 € as a single person.
- Tax parameters are dated; the 1 November 2026 formula lowers singles' withholding by 8.47 € and raises Marie's by 28.86 €.
- The marginal loss is not the band rate: Jurgen keeps only 16.40 € of a 100 € raise.
- Bonuses and other irregular pay need their own method; an annualising formula would overtax them.

1. SPF Finances, withholding tax return (withholding is an advance credited against the final tax): https://finances.belgium.be/fr/entreprises/personnel_et_remuneration/precompte_professionnel/declaration. ↩
2. SPF Finances, Annex III AR/CIR 92 version history (AR 11.12.2025, AR 14.02.2026, AR 24.07.2026, AR 27.09.2026 published 2.10.2026): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/439228c6-67bb-4e75-a9e1-4cabe4dc62ec. ↩
3. SPF Finances, key formula from 1 January 2026 (n° 3, 5, 7–21 and annexes 1, 3, 4): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/389035ca-432e-422b-859d-2d32fbdaa4cf. ↩↩↩
4. SPF Finances, Annex III consolidated with the AR of 27 September 2026 (formula from 1 November 2026, n° 32, 34, 38–40, 54): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/0c174ec5-448b-4d1e-983b-fc457e5e7e28. ↩
5. SPF Finances withholding simulators 2026, January version https://www.minfin.fgov.be/myminfin-rest/fisconetPlus/public/document/addenda/SimulateurPrP2026verrouilleFR.xlsx and November version https://www.minfin.fgov.be/myminfin-rest/fisconetPlus/public/document/addenda/SimulateurPrP2026novembreverrouill%C3%A9.xlsx. ↩
6. SPF Finances, Annex III from 1 January 2026, n° 53–62 (exceptional income, arrears and indemnities): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/536a741e-81d6-4605-ad11-8e07ebc4a440. ↩
7. Independent calculator, `research/numbers_be_calc.md`, sections 10.3 to 10.6. ↩↩↩
8. Payslips computed by Odoo master on the demo database, `research/numbers_be.md`: Marie's February 2026 lines (`TRANSPORT_TAX_DED`), the October 2026 summary, the double holiday pay and Emma's 13th month. ↩↩↩

Time, proration and overtime

Partial months, absences, and what an hour is worth

AFTER THIS CHAPTER YOU CAN

- Prorate a monthly salary with the common methods and explain why they differ
- Compute a mid-month joiner's first payslip, and test thresholds on a full-month equivalent
- Classify absences as paid by the employer, paid by a fund, or unpaid
- Compute an hourly rate and an overtime premium
- Describe how Odoo turns time into payslip lines

Yannick joins MyVidangeCompany on Thursday 12 March 2026 at 3,600.00 € a month. How much does he earn for March? Four payroll teams, four answers: 2,270.77 €, 2,290.91 €, 2,322.58 € or 2,326.15 €. All four are defensible. This chapter is about choosing a denominator, and about everything else that turns time into money.

What proration is

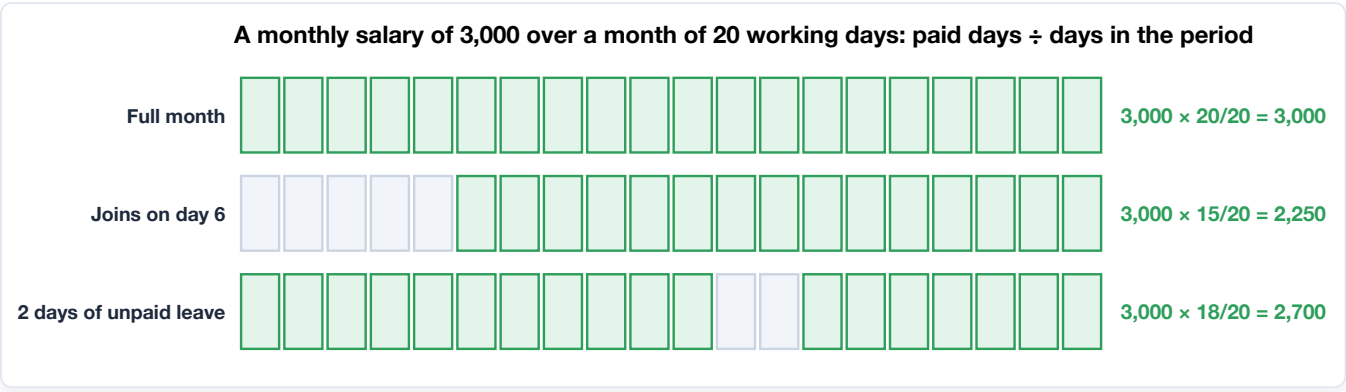
A monthly salary pays for a full month. When the employee is paid for only part of it (he joins or leaves mid-month, or takes unpaid days), the payroll pays the same share of the salary. That is **proration** (*pro rata*, *prorating*, *partial-period pay*):

$$\text{Prorated salary} = \text{Monthly salary} \times \text{paid time} \div \text{time in the period}$$

The **paid time** is the part of the period the employer pays for: days worked, plus paid days off such as public holidays or paid leave.

Three things follow from that definition:

- **Time only matters when the month is incomplete.** A monthly employee gets the same salary every full month, whatever the number of days in it.
- **The denominator is a choice.** Working days, calendar days, a fixed 30 days: each gives a different amount for the same absence.
- **Every absence needs an answer to "who pays?"** A paid day stays in the paid time; an unpaid day leaves it, and a day paid by a fund leaves the payslip too.



Illustrative numbers: 3,000 a month, 20 working days in the month, the working-days method. A joiner on day 6 is paid 15 days; two unpaid days cost 2/20 of the salary.

Time: the inputs

Three inputs decide how time becomes money:

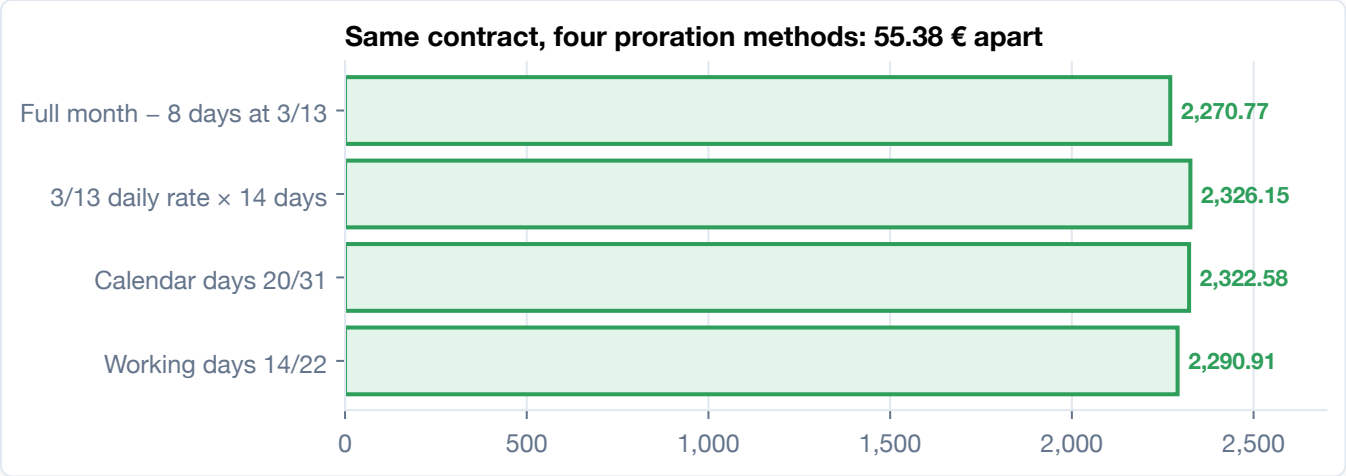
- **The work schedule**: hours per day, days per week. The cast works 38 hours, Monday to Friday (`UNVERIFIED` as the sector's working time).
- **Public holidays**: paid days off. Belgium has ten.[2]
- **The pay basis**: monthly, or hourly and daily, where every hour or day worked is paid.

Proration: the denominator is a policy



March 2026: 22 working days. Yannick is under contract for 14 of them (and 20 of the 31 calendar days).

Method	Formula	Yannick, March 2026
Working days	3,600.00 × 14 / 22	2,290.91
Calendar days	3,600.00 × 20 / 31	2,322.58
Fixed daily rate × days worked	3,600.00 × 3/13 ÷ 5 × 14	2,326.15
Full month – missing days at a fixed daily rate	3,600.00 – 3,600.00 × 3/13 ÷ 5 × 8	2,270.77



The same contract and the same start date give four March salaries.

The "fixed daily rate" is a Belgian convention: a week is worth monthly × 3/13 (a quarter has 3 months and 13 weeks), a day a fifth of that.[3] Other countries use a 30-day month, 30.4 days, or a 12-month average.

🔑 **Key idea**

Proration is a ratio, and the **denominator is a policy**. Belgian law sets no method for a monthly employee's incomplete month (`UNVERIFIED` : practice, not law). Choose one, write it down, and use it for joiners, leavers and unpaid days alike.

🌍 **Around the world**

- **Saudi Arabia**: real calendar days or a fixed 30-day month.
- **Switzerland**: a company option for the 30-day method.
- **Mexico**: 30.4 days per month for tax.
- **Hong Kong**: leave pay uses the average daily wage of the last 12 months.
- **Philippines**: a daily wage on the contract, converted between pay frequencies.

Case study: Yannick, a mid-month joiner

Worked example: Yannick, March 2026 (working-days method)

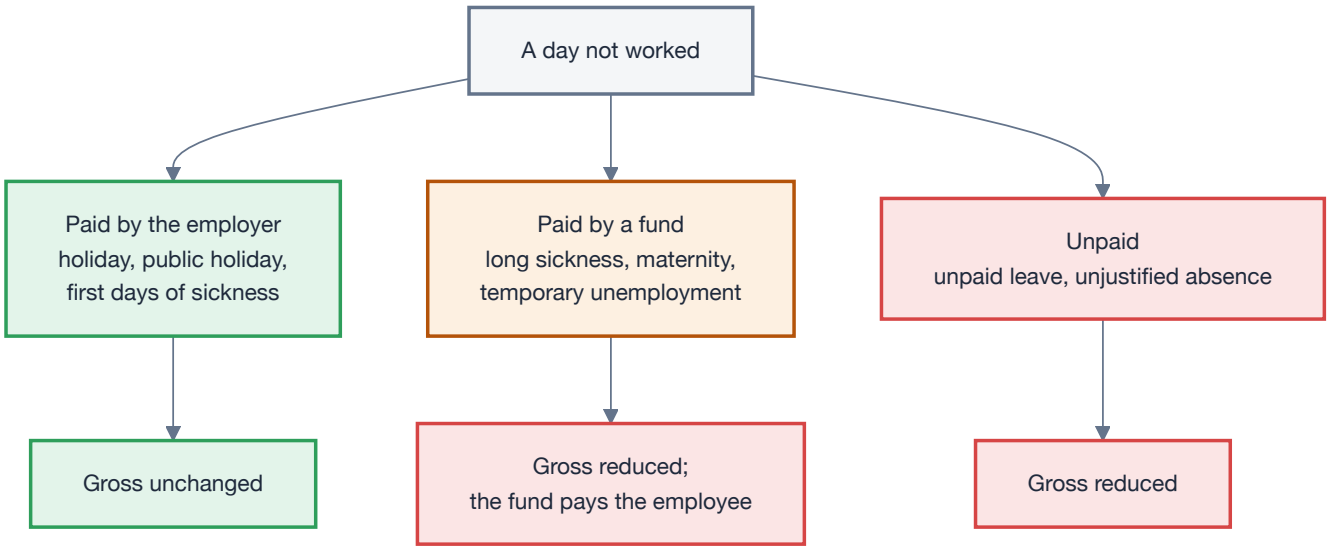
Line	Amount (€)
SALARY Gross 3,600.00 × 14/22	2,290.91
ONSS Employee contributions 13.07 %	–299.42
Low-wage reduction (reference wage 3,600.08, above the ceiling)	0.00
GROSS Taxable base	1,991.49
P.P Withholding (January formula)	–163.37
M.ONSS Special contribution (quarter base too low)	0.00
NET **Net pay**	**1,828.12**
Employer contributions (Q1 rate 25.13 %)	575.71
Employer cost	**2,866.62**

A partial month sets a second trap: Yannick's 2,290.91 € **looks** like a low wage. Compare it with the low-wage threshold and every joiner gets a reduction. Scale back to a full-month equivalent first: (2,290.91 / 14) × 22 = 3,600.08 €, above the 3,336.98 € ceiling. No reduction.[4]

In Odoo. Yannick's March payslip uses the fourth method: 2,270.77 € gross and 1,818.11 € net. Neither is wrong; the method is a policy to confirm with the customer (Chapter 10).[6]

Absences: who pays?

The payroll question is not "is he absent?" but "who pays for this day?"



An unpaid day is a proration in the other direction. Jurgen takes one unpaid day in October 2026.

Jurgen, October 2026, 1 unpaid day	Amount (€)
Daily rate 2,300.00 × 3/13 ÷ 5	106.15
Gross , 2,300.00 – one day	2,193.85
Employee contributions after the low-wage reduction	–2.19
P.P Withholding	–105.95
M.ONSS Special contribution	–10.37
NET **Net pay**	**2,075.34**

His usual net is 2,143.05 €. The unpaid day also scales his low-wage reduction by 21/22.[4]

A sick day is usually paid by the employer for a first period, then by a fund. In Belgium a white-collar employee keeps his full salary for the first 30 days (the "guaranteed salary").[3] If Jurgen is ill for a day, his gross stays 2,300.00 €; only the day's type changes, and with it per-day benefits such as meal vouchers.

> ⚠️ **Common pitfall**
>
> Mixing methods gives two values of "a day" in one company: Jurgen's day is worth 104.55 € by working days and 106.15 € at the fixed rate.[5] Pick one and use it everywhere.

Overtime

Overtime is time above a daily or weekly limit, paid at a premium and often taxed specially.

$$\text{Overtime pay} = \text{hours} \times \text{hourly rate} \times (100\ \% + \text{premium})$$

The hourly rate is again a convention; in Belgium, monthly × 3 / 13 / weekly hours. Belgian law sets a premium of at least 50 % (100 % on Sundays and holidays).[1]

> 🇧🇪 **Worked example: Jurgen, 4 hours at 150 %**
>
> | Line | Amount (€) |
> |---|---|
> | Hourly rate 2,300.00 × 3 / 13 / 38 | 13.97 |
> | **Overtime** 4 h × 13.97 × 150 % | 83.82 |
>
> The hourly-rate convention is common practice, `UNVERIFIED` as a legal rule.[5] Odoo pays 83.81 €: it does not round the hourly rate first.[6]

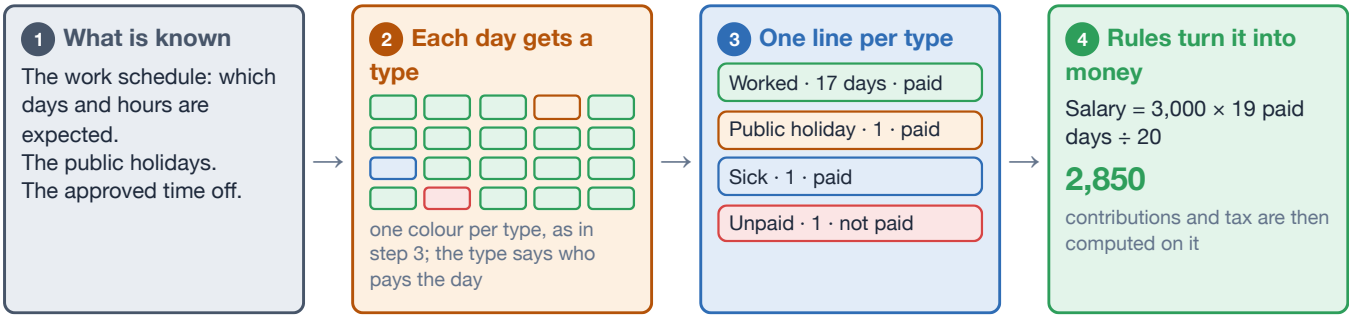
> 🌍 **Around the world**
>
> - **Switzerland**: 125 %, 150 % or 200 %.
> - **Morocco**: 25 %, 50 % or 100 %.
> - **United States**: 150 % as a work entry; a federal deduction for overtime, capped year to date.
> - **UAE**: weekday and rest-day overtime, each at its work-entry rate.
> - **Kenya, Netherlands, India**: no overtime rule in the module.

In Belgium

Some overtime hours are free of contributions or carry a tax reduction, but only up to a yearly number of hours.[7,8] The same four hours can be treated three ways, so the payroll needs an hour counter per employee and year.

How time reaches the payslip

Every payroll system, Odoo included, turns time into money in four moves. Illustrative month: 3,000 a month, 20 working days, of which 17 worked, a public holiday, a sick day paid by the employer and an unpaid day.



In Odoo. Work entries are generated from the employee's version (`hr.version`), its calendar and the time off, then summed into `hr.payslip.worked_days` lines, one per work entry type; the type says whether the day is paid. The **localization** decides how a line becomes money: Belgium values non-attendance lines at the 3/13 rate and gives the attendance line the rest, so a full month always pays the full salary. Chapter 14 walks through it in Odoo.

In practice

Before configuring, write the time policy on one page: proration method, daily and hourly rates, who pays each absence, how overtime is counted. Have the customer's accountant confirm it. Most disputes about joiners and leavers are disputes about this page.

Summary

- A monthly employee earns the same salary every full month; time matters for joiners, leavers, absences and overtime.
- Proration's denominator is a policy: Yannick's March is 2,270.77 € to 2,326.15 € depending on the method.
- Thresholds tested on monthly pay must use a full-month equivalent, or every joiner looks like a low earner.
- Every absence is paid by the employer, paid by a fund, or unpaid. An unpaid day lowers Jurgen's net from 2,143.05 € to 2,075.34 €; a short sick day leaves his gross unchanged.
- Overtime = hours × hourly rate × (1 + premium); the hourly rate is also a convention.
- In Odoo, time becomes worked-days lines per work entry type; the localization decides how they become money.

1. Labour Law of 16 March 1971, art. 19, 26bis, 29 (Justel): https://www.ejustice.just.fgov.be/cgi_loi/article.pl?language=fr&lg_txt=f&type=&sort=&numac_search=1971031602&cn_search=1971031602&caller=&&view_numac=1971031602fr. ↩
2. SPF Emploi, public holidays: https://emploi.belgique.be/fr/themes/jours-feries-et-conges/jours-feries. ↩
3. Law of 3 July 1978 on employment contracts, art. 39 §1 (3/13) and art. 70 (guaranteed salary) (Justel): https://www.ejustice.just.fgov.be/cgi_loi/article.pl?language=fr&lg_txt=f&type=&sort=&numac_search=1978070303&cn_search=1978070303&caller=&&view_numac=1978070303fr. ↩↩
4. ONSS, work bonus (incomplete months, S = (W / J) × D, P = R × J / D): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/deductions/workers_reductions/workbonus.html. ↩↩
5. Independent calculator, `research/numbers_be_calc.md`, sections 4 and 10.8 to 10.11. ↩↩
6. Payslips computed by Odoo master on the demo database, `research/numbers_be.md`, sections 2 (Yannick, March 2026) and 3 (Jurgen, April and May 2026). ↩↩
7. ONSS, voluntary overtime (law of 18 May 2026, AR of 9 July 2026): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/salary/salary_exclusions/voluntary_overtime.html. ↩
8. SPF Finances, Annex III consolidated, n° 45 (overtime reduction): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/0c174ec5-448b-4d1e-983b-fc457e5e7e28. ↩

Irregular pay, corrections and leavers

Bonuses, holiday pay, mistakes, and the last payslip

AFTER THIS CHAPTER YOU CAN

- Recognise the patterns behind irregular pay: 13th months and bonuses, holiday pay, accruals and provisions
- Choose a correction method, and know why some errors cannot be recovered from the employee
- Apply a garnishment ceiling on net pay
- Tell notice, payment in lieu of notice, severance and end-of-service gratuity apart
- Assemble a leaver's final pay from its components

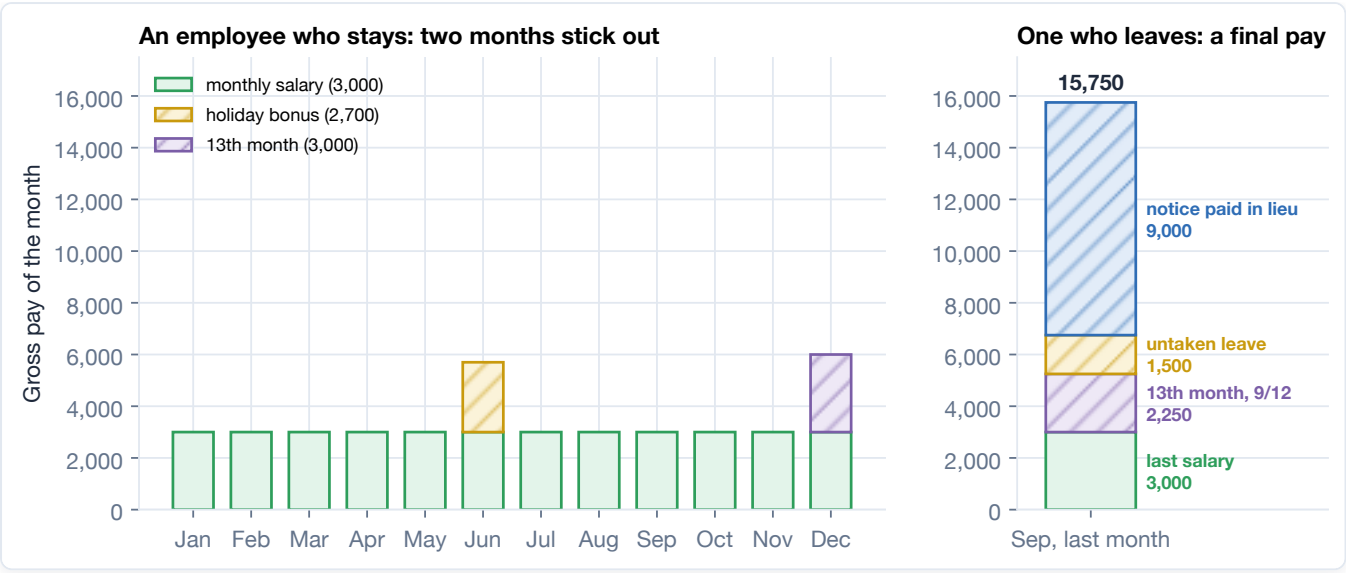
On 31 October 2026 MyVidangeCompany dismisses Emma after seven years, with immediate effect and an indemnity in lieu of notice. Her last pay is the biggest of her career: 48,959.75 € gross, five different components, three different tax rates, and 25,311.63 € net. None of it follows the monthly formula of Chapters 2 to 4. This chapter covers the pay that does not come every month: bonuses, holiday pay, corrections and leavers.

What irregular pay is

Irregular pay (*non-recurring pay*, *one-off payments*; tax laws often say *non-periodic income*) is any pay that does not come with every regular payslip: a 13th month, a bonus, a holiday bonus, a correction of an earlier month. A leaver's **final pay** (*final settlement*, *full and final settlement*) is the last pay of a contract. It collects several irregular items at once: the last salary, the bonus and leave earned but not yet paid, and money for the end of the contract itself.

Three things follow from that definition:

- **It does not fit the monthly formula.** Annualise a double month and the employee looks like a high earner for a year. Irregular pay usually gets its own tax method (Chapter 4) and often its own payslip.
- **It is earned before it is paid.** A 13th month is earned month by month; a leaver is owed the share earned so far.
- **A final pay is a list of items.** The items are the same everywhere; the law of each country says which are due (see Leavers below).



Illustrative numbers, 3,000 a month. Left: an employee who stays gets a holiday bonus in June and a 13th month in December. Right: one who leaves at the end of September gets, in one month, the last salary, 9/12 of the 13th month, the untaken leave and the notice paid in lieu: more than five normal months.

13th months and bonuses

Many countries add a payment once or twice a year: a 13th month, a Christmas bonus, a religious-holiday allowance. They share three design questions:

Question	Options
When is it paid?	once a year; in two halves; pro rata at exit
How is it accrued?	not at all; or a **monthly provision**, so the cost lands in the month it was earned
How is it taxed?	its own method, not the monthly formula (Chapter 4)

In Odoo, such a payment is usually a **separate payslip on its own structure**, with its own rules and tax method, rather than an extra line on the monthly payslip.

> 🌍 **Around the world**
>
> - **Mexico**: the Christmas bonus has its own structure, an exempt part and monthly provisions.
> - **Philippines**: a 13th-month provision every month.
> - **Switzerland**: 13th and 14th month, with provisions.
> - **Luxembourg**: separate structures with a tax rule for non-periodic income.
> - **Indonesia**: the religious-holiday allowance, entered as a payslip input.

Case study: Mélanie's year-end premium

MyVidangeCompany's sector agreement grants a year-end premium of one monthly salary in December. It carries ordinary contributions, a flat tax rate (Chapter 4), and pushes up the quarterly special contribution (Chapter 3).

Worked example: Mélanie's December 2026

Line	Amount (€)
SALARY December salary	4,000.00
Year-end premium	4,000.00
ONSS Employee contributions 13.07 % on both	−1,045.60
P.P Withholding on the salary (November formula)	−818.23
Withholding on the premium : 3,477.20 × 46.44 %	−1,614.81
M.ONSS Special contribution: quarter's 159.53 − 72.48 already withheld	−87.05
NET **Net pay December**	**4,434.31**
Employer contributions 25.12 %	2,009.60
Employer cost	**10,009.60**

The premium lifts the quarterly base to 16,000.00 €; December withholds the quarter's 159.53 € minus what October and November took.[9] The premium's rules come from the sector agreement, not the law (`UNVERIFIED`, secondary source).[5]

Holiday pay

Paid leave is the employee's right to a number of days off a year, set by law or contract, without losing pay. **Holiday pay** is the money tied to those days. It comes in up to three forms, and each one lands on a different payslip:

The salary continues

The employee is on leave; the salary keeps running as if he worked.

WHEN	the month of the days off
PAYSLIP	same gross; the days show as paid leave instead of worked (Chapter 5)
WHERE	almost everywhere

A holiday bonus

Extra money on top of the salary, so the employee can afford the holiday.

WHEN	once a year, often before summer
PAYSLIP	an extra amount, often its own payslip and tax method
WHERE	Belgium, Mexico, Australia (leave loading)

Untaken leave paid out

Days earned but not taken when the contract ends become money.

WHEN	the final pay
PAYSLIP	days left × daily rate, one line of the final pay
WHERE	Saudi Arabia, UAE, Belgium

In a few systems a fund pays the holiday money instead of the employer (Belgian blue-collar workers); then it never appears on the payslip.

Case study: Mélanie's holiday bonus

Belgian white-collar employees get a holiday bonus of 92 % of a monthly salary, earned in the previous year. Instead of ordinary contributions, a special 13.07 % deduction applies to part of it, and it is taxed at a flat rate.[2,4]

Worked example: Mélanie's holiday bonus, June 2026

Line	Amount (€)
Holiday bonus 92 % × 4,000.00	3,680.00
Special 13.07 % on 85 % of the salary (3,400.00)	−444.38
Taxable	3,235.62
Withholding , holiday-pay rate 42.39 %	−1,371.58
Net holiday bonus	**1,864.04**

Odoo's June payslip gives the same special deduction, withholding and net.[1]

In Belgium

Three variants: **simple** (the salary during the days off), **double** (the 92 % bonus) and **exit** holiday pay, which settles both when a contract ends.[3] The 85 % base is practice, not a legal figure (`UNVERIFIED`).

Corrections

A **correction** fixes a payslip that was computed with wrong or late information: an unpaid day reported after the payroll ran, a raise signed late, a wrong rate. It happens whenever the error is found, often weeks later. What it means for the payslip depends on how far the payslip has already travelled: before the bank transfer nothing has left the company; after it, the money and the declarations have, and only the **difference** can still be settled.

1 Computed, not paid yet
Fix the data, **cancel and recompute**.
Nothing has left the company, so nothing needs undoing.

→

2 Paid, small error, same year
Adjust on the next payslip: one extra line for the difference, plus or minus.
The declared month stays wrong until its return is corrected.

→

3 Large, or an earlier year
Refund and reissue: a negative copy cancels the original, a new payslip replaces it.
More documents; the declarations must be amended.

The further the payslip went (paid, declared, a closed year), the heavier the fix. The law adds limits: some deductions cannot be recovered from the employee later (in Belgium, a contribution not withheld in time),[6] a closed tax year needs a special form, and arrears have their own tax rule.[7]

Worked example: Jurgen's forgotten unpaid day

Jurgen's October unpaid day is entered after October was paid. MyVidangeCompany deducts the 106.15 € (Chapter 5's daily rate) on the November payslip, and November's reduction, withholding and special contribution follow the lower gross. October's line in the quarterly return is corrected when it is filed.

When deductions exceed earnings, the net goes negative and a bank cannot pay it. The usual policy is to pay zero and carry the rest forward; the Belgian structure has a `NET_TO_RECOVER` line for that.

Around the world

- **India, Turkey, Poland**: the cumulative method corrects itself next month.
- **Egypt, Pakistan**: a year-to-date tax adjustment wizard adds a correction line.
- **Australia**: a pay-event report already sent can be replaced in full.
- **Philippines**: a year-end annualisation trues up the year.

In Odoo. A validated payslip can be refunded: a negated copy linked to the original. Chapter 15 corrects a payslip after validation.

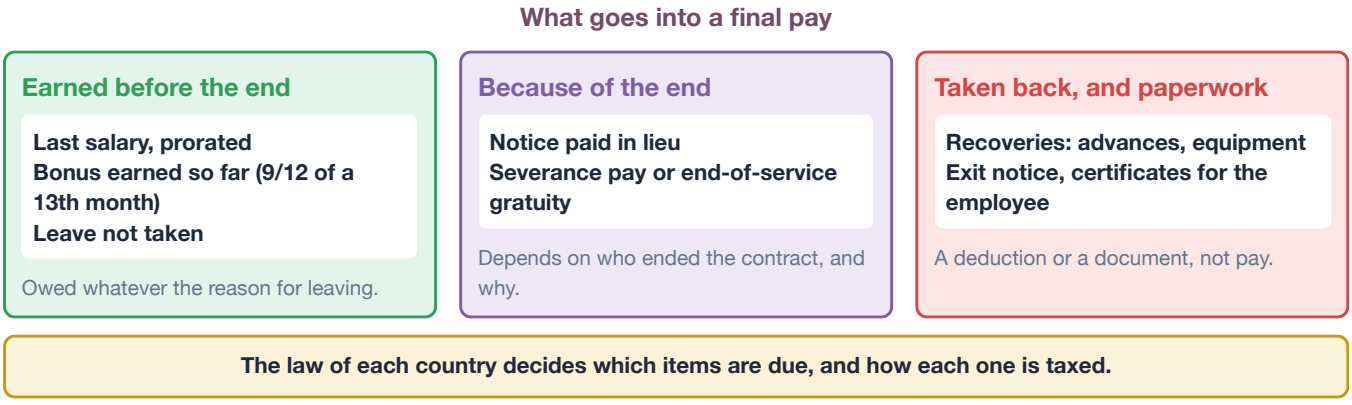
Garnishment

A court can order the employer to pay part of the salary to a creditor. Almost every country protects a minimum: only the **net** above legal thresholds can be seized, in slices. In Belgium nothing below 1,419.00 € of net can be seized in 2026, then growing slices; on Mélanie's 2,614.26 €, at most 906.46 €.[8,9]

In Odoo. A garnishment is a payslip adjustment (`hr.salary.attachment`); the rule applies the legal ceiling.

Leavers

A **leaver** is an employee whose contract ends: he resigns, is dismissed, reaches the end of a fixed-term contract, or retires. It happens on a known last day, often after a notice period. For the payroll, the leaver's last pay, the **final pay**, must settle everything still owed in one go: there is no next month to fix a forgotten item. The items fall into three groups:



Four terms are often confused:

Term	What it is	Typical formula
Notice	a period between announcement and end, worked and paid as usual	length by seniority
Payment in lieu of notice	the notice not worked, paid at once	weekly pay × weeks of notice
Severance pay	compensation for losing the job, on top of notice	an amount per year of service
End-of-service gratuity	earned through the years, paid at any exit	a share of salary per year, accrued monthly

Around the world

- **Saudi Arabia, UAE**: end-of-service gratuity, accrued monthly, paid when a departure reason is set.
- **India**: a monthly gratuity line; "full-and-final" payslips on departure.
- **Hong Kong**: severance reduced by the employer's provident-fund contributions.
- **Australia**: lump sums with tax-free limits by years of service.
- **Turkey**: severance pay with a legal ceiling.

Case study: Emma, dismissed with an indemnity in lieu of notice

Emma's notice depends on her seniority: 7 years and 1 month gives **24 weeks**.[10] MyVidangeCompany pays them as an indemnity, valued on her annual pay, benefits included:

$$\text{Indemnity} = (12 \times \text{monthly} + \text{year-end premium} + \text{holiday bonus}) \div 52 \times \text{weeks of notice}$$

For Emma: (60,000.00 + 5,000.00 + 4,600.00) ÷ 52 × 24 = **32,123.08 €**. It carries ordinary contributions[11] and is taxed on a separate indemnity scale (38.96 % for Emma).

Common pitfall

Getting the hire date wrong by one month. Counted from 1 November 2019, Emma's 24 weeks become 21: 4,015.39 € less. Check the contract start, earlier contracts with the same employer, and the notice start before computing.

In Belgium

Notice grows with seniority by steps (3 more weeks per started year from 5 years), with a cap for recent contracts. An employee who resigns has a shorter notice. The employer also pays a special contribution of 1 % to 3 % on dismissal indemnities (`UNVERIFIED` band; the calculator applies 1 %).[12]

Emma's final pay

Her other items: the October salary, the year-end premium pro rata (10/12), and the **exit holiday pay** she earned in 2026.



Emma's final pay, October 2026: four earnings, three deductions. The indemnity is by far the largest part.

Component	Pattern	Gross	Employee contributions	Withholding
October salary	last salary, monthly formula	5,000.00	653.50	1,245.27
Year-end premium 10/12	pro-rata bonus, flat rate 51.48 %	4,166.67	544.58	1,864.65
Exit holiday pay, simple	accrued leave	3,835.00	501.23	with next row
Exit holiday pay, double	accrued holiday bonus, 47.44 % on both	3,835.00	444.38	3,190.05
Indemnity, 24 weeks	payment in lieu of notice, 38.96 %	32,123.08	4,198.49	10,879.42
Total		**48,959.75**	**6,342.18**	**17,179.39**

Final pay	Amount (€)
Gross	48,959.75
Employee contributions	–6,342.18
Withholding	–17,179.39
Special social security contribution (quarter, indemnity left out)	–126.55
Net paid	**25,311.63**
Employer contributions 25.12 % on 45,124.75	11,335.34
Special contribution on the indemnity 1 %	321.23
Employer cost	**60,616.32**

Five components, four tax treatments, two contribution treatments: a final pay is never "one more payslip". Exit holiday pay is 15.34 % of the year's gross, half simple, half double.[3]

❓ Open questions: Emma's final pay in Odoo

Odoo finds the same 24 weeks and the same premium withholding. Other lines differ; the main ones:[1]

Item	Calculator	Odoo	Why
Withholding on the indemnity	10,879.42	8,771.79	38.96 % against about 31.3 %: **open**
Exit holiday pay, simple part	3,835.00	3,476.88	Odoo's base covers January to September only: **open**
Net of the five payslips	**25,311.63**	**27,099.24**	

Chapter 10 shows how such differences are taken to an expert.

In Odoo. Each component is a separate payslip on its own structure (*Monthly Pay*, *13th Month*, *Termination Holidays*, *Termination Fees*). The departure dialog computes the notice; the exit payslips join the leaver's last pay run. Chapter 17 follows Emma's departure in Odoo.

💡 In practice

Build a final pay as a checklist before computing: last salary, bonus pro rata, accrued leave, notice or payment in lieu, severance, recoveries (advances, a laptop), documents. Most final-pay errors are a forgotten line, not a wrong rate.

Summary

- Irregular pay (13th months, bonuses, holiday bonuses) usually gets its own structure and its own tax method, and is often accrued monthly with a provision.
- Mélanie's December: 4,000.00 € of premium taxed at a flat 46.44 %, and a quarterly catch-up of 87.05 €; net 4,434.31 €.
- Holiday pay is salary that continues, sometimes plus a bonus; untaken leave is paid at exit. Mélanie nets 1,864.04 € of holiday bonus.
- Correct before payment by recomputing; after, adjust next month or refund and reissue.
- Garnishment is capped on the net: at most 906.46 € of Mélanie's 2,614.26 €.
- Notice, payment in lieu, severance and gratuity are four different things. Emma gets 24 weeks paid in lieu: 32,123.08 €.
- Emma's final pay: 48,959.75 € gross, 25,311.63 € net, 60,616.32 € of employer cost.

---

1. Payslips computed by Odoo master on the demo database, `research/numbers_be.md`, sections 4 (Emma's departure and exit payslips) and 5 (double holiday pay); differences in `research/odoo_vs_calc.md`. ↩↩
2. socialsecurity.be, annual holidays (20 days, double holiday pay 92 %): https://www.socialsecurity.be/citizen/fr/conges-credit-temps-et-interruption-de-carriere/vacances-annuelles. ↩
3. ONSS, holiday pay (exit holiday pay 15.34 %, new employer advance 10 %): https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/salary/particularcases/holidaypay.html. ↩↩

4. ONSS, special contribution on double holiday pay, private sector: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/special_contributions/other_specialcontributions/doubleholiday_privatesector.html. ↩
5. Secondary source: ACCG, *Tool CP 200 prime de fin d'année 2025*: https://www.accg.be/sites/default/files/sectors/cp-200/accg-cp-200-tool-cp-200-prime-de-fin-d-annee-2025.pdf. ↩
6. ONSS, general rule on contribution payment: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/contributionpayment_obligations_periodicity/generalrule.html. ↩
7. SPF Finances, Annex III from 1 January 2026, n° 53–62: https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/536a741e-81d6-4605-ad11-8e07ebc4a440. ↩
8. SPF Emploi, garnishment and assignment of salary (2026 amounts): https://emploi.belgique.be/fr/themes/remuneration/protection-de-la-remuneration/saisie-et-cession-sur-salaires. ↩
9. Independent calculator, `research/numbers_be_calc.md`, sections 5 to 7, 10.11 and 10.13. ↩↩
10. Law of 3 July 1978, art. 37/1, 37/2 (version from 1 August 2026), 37/4 and 39 (Justel): https://www.ejustice.just.fgov.be/cgi_loi/article.pl?language=fr&lg_txt=f&type=&sort=&numac_search=1978070303&cn_search=1978070303&caller=&&view_numac=1978070303fr. ↩
11. ONSS, end of the employment contract: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/salary/particularcases/end_of_contract.html. ↩
12. ONSS, special contribution on dismissal indemnities: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/special_contributions/other_specialcontributions/terminationfeecontribution.html. ↩

Declarations and the compliance calendar

Who gets which report, how often, and what must match

AFTER THIS CHAPTER YOU CAN

- Name the four kinds of payroll filing every country has, and the newer per-payslip and per-pay-run models
- Explain why every declaration is a sum or a list of validated payslip lines
- Say which filings must agree with each other
- Place filings, payments and yearly events on one calendar
- Name the Odoo models that produce each of MyVidangeCompany's filings

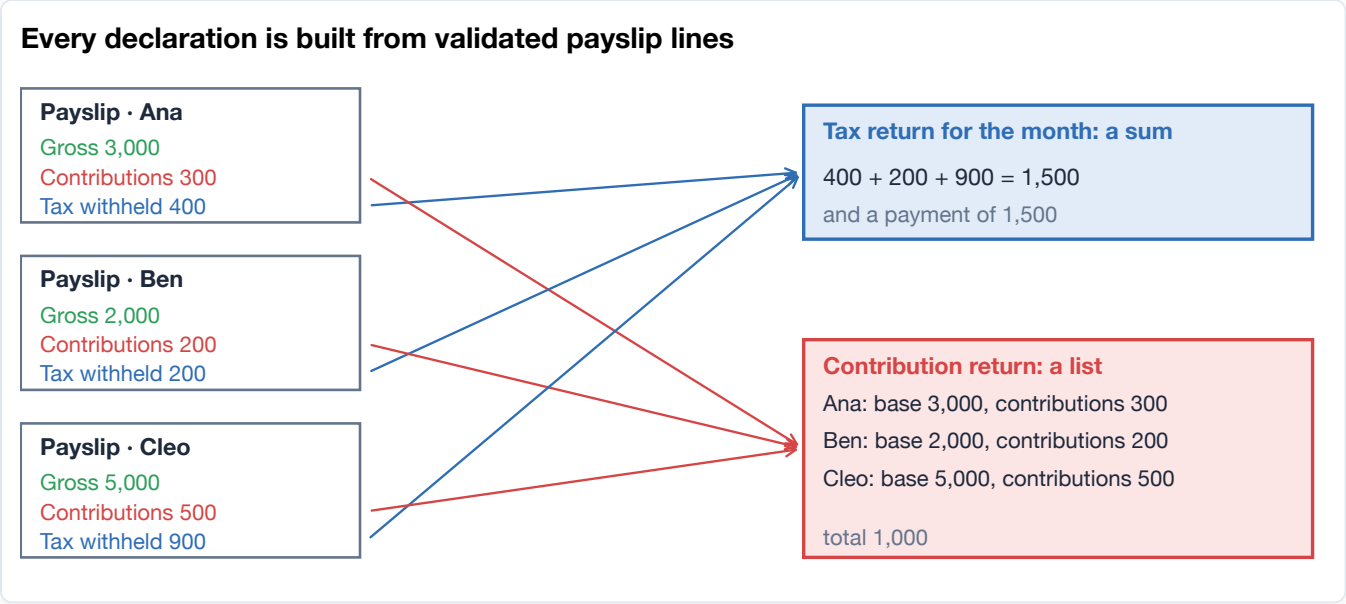
Payroll does not end at the bank transfer. MyVidangeCompany notifies each hire before the first hour, reports the tax withheld every month, the contributions every quarter, and each employee's yearly totals every February. All must agree with the payslips and with each other. The form names are Belgian; the four kinds of filing exist almost everywhere.

What a payroll declaration is

A **payroll declaration** (*payroll return*, *filing*, *payroll report*) is a report the employer sends to an authority about its payroll: who works for it, what they earned, and what was withheld or is owed. Most declarations come with a payment, or are checked against one. They go to the social security office, the tax authority, and sometimes the employee.

Three things follow from that definition:

- **It is built from the payslips.** Every declaration is a **filtered sum or list** of validated payslip lines. If a declaration needs a number the payslips do not have, the payslips are missing a rule.
- **It runs on the authority's calendar,** not the payroll's: at each hire or exit, each month or quarter, each year (the four clocks of Chapter 1).
- **Declarations must agree** with each other and with the payments. An authority that receives two numbers compares them.



Illustrative numbers, three employees. The tax return adds up the tax withheld on the three payslips and comes with the payment. The contribution return lists each employee's base and contributions. Nothing in either return is typed by hand.

Who receives which filing

Three parties receive payroll reports, and each needs them for a different reason. **Social security** must know who is insured and on what pay: it collects the contributions, and it computes each worker's pension, unemployment and sick pay from what the employer declared. The **tax authority** collects the tax withheld, which is the employee's income tax paid in advance, and must credit it to the right person. The **employee** needs his yearly totals to file his own tax return.

Who receives what, and why

Social security

Hire and exit notices
Contribution return, per worker

To collect the contributions and to compute each worker's rights.

Tax authority

Withholding return, with the payment
Annual certificate per employee

To collect the tax withheld and credit it to each employee.

Employee

The payslip, every period
A copy of his annual certificate

To check his pay and file his own tax return.

All of them are built from the same validated payslips.

Four kinds of filing

Kind	Purpose	Frequency	MyVidangeCompany (Belgium)
Hire and exit notification	tells the state someone works, before anything is paid	at each event	Dimona
Periodic contribution return	per worker: pay, working time, contributions	monthly or quarterly	DmfA, quarterly

Kind	Purpose	Frequency	MyVidangeCompany (Belgium)
Periodic withholding return	total tax withheld, paid with the return	monthly	form 274
Annual employee certificate	per employee: the year's taxable pay and tax withheld	yearly	tax slip 281.10

Newer models report **every payslip or every pay run**, so a correction is a resend rather than an amendment. Some countries add **payment-control files**, where the salary transfer itself is reported.

🌍 **Around the world**

- **United States**: quarterly withholding return (941), annual wage statement per employee (W-2).
- **Mexico**: one signed electronic payslip (CFDI) per payslip, sent to the tax office.
- **Australia**: a pay-event report every pay run (Single Touch Payroll).
- **UAE**: a Wage Protection System salary file; no tax filing.
- **Hong Kong**: employer's returns at year end, hire and exit.

Case study: MyVidangeCompany's filings

The hire and exit notice

Belgium's **Dimona** is due at the latest when the worker starts, and the exit notice on the first working day after the end.[1] Yannick's is due on 12 March 2026; Emma's exit notice on Monday 2 November.

The monthly withholding return

MyVidangeCompany files **form 274** monthly and pays the tax with it, around the 15th of the next month.[3,4]

🧱 **Worked example: the withholding return for October 2026**

In a standard month the five regular payslips withhold 4,646.49 €. October 2026 is not standard: Emma's final pay replaces her regular payslip.[6]

Withholding, October 2026	Amount (€)
Mélanie, Jurgen, Marie, Yannick	3,401.22
Emma's final pay (salary, premium, exit holiday pay, indemnity)	17,179.39
Return for October 2026, due 13 November 2026	20,580.61

The return separates the amounts by kind of income (salary, holiday pay, indemnities), so each payslip line must carry that classification.

In Odoo. The return is literally a sum of payslip lines: Odoo's February 2026 return, 3,978.86 €, equals that month's withholding lines. For October Odoo withholds 18,216.35 €; the gap comes from Emma's final pay (Chapter 6).[5]

The quarterly social security return and the annual tax slip

The quarterly return (**DmfA**) lists, per worker, the pay by code, working time, contributions and reductions; it is due on the last day of the month after the quarter.[2] Each February, the employer sends one annual tax slip (**281.10**) per employee to the tax administration, with a copy to the employee.[7]

What must match

An authority often receives the same money twice, by two different roads, and compares the two. Withholding tax is the clearest case:

- **Every month**, the withholding return gives the month's total for all employees, and the employer pays it. The tax office knows what it received.
- **After the year**, the annual certificates give each employee's total for the year. The tax office knows how much to credit to each person against his final tax.

Both are sums of the same payslip lines, one by month and one by employee, so their grand totals must be equal. If the certificates add up to more than the returns, employees are credited with tax that nobody paid, and the tax office asks why.

Tax withheld	Jan	Feb	…	Dec	Annual certificate
Employee A	300	300	…	300	3,600
Employee B	200	200	…	200	2,400
Employee C, joins in July	0	0	…	100	600
Monthly return	500	500	…	600	6,600 = 6,600

Illustrative numbers. Each column total is a monthly return; each row total is an annual certificate. The twelve returns (6 × 500 + 6 × 600) and the three certificates (3,600 + 2,400 + 600) must both come to 6,600.

Contributions follow the same logic: the contribution return lists each worker's contributions, and their sum must equal what was paid for the period. Both must also equal the payroll debts in the accounts (Chapter 8). At MyVidangeCompany, the twelve monthly withholding returns of 2026 must add up to the annual tax slips, and each quarterly return must match the contributions paid. A localization makes these matches true by construction: every filing reads the same validated lines.

> ⚠ **Common pitfall**
>
> Filing from a PDF or a spreadsheet. A payslip corrected after filing leaves the return silently out of date. Generate each declaration from the validated lines and regenerate it after a correction.

The payslip and the records to keep

Every country also sets the **content of the payslip** and the **records to keep**. In Belgium the payslip must show how pay is computed, and the employer keeps an individual account per worker.[8,9] Retention periods were not researched (`UNVERIFIED`). A localization covers them with the payslip report and a yearly per-employee report.

The calendar

Put together, the filings give a calendar with four rhythms: the event, the month, the quarter and the year (Chapter 1).

Worked example: MyVidangeCompany's fourth quarter 2026

Date	Event
1 Nov	new withholding formula
2 Nov	exit notice for Emma
early Nov	October net pay transferred: 37,177.22 €
13 Nov	withholding return and payment for October: 20,580.61 €
December	year-end premiums
31 Jan 2027	quarterly social security return for Q4
28 Feb 2027	annual tax slips for 2026

In practice

Order the filings by the cost of being late: the hire notice first, then the withholding return, then the contribution return, then the annual certificates. If a payroll bureau keeps filing one, agree how Odoo's numbers reach it.

In Odoo. Each localization adds one model per filing: Belgium has `l10n_be.dmfa`, `l10n_be.274_xx` and `l10n_be.281_xx`, among others. The yearly per-employee ones share `hr.payroll.declaration.mixin` (Chapter 22). Many smaller localizations ship no declaration at all. Chapter 17 produces the Belgian filings.

Summary

- Declarations are sums or lists of validated payslip lines; design the lines first.
- Four kinds of filing exist almost everywhere: hire and exit notification, periodic contribution return, periodic withholding return, annual employee certificate.
- Some countries report every payslip (Mexico) or every pay run (Australia); some add payment-control files (the UAE's Wage Protection System).
- MyVidangeCompany: Dimona at each hire and exit, a monthly withholding return (20,580.61 € for October 2026), a quarterly social security return, annual tax slips by 28 February.
- The year's withholding returns must agree with the annual certificates; contribution returns must agree with payments.
- Payslip content and record keeping are legal duties too.

---

1. ONSS, Dimona: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/dimona/general.html and

https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/dimona/declarations_modifications_cancellations.html. ↩

2. ONSS, DmfA general: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_nsso/dmfa/general.html. ↩
3. SPF Finances, withholding tax return: https://finances.belgium.be/fr/entreprises/personnel_et_remuneration/precompte_professionnel/declaration. ↩
4. SPF Finances, withholding calendar: https://finances.belgium.be/fr/entreprises/personnel_et_remuneration/precompte_professionnel/calendrier. ↩
5. Payslips and declarations computed by Odoo master on the demo database, `research/numbers_be.md`, sections 6 and 8. ↩
6. Independent calculator, `research/numbers_be_calc.md`, sections 5 and 10.2. ↩
7. SPF Finances, Belcotax-on-web deadlines: https://finances.belgium.be/fr/E-services/Belcotaxonweb/delais-de-rentree-des-fiches; Belcotax-on-web: https://finances.belgium.be/fr/E-services/Belcotaxonweb. ↩
8. SPF Emploi, payslip and receipt for the balance: https://emploi.belgique.be/fr/themes/remuneration/protection-de-la-remuneration/decompte-de-paie-et-quittance-pour-solde-de. ↩
9. ONSS, social documents: https://www.socialsecurity.be/employer/instructions/dmfa/fr/latest/instructions/obligations/obligations_other/socialdocuments.html. ↩

Accounting for payroll

One payslip, one balanced entry, and liabilities that wait to be paid

AFTER THIS CHAPTER YOU CAN

- Build the journal entry of a payslip: costs on one side, liabilities on the other
- Explain why employee deductions are liabilities, and only the employer charges add cost
- Follow MyVidangeCompany's October 2026 through T-accounts until every liability is paid
- Explain provisions for holiday pay, 13th months and gratuities
- Describe how Odoo creates the entry and the payment file

At month end the accountant wants one thing from payroll: a journal entry. The salary and the employer's charges are costs. Everything withheld is money owed to someone else until it is paid. Payroll accounting is mostly the story of those debts. The shape is the same in every country; only the account numbers change.

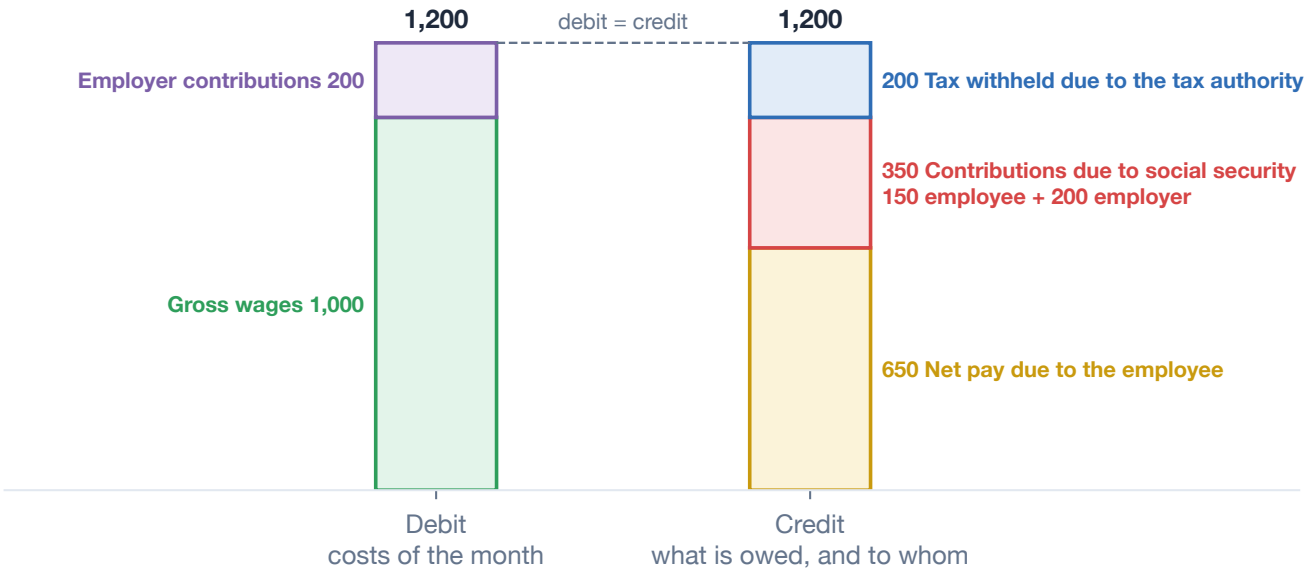
What payroll accounting is

Payroll accounting (*payroll bookkeeping*, *booking the payroll*) records the pay run in the company's books. Each period it posts one balanced **journal entry**: the cost of the period on the debit side, and on the credit side what the company now owes, and to whom. Later payments close those debts one by one.

Three things follow from that definition:

- **The cost is the employer cost.** Gross wages plus the employer's own charges; nothing withheld from the employee adds to it.
- **Withheld money is a debt, not a cost.** Until it is paid, the tax and contributions kept back sit on liability accounts, like the net pay not yet transferred.
- **The entry is a sum of payslip lines.** Each salary rule says which account it goes to; the entry adds them up.

One payslip, one balanced entry: 1,200 of cost, 1,200 of debts



Illustrative numbers: the payslip of Chapter 2's figure (1,000 gross, 200 employer contributions). 1,200 of cost on the debit side; on the credit side, 650 owed to the employee, 350 to social security and 200 to the tax authority.

Case study: MyVidangeCompany's entry

Belgian companies use a standard chart of accounts (legal text not retrieved, `UNVERIFIED`): 6202 remuneration, 621 employer social security, 623 other personnel costs, and the liabilities 453 (tax withheld), 454 (social security), 455 (net pay due) and 440 (suppliers).

Worked example: Mélanie and Marie, October 2026

Account	Mélanie debit	Mélanie credit	Marie debit	Marie credit
6202 Gross wages	4,000.00		7,500.00	
621 Employer contributions (– reduction + CO2)	1,004.80		1,926.34	
623 Meal vouchers, employer part			196.02	
454 Social security due		1,563.84		2,967.53
453 Tax withheld due		826.70		1,770.92
455 **Net pay due**		**2,614.26**		**4,663.91**
440 Meal-voucher issuer				220.00
Total	**5,004.80**	**5,004.80**	**9,622.36**	**9,622.36**

For Mélanie, 454 collects the employee, special and employer contributions. Marie's car benefit does not appear at all: no money moves for it. Each total equals the employer cost of Chapter 3.[1]

The five regular payslips of October 2026 make one entry of 27,849.02 € on each side: 14,917.31 € of net pay due, 8,065.22 € of social security, 4,646.49 € of tax withheld and 220.00 € to the voucher issuer.[1] The debit side is the month's employer cost; the credit side says where the money will go.

In Odoo. The demo database books the same accounts with six digits (620200, 454000…); its February 2026 entry balances at 23,426.76 €. It books only the employee part of the meal vouchers, so agree with the customer's accountant how vouchers are booked.[2]

From book to bank: the T-accounts

Accountants follow each account with a **T-account**: the account drawn as a large T, its name on top. Amounts that increase the account go on one side, amounts that decrease it on the other, and the difference between the two sides, the **balance**, is what the account holds. For a debt such as *net pay due*, the right side (credit) records what the company comes to owe and the left side (debit) what it pays off, so the balance is what is **still owed**.

Take account 455, the net pay MyVidangeCompany owes its employees for October 2026:

455 Net pay due

DEBIT (paid off)		CREDIT (now owed)	
Transfer, early Nov	14,917.31	Payroll entry, 31 Oct	14,917.31
Balance: still owed	**0.00**		

After the payroll entry
14,917.31 owed

↓ the bank pays the employees

After the salary transfer
0.00 owed

The payroll entry fills the account; the bank payment empties it, and books the same amount as money leaving the bank account (550). The three other debts of the entry work the same way, each emptied on its own date:

Account	Emptied by	When
453 Tax withheld, 4,646.49 €	payment with the withholding return	by 13 November
454 Social security, 8,065.22 €	monthly advances, then the balance	balance by 31 January
440 Voucher issuer, 220.00 €	payment of its invoice	on the invoice terms

A balance left on a payroll liability after its due date is the first thing an accountant checks: a payment that did not happen, or a line booked on the wrong account.

> ⚠ **Common pitfall**
>
> Booking the employee contributions and the withholding as expenses. They are liabilities; the cost is already in the gross wages. Booking them as expenses counts them twice.

Provisions: cost now, cash later

A **provision** is a debt the company records before the bill arrives: it knows it will have to pay an amount later, so it books the cost now, a little each month. Take a 13th month of 1,200, earned over the year and paid in December (illustrative numbers, contributions left out):

Month	Cost booked	Provision (owed so far)	Cash paid
January	100	100	0
February	100	200	0
…	…	…	…

Month	Cost booked	Provision (owed so far)	Cash paid
December, before payment	100	1,200	0
December, payment	0	0	1,200

Without the provision, December would carry 1,200 of cost and the other months none. With it, each month carries its share, and the December payment simply pays off the debt already recorded.

> Each month: debit provision cost / credit provision liability. On payment: debit provision liability / credit net pay due

Saudi Arabia and the UAE provision the gratuity, Mexico and the Philippines the bonuses (Chapter 6). Many Belgian employers provision holiday pay (account 625 against 456), an accounting policy the book does not compute.

Payment files

The net pay leaves the company as one bank file with a line per employee; the legal payment date applies (Chapter 1).[3] Tax and contributions are paid separately, each with its own reference.[4] MyVidangeCompany uses a SEPA credit-transfer file.

Around the world

- **Belgium, Netherlands, Luxembourg**: SEPA credit transfer.
- **United States**: a NACHA (ACH) file.
- **Australia**: an ABA bank file.
- **UAE, Saudi Arabia**: a Wage Protection System salary file, checked by the labour ministry.
- **India**: a payment advice and a bank e-NET file.

In Odoo. `hr_payroll_account` gives each salary rule an `account_debit` and `account_credit`, and each structure a journal; each localization's `_account` module sets them for its chart. At validation Odoo builds the entry from the lines and groups them by account. If they do not balance (a rule without an account), Odoo adds an *Adjustment Entry* line instead of refusing: the entry posts and looks fine, and it is wrong. `hr_payroll_account_iso20022` exports the SEPA file. Chapter 16 posts and pays February 2026.

In practice

Before go-live, validate one payslip per structure and read each entry against the chart of accounts. Agree the accounts with the accountant before the first pay run; changing them later means reversing entries.

Summary

- A payslip becomes one balanced entry: costs (gross wages, employer contributions, other personnel costs) against liabilities (net pay, social security, tax withheld, third parties).
- Employee deductions are liabilities held for third parties; only employer charges add cost.

- MyVidangeCompany's October 2026: 27,849.02 € on each side. Each liability is closed by a payment on its own date.
- A benefit in kind moves no money and has no line in the entry.
- Pay earned monthly but paid later (holiday pay, 13th month, gratuity) is provisioned each month.
- Odoo books the entry at validation from the accounts on the salary rules, and balances any gap with an *Adjustment Entry* line that you should treat as a bug.

---

1. Independent calculator, `research/numbers_be_calc.md`, sections 10.1 and 10.12. ↩↩
2. Journal entry of the February 2026 pay run on the demo database, `research/numbers_be.md`, section 7. ↩
3. SPF Emploi, time and date of payment: https://emploi.belgique.be/fr/themes/remuneration/protection-de-la-remuneration/epoque-et-date-de-paiement. ↩
4. SPF Finances, withholding tax return and payment account: https://finances.belgium.be/fr/entreprises/personnel_et_remuneration/precompte_professionnel/declaration. ↩

PART II

Localization theory

What changes from one country to the next, how to find out what the law says, and how to keep up when it moves. The comparison points are Odoo's own localizations, from the United States, India and Mexico to the Gulf, Australia, Switzerland, Hong Kong and Kenya, read from the code. Belgium is one worked example among them.

What varies between countries, and what doesn't

Four layers, ten axes of variation, and a comparison grid of Odoo's localizations read from the code

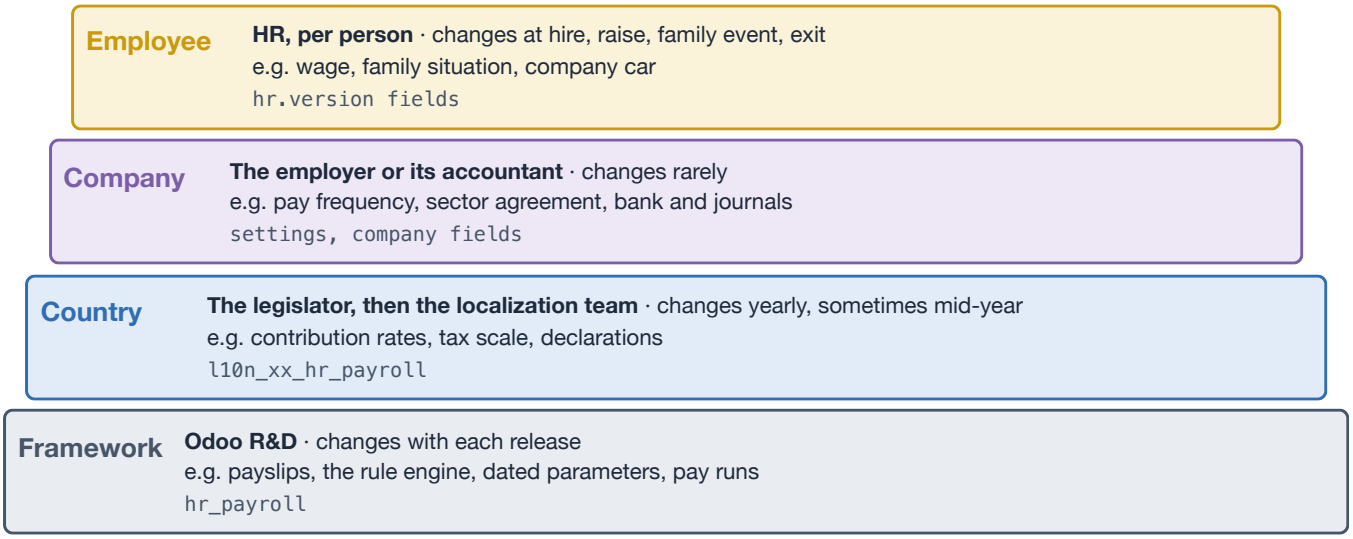
AFTER THIS CHAPTER YOU CAN

- Place any payroll fact in one of four layers: framework, country, company, employee
- Name what every country shares and the axes along which countries differ
- Read the comparison grid and say what each module teaches
- Recognise the four places a rate can live: XML parameters, downloaded parameters, customer-maintained records, contract fields

Open three payslips in Odoo: Monterrey, Sydney, Namur. Each has earnings, contributions, a tax line and a net, computed by the same engine. Yet the Mexican one caps its social-security wage in a legal reference unit, the Australian one has no employee contribution, and the Belgian one annualises the month. Before you write or review a localization, know which part Odoo gives you and which part is yours to build.

The four layers

Every payroll fact lives in one of four layers. The question to ask is: *who decides this value, and how often does it change?*



Each layer is built on the one below it: a company configures what its country's module allows, and an employee's values fill fields the country declared. When a value has to go somewhere, put it in the **highest layer where it is still true**: a legal rate in the country, an employer's choice in the company, a person's situation on the employee.

What is the same everywhere

However exotic a country's payroll looks, it is built from the same five ideas. Odoo implements each idea **once**, in the framework, and every localization reuses it. Learn these five and you can open any country module and know where to look.

1 · Time becomes money

A month of salary pays a month of work. Days not worked, paid leave and overtime change the amount, each at its own rate.

FRAMEWORK PIECE
work entries and worked days — Chapters 5, 14

2 · Totals feed the formulas

A contribution or a tax is a percentage of a **total** of payslip lines: the gross, the contribution base, the taxable base.

FRAMEWORK PIECE
salary rule categories — Chapters 2, 20

3 · Rates change over time

Every year brings new rates and tax scales, but an old payslip must keep the rates of its own period.

FRAMEWORK PIECE
dated rule parameters — Chapters 11, 21

4 · Every period ends in an entry and a payment

The pay run is booked in the accounts and the net is paid to the bank, every period, the same way.

FRAMEWORK PIECE
pay runs and payroll accounting — Chapters 8, 16

5 · The authorities want reports

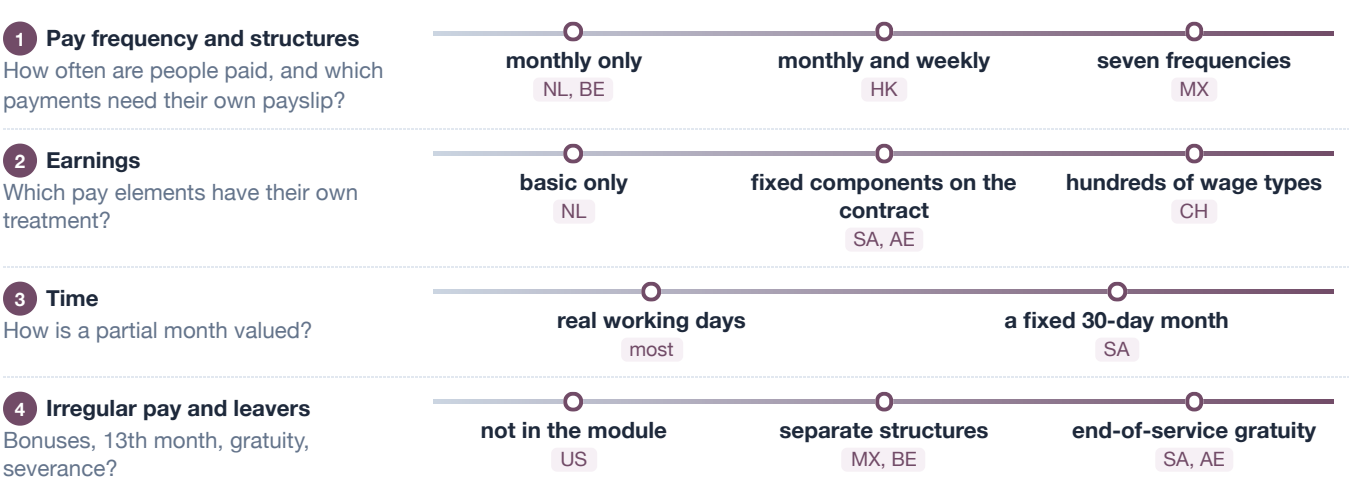
What was paid and withheld is reported to the tax office and social security, monthly, quarterly or yearly.

FRAMEWORK PIECE
the declaration model — Chapters 7, 17

The axes of variation

Everything outside those five ideas varies, and ten questions cover it. They are the checklist for any new country: answer each one by placing the country on its line, and you know the shape of the module before reading a law text. Each line runs from the simplest answer found in Odoo's localizations (left) to the heaviest (right).

WHAT IS PAID





The further right a country sits, the more code it needs: a heavy answer on the **outputs** (declarations, payment files, accounting) and on the **data feeds** costs far more than a heavy answer on earnings or contributions, which are only more rules.

The comparison grid

Six localizations, read from the master code.[1] Each has a personality; together they cover almost every answer on the ten axes. "None" means the module does not do it, not that the country has no such rule.

NL Netherlands — *the minimalist*

CONTRIBUTIONS
national insurances, capped yearly ÷ 12

INCOME TAX
monthly brackets

DECLARATIONS
none

AE United Arab Emirates — *no income tax*

CONTRIBUTIONS
pension for nationals only

INCOME TAX
none

DECLARATIONS
a wage-protection salary file

IN India — *the year-to-date one*

CONTRIBUTIONS
provident fund on basic; state insurance below a threshold

INCOME TAX
cumulative over the financial year

DECLARATIONS
quarterly tax-at-source statement

MX Mexico — *the e-payslip*

CONTRIBUTIONS
social security on an integrated daily wage, capped at 25 × UMA

INCOME TAX
period tables, or monthly table × period factor

DECLARATIONS
one signed e-payslip (CFDI) per payslip

AU Australia *the real-time reporter*

CONTRIBUTIONS
no employee share; employer superannuation

INCOME TAX
weekly-equivalent coefficients

DECLARATIONS
a report to the tax office every pay run

BE Belgium *the heavyweight*

CONTRIBUTIONS
one uncapped employee rate; employer rates downloaded from official files

INCOME TAX
annualise the month, ÷ 12

DECLARATIONS
hire/exit notice, quarterly, monthly and yearly returns

What the six teach:

- **Size follows declarations and data feeds, not rules.** Australia has fewer rules than Mexico but far more code: it reports to the tax office in real time.
- **A rate has four possible homes:** a data file (NL, MX), a downloaded official file (BE), records the customer maintains (CH insurers), or a field on the employee's version (SA).
- **The employee side can be empty:** Australia has no employee contribution; the UAE has one for nationals only.
- **The accounting module can carry the law:** Mexico puts its e-payslip there, Australia its pay-run reporting.

Others (US, CH, SA, HK, KE…) are in `research/world_grid.md`.

> 💡 **In practice**
>
> Before estimating a new country, count its **statutory outputs** and its **sources of rates**. Each declaration is a model, an export, a test and a legal source to verify. Rules are cheap; declarations and data feeds are where the weeks go.

> ⚠️ **Common pitfall**
>
> Trusting a grid cell because the rule exists. Reading these modules found an income-tax rule that over-taxes incomes above the second band, tested only on lower salaries. A cell is verified when a golden payslip covers it (Chapter 10), not when the code is there. Chapter 22 lists these patterns.

> 🇧🇪 **In Belgium**
>
> Belgium sits at the top of most columns: more than 200 sectors with their own agreements, quarterly declarations, a dozen filings. Two of its answers travel: switching rules on and off by sector, and downloading official rates into dated parameters.

Summary

- Four layers: framework, country, company, employee. Put each fact in the highest layer where it is still true; every payslip needs all four.
- The framework (`hr_payroll`) holds the engine, categories, dated parameters, pay runs and hooks; a localization ships rules, data, fields and declarations.
- Ten axes describe a country, from pay frequency to data feeds.
- Rates live in four places: XML parameters, downloaded parameters, customer-maintained records, version fields.

- Size follows declarations and data feeds, not the number of rules.
- A grid cell is verified by a golden payslip, not by the presence of a rule.

---

1. Read from the `l10n_*_hr_payroll*` modules of enterprise master; counts by script over XML records, Python classes and test methods. `research/world_grid.md`, sections 1 to 11. ↩

Researching a new country

From the official gazette to a golden payslip: a source hierarchy, a template and a list of questions

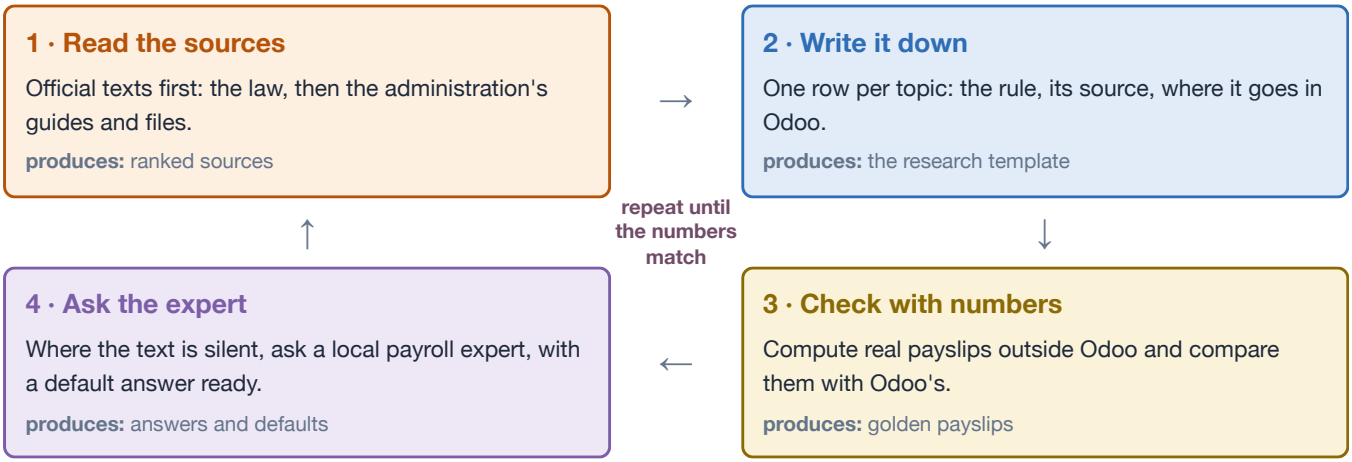
AFTER THIS CHAPTER YOU CAN

- Rank payroll sources in any country and know which one wins
- Fill the research template for a country and map each row to an Odoo target
- Run the template backwards on an existing module to find gaps and risks
- Handle an UNVERIFIED item: write it down, pick a default, ask, and keep it visible
- Build golden payslips from an independent calculation, and read the differences

Sales says: "We need payroll for country X by January." Nobody has read its tax code. Code from a provider's blog and you will discover in October that the tables change on 1 November and the tax office wants one signed file per payslip. Research is not a phase you finish; it is a loop that ends in numbers you can test.

The research loop

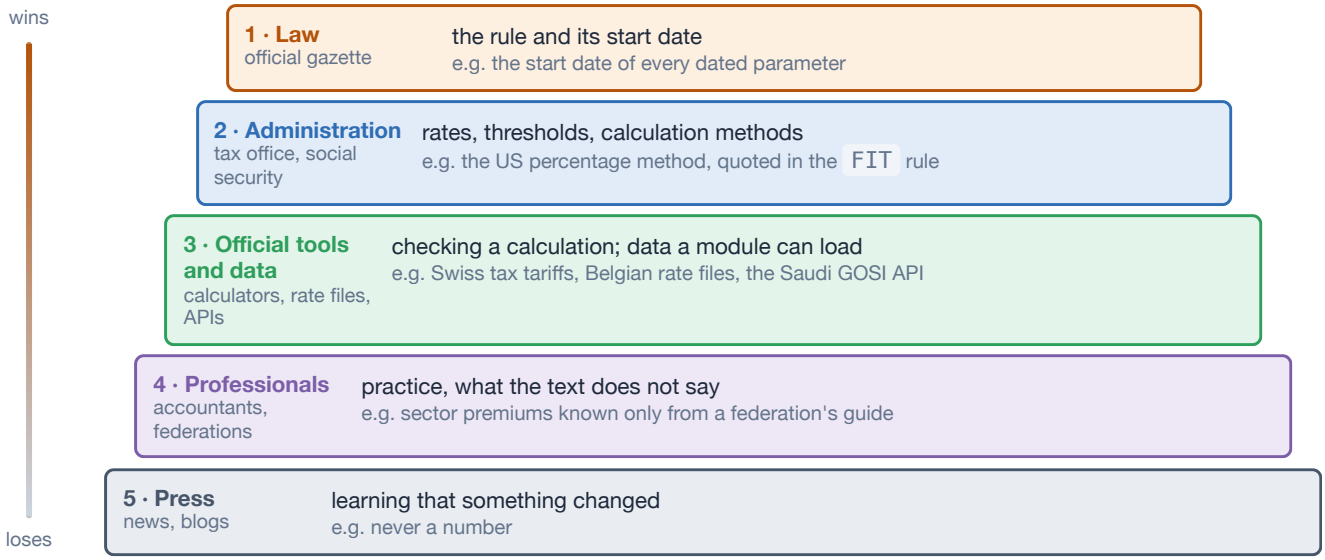
Researching a country is a loop of four steps, run again every time the numbers disagree. The rest of the chapter takes them one by one.



The examples come from this book's own loop on Belgium.

The source hierarchy

When two sources disagree, the higher one wins. Always write down which level a number comes from.



Level 3 deserves attention: an official file or API can be loaded by the module instead of copied by a developer (Chapter 11), and a file specification is a legal source in its own right. Belgium's tax office publishes a version history of its withholding rules, listing every amending decree.[2]

> ⚠ **Common pitfall**
>
> Treating the text as the whole truth. A rule for "income paid from 1 August" does not say what to do with a July salary paid on 3 August. Separate **what the text says** from **what practice does**, and note which one each number comes from.

The research template

Copy this table for each country, one row per axis of Chapter 9 (frequencies, earnings, time, payment and the others follow the same pattern). The last two columns turn research into work: where it goes in Odoo, and how much to trust it.

Topic	Question to answer	Typical Odoo target	Source level
Structures	which payments need their own payslip	`hr.payroll.structure` records	1–2
Contributions	funds, rates, bases, caps	rules + dated parameters	2–3
Income tax	method, tables, family situation	rules + parameters, version fields	2–3
Leavers	notice, severance, final pay	structure + departure flow	1–2
Declarations	each filing, period, deadline, format	declaration model, export	2–3

Two examples: one filled from the law, one filled backwards from an existing module.

Example 1: Belgium, filled from the law (excerpt)

Topic	Rule (2026)	Source (level)	Odoo target	Status
Employee social security	13.07 %	social-security instructions (2)	downloaded rate parameter	VERIFIED
Withholding method	annualise, flat expenses, scale, reductions, ÷ 12	tax office's key formula (2)	rule chain + helpers	VERIFIED
Proration of a joiner's month	no statutory method	none (practice)	worked-days valuation	**UNVERIFIED**
Sector end-of-year premium	one monthly salary, pro rata	accountants' federation (4) only	separate structure	**UNVERIFIED**

Example 2: Mexico, run backwards from `l10n_mx_hr_payroll`

Here the module exists and the question is "where does it do this?". The status column says what the code shows; the law itself was not checked for this book.

Topic	What the module does	Odoo target	Status
Frequencies	seven, from daily to bi-monthly	`_get_selection_schedule_pay`	in module
Contributions	on the integrated daily wage, capped at 25 × UMA	`SBC` rule	in module; the cap is hard-coded (see below)
Income tax	period tables	`ISR`, `l10n_mx_isr_tables`	in module
Leavers	severance and final settlement	—	**gap**

Running the template backwards found one gap (no severance flow) and one risk: the module declares a parameter for the 25 × UMA cap but the `SBC` rule writes `25` in its code, so the parameter is never read.

In practice

Run the template backwards on a module before you review or extend it. An empty "Odoo target" cell is a gap; an UNVERIFIED status is a risk.

Handling an UNVERIFIED item

The Belgian research ended with sixteen UNVERIFIED items,[3] which is normal. What matters is how each is handled.

Item	Default chosen	What happened next
Closure-fund rate (web page unreadable)	left out of the calculation	Odoo imports it from the official rate file: a machine-readable file answered what a web page could not
Proration of Yannick's March (no statutory method)	working days	Odoo uses a third method: question for the expert

> 🔑 **Key idea**
>
> An UNVERIFIED item is not a failure; a hidden one is. Give each a default, mark it wherever the number is used (docs, comments, tests), and tie it to a question with an owner. None should block the build; all should block the release.

Questions for a payroll expert

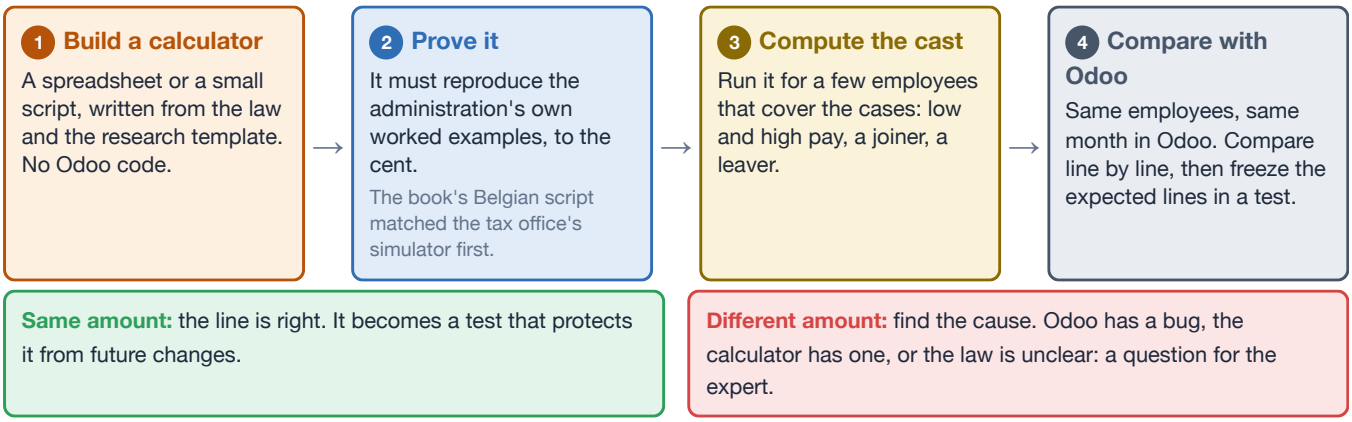
Research ends where the text is silent. Collect the questions, each with a **default** and a **design consequence**, so the answer changes data, not architecture. The same families come up everywhere:

Question family	Example	If the answer differs
Which proration method?	working days or a fixed daily rate?	another worked-days valuation
Which date decides?	does a July salary paid in August get the August rate?	the rule passes the payment date to the lookup (Chapter 11)
What enters a base?	is the pro-rata premium in the indemnity base?	one category added to a base
How is it rounded?	half-up at every step, or down?	one `float_round`; one cent on many payslips

Golden payslips

A test needs an **expected value**. If that value comes from Odoo itself, the test only proves that Odoo agrees with Odoo. A **golden payslip** is a payslip whose every line was computed **outside Odoo**, from the law, by something that does not share Odoo's code. When Odoo's payslip matches it line by line, the localization is right for that case; when it does not, one of the two is wrong, and you know exactly which line to look at.

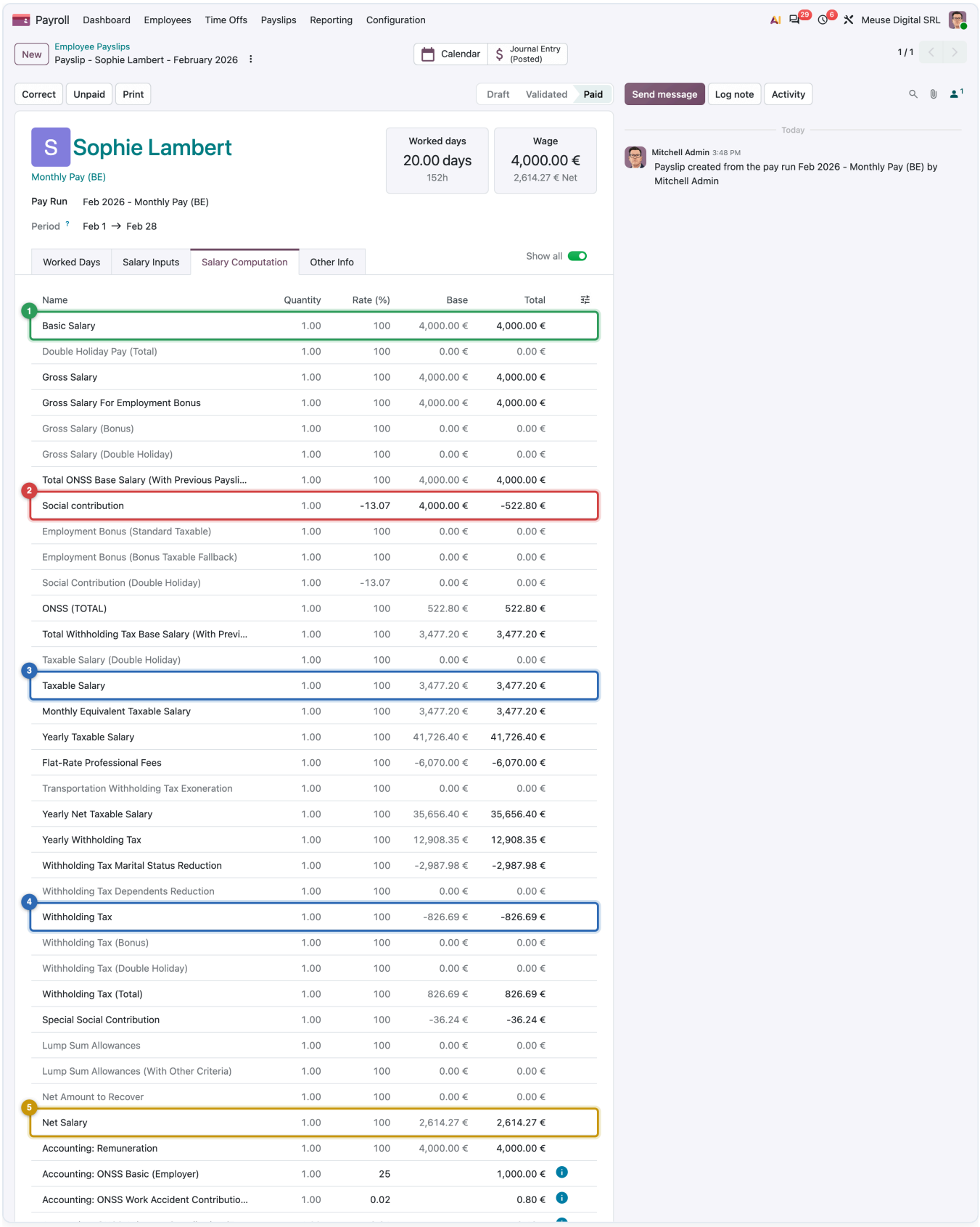
How to get one:



Worked example: Mélanie and Marie, calculator against Odoo

Line	Mélanie, calculator	Mélanie, Odoo	Marie, calculator	Marie, Odoo
Gross salary	4,000.00	4,000.00	7,500.00	7,500.00
Employee social security	522.80	522.80	980.25	980.25
Car benefit in kind	—	—	148.57	151.42
Taxable salary	3,477.20	3,477.20	6,668.32	6,671.17
Withholding tax	826.70	826.69	1,770.92	1,750.15
Net	**2,614.26**	**2,614.27**	**4,663.91**	**4,684.68**

October 2026 payslips.[4,5] Where both agree, the payslip is correct. Where they differ, find the cause and take it to the expert: Odoo rounds the monthly tax down where the official formula rounds half up (Mélanie's cent),[1] spreads the car benefit by days (Chapter 2), and deducts a transport exemption (Chapter 4).



	Name	Quantity	Rate (%)	Base	Total
1	Basic Salary	1.00	100	4,000.00 €	4,000.00 €
	Double Holiday Pay (Total)	1.00	100	0.00 €	0.00 €
	Gross Salary	1.00	100	4,000.00 €	4,000.00 €
	Gross Salary For Employment Bonus	1.00	100	4,000.00 €	4,000.00 €
	Gross Salary (Bonus)	1.00	100	0.00 €	0.00 €
	Gross Salary (Double Holiday)	1.00	100	0.00 €	0.00 €
	Total ONSS Base Salary (With Previous Paysli...	1.00	100	4,000.00 €	4,000.00 €
2	Social contribution	1.00	-13.07	4,000.00 €	-522.80 €
	Employment Bonus (Standard Taxable)	1.00	100	0.00 €	0.00 €
	Employment Bonus (Bonus Taxable Fallback)	1.00	100	0.00 €	0.00 €
	Social Contribution (Double Holiday)	1.00	-13.07	0.00 €	0.00 €
	ONSS (TOTAL)	1.00	100	522.80 €	522.80 €
	Total Withholding Tax Base Salary (With Previ...	1.00	100	3,477.20 €	3,477.20 €
	Taxable Salary (Double Holiday)	1.00	100	0.00 €	0.00 €
3	Taxable Salary	1.00	100	3,477.20 €	3,477.20 €
	Monthly Equivalent Taxable Salary	1.00	100	3,477.20 €	3,477.20 €
	Yearly Taxable Salary	1.00	100	41,726.40 €	41,726.40 €
	Flat-Rate Professional Fees	1.00	100	-6,070.00 €	-6,070.00 €
	Transportation Withholding Tax Exoneration	1.00	100	0.00 €	0.00 €
	Yearly Net Taxable Salary	1.00	100	35,656.40 €	35,656.40 €
	Yearly Withholding Tax	1.00	100	12,908.35 €	12,908.35 €
	Withholding Tax Marital Status Reduction	1.00	100	-2,987.98 €	-2,987.98 €
	Withholding Tax Dependents Reduction	1.00	100	0.00 €	0.00 €
4	Withholding Tax	1.00	100	-826.69 €	-826.69 €
	Withholding Tax (Bonus)	1.00	100	0.00 €	0.00 €
	Withholding Tax (Double Holiday)	1.00	100	0.00 €	0.00 €
	Withholding Tax (Total)	1.00	100	826.69 €	826.69 €
	Special Social Contribution	1.00	100	-36.24 €	-36.24 €
	Lump Sum Allowances	1.00	100	0.00 €	0.00 €
	Lump Sum Allowances (With Other Criteria)	1.00	100	0.00 €	0.00 €
	Net Amount to Recover	1.00	100	0.00 €	0.00 €
5	Net Salary	1.00	100	2,614.27 €	2,614.27 €
	Accounting: Remuneration	1.00	100	4,000.00 €	4,000.00 €
	Accounting: ONSS Basic (Employer)	1.00	25		1,000.00 €
	Accounting: ONSS Work Accident Contributio...	1.00	0.02		0.80 €

*Mélanie, February 2026, with *Show all* on. ① basic salary 4,000.00. ② social security −13.07 %. ③ taxable salary 3,477.20, equal to the calculator. ④ withholding −826.69, one cent below the calculator; the cent is already visible in the yearly tax line. ⑤ net 2,614.27.*

> ⚠️ **Common pitfall**
>
> Generating the "expected" values by running Odoo and pasting the output into the test. That test can never fail for a wrong rule, only for a changed one. The **first** snapshot of each case must be checked against an independent source (Chapter 21).

> 🌍 **Around the world**
>
> - **MX**: tests write the hand computation in a comment next to each expected value.
> - **US**: the `FIT` rule quotes the tax office's worksheet in its comments.
> - **BE**: JSON snapshots hold the expected lines of large payslips.
> - **NL**: no test reaches the upper tax bands.

Summary

- Research is a loop: scope, read, fill the template, list the UNVERIFIED items, ask, compute golden payslips, build, compare.
- Rank sources: law, administration, official tools and data, professionals, press. The higher level wins; official files and APIs are sources a module can load.
- One template row per topic, with its source level, its Odoo target and a status. Run it forwards from the law, or backwards on an existing module to find gaps and risks.
- An UNVERIFIED item gets a default, stays visible wherever it is used, and is tied to a question with an owner.
- Expert questions fall into families (proration, deciding date, base, rounding), each with a default and a design consequence.
- Golden values come from an independent calculation that first reproduces official examples.

---

1. SPF Finances (Belgian tax office), key formula from 1 January 2026 (n° 3: every amount rounded to the cent): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/389035ca-432e-422b-859d-2d32fbdaa4cf. ↩
2. SPF Finances, Annex III AR/CIR 92 version history: https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/439228c6-67bb-4e75-a9e1-4cabe4dc62ec. ↩
3. Research notes `research/belgium_law.md`, sections 11 (UNVERIFIED items) and 12 (open questions). ↩
4. Independent calculator, `research/numbers_be_calc.md`, sections 2, 4 and 10.8. ↩
5. Payslips computed by Odoo master on the demo database, `research/numbers_be.md`, sections 1 and 2. ↩

Laws that change over time

Effective dates, indexation, yearly tables, reference units, mid-year and retroactive changes, downloaded rates, and the dated parameter that absorbs them

AFTER THIS CHAPTER YOU CAN

- Model any legal value as a timeline of effective dates
- Tell indexation of wages (employee data) from a new table or unit (country data)
- Say which parameter value Odoo uses for a payslip, and why the period end decides
- Handle a scheduled reform, a mid-year change, and a law that applies to the past
- Choose between typing rates into data files and downloading them from the administration

Every country changes its payroll law on its own calendar. In 2026 Mexico got new tax tables on 1 January and a new reference unit on 1 February; Australia switches its superannuation basis on 1 July; Belgium needed a new withholding formula on 1 November. A payslip of the old period, reopened a year later, must still show the old amount. Payroll must always answer: *what was the law on that date?*

Where a change goes

When the law changes, the first question is **what** changed. Each kind of change has one home in Odoo, and in all three cases the payslip reads what was valid on its own dates.

The law changed. What changed?

A person's wage

a new `hr.version`, dated

e.g. wages indexed by 2.21 % on 1 January

A rate, a cap, a table

a new dated value of a `hr.rule.parameter`

e.g. next year's tax scale, a new contribution rate

The method itself

rule or helper code, reading new parameters

e.g. a new withholding formula with an extra step

Every payslip, old or new, is computed with the law of its own period.

Every legal value is a timeline

A rate is never "13.07 %". It is "13.07 % from a date, until a later value replaces it". The law changes on several rhythms:

Rhythm	Example	What the module stores
Yearly tables and units	MX tax tables each 1 January, reference unit each 1 February	one new value per year
Wage indexation	BE sector wages +2.21 % on 1 Jan 2026[3]	a new version of each contract
Scheduled reform	MX employer pension rates loaded up to 2030	future values, loaded in advance

Rhythm	Example	What the module stores
Mid-year change	AU super on 1 Jul; BE withholding on 1 Nov 2026	a dated value, sometimes a rule branch
Retroactive change	BE overtime reduction published in July, in force from 1 Jan	a past-dated value, plus corrections

> 🔑 **Key idea**
>
> Store every legal value with its start date, and **never edit a value that has already been used**. A new law is a new dated record. Then every payslip, old or new, recomputes with the law of its own period.

1 Jan 2026: MX tax tables, MX minimum wage, BE scale and index → **1 Feb**: MX reference unit → **1 Jul**: AU super basis → **1 Aug**: BE notice table → **1 Nov**: BE second tax formula of the year (and in July, a BE decree reaching back to 1 Jan).

Indexation: a new wage, not a new parameter

Some countries raise wages automatically with prices. The percentage is law; the result is a new wage for each person.

> 🧱 **Worked example: Mélanie on 1 January 2026**
>
> Belgian sector wages rose by 2.21 % on 1 January 2026.[3] Mélanie's 2025 salary of 3,913.51 € became 4,000.00 €.[4]

	Before 1 Jan 2026	From 1 Jan 2026
Mélanie's monthly salary	3,913.51	4,000.00
Index factor		× 1.0221

Indexation is a law about **every employee's wage**, so its result is employee data. Odoo stores it as a new version of the contract, not as a rule parameter.



In Odoo. The *Salary Increase* wizard (`hr.payroll.salary.increase`, Chapter 13) takes employees, a date and a rate. It does not change the old version: it **adds** one per employee, starting on the indexation date. Each payslip then picks the version valid in its period, so December keeps the old wage and January gets the new one. If January was already validated when the indexation is entered, the wizard opens the correction wizard (Chapter 15).

Yearly tables: one dated value per year

Most countries reissue their tax tables every year. The design is the same everywhere: one parameter, one value per year, the same shape every year (a list of brackets, or a dict of lists by frequency in Mexico). The Belgian value for 2026:

```
<record id="rule_parameter_value_prp_basic_bareme_2026" model="hr.rule.parameter.value">
    <field name="parameter_value">[(16710.0, 0.2675), (29500.0, 0.428), (51050.0, 0.4815), (None,
0.535)]</field>
    <field name="rule_parameter_id" ref="rule_parameter_prp_basic_bareme"/>
    <field name="date_from" eval="datetime(2026, 1, 1).date()"/>
</record>
```

Its bands match the official formula (16,710 / 29,500 / 51,050).[2] That check, value by value against the source, is the yearly upkeep of a localization (Chapter 22).

> ⚠️ **Common pitfall**
>
> Editing last year's value to this year's bands "because it is January". Every recomputed payslip of last year (a correction, a year-end certificate, a test) then silently uses this year's law. Add a record; never overwrite history.

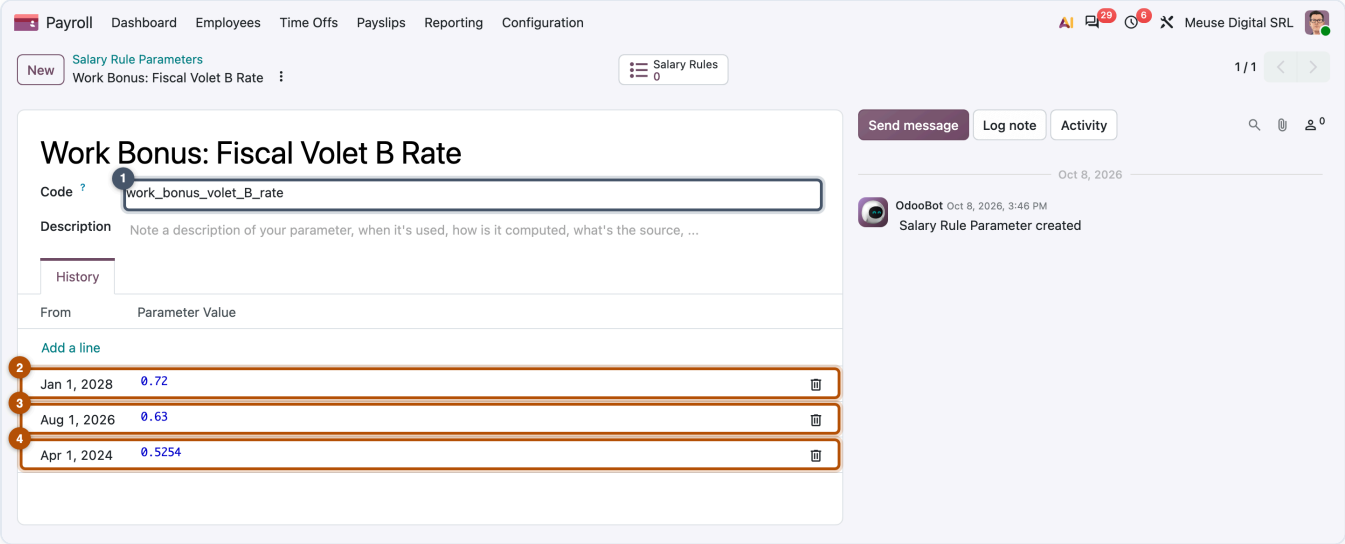
How Odoo picks the value for a payslip

A salary rule asks for a parameter by its code. It never mentions a date.

```
# in a rule or a helper on hr.payslip
uma = payslip._rule_parameter('l10n_mx_uma')['daily']
```

The engine returns the latest value whose `date_from` is on or before the payslip's **end date** (`date_to`). Two consequences for the law: a value starting on the 16th applies to the whole period, and a value has no end date, so a temporary measure is **two** values (the measure, and the one restoring the baseline). Chapter 21 shows how a rule reads it.

> 🇧🇪 **Worked example: Jurgen, July and August 2026**
>
> A Belgian tax reduction for low wages rises on 1 August 2026. Jurgen's July payslip reads the old value, his August payslip the new one.[4]
>
> | | July 2026 | August 2026 |
> |---|---|---|
> | Withholding before the reduction | 294.95 | 294.95 |
> | Low-wage tax reduction | −132.63 | −150.62 |
> | Withholding | 162.32 | 144.33 |
> | **Net pay** | **2,125.06** | **2,143.05** |
>
> The demo database agrees: the August pay run withholds 17.99 € less than July, exactly 150.62 − 132.63.[5]



The parameter behind that reduction, in Configuration > Rule Parameters. ① its code, the only thing the rule knows. ② a value already loaded for 2028. ③ the value from 1 August 2026 that Jurgen's August payslip reads. ④ the older value his July payslip reads.

> ⚠️ **Common pitfall**
>
> Assuming the work period is what the law looks at. Many texts speak of income **paid** from a date: a July salary paid on 3 August gets the July value in Odoo. If the payment date decides, the rule must pass it as reference date.

Mid-year changes

A mid-year change of a **number** is another dated value. A change of **method** needs code, and the question is how to keep that code dated.

> 🌍 **Around the world: Australia, 1 July 2026**
>
> Superannuation moves to a new base with a year-to-date cap. The parameter changes shape (a number before, a dict after) and the rule branches on a date in its code:

```
rule = payslip._rule_parameter("l10n_au_withholding_super")
if payslip.date_from >= date(2026, 7, 1):
    prior_slips = payslip._l10n_au_get_year_to_date_slips()
    prior_ote = prior_slips._get_line_values(['QE'], compute_sum=True)['QE']['sum']['total']
    remaining = max(rule['cap'] - prior_ote, 0)
    result = min(categories['QE'], remaining) * rule['rate'] / 100
else:
    result = categories['OTE'] * (rule / 100)
```

> But the date now lives in two places, and the rule keys on `date_from` while the lookup keys on `date_to`. A weekly payslip from 29 June to 5 July reads the new dict but takes the old branch, which divides a dict by 100 (read from the code, not run: UNVERIFIED). Branching on the parameter's own shape keeps one source of truth.

Worked example: Marie's November rise (Belgium)

The 1 November 2026 formula (Chapter 4) changed three family levers: Marie's withholding rises by 28.86 €, the singles' falls by 8.47 €.[1] In Odoo it needs no code: three new records dated 1 November, next to the January ones.

```
<record id="rule_parameter_value_prp_max_spouse_income_2026_11"
model="hr.rule.parameter.value">
    <field name="parameter_value">11780.0</field>
    <field name="rule_parameter_id" ref="rule_parameter_prp_max_spouse_income"/>
    <field name="date_from" eval="datetime(2026, 11, 1).date()"/>
</record>
```

In practice

On 8 October 2026 master had no value dated 1 November for those parameters: Marie would be under-withheld by 28.86 € a month. The weekly payroll data cron (Chapter 21) delivers the fix without a module upgrade, if it is merged before the November pay runs. Track announced changes with their dates.

Retroactive changes

Sometimes the law is published after its start date. A Belgian decree of 24 July 2026 extended an overtime tax reduction to all sectors **from 1 January 2026**.[1]

Start of the rule in the past → Publication → Data update value dated at the start → Computed with the old rule → New and recomputed payslips see the new rule → Correction payslips (Chapter 6, Chapter 15)

A past-dated value fixes **new** and **recomputed** payslips. Validated ones record what was paid; the difference becomes a correction. Odoo flags validated payslips when a version, the company settings or the time off change after validation, but **not** when a rule parameter gets a past-dated value: find those payslips yourself.

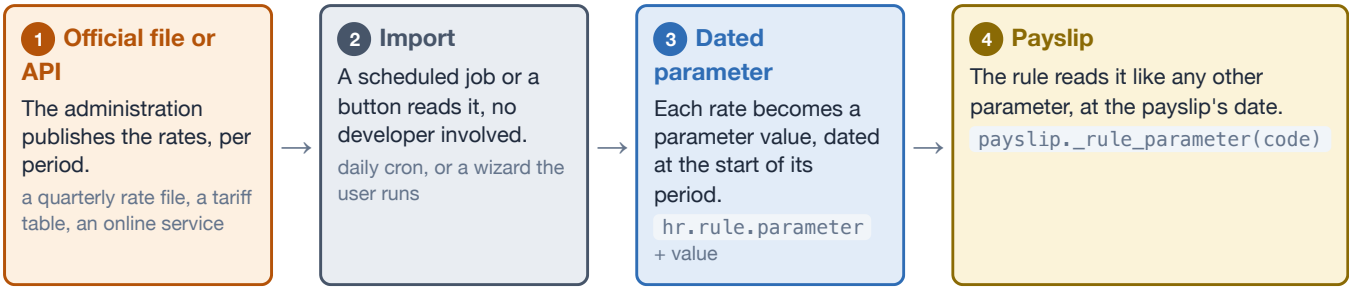
Common pitfall

Assuming the system will "catch up" after a retroactive value. It won't. List the validated payslips of the period, recompute them in a correction run, and check whether filed declarations need an amendment.

Rates you download

When the administration publishes rates as a file or an API, a module can load them instead of a developer typing them:

Country	Source	How it arrives	Where it lands
BE	social-security rate files, per quarter	daily cron + wizard	dated rule parameters
CH	cantonal source-tax tariffs	wizard run by the user	dated rule parameters
SA	the GOSI API	optional module with a cron	fields on the version



Imported data still goes through dated parameters, so an old quarter's payslip recomputes with that quarter's rates. The closure-fund rate the research could not read on a web page (Chapter 10) arrives this way.[5]

> ⚠ **Common pitfall**
>
> A test database or offline server has no downloaded rates, and the lines that read them may silently compute 0. Tests load fixed copies instead. Also, a corrected file for a past period updates that value in place: an intended exception to "never edit history".

Summary

- Every legal value is a timeline. Countries change values yearly (tables, units), quarterly, mid-year, on a multi-year schedule, and sometimes retroactively.
- Indexation changes the wage, so it is a new `hr.version` (wizard *Salary Increase*); a new rate, table or unit is a new `hr.rule.parameter.value`.
- Store a reference unit plus multipliers: one value a year moves every cap.
- A payslip reads the latest value whose `date_from` is on or before its `date_to`; a temporary measure needs a closing value.
- A change of number is data; a change of method needs code, and the date should live in one place.
- A retroactive value fixes new and recomputed payslips; validated ones are corrected, and nothing flags them for you.
- Rates published as files or APIs can be imported into dated parameters (BE, CH) or fields (SA); payslips stay reproducible.

---

1. SPF Finances (Belgian tax office), Annex III AR/CIR 92 for income 2026, consolidated with the AR of 24 July 2026 and the AR of 27 September 2026 (n° 32, 34, 38, 45, 46.2): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/0c174ec5-448b-4d1e-983b-fc457e5e7e28. ↩↩
2. SPF Finances, key formula from 1 January 2026 (annex 1, basic scale): https://www.minfin.fgov.be/myminfin-web/pages/public/fisconet/document/389035ca-432e-422b-859d-2d32fbdaa4cf. ↩
3. SPF Emploi (Belgian labour ministry), minimum wages of joint committee 200, "Indexation de 2,21% en 1/2026": http://www.salairesminimums.be/document.html?jcId=8e8cddf22c5d4cc9b9fe6234b4ea569e&date=01%2F03%2F2026. ↩↩
4. Independent calculator, `research/numbers_be_calc.md`, sections 3, 8 and 10.7. ↩↩
5. Payslips computed by Odoo master on the demo database, `research/numbers_be.md`, sections 1 and 6. ↩↩

PART III

Odoo payroll, hands-on

The screens a payroll officer meets every month, in the order the data flows: setup, employees, time, pay runs, accounting, declarations. Each chapter ends with missions played on the master runbot, from an empty database to a departure in October.

Tour and setup

Which modules make up payroll, what each menu holds, and where the company settings live

AFTER THIS CHAPTER YOU CAN

- Name the modules of Odoo payroll and say what each one adds
- Find your way in the Payroll app: dashboard, employees, time off, payslips, reporting, configuration
- Tell a structure type, a structure, a salary rule and a rule parameter apart
- Configure a company's payroll settings and read the dashboard warnings
- Set up MyVidangeCompany's payroll from scratch on the master runbot

It is your first morning on a payroll project. Someone gives you an empty Odoo and says: "Set up payroll for our company and run the year." Before you click anything you need a map: which module owns which screen, where the wage lives, where the law lives, and what the dashboard is telling you. This chapter is that map, and its missions set up the company you will pay all year.

Part III follows the data. The employee's **version** (Chapter 13) says *what* to pay. **Time** (Chapter 14) says *how much of it was earned*. A **pay run** (Chapter 15) turns both into payslips. **Accounting** (Chapter 16) books and pays the result. **Declarations** (Chapter 17) tell the authorities.

Each chapter ends with missions, played on one master runbot database that you build from an empty Odoo (the preface explains how). They follow MyVidangeCompany through 2026:

January setup, first run, a late correction → March Yannick joins → April sick day, unpaid day → May raise, overtime → June holiday-pay run → October Emma leaves, capstone month

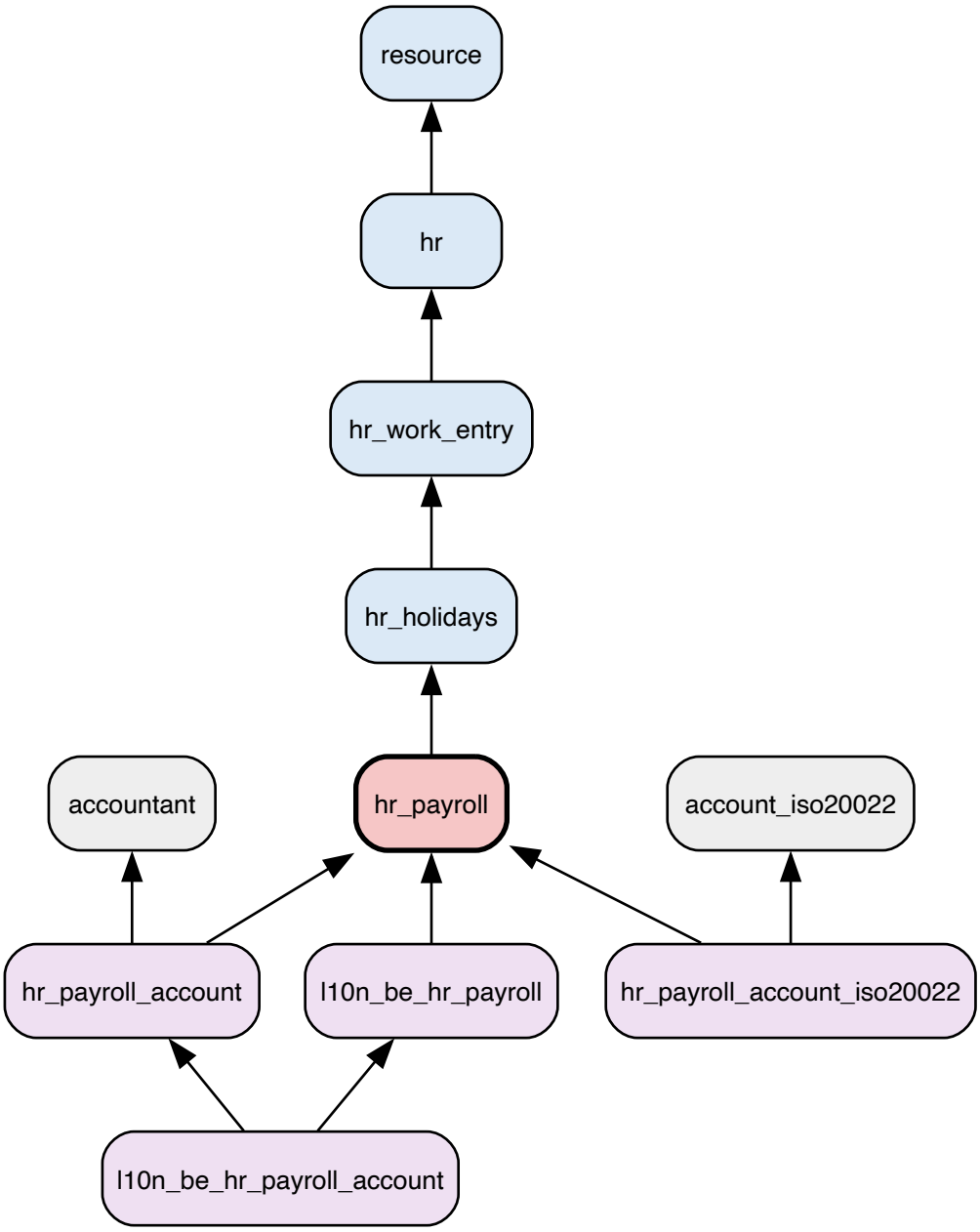
> ⚠️ **Common pitfall: the wrong company**
>
> A runbot database also holds Odoo's demo companies. If the company switcher (top right) is not on *MyVidangeCompany*, the Payroll app shows other employees, structures and warnings, and no mission check will match.

The module stack

Payroll is not one module but a stack. Each layer adds one idea.

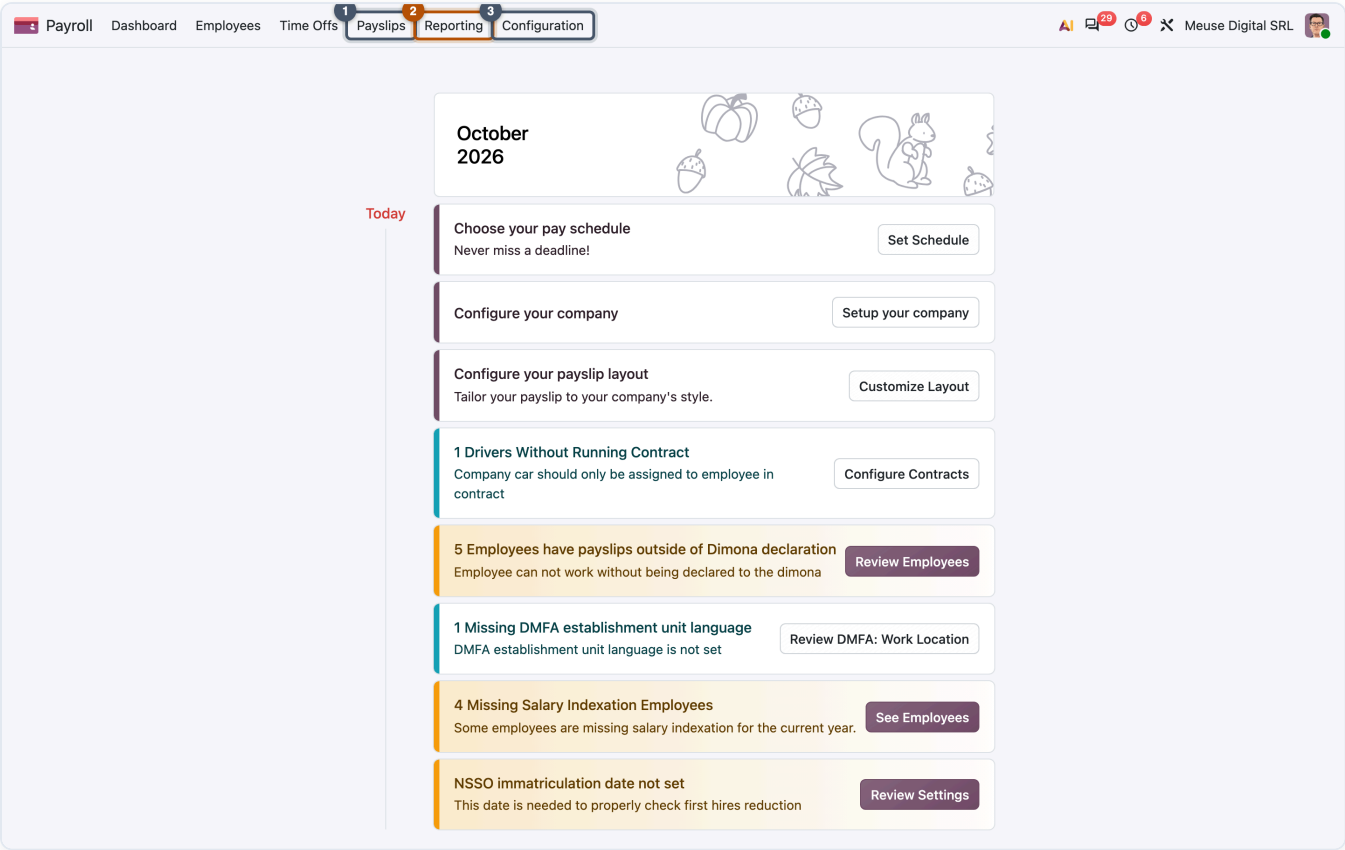
Module	Adds
`resource`	working schedules, public holidays, the time-interval engine
`hr`	employees and **hr.version**, the dated record of the employment terms
`hr_work_entry`, `hr_holidays`	time types, the work-entry generator, time off
`hr_payroll` (enterprise)	structures, rules, parameters, payslips, pay runs, warnings, declarations
`hr_payroll_account`, `hr_payroll_account_iso20022`	journal entries from payslips; the SEPA bank file

Module	Adds
`l10n_be_hr_payroll` (+ `_account`)	the Belgian structures, rules, parameters, declarations and accounts



Arrows point to what a module depends on. The localization sits *on top of* `hr_payroll` and never replaces it.

The Payroll app, menu by menu



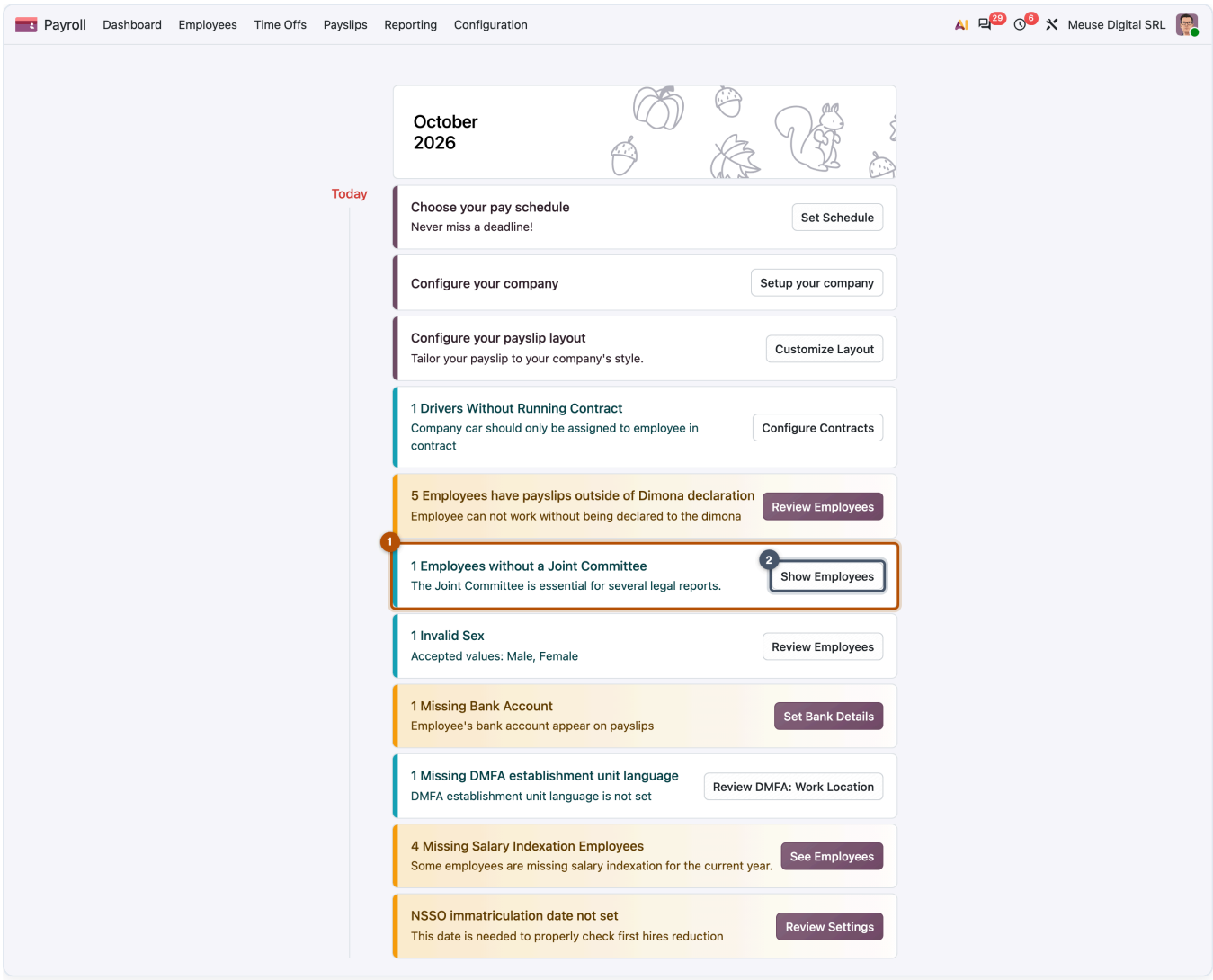
*The Payroll dashboard of MyVidangeCompany (October 2026, from the full demo database). ① **Payslips**, where pay runs and payslips live. ② **Reporting**, where the country's declarations hang. ③ **Configuration**, where schedules, structures, rules and settings live. The cards are warnings: purple ones are onboarding to-dos, blue ones information, orange ones problems to fix.*

Menu	Opens	Chapter
Employees	the employee form and its *Payroll* tab (`hr.version`)	13
Time Offs	the absences of each employee (`hr.leave`)	14
Payslips > Pay Runs / Payslips	`hr.payslip.run` , `hr.payslip`	15
Reporting	pivots on payslip lines, and the country's declarations	17
Configuration > Settings, Company data	the company's payroll settings	this chapter
Configuration > Salary	structures, rules, parameters, categories	Part IV

Configuration is for payroll **managers**; payroll **officers** see the other menus.

The dashboard is a list of checks

Every card on the dashboard is a record of `hr.payroll.warning`: a name, a colour, and a check written as a domain or a few lines of Python. The dashboard runs the checks for the current company and shows those that find something; the card's button opens the records found. Some warnings also **block validation** of the payslips they concern (Chapter 15).

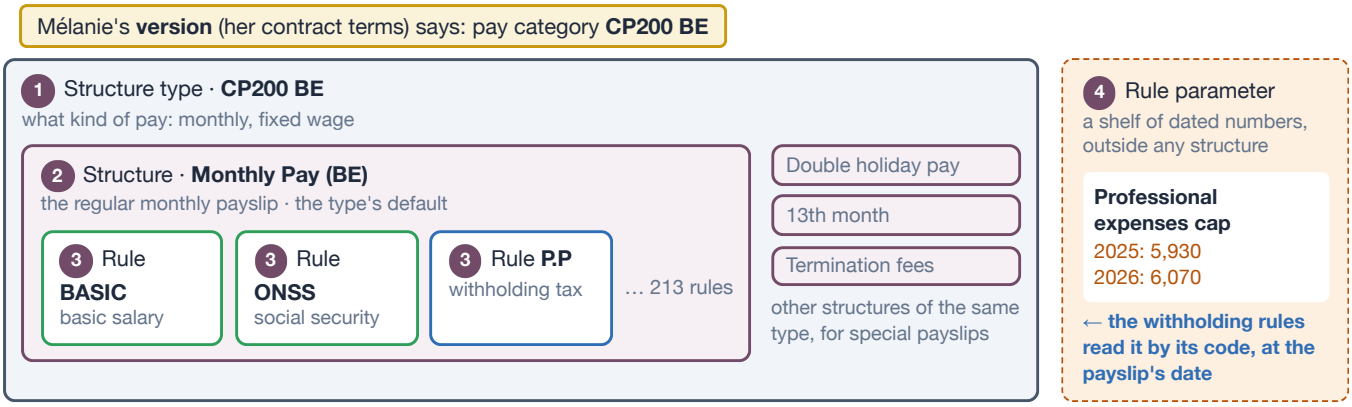


*A check catching a data problem: ① the card "Employees without a Joint Committee" (one employee is missing the Belgian sector field, see the box below) and ② its **Show Employees** button, which opens exactly the employees the check found.*

> 🔑 **Key idea: a warning is configuration, not code**
>
> Because a warning is a record, a localization adds its legal checks as data, and a customer can add their own under *Configuration* > *Dashboard* > *Warnings*. Chapter 21 places warnings among the records a localization ships.

Four words that sound alike

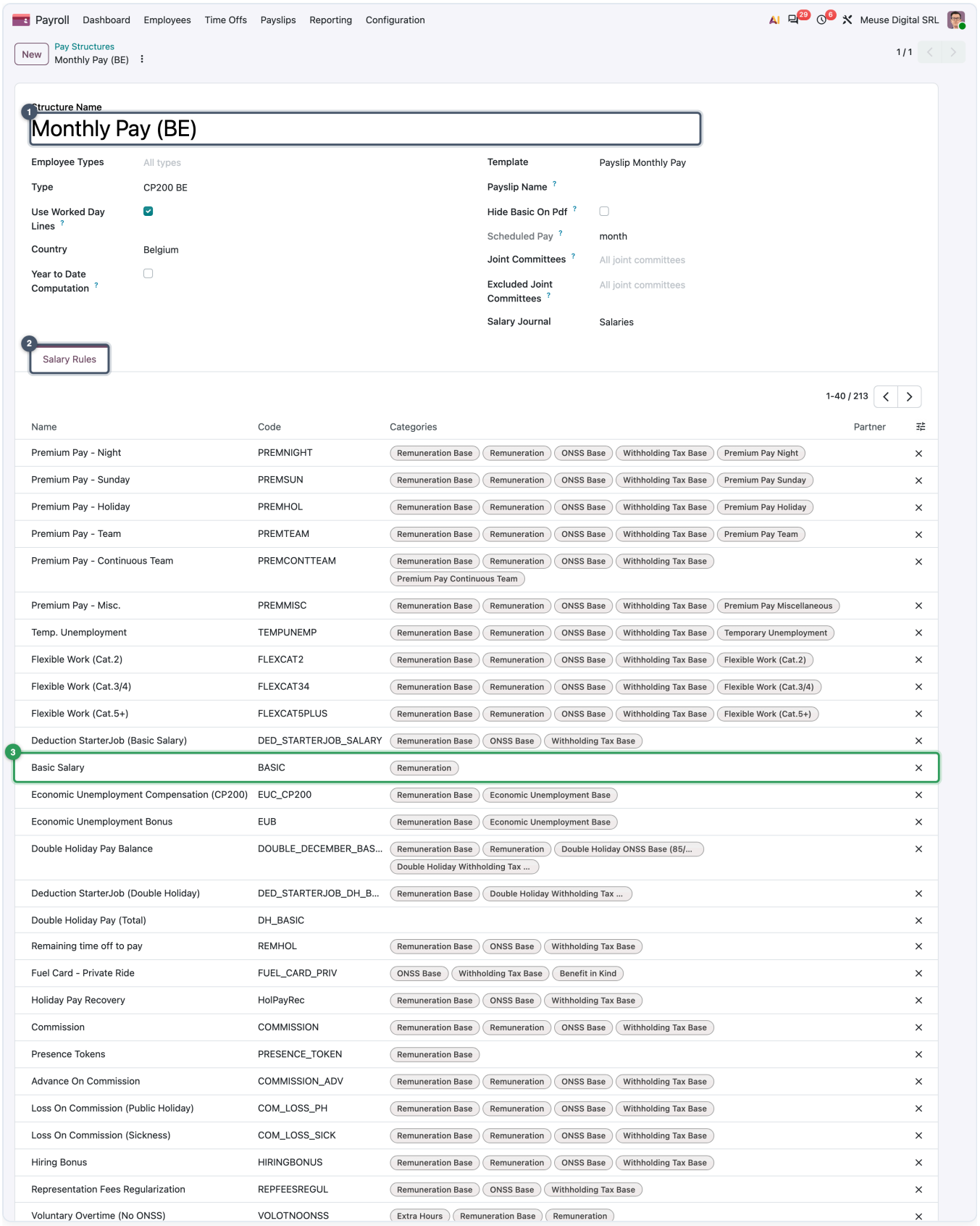
Four Odoo objects decide how a payslip is computed: the **structure type**, the **structure**, the **salary rule** and the **rule parameter**. Their names sound alike and everyone mixes them up at first. The easiest way to keep them apart is to follow one payslip and ask: *how does Odoo know what to compute for Mélanie in October?*



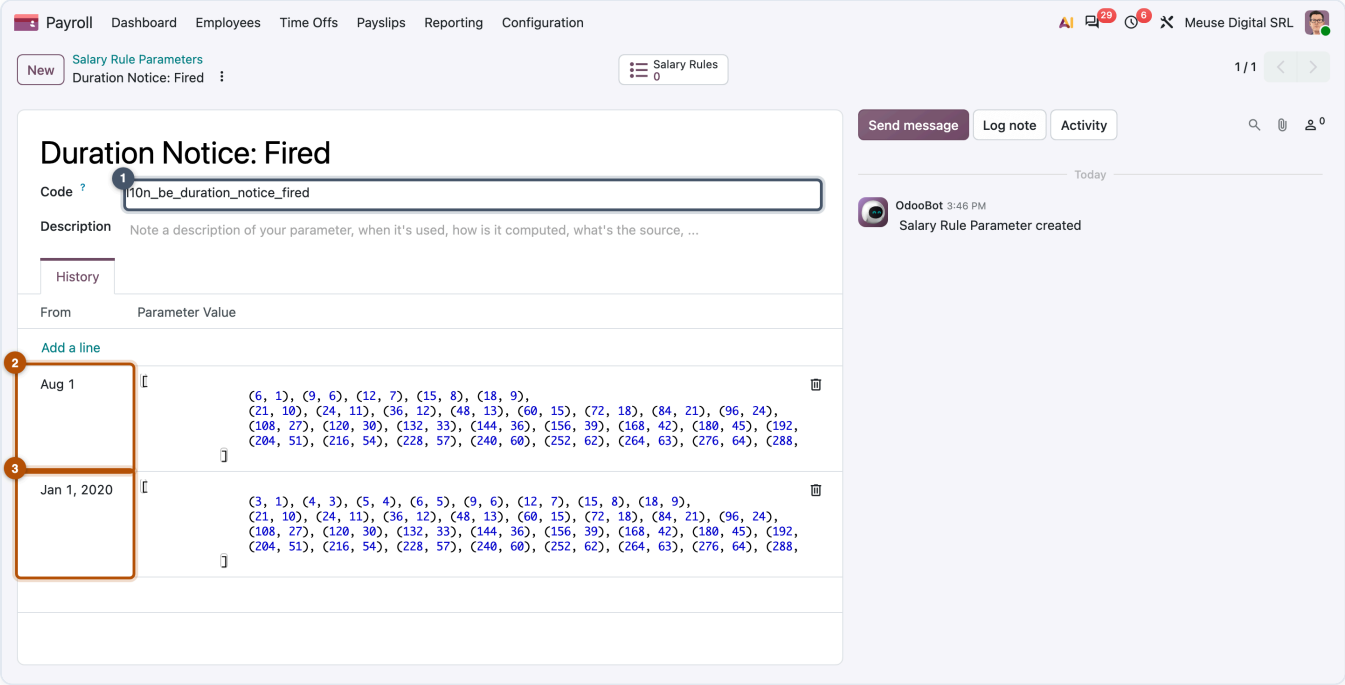
Read the figure in order of the numbers:

1. **The structure type is the kind of pay.** It is stored on the employee's version and answers *who is paid how, and how often*: here, a monthly fixed wage under the sector agreement CP 200. Change an employee's type and a different set of rules runs, without touching a single rule.
2. **A structure is the recipe for one kind of payslip.** The type has a **default** structure for the regular monthly payslip, and others for payslips that have their own tax treatment: double holiday pay, the 13th month, termination fees. Each runs on its own payslip (Chapters 15 and 17).
3. **A salary rule computes one line of the payslip**: when it applies (its condition), how much (its formula) and which totals it feeds (its categories). The regular Belgian structure has 213 of them; `P.P` computes the withholding tax.
4. **A rule parameter holds a legal number with its dates.** A rule never writes the law's numbers in its code: the withholding rules ask for the parameter by its code, and get the value valid on the payslip's date, 6,070 in 2026. Next year's law is a new value, not a new rule (Chapters 11 and 21).

> 🔑 **Key idea**
>
> Each object changes for a different reason: the **type** when the employee's kind of pay changes, a **structure** when a new kind of payslip is needed, a **rule** when the calculation method changes, a **parameter** when the law's numbers change. Keeping them apart is what lets a yearly law change be a one-line data update.



*Configuration > Salary > Structures > Monthly Pay (BE). ① the structure name. ② the **Salary Rules** tab, with the rules and their categories. ③ the **BASIC** rule. The header holds the type (CP200 BE), Use Worked Day Lines (this structure is driven by time), the scheduled pay and the salary journal.*



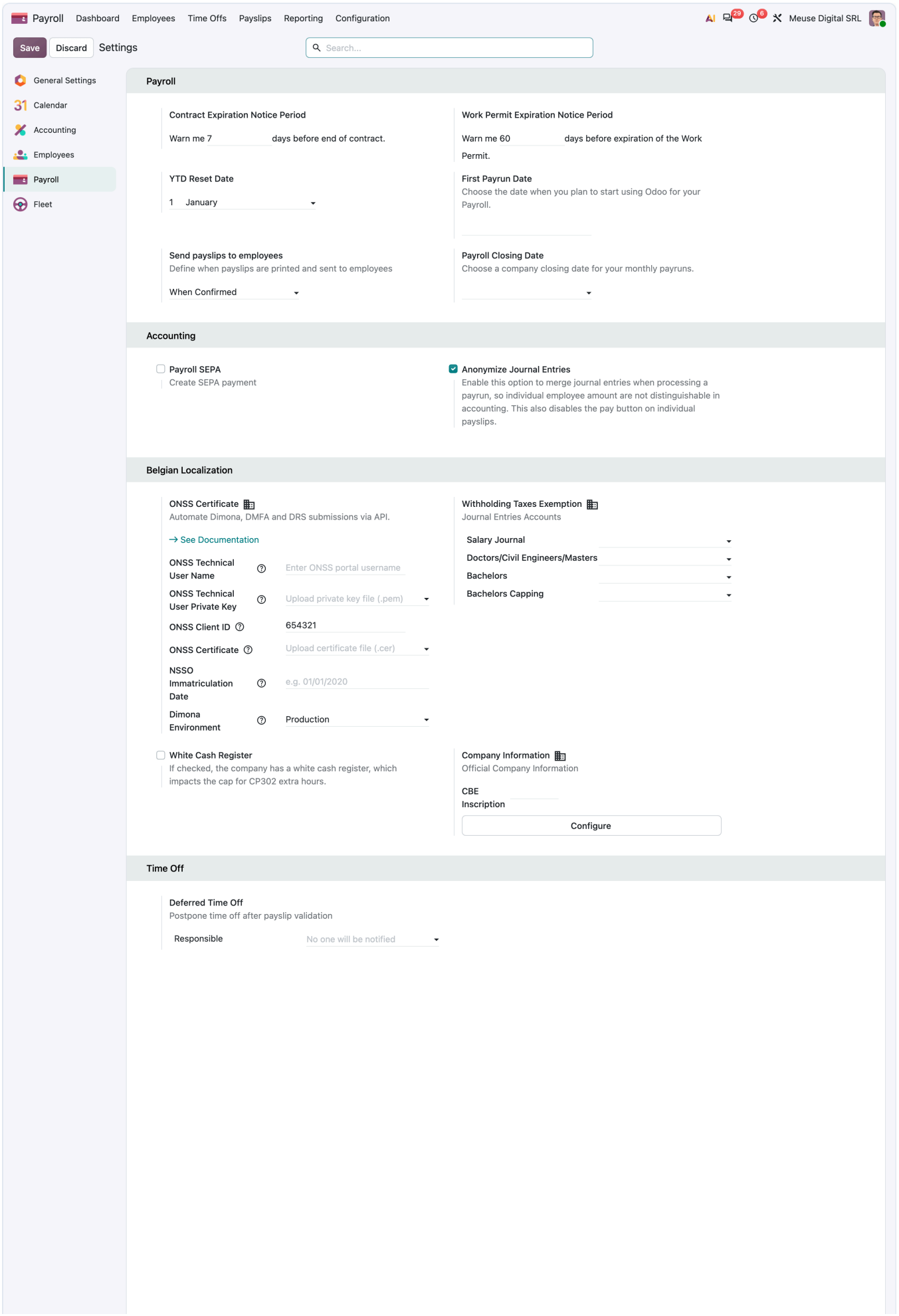
Configuration > Salary > Rule Parameters, the notice length after a dismissal: ① its code, which rules use to read it, ② the value valid from 1 August and ③ the value valid from 1 January 2020. A payslip reads the value in force at its end date (Chapters 11 and 21).

🌍 Around the world

- **Belgium, UAE, Saudi Arabia**: monthly structure types.
- **Philippines**: a semi-monthly type.
- **Hong Kong**: an extra weekly type for casual employees.
- **India**: a second type for interns.
- **Belgium, Luxembourg**: several structures per type, because holiday pay, bonuses and termination are separate payslips.

Company settings

Configuration > *Settings* holds the operational choices; *Configuration* > *Company data* holds the legal identity of the employer, as a dated record.


Payroll
Dashboard
Employees
Time Offs
Payslips
Reporting
Configuration
Meuse Digital SRL
Save
Discard
Settings
Search...
General Settings
Calendar
Accounting
Employees
Payroll
Fleet
Payroll
Contract Expiration Notice Period
Warn me 7 days before end of contract.
Work Permit Expiration Notice Period
Warn me 60 days before expiration of the Work Permit.
YTD Reset Date
1 January
First Payrun Date
Choose the date when you plan to start using Odoo for your Payroll.
Send payslips to employees
Define when payslips are printed and sent to employees
When Confirmed
Payroll Closing Date
Choose a company closing date for your monthly payruns.
Accounting
Payroll SEPA
Create SEPA payment
Anonymize Journal Entries
Enable this option to merge journal entries when processing a payrun, so individual employee amount are not distinguishable in accounting. This also disables the pay button on individual payslips.
Belgian Localization
ONSS Certificate
Automate Dimona, DMFA and DRS submissions via API.
See Documentation
ONSS Technical User Name
Enter ONSS portal username
ONSS Technical User Private Key
Upload private key file (.pem)
ONSS Client ID
654321
ONSS Certificate
Upload certificate file (.cer)
NSSO Immatriculation Date
e.g. 01/01/2020
Dimona Environment
Production
Withholding Taxes Exemption
Journal Entries Accounts
Salary Journal
Doctors/Civil Engineers/Masters
Bachelors
Bachelors Capping
White Cash Register
If checked, the company has a white cash register, which impacts the cap for CP302 extra hours.
Company Information
Official Company Information
CBE Inscription
Configure
Time Off
Deferred Time Off
Postpone time off after payslip validation
Responsible
No one will be notified

*Configuration > Settings for MyVidangeCompany. **Payroll**: expiry notices, year-to-date reset date, when payslips are sent, payroll closing date. **Accounting**: Payroll SEPA and Anonymize Journal Entries. **Belgian Localization**: the social security office connection and the employer's registration date. **Time Off**: Deferred Time Off with its responsible.*

Setting	What it does
YTD Reset Date	the day the year-to-date totals restart
Send payslips to employees	when the PDF is e-mailed: when confirmed, when paid, never
Payroll SEPA	installs the SEPA module, so the *Pay* button can produce a bank file
Anonymize Journal Entries	one journal entry per pay run instead of one per payslip
Deferred Time Off > Responsible	who is warned when time off arrives after its month was validated (Chapter 14)

Company data (`payroll.config.settings`) has an *Effective Date*. A localization stores there the employer data the law attaches to a period: registration numbers, sector, size class. A 2025 payslip still reads the 2025 configuration after the company changes it.

In Belgium

The localization installs its rules, structures and warnings **archived**. Choosing the employer's sector agreement (the *joint committee*, CP 200 for most office employees) in *Company data* activates the ones that apply. A chart of accounts loaded before that leaves those rules without accounts (Chapter 22 gives the fix), which is why Mission 12.2 chooses the committee first.

In practice: the order of setup for a new company

1. Install the localization, then fill *Company data* before loading the chart of accounts.
2. Check the structure type, the working schedule and the public holidays (Chapter 14).
3. Check the salary journal on the structure and the accounts on the rules (Chapter 16).
4. Create employees and their versions (Chapter 13), then the first pay run (Chapter 15).
5. Read the dashboard after each step: a card that does not go away is a step you skipped.

Missions

Mission 12.1 · Install Payroll and create the company ★

Situation. MyVidangeCompany (Rue des Brasseurs 10, 5000 Namur, Belgium, VAT BE0765432146) wants to run its payroll in Odoo. Your runbot database is still empty (Mission 0.1).

Your goal. Install Payroll, create MyVidangeCompany and switch to it.

Check. The company switcher lists *MyVidangeCompany*, and *Apps* shows **Belgium - Payroll** as installed, although you never clicked it.

▼ Hint

Install *Payroll* only, not *Accounting* yet (see Mission 12.2). Companies live in the *Settings* app; a Belgian company makes Odoo install the Belgian payroll module by itself.

🎯 Mission 12.2 · Switch on the Belgian localization ★★★

Situation. The accountant sends the employer data: social security number 119753095, employer category 010, joint committee 200, 5 to 9 employees, revenue code 1293, payroll valid since 1 January 2019. Staff work 38 hours a week, Monday to Friday, 8:00–12:00 and 12:30–16:06, and payroll starts in January 2026.

Your goal. Fill *Company data*, install Accounting with the Belgian chart of accounts, make *Standard 38 hours/week* the company's schedule and load the 2026 public holidays.

Check. *Monthly Pay (BE)* uses the journal *Salaries* and its rule *Net Salary* credits **455000 Remuneration**; the currency is **EUR**; the rule parameters include the *ONSS Rates: Employer Class (010) …* lines (**97** when this book was played). The schedule has **38 hours a week**, 7.6 hours a day, work time rate **100 %**; the company has **10 public holidays** in 2026.

▼ **Hint**

Order matters: the joint committee before the chart (the *Fiscal Localization* of the Accounting settings), and a company-data *Effective Date* of 01/01/2019. The rates download a few seconds after you save; if none appear, or you play after 2026, use *Configuration* > *Belgium* > *Import ONSS Rates* (debug mode), 2026, Q1. A new schedule takes the 40-hour one as *Reference Calendar*: empty that field, or everybody is 95 % part-time.

🎯 Mission 12.3 · Configure MyVidangeCompany's settings ★

Situation. MyVidangeCompany registered as an employer with the social security office on 1 October 2019; the payroll manager (Mitchell Admin) wants to hear about late time off; salaries will be paid by SEPA transfer.

Your goal. Record the three facts in the payroll settings.

Check. The card *NSSO immatriculation date not set* is gone, the dashboard shows **3 cards** (*Choose your pay schedule*, *Configure your payslip layout*, *Create your first employee*) and **SEPA Payments for Payroll** is installed.

▼ **Hint**

All three are on one settings page, in three blocks. The SEPA checkbox installs a module, so the page reloads.

🎯 Mission 12.4 · Find where the law lives ★★

Situation. A developer from another office asks: "How many rules does the regular monthly structure run, and what is the 2026 cap on flat-rate professional expenses?"

Your goal. Answer both from the screens, without reading code.

Check. *Monthly Pay (BE)* has **213** salary rules; *CP200: Maximum Professional Flat Fee* is **6,070.00** from 1 January 2026 (5,930.00 from 1 January 2025).

▼ **Hint**

Both answers are under *Configuration* > *Salary*; search the parameters for "Professional".

Summary

- Payroll is a stack: `resource`, `hr` (with **hr.version**), work entries and time off, `hr_payroll`, then accounting, SEPA and the localization on top.
- The Payroll app has six menus: Dashboard, Employees, Time Offs, Payslips, Reporting and Configuration.
- Dashboard cards are **hr.payroll.warning** records: a check, a colour, a button to the records found; some block validation.
- Structure type (kind of pay) > structure (set of rules) > salary rule (one line) > rule parameter (a dated number).
- *Settings* holds operational choices; *Company data* holds the employer's legal identity as a dated record.
- In Belgium, choosing the joint committee activates the localization's rules: do it before loading the chart of accounts.

Employees and versions

One employee, a dated record per change, and every pay element in its right place

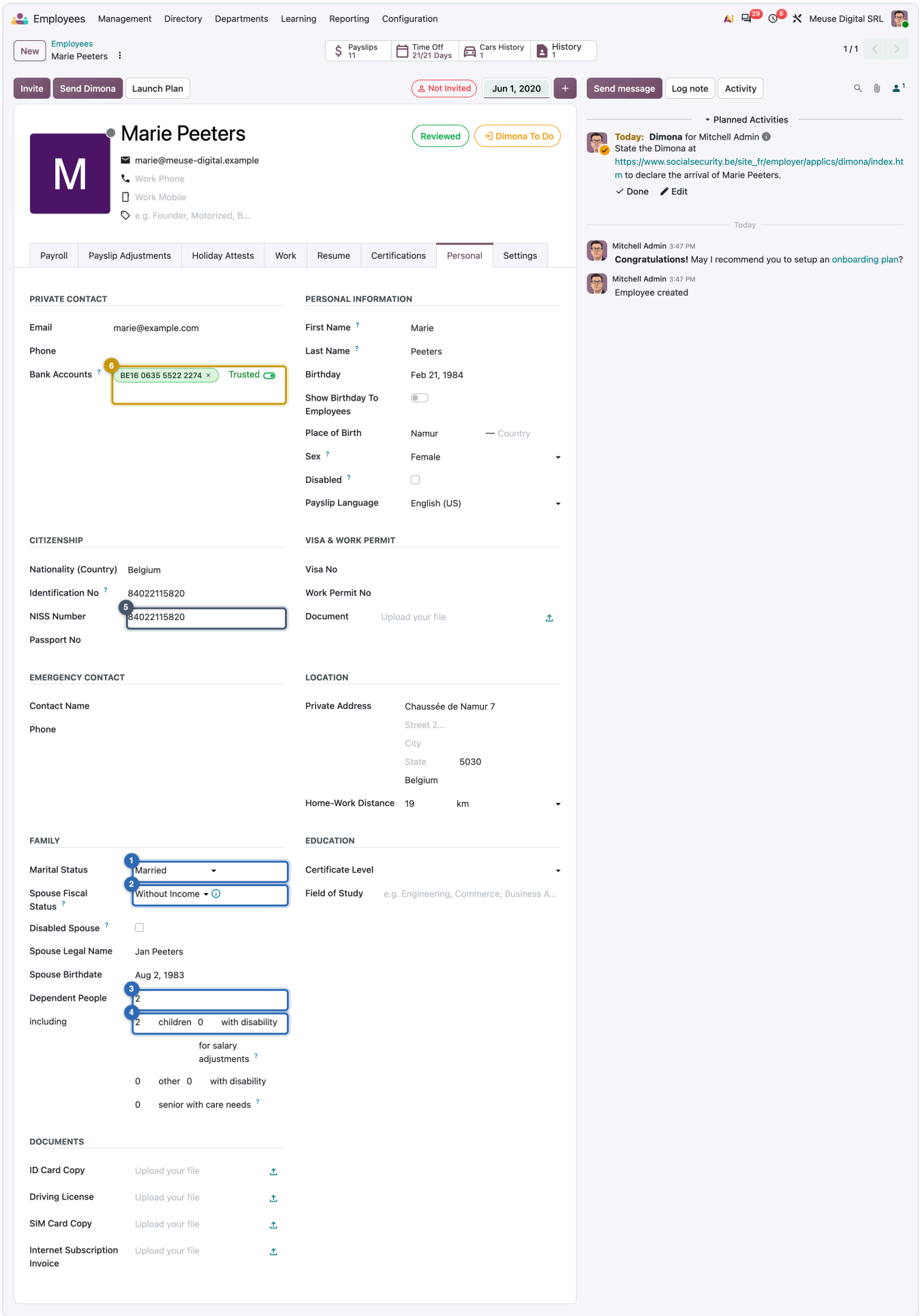
AFTER THIS CHAPTER YOU CAN

- Read an employee form and say which fields payroll uses, and why
- Explain what a version is, how versions follow each other, and what a contract is on top of them
- Distinguish wage, wage type, pay frequency and pay category
- Choose between a version field, a payslip input and a payslip adjustment for a pay element
- Hire an employee and give a raise in Odoo without rewriting history

This spring Yannick joins MyVidangeCompany on 12 March, and Jurgen gets a raise from 1 May. Each time payroll must know *what* was agreed and *from when*. Odoo answers with one idea: the employee's terms live in **versions**, dated records that follow each other.

The employee: who is paid

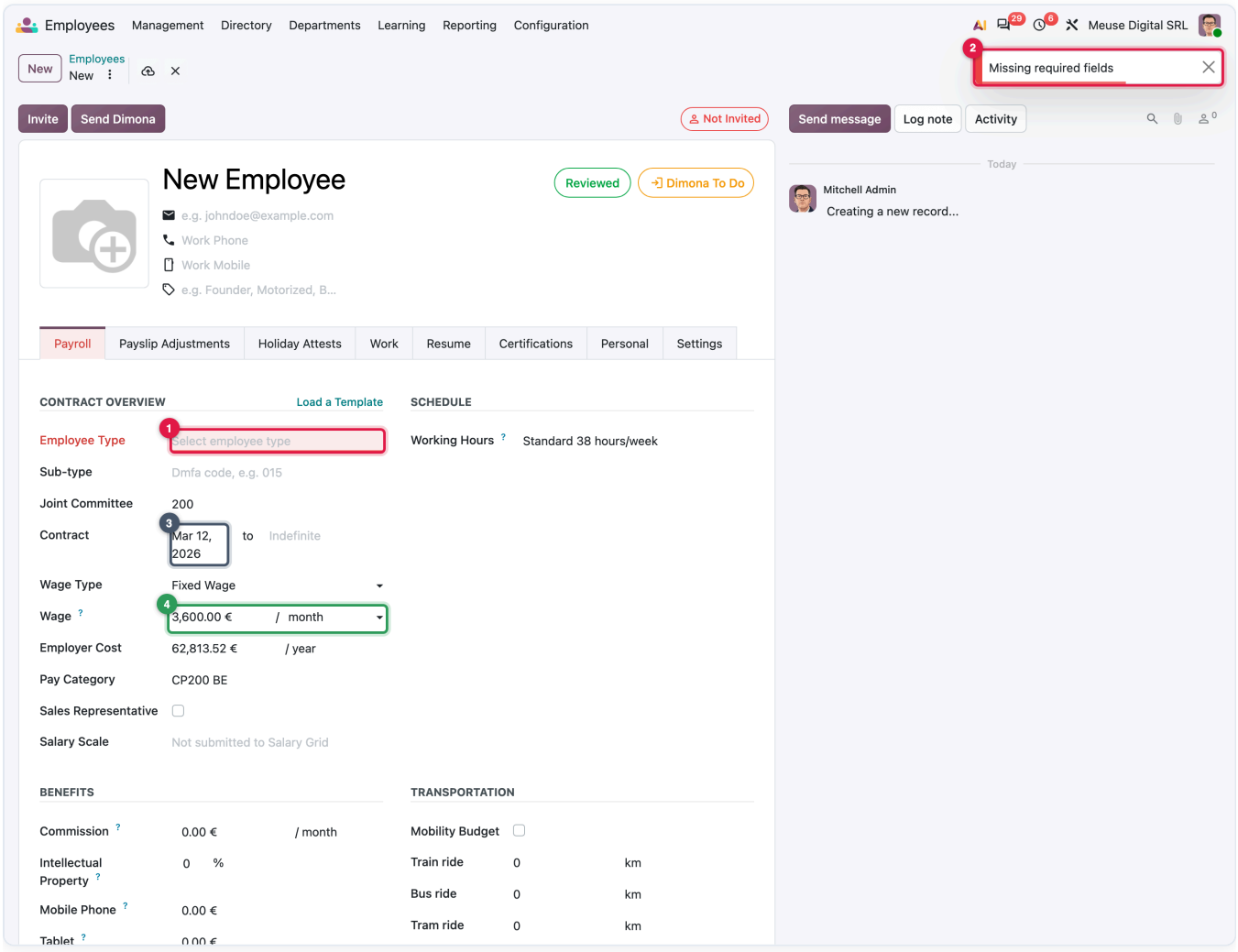
The employee form holds the person: identity, family, address, bank account. Tax and social security depend on several of these fields.


Employees
Management
Directory
Departments
Learning
Reporting
Configuration
Meuse Digital SRL
New
Employees
Marie Peeters
Payslips 11
Time Off 21/21 Days
Cars History 1
History 1
1 / 1
Invite
Send Dimona
Launch Plan
Not Invited
Jun 1, 2020
Send message
Log note
Activity
M
Marie Peeters
marie@meuse-digital.example
Work Phone
Work Mobile
e.g. Founder, Motorized, B...
Reviewed
Dimona To Do
Payroll
Payslip Adjustments
Holiday Attests
Work
Resume
Certifications
Personal
Settings
PRIVATE CONTACT
Email
marie@example.com
Phone
Bank Accounts
6
BE16 0635 5522 2274
Trusted
PERSONAL INFORMATION
First Name
Marie
Last Name
Peeters
Birthday
Feb 21, 1984
Show Birthday To Employees
Place of Birth
Namur
Country
Sex
Female
Disabled
Payslip Language
English (US)
CITIZENSHIP
Nationality (Country)
Belgium
Identification No
84022115820
NISS Number
5
84022115820
Passport No
VISA & WORK PERMIT
Visa No
Work Permit No
Document
Upload your file
EMERGENCY CONTACT
Contact Name
Phone
LOCATION
Private Address
Chaussée de Namur 7
Street 2...
City
State
5030
Belgium
Home-Work Distance
19
km
FAMILY
Marital Status
1
Married
Spouse Fiscal Status
2
Without Income
Disabled Spouse
Spouse Legal Name
Jan Peeters
Spouse Birthdate
Aug 2, 1983
Dependent People
3
2
including
4
2 children 0 with disability
for salary adjustments
0 other 0 with disability
0 senior with care needs
EDUCATION
Certificate Level
Field of Study
e.g. Engineering, Commerce, Business A...
DOCUMENTS
ID Card Copy
Upload your file
Driving License
Upload your file
SIM Card Copy
Upload your file
Internet Subscription Invoice
Upload your file
Planned Activities
Today: Dimona for Mitchell Admin
State the Dimona at https://www.socialsecurity.be/site_fr/employer/applics/dimona/index.htm to declare the arrival of Marie Peeters.
Done
Edit
Today
Mitchell Admin 3:47 PM
Congratulations! May I recommend you to setup an onboarding plan?
Mitchell Admin 3:47 PM
Employee created

Marie, Personal tab. ① marital status (Married). ② the spouse's tax status (Without Income). ③ dependent people (2) and ④ of which children (2). ⑤ the national identification number used by the social security office. ⑥ the bank account that receives the net pay, marked Trusted.

Field group	Why payroll needs it
Family situation ①–④	the withholding tax grants reductions for a dependent spouse and children (Chapter 4)
National number ⑤	identifies the worker in every social security and tax declaration (Chapter 17)
Bank account ⑥	where the net goes; the SEPA file reads it (Chapter 16)

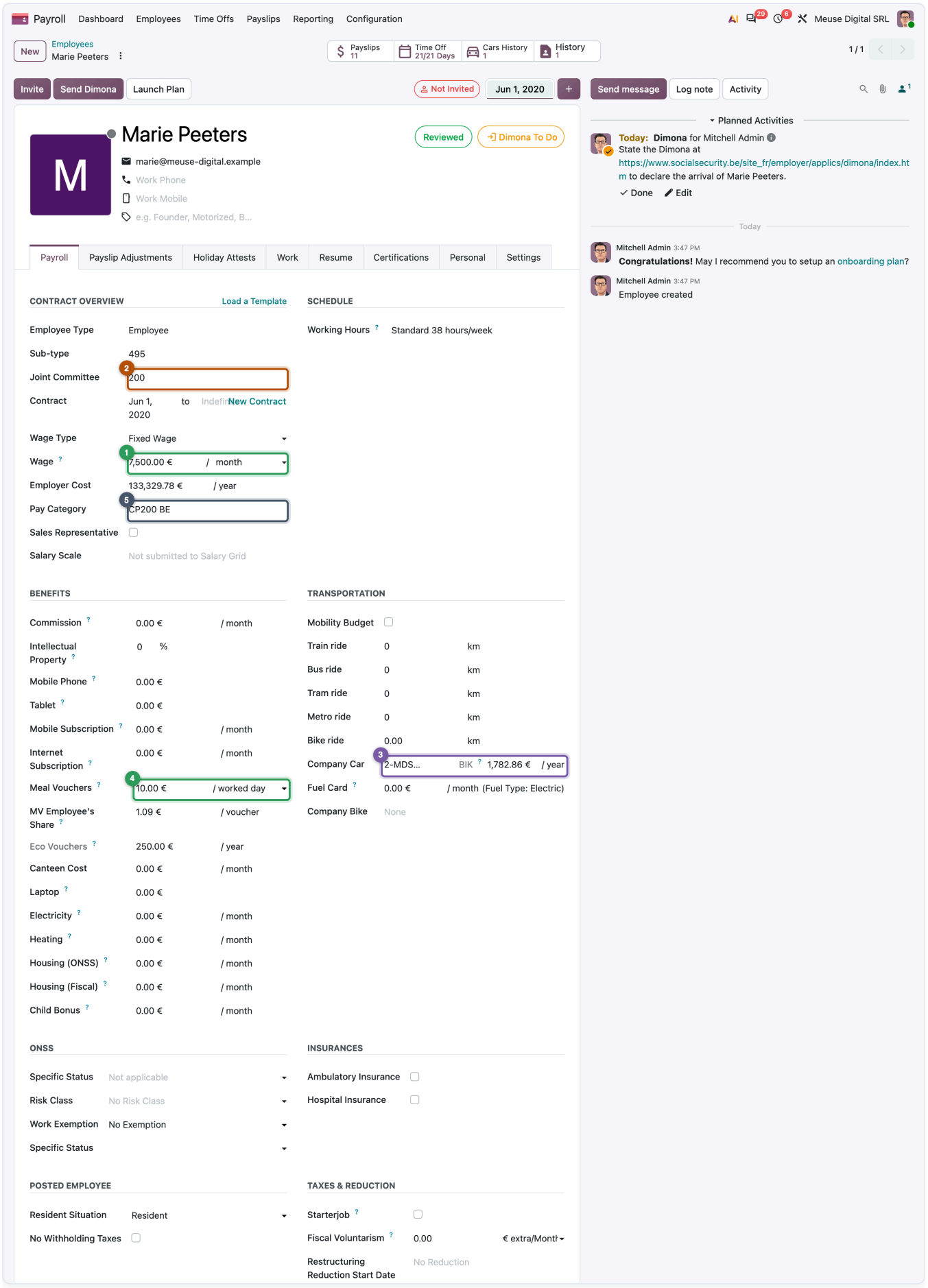
> ⚠ **Common pitfall: an employee is not complete when the form saves**
>
> Odoo saves an employee without a national number or a bank account. The dashboard then shows one card per gap, and declarations refuse the employee later. After every hire, look at the dashboard. (The reverse also holds: a few contract fields are required.)



A new employee with ③ a contract start of 12 March 2026 and ④ a wage of 3,600.00 € a month, saved with ① Employee Type left empty: Odoo refuses with ② "Missing required fields".

The version: what is agreed, from when

The *Payroll* tab belongs to a **version** (`hr.version`); the employee shows the one current today. A version starts on its *effective date* and stays valid until the next one starts.

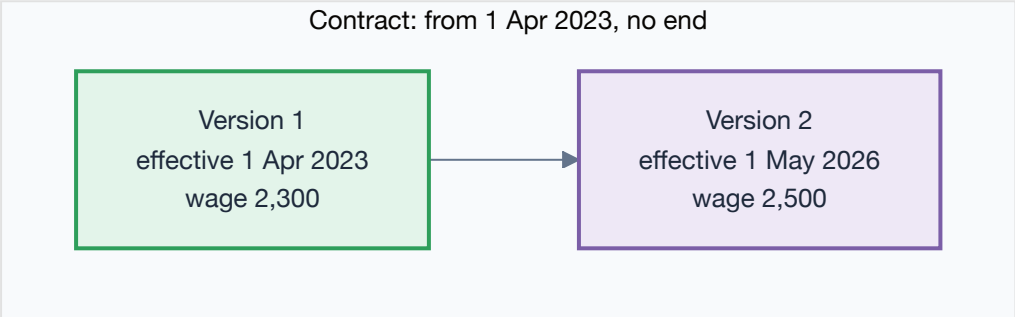


*Marie's Payroll tab, her current version (the date button **Jun 1, 2020** and the **+** next to it select and add versions). ① wage 7,500.00 € per month. ② joint committee 200, the Belgian sector field. ③ company car with its yearly benefit in kind, 1,782.86 €. ④ meal*

vouchers, 10.00 € per worked day, with an employee share of 1.09 €. ⑤ pay category CP200 BE, the structure type. The header also shows the employer cost, 133,329.78 € a year.

Field	Meaning
Effective date (`date_version`)	the day this version starts
Contract (`contract_date_start` , `contract_date_end`)	the employment contract's dates, shared by all its versions
Wage Type, Wage	a fixed amount per pay period, or an hourly rate
Working Hours	the schedule; the denominator of every proration (Chapter 14)
Pay Category (`structure_type_id`)	the structure type, so the rules that will run

How versions follow each other



This is Jurgen after Mission 13.4. Version 1 ends the day before version 2 starts; you never type an end date. Both belong to the same **contract** because they share the contract dates: a contract is not a separate record but the run of versions with the same start and end.

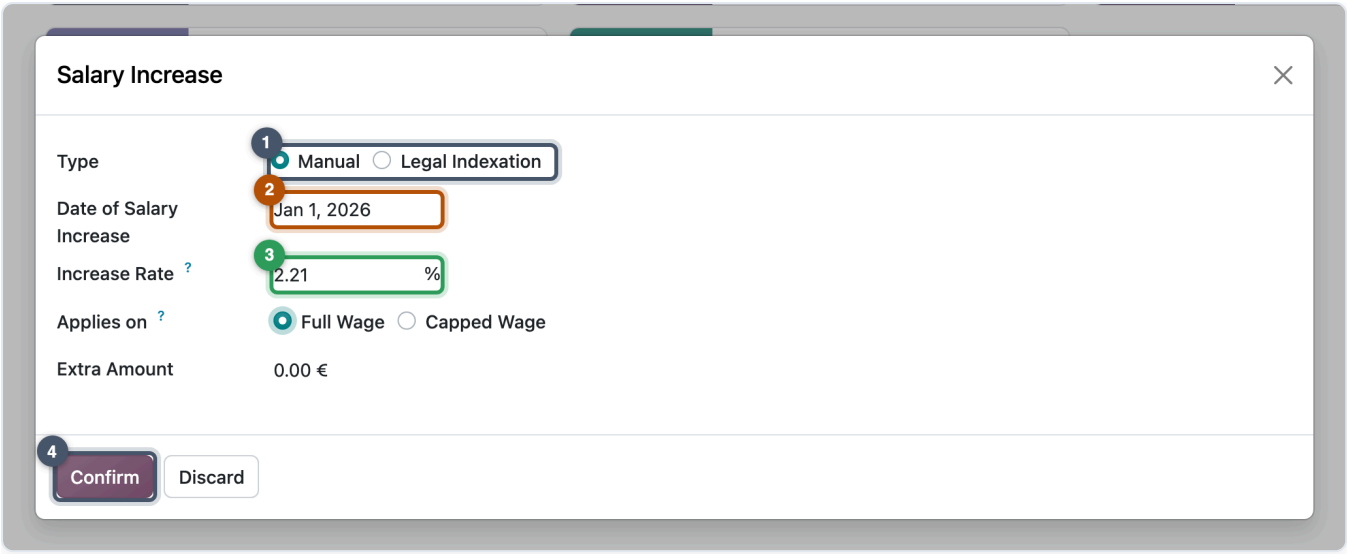
> 🔑 **Key idea: one payslip may read several versions**
>
> A payslip belongs to the version valid at its start, but its worked days are computed for **every version of the same contract** active in the period. A raise on 15 May gives one May payslip with two sets of worked-days lines, each valued at its own wage. A new *contract* gives a separate payslip.

> 🇧🇪 **In Belgium**
>
> Two versions share a payslip when they have the same Dimona category (the class of worker declared at hiring) and worker code, whatever their contract dates. The *Sub-type* field (495 for a white-collar employee) is that worker code.

> ⚠️ **Common pitfall: editing the wage instead of adding a version**
>
> Typing 2,500 into Jurgen's current version rewrites the past: January to April would look as if he always earned 2,500. Add a version dated 1 May with the **+** button instead.

Raising many employees at once

A sector-wide indexation is a new version per employee. The *Salary Increase* action (select employees, then *Actions*) creates them in one go (Chapter 11 explains indexation).



The Salary Increase wizard on MyVidangeCompany's five employees. ① type: Manual or Legal Indexation. ② the date of the increase, 1 January 2026. ③ the rate, 2.21 %, the sector indexation of January 2026. ④ Confirm, not clicked here: the cast's 2026 wages are already indexed.

Wage, wage type, pay frequency

The **wage type** is a fixed amount or an hourly rate. The **pay frequency** comes from the structure type. The **wage** is an amount *per pay period*: on a semi-monthly type it is a half-month amount. Always read the unit next to the number.

Benefits are fields, read by rules

Marie's car and meal vouchers are fields of her version, read by the rules every month:

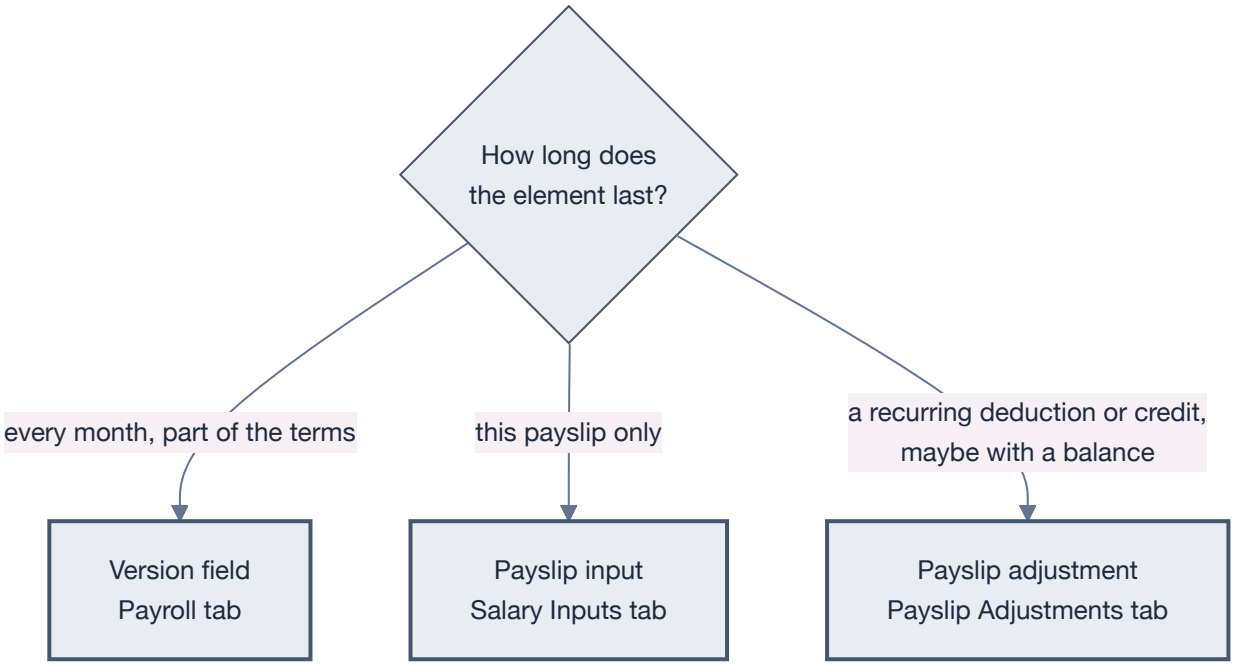
On the version	On Marie's February payslip	Line code
Company car, yearly benefit 1,782.86 €	benefit in kind added to the taxable base: 136.77 €, then deducted again from net	`ATN.CAR`, `ATN_DED`
Meal voucher 10.00 €, employee share 1.09 €	20 worked days × 1.09 € = –21.80 € on net; employer part 178.20 €	`MEAL_V_EMP`, `MEAL_VOUCHER_EMPLOYER`

A benefit in kind is taxed as if paid in cash, then removed before net pay (Chapter 2).

> 🌍 **Around the world**
>
> - **Belgium**: about 250 fields on the version (cars, vouchers, insurances, family, Dimona).
> - **Switzerland**: about 100.
> - **India**: about 50.
> - **United States**: about 30.
> - **Netherlands**: one. Same engine; only the legal data around the employee varies.

A **contract template** is a version without an employee; *Load a Template* copies its fields onto the current version.

Three places for a pay element



Level	Stored in	MyVidangeCompany example
Version field	a field on `hr.version`, read by the rules	Marie's car and meal vouchers
Payslip input	`hr.payslip.input`, read as `inputs['CODE']`	Emma's notice of 24 weeks (Chapter 17)
Payslip adjustment	`hr.salary.attachment`, an input on every payslip until the balance is reached	a wage garnishment, a loan repaid monthly

> 💡 **In practice: one change, one version**
>
> Date each version at the day the change takes effect, not the day you type it. If that day is in a validated month, add the version anyway: Odoo flags the payslips as outdated (Chapter 15 corrects them).

Missions

🎯 Mission 13.1 · Hire the first four ★★

Situation. On 1 January 2026 MyVidangeCompany employs four people on open-ended contracts. Marie is married to a spouse without income and has two dependent children; the others are single without children.

Employee	Job	Contract start	Wage (€/month)	Born	Sex	National number	IBAN
Mélanie	Developer	01/02/2021	4,000.00	14/05/1993	F	93051412288	BE14 0635 5500 0083
Jurgen	Support Agent	01/04/2023	2,300.00	03/09/1998	M	98090321174	BE15 0635 5511 1130

Employee	Job	Contract start	Wage (€/month)	Born	Sex	National number	IBAN
Marie	Engineering Manager	01/06/2020	7,500.00	21/02/1984	F	84022115820	BE16 0635 5522 2274
Emma	Ops Lead	01/10/2019	5,000.00	09/11/1981	F	81110920443	BE18 0635 5544 4465

Your goal. Create the four employees, each with a version at the right wage and start date, and a bank account Odoo may pay to.

Check. Each version is on **CP200 BE**, sub-type **495**, *Standard 38 hours/week*. Employer cost per year: Mélanie **69,792.80 €**, Jurgen **40,130.86 €**, Emma **87,241.00 €**, Marie **130,861.50 €**.

▼ **Hint**

The contract start is the "Not Employed" placeholder on the Payroll tab, and the form needs an *Employee Type*. A bank account must be *trusted* (Send Money) before a bank file can use it.

🎯 Mission 13.2 · Marie's car and meal vouchers ★★

Situation. Marie drives an electric Volkswagen ID.4, plate 2-MDS-726, catalogue value 52,000.00 € including VAT, CO_2 0 g/km, first registered on 5 January 2026. She gets meal vouchers of 10.00 € a worked day, of which she pays 1.09 €.

Your goal. Register the car in Fleet with Marie as driver, and put the car and the vouchers on her version.

Check. The car's benefit in kind is **1,782.86 € / year**; Marie's employer cost rises to **133,329.78 € / year**; meal vouchers **10.00 €** per worked day, employee share **1.09 €**.

▼ **Hint**

The model ID.4 does not exist yet: create it in Fleet's configuration. On the version, tick the company car before you choose it.

🎯 Mission 13.3 · Read the dashboard ★

Situation. Your manager asks: "What is Odoo complaining about, now that we have staff?"

Your goal. List the dashboard cards, open the records behind each blue or orange one, and fix the card about the work address.

Check. 6 cards. *Missing Salary Indexation Employees* lists **4 employees**; *Drivers Without Running Contract* points at the car **2-MDS-726**. Once the company has a social security establishment unit, *Missing ONSS identification code* is gone (a blue *Missing DMFA establishment unit language* card takes its place: leave it).

▼ **Hint**

The address card wants a work location of type *DMFA Unit* at MyVidangeCompany's address (*Configuration* > *Belgium* > *Work Locations*). For the car, read what the warning really checks (*Configuration* > *Dashboard* > *Warnings*): a lease, not an employment contract.

🎯 Mission 13.4 · Yannick joins, Jurgen gets a raise ★★

Situation. Yannick (born 30 July 1996, male, national number 96073031772, IBAN BE17 0635 5533 3321) starts as a designer at 3,600.00 € a month on Thursday 12 March 2026. Jurgen's wage goes from 2,300.00 € to 2,500.00 € from 1 May 2026; his earlier months must stay as they are.

Your goal. Hire Yannick from 12 March, and record Jurgen's raise so that January to April still use 2,300.00 €.

Check. Yannick's version is effective **12/03/2026**, **CP200 BE**, sub-type **495**, employer cost **62,813.52 € / year**. Jurgen has **2 versions**: the first ends on **30/04/2026** (40,130.86 € / year), the second starts on **01/05/2026** at 2,500.00 € (**43,620.50 € / year**); his contract start stays **01/04/2023**.

▼ **Hint**

Yannick: same form as Mission 13.1. Jurgen: do not type over the wage; use the **+** next to the version date in the header.

Summary

- The employee holds the person (identity, family, bank); the version holds the agreed terms (wage, schedule, pay category, benefits).
- A version starts on its effective date and ends the day before the next one; a contract is the run of versions sharing the same contract dates.
- A payslip reads every version of the same contract active in its period; a new contract means a separate payslip.
- The wage is an amount per pay period; the pay frequency comes from the structure type.
- Benefits are version fields read by rules; one-off amounts are payslip inputs; recurring deductions with a balance are payslip adjustments.
- Hire with the contract start date, raise with a new version, and read the dashboard after each.

Time: schedules, time off, worked days

How a calendar and a leave request become paid and unpaid days on a payslip

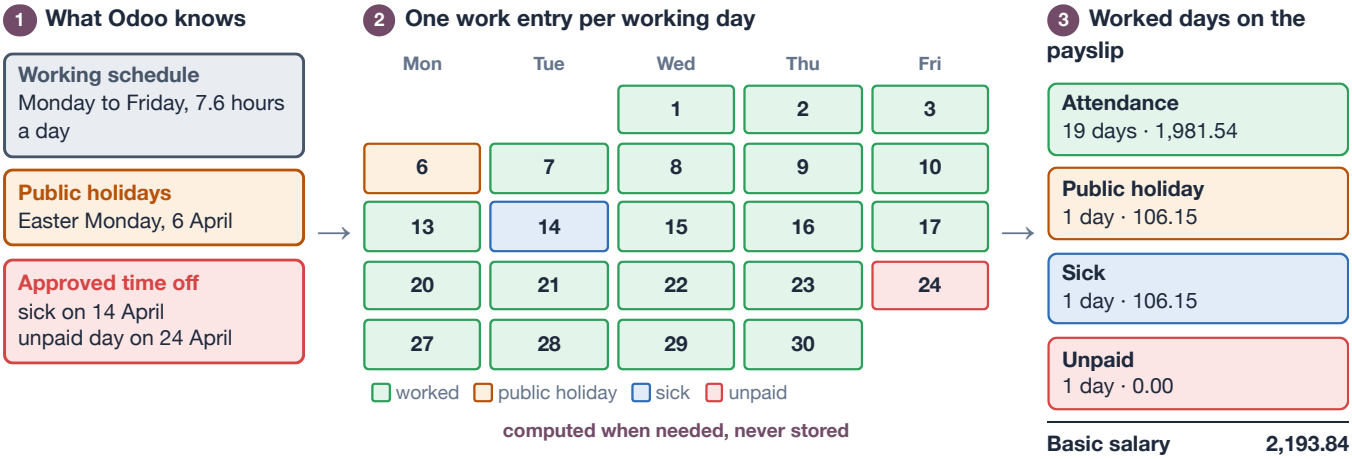
AFTER THIS CHAPTER YOU CAN

- Read a working schedule and compute the hours of a month from it
- Explain what a time type is and which of its fields change pay
- Follow time off and public holidays into generated work entries, then into worked-days lines
- Explain how a localization values an absence, with Jurgen's April as the example
- Record sick days, unpaid days, leave and overtime so that the payslip is right without typing a number

Jurgen was ill on 14 April and took an unpaid day on 24 April; April also has Easter Monday. His payslip must show 19 days of work, a paid sick day, an unpaid day and a paid holiday, and nobody types those numbers. They come from his **schedule** (what he was expected to work), his **time off** (what he did not) and the **time types** that say how each hour is paid.

How time becomes pay

A monthly salary is not paid "for the month": Odoo pays it **day by day**, according to what each day was. This chapter follows one month, Jurgen's April 2026, through the three steps below.



1. **What Odoo knows.** The working schedule says which days and hours were expected. Public holidays and approved time off say what happened instead.
2. **Work entries.** For each expected day, Odoo derives one *work entry* with a **time type**: worked, public holiday, sick, unpaid. Nobody types them, and they are not stored: Odoo computes them again whenever a payslip needs them, so a late change of time off is never forgotten.
3. **Worked days.** The payslip groups the entries by type into *worked-days* lines. Each type says whether it is paid, so the unpaid day is worth 0, and the total of the lines is the basic salary. The rules then compute everything else from it.

Working schedules

A **working schedule** (`resource.calendar`) lists the weekly attendance slots. It says which hours were expected, and is the denominator when a wage is cut into hours. MyVidangeCompany's *Standard 38 hours/week* runs Monday to Friday, 8:00–12:00 and 12:30–16:06: 7.6 hours a day, so 167.2 expected hours in April 2026 (22 weekdays, Easter Monday included).

Public holidays are *closure days*: leaves of the whole company, with the time type *Public Holiday*.

Time types

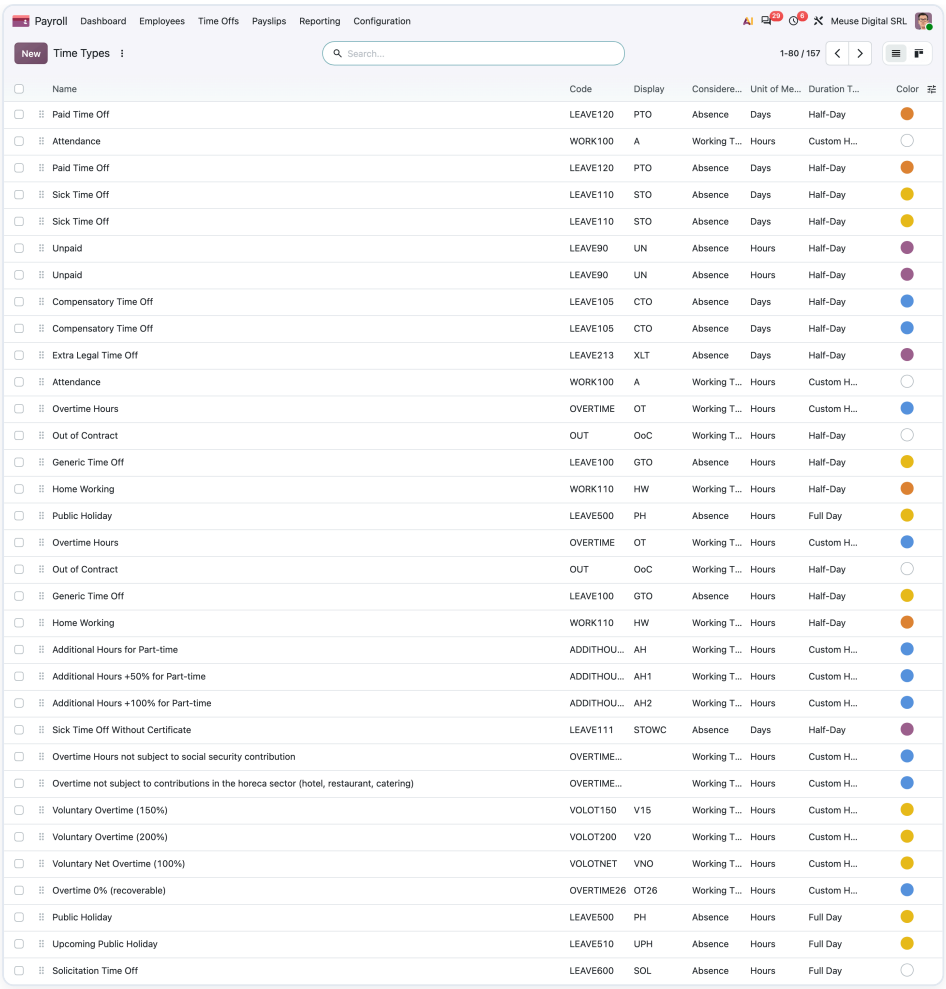
Every hour of the month has a **kind**: worked, sick, holiday, unpaid. A **time type** (`hr.work.entry.type`) names that kind and says how it is paid. Time off types and schedules point to it, and each worked-days line of the payslip is one time type.

Sick Time Off `LEAVE110`

Code	the key rules read it with: `worked_days['LEAVE110']`
Considered as	*absence*: its hours come from the schedule (7.6 h a day), not from a clock
Pay rate	100 %: the day is paid. Unpaid leave is 0 %, overtime can be 150 %
Salary categories	which totals of the payslip the line adds to

Jurgen's April uses four:

- **Attendance** `WORK100 · 100 %`
- **Public Holiday** `LEAVE500 · 100 %`
- **Sick Time Off** `LEAVE110 · 100 %`
- **Unpaid** `LEAVE90 · 0 %`

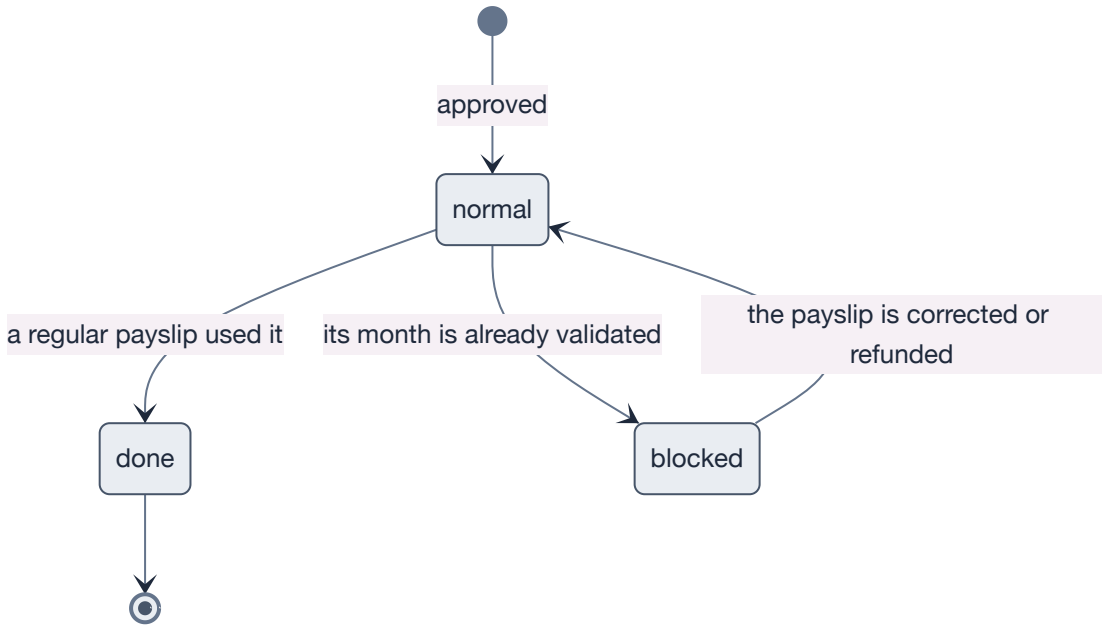


Configuration > Time Management > Time Types: name, code, considered as (absence or working time), unit.

> ⚠️ **Common pitfall: the same name twice**
>
> *Paid Time Off*, *Sick Time Off*, *Unpaid* and *Public Holiday* each appear twice: one generic row, one Belgian. Before you edit one, add the *Country* column to the list and pick your company's country.

Time off

Payroll only uses the result of the Time Off app: an approved request becomes a leave on the employee's calendar, with its time type. Paid leave consumes an *allocation* (20 days a year at MyVidangeCompany); sick and unpaid days do not.



The payroll status of a leave (`payslip_state`) protects validated months. A leave approved after its month was validated is **blocked**: a dashboard card says *Time Off missing from period's payslip*, the *Deferred Time Off* responsible (Chapter 12) is warned, and you correct that month's payslip (Mission 15.2).

Work entries are generated, not stored

A **work entry** is "this employee, this day, this time type, this many hours". Odoo does not store them: `hr.version` generates them whenever a payslip needs its worked days.

	Mon 13	Tue 14	Wed 15	Thu 16	Fri 17
1 **The schedule expects**	7.6 h	7.6 h	7.6 h	7.6 h	7.6 h
2 **Approved time off**		**sick, full day**			
3 **Work entries**	Attendance 7.6 h	**Sick 7.6 h**	Attendance 7.6 h	Attendance 7.6 h	Attendance 7.6 h

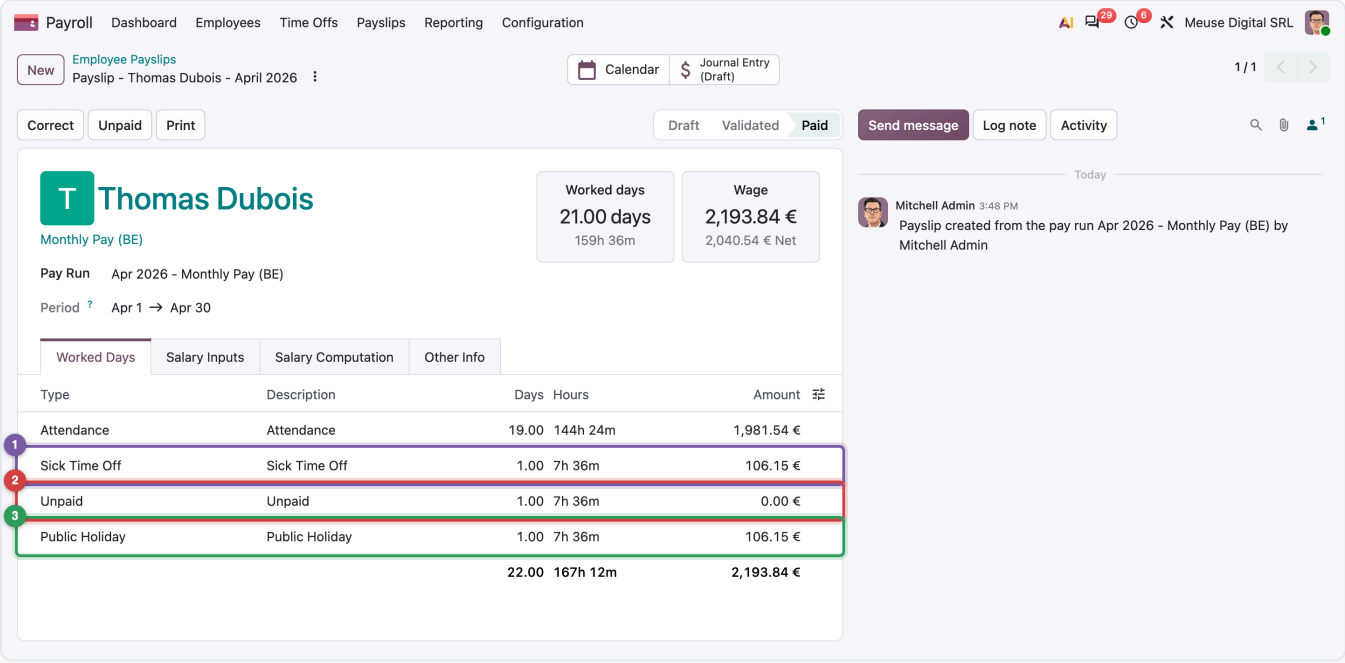
One week of Jurgen's April. Odoo starts from what the schedule expects (1), lays the approved time off and public holidays over it (2), and what remains is attendance (3). Each day gets one entry per kind of time, with its hours.

- **An absence counts the scheduled hours**, not the clock: a full sick day on a 38-hour week is 7.6 hours.
- **Overlaps have a winner.** A public holiday beats a personal leave on the same day, unless the localization lists the leave as taking precedence (in Belgium: long sickness, parental leave, unpaid…).

Because nothing is stored, a draft payslip always follows today's data. Only validated payslips are frozen.

From work entries to worked days

The payslip groups the entries by time type, converts hours to days with the schedule, and writes one **worked-days line** (`hr.payslip.worked_days`) per type, with an amount.



Jurgen, April 2026, Worked Days tab. ① the sick day: 1 day, 7 h 36 min, paid 106.15 €. ② the unpaid day: 1 day, 0.00 €. ③ Easter Monday, paid 106.15 €. Attendance shows 19 days and 1,981.54 €. The total is 22 days and 2,193.84 €, which becomes the basic salary.

Line	Days	Hours	Amount (€)
WORK100 Attendance	19	144.4	1,981.54
LEAVE110 Sick Time Off	1	7.6	106.15
LEAVE90 Unpaid	1	7.6	0.00
LEAVE500 Public Holiday	1	7.6	106.15
Basic salary	22	167.2	**2,193.84**

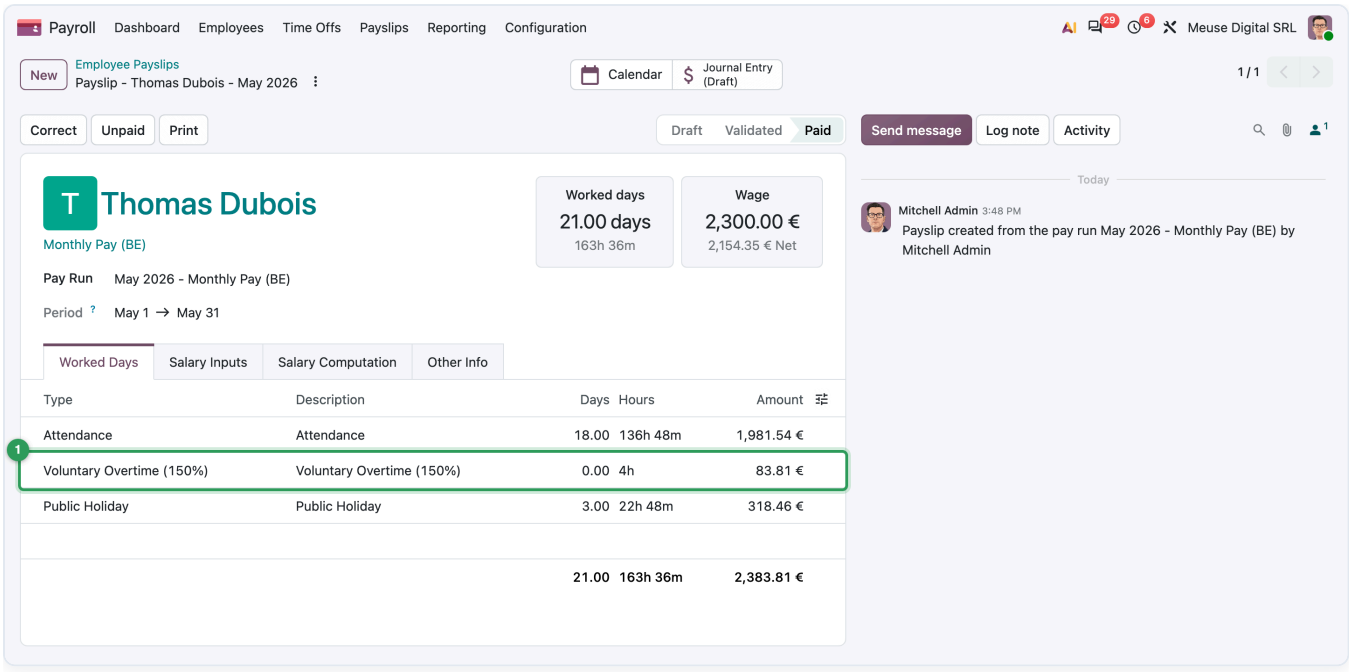
The generic engine values each line at *wage ÷ hours of the month × hours × pay rate*. A localization may replace that convention. The Belgian one values every non-attendance line at a fixed hourly rate and gives attendance the rest of the wage, so one missing day always costs the same.

> **In Belgium**
>
> The fixed hourly rate is wage × 3 ÷ (13 × weekly hours): 2,300 × 3 ÷ (13 × 38) = 13.97 € for Jurgen, so 7.6 hours are worth 106.15 € and his basic salary is 2,300.00 − 106.16 = 2,193.84 €. Chapter 5 compares the proration methods.

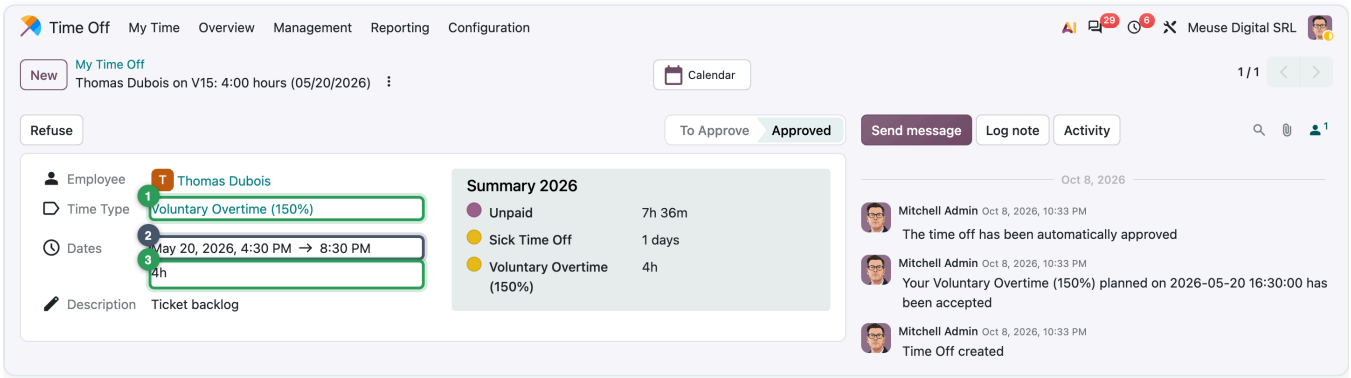
Yannick's March works the same way: the 8 days before his start become an unpaid `OUT` line (*Out of Contract*).

Overtime is a time type too

Overtime is a working-time type with a pay rate above 100 %, here *Voluntary Overtime (150%)* (`VOLOT150`), entered as a *time off* request of that type. It reads oddly, but it is the same mechanism.



Jurgen, May 2026 (demo database, before any raise). ① 4 hours of voluntary overtime at 150 %: 0 days, 4 h, 83.81 €. The line feeds the salary rule `VOLOT150`, which adds it to gross pay.



The request behind the overtime line, approved. ① type Voluntary Overtime (150%). ② 20 May 2026, 4:30 PM → 8:30 PM. ③ duration 4 h: the type is counted in hours, so the form asks for a start and an end time.

🌍 Around the world

- **Belgium**: 143 time types, because each absence has its own social security code.
- **Philippines**: 50 (night shifts, rest days, holiday premiums).
- **Australia, Switzerland**: 31 and 25.
- **United States**: 12.
- **Netherlands, India, France**: about 10 each.

💡 In practice: fix time at the source

If a worked-days line is wrong, fix the leave, the schedule or the version and refresh the draft. A hand-edited line stops Odoo from recomputing the worked days, so later changes will not reach it.

Missions

🎯 Mission 14.1 · Paid leave for 2026 ★

Situation. Mélanie, Jurgen, Marie and Emma each earned 20 days of paid leave for 2026. Yannick, hired in 2026, earned none.

Your goal. Give the four their 2026 paid leave, valid from 1 January to 31 December 2026.

Check. Four approved allocations of **20 days** of *Paid Time Off*; each employee's *Time Off* smart button shows **21/21 Days**.

▼ **Hint**

Leave rights are *allocations*, not requests. The 21st day is Odoo's compensation for 15 August 2026, a Saturday.

🎯 Mission 14.2 · Jurgen's sick day and unpaid day ★

Situation. Jurgen is sick on Tuesday 14 April 2026 (flu) and takes an unpaid day on Friday 24 April for personal reasons.

Your goal. Record both absences so that April's payslip reflects them.

Check. Both requests are **approved**, each **1 day (7.6 hours)**; Jurgen's counter still shows **21/21 Days**.

▼ **Hint**

Pick the time types whose codes are `LEAVE110` and `LEAVE90`.

🎯 Mission 14.3 · Preview April before the pay run ★★

Situation. Before April's pay run, your manager wants to see what Jurgen's absences do to his pay, and how Yannick's first month looks.

Your goal. Create draft payslips for Jurgen (April 2026) and Yannick (March 2026), read their worked days and net, then delete both drafts.

Check. Jurgen, April: attendance **19 days, 1,981.54 €**; sick **106.15 €**; unpaid **0.00 €**; public holiday **1 day**; net **2,040.54 €**. Yannick, March: attendance **14 days, 2,270.77 €**, out of contract **8 days**; net **1,818.11 €**.

▼ **Hint**

Payslips > *Payslips* > *New*: an employee and a period are enough; *Compute Sheet* gives the net.

🎯 Mission 14.4 · Emma's summer holiday ★

Situation. Emma books her summer holiday from Monday 3 to Friday 14 August 2026.

Your goal. Record the holiday against her 2026 legal leave.

Check. The request is approved for **10 days**; Emma's counter shows **11/21 Days**.

▼ **Hint**

Paid Time Off needs an allocation, so the request shows the balance while you type.

Summary

- The schedule says which hours were expected; MyVidangeCompany's has 7.6 hours a day, 167.2 hours in April 2026.
- A time type is the single record behind leave types, schedule slots and worked-days lines; its code, pay rate and categories decide the money.
- Approved time off and public holidays become leaves on the calendar; a leave that arrives after its month was validated is blocked until the payslip is corrected.
- Work entries are generated on demand and never stored; absences count scheduled hours; overlaps follow a precedence.
- Worked-days lines carry days, hours and amounts; a localization decides how absences are valued (Belgium: a fixed 3/13 hourly rate).
- Fix time at its source and refresh the draft; never hand-edit a worked-days amount.

Pay runs and payslips

Generate, compute, review, validate, and correct without breaking the audit trail

AFTER THIS CHAPTER YOU CAN

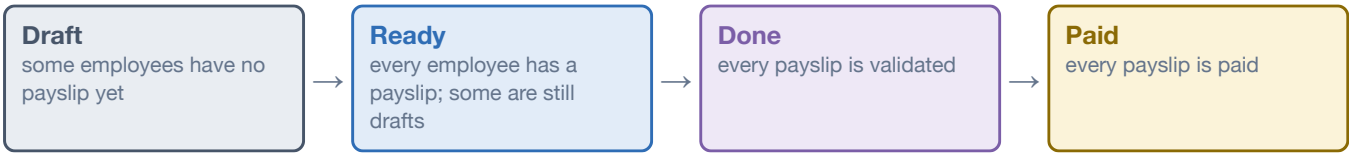
- Follow a pay run through its states and say what each button does
- Read a payslip tab by tab: worked days, inputs, computation, other info
- Use warnings to decide whether a run can be validated
- Pick the right tool for a mistake: recompute, cancel, correct, or an off-cycle payslip
- Run January to May 2026 for MyVidangeCompany, including a late correction and overtime

End of January 2026: four employees, no payslip yet, and the bank wants a file by the 28th. This chapter is the monthly routine: open a pay run, let Odoo build the payslips, review, validate, and correct a payslip after it was validated.

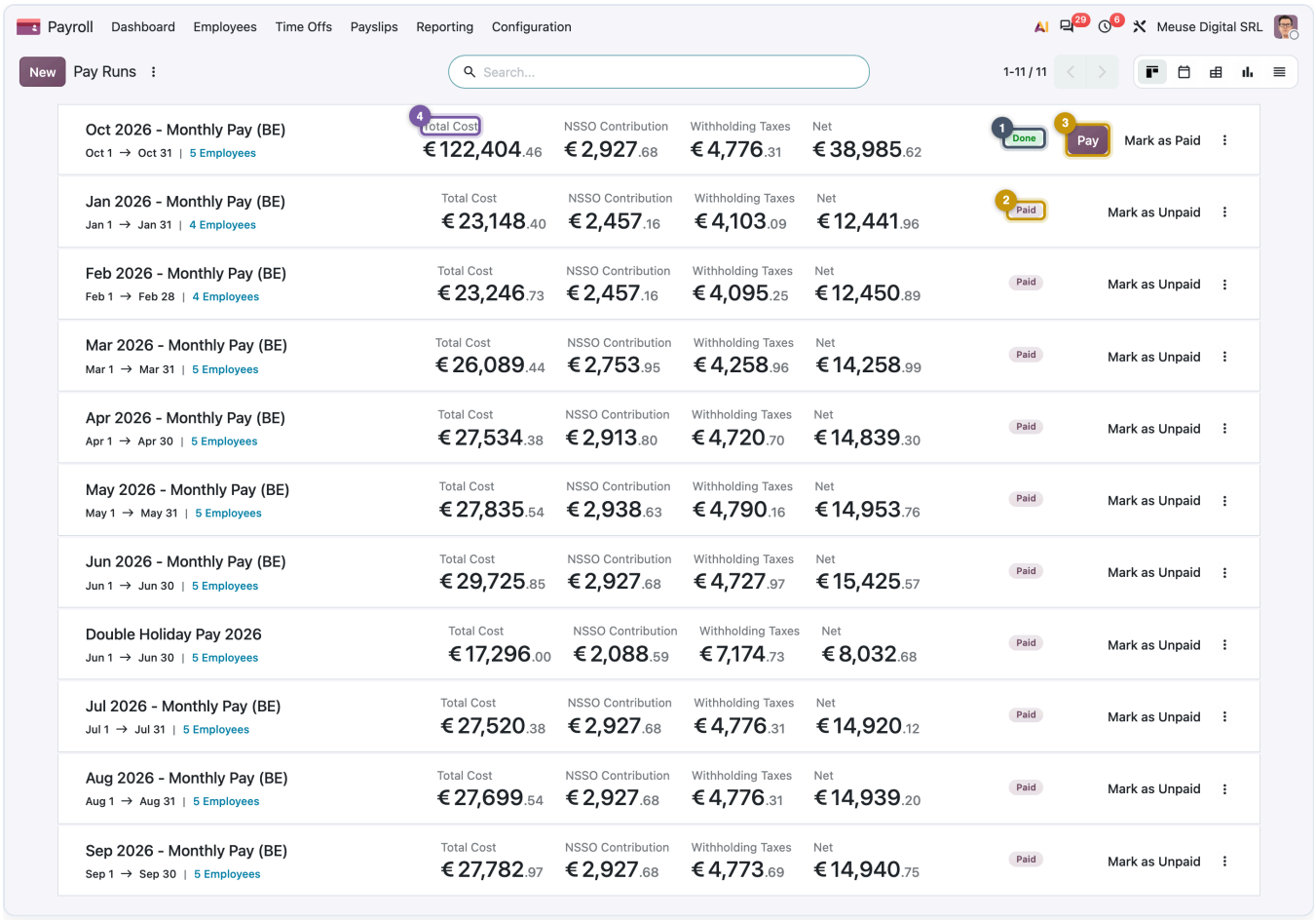
The pay run

A **pay run** (`hr.payslip.run`) is one batch of payslips: everyone paid on the same structure for the same period, for example *Monthly Pay, February 2026*. You work on the run instead of on fifty payslips one by one.

You never set a run's status yourself. It **follows its payslips**, and tells you at a glance where the month stands:

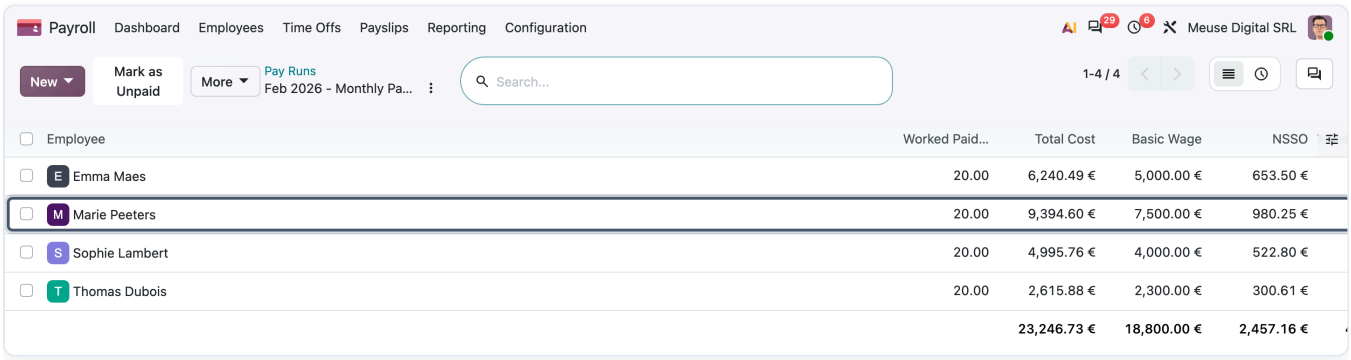


(If every payslip is cancelled, the run is *Cancelled*.)



*Payslips > Pay Runs on the full demo database. ① the October run is **Done**: validated, not paid yet. ② the January run is **Paid**. ③ **Pay** opens the payment wizard (Chapter 16). ④ **Total Cost**, the employer cost of the run.*

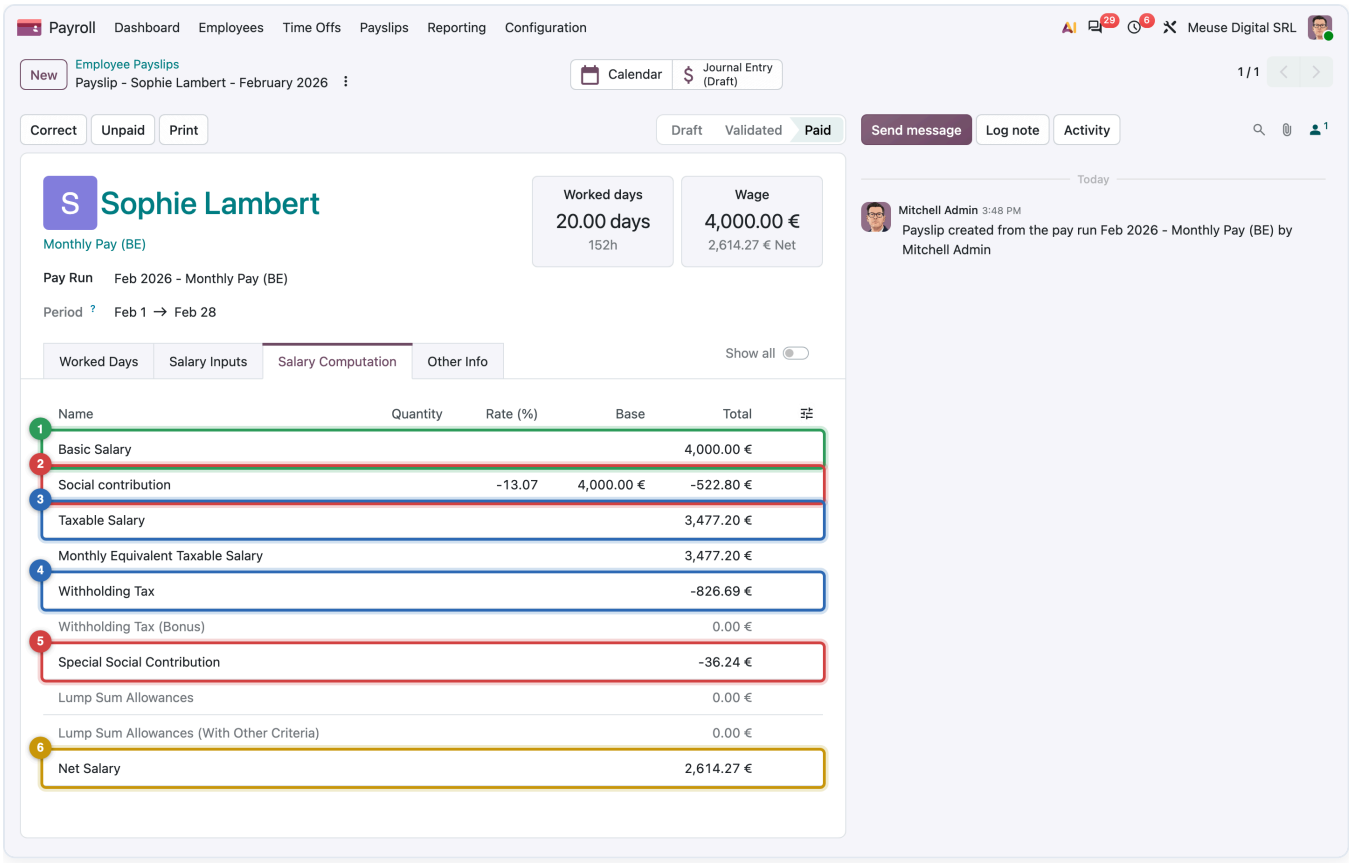
When you create a run, Odoo finds the **valid versions** (running in the period, on the structure's pay category, not already paid in another regular run) and creates one draft payslip per version, with its worked days (Chapter 14). The salary lines come with *Compute*.



The February 2026 run, one row per payslip. ① Marie's payslip. The footer sums the run: 23,246.73 € total cost, 18,800.00 € basic wage.

Reading a payslip

A payslip has four tabs, in the order of the calculation: **Worked Days** (Chapter 14), **Salary Inputs** (values typed for this payslip, or brought by adjustments), **Salary Computation** (the lines the rules computed, `hr.payslip.line`) and **Other Info** (accounting date, journal).



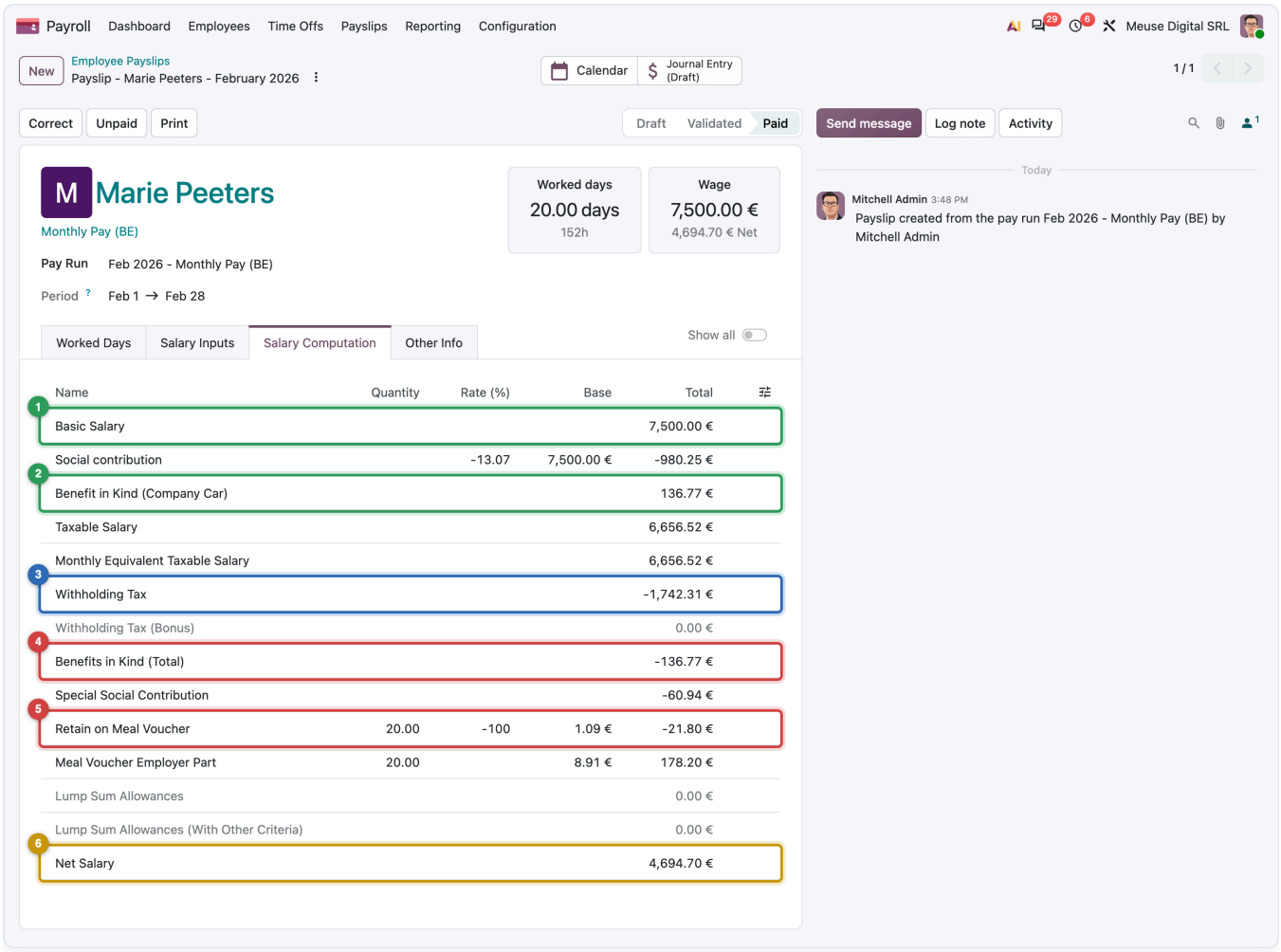
Mélanie, February 2026, Salary Computation. ① basic salary 4,000.00 €. ② employee social security, 13.07 % of 4,000.00. ③ taxable salary. ④ withholding tax. ⑤ special social security contribution. ⑥ net salary 2,614.27 €. Show all reveals the hidden helper and employer lines.

Worked example: Mélanie, February 2026

Line	Code	Amount (€)
Basic salary	`BASIC`	4,000.00
Employee social security 13.07 %	`ONSS`	−522.80
Taxable salary	`GROSS`	3,477.20
Withholding tax	`P.P`	−826.69
Special social security contribution	`M.ONSS`	−36.24
Net salary	`NET`	**2,614.27**
Employer social security (hidden line)	`ONSSEMPLOYER`	1,032.00

Hidden lines (*Show all*) are intermediate steps and employer lines, which only accounting reads. Chapter 2 explains the bases, Chapter 19 the rules.

Marie's payslip adds what her version holds: the car and the meal vouchers.



Marie, February 2026. ① basic salary 7,500.00 €. ② the car's benefit in kind, 136.77 €, added to the taxable base. ③ withholding tax −1,742.31 €, lowered by her family situation. ④ the same benefit removed again before net. ⑤ her meal-voucher share, 20 × 1.09 € = −21.80 €. ⑥ net 4,694.70 €.

Compute deletes the draft lines and runs the structure's rules in sequence (Chapter 19). Drafts also refresh by themselves when a wage changes or a leave is approved.

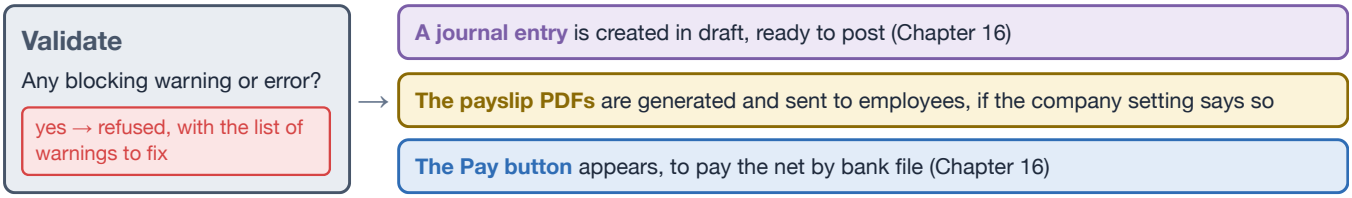
Warnings before you validate

Each payslip shows its problems as a banner and a badge: **red** (blocked) or **orange** (read it first). Validation also refuses if a dashboard warning marked *block payslips* concerns these payslips.

> ⚠️ **Common pitfall: a banner that is not an error**
>
> Emma's termination payslip shows "Payslip period does not match payrun": it covers 31 October only. That is expected for exit payslips (Chapter 17). Read each warning; do not try to make every banner disappear.

Validation, and what it starts

Validate is the point of no return of a payroll month. Before it, payslips are drafts that change with the data. After it, they are the official record of what each employee was paid: the amounts are frozen, and three things start.



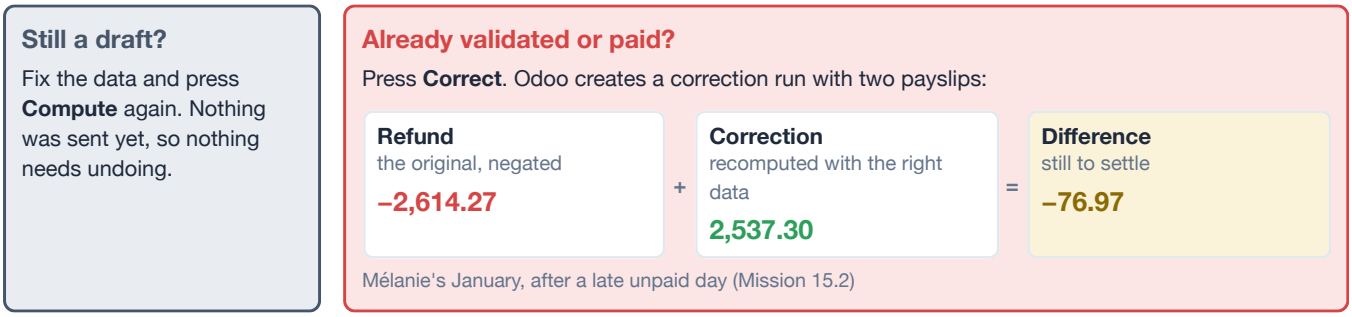
A validated payslip is never recomputed. If its data changes later, Odoo flags it as *outdated* and offers a correction.

Corrections

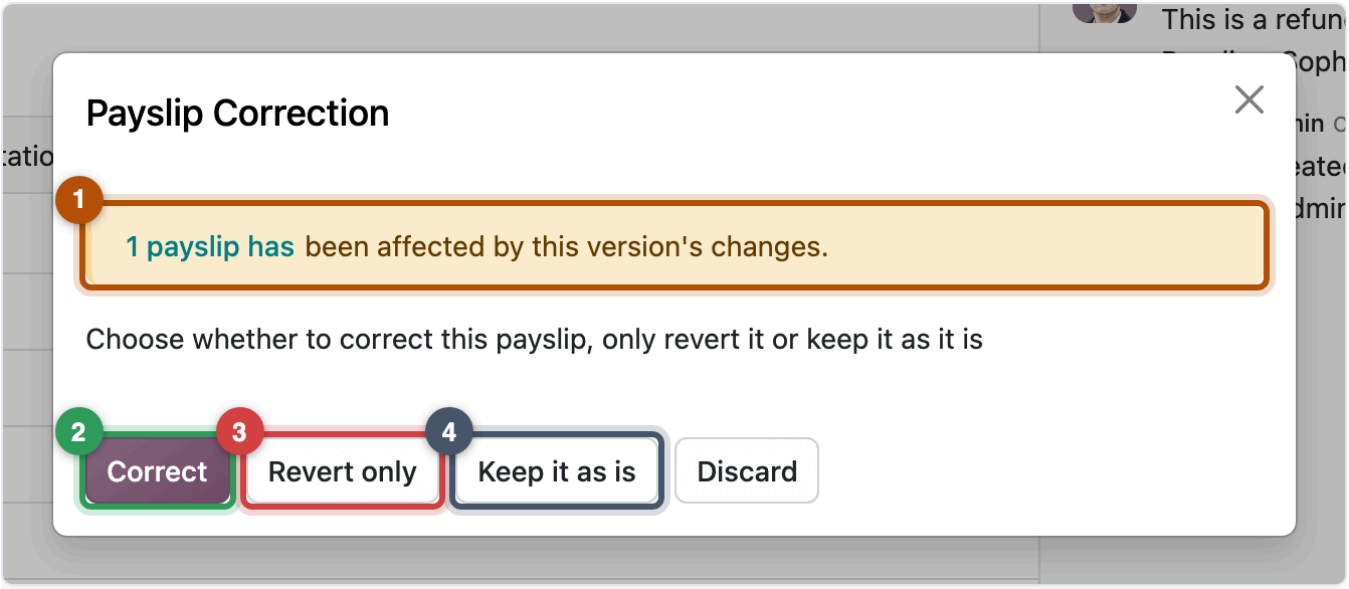
A payslip needs a correction when what it says is no longer true. That happens more often than mistakes in the rules:

- **information came late**: a sick day or an unpaid day approved after the month was validated;
- **a change was backdated**: a raise agreed in March, effective from 1 January;
- **the data was wrong**: a wrong wage, a forgotten input, the wrong pay category;
- **the law changed retroactively**: a new rate published in July, valid from January (Chapter 11).

A validated payslip has already been sent to the employee, booked in the accounts and counted in the declarations, so payroll never edits it. It cancels it with a negative copy and issues the right one; only the difference between the two is still to pay or to recover.



Three other tools exist for other situations: **delete** a draft payslip that should not exist; **cancel** a validated one that should never have existed (payroll managers only; its entry is reversed); and use an **off-cycle payslip** on another structure for a payment outside the regular month, such as a 13th month, holiday pay, an advance or a leaver's exit pay.



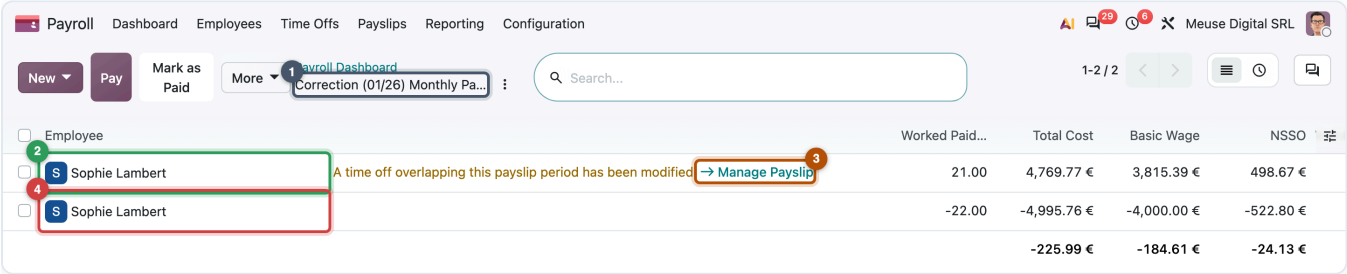
*The Payslip Correction wizard, opened from the Manage Payslip link of Mélanie's January payslip. ① the payslips affected by the change. ② **Correct**: a refund of the original and a recomputed payslip. ③ **Revert only**: the refund alone. ④ **Keep it as is**: Odoo stops flagging this payslip as outdated.*

Worked example: Mélanie's late unpaid day (Mission 15.2)

Mélanie's unpaid day of 23 January is approved after January was validated. Her payslip shows an issue with a *Manage Payslip* link, and *Correct* creates a run *Correction (01/26) Monthly Pay (BE)*:

Payslip	Net (€)
Refund: Mélanie, January (the original, negated)	–2,614.27
Correction: Mélanie, January (with the unpaid day)	2,537.30
Difference	**–76.97**

The unpaid day removes one day at the fixed hourly rate (Chapter 14): basic 3,815.39 €.



The run that Correct created. ① its name, Correction (01/26) Monthly Pay (BE). ② the correction payslip: 21 paid days, basic 3,815.39 €. ③ it carries the issue until it is validated. ④ the refund: –22 days, –4,000.00 € basic. The footer is the net effect: –184.61 € of basic wage.

Common pitfall: validating a Belgian correction run

The Belgian localization marks a payslip with a negative net as *Paid* at once, so *Validate* on the correction run fails with "You can't create a journal entry for a paid payslip". Validate the correction payslip itself.

🌍 **Around the world**

- **Belgium**: ten structures (monthly pay, double holiday pay, 13th month, termination fees, exit holiday pay…).
- **Luxembourg**: a gratification and a 13th month.
- **Mexico**: a Christmas bonus.
- **Saudi Arabia**: a salary advance and loan structure.
- **Most others**: one regular structure.

💡 **In practice: three checks before Validate**

1. **Headcount**: one payslip per person you expect (Yannick from March, Emma's exit payslips in October).
2. **Delta**: compare net and total cost with last month; explain every difference by an event.
3. **Issues**: open every orange or red badge and read it.

Missions

🎯 Mission 15.1 · Run January ★

Situation. End of January 2026: MyVidangeCompany's first pay run in Odoo.

Your goal. Create, compute and validate the January pay run on *Monthly Pay (BE)*.

Check. 4 payslips, net **12,441.96 €**, total cost **23,082.89 €**, state **Done**.

▼ **Hint**

Odoo proposes the current month: set 1–31 January 2026.

🎯 Mission 15.2 · A late unpaid day ★★★

Situation. On 5 February, HR remembers that Mélanie took Friday 23 January off, unpaid, for a family matter. January is already validated.

Your goal. Record the leave and correct Mélanie's January pay, keeping the original payslip.

Check. A run **Correction (01/26) Monthly Pay (BE)** holds a refund of **–2,614.27 €** and a correction of **2,537.30 €** net, both validated; the January run stays **Done** at 12,441.96 €.

▼ **Hint**

After approval, Mélanie's January payslip offers a way to the correction wizard. Read the pitfall above before you validate.

🎯 Mission 15.3 · February to April, with Yannick ★★

Situation. Three ordinary months, except that Yannick joins on 12 March and Jurgen is absent twice in April (Mission 14.2).

Your goal. Run, compute and validate February, March and April.

Check. February net **12,450.89 €** (4 payslips); March **14,258.99 €** (5 payslips, Yannick **1,818.11 €**); April **14,839.30 €** (Jurgen **2,040.54 €**).

▼ **Hint**

If Yannick is missing from March, check his contract start (Mission 13.4), not the pay run.

🎯 Mission 15.4 · Overtime in May ★★

Situation. On Wednesday 20 May Jurgen stays from 16:30 to 20:30 to clear a ticket backlog: 4 hours of voluntary overtime at 150 %, in his first month at 2,500.00 € (Mission 13.4).

Your goal. Record the overtime and run May.

Check. Jurgen's May payslip: basic **2,500.00 €**, `VOLOT150` **4 hours, 91.09 €**, net **2,196.55 €**; the May run's net is **14,995.96 €**.

▼ **Hint**

Overtime is a request of type *Voluntary Overtime (150%)* with a start and end time (Chapter 14). Why is it more than the 83.81 € of Chapter 14's screenshot?

Summary

- A pay run is a period, a structure and a list of versions; its state follows its payslips (Draft, Ready, Done, Paid).
- Generating creates one draft payslip per valid version, with worked days; Compute adds the lines; Validate locks them and creates the journal entry.
- Issues and blocking warnings decide whether a payslip can be validated; banners are information, not always errors.
- A validated payslip is never edited: *Correct* creates a refund and a recomputed payslip in a correction run.
- Holiday pay, bonuses, advances and exit pay are off-cycle payslips on their own structures.

Accounting and payment

From payslip lines to one balanced entry, then to a bank file

AFTER THIS CHAPTER YOU CAN

- Explain how validated payslips become a journal entry, and which settings shape it
- Read a real payroll entry line by line and check that it balances
- Use the Adjustment Entry line as a configuration smoke test
- Pay a pay run with a SEPA file, and know what Paid does and does not mean in accounting

Chapter 8 drew a payroll month as T-accounts on paper. Here Odoo does it: the structure and its rules hold the configuration, *Validate* triggers the entry, and a bank file pays the net.

From payslip to bank

Validating the payslips is not the end of the month. Two things still have to happen in the real world, and Odoo handles both:

- **Book it.** The company records in its accounts what the month cost and what it now owes, and to whom: the employees, the social security office, the tax office. That record is a **journal entry**.
- **Pay it.** The company sends the money. For the employees' net pay, Odoo produces a bank file the bank executes in one go.

Book it	Payslips validated	→	Journal entry created, in draft	→	The accountant checks and posts it
Pay it	Pay button on the run	→	Bank file (SEPA)	→	The bank pays each employee

Both meet on the **bank statement**: the payment is matched with the net pay owed in the entry, and the debt is closed.

What you configure

Four settings decide what the journal entry looks like. They are easiest to understand on the result, a simplified version of MyVidangeCompany's February entry:

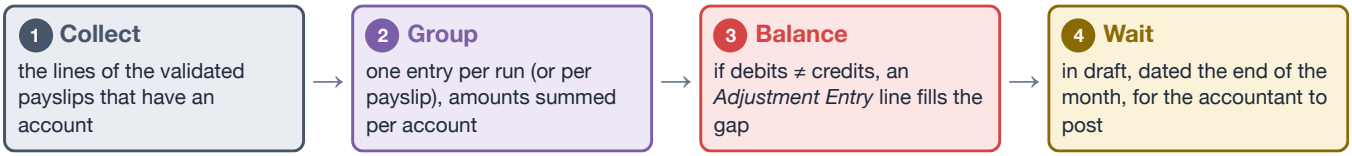
1 Journal: **Salaries (SLR)** · entry SLR/2026/02/0001 · 4 one entry for the whole run			
2 Account	**3 Partner**	**Debit**	**Credit**
620200 Remuneration		18,800.00	
621000 Employer contributions		4,626.76	
454000 Social security office	the social security office		6,975.21
453000 Taxes withheld	the tax office		3,978.86
455000 Remuneration owed			12,450.89
…			

1. **The journal** is the book the entry is written in; payroll has its own, *Salaries*. You set it on the **structure**. A structure without a journal produces no entry at all, and no error.
2. **The accounts** are the lines of the company's chart of accounts: an expense (*Remuneration*) or a debt (*Social security office*). Each **salary rule** says which account its amount goes to: a **debit** account for what the company spends, a **credit** account for what it owes.
3. **The partner** says to whom a debt is owed. It is also set on the rule.
4. **One entry per run, or one per payslip.** The company setting *Anonymize Journal Entries* (Chapter 12) groups the whole run into one entry, so individual salaries cannot be read in the books.

With the Belgian chart of accounts, the localization fills all four for you: the *Salaries* journal, the rule accounts and the anonymized entries.

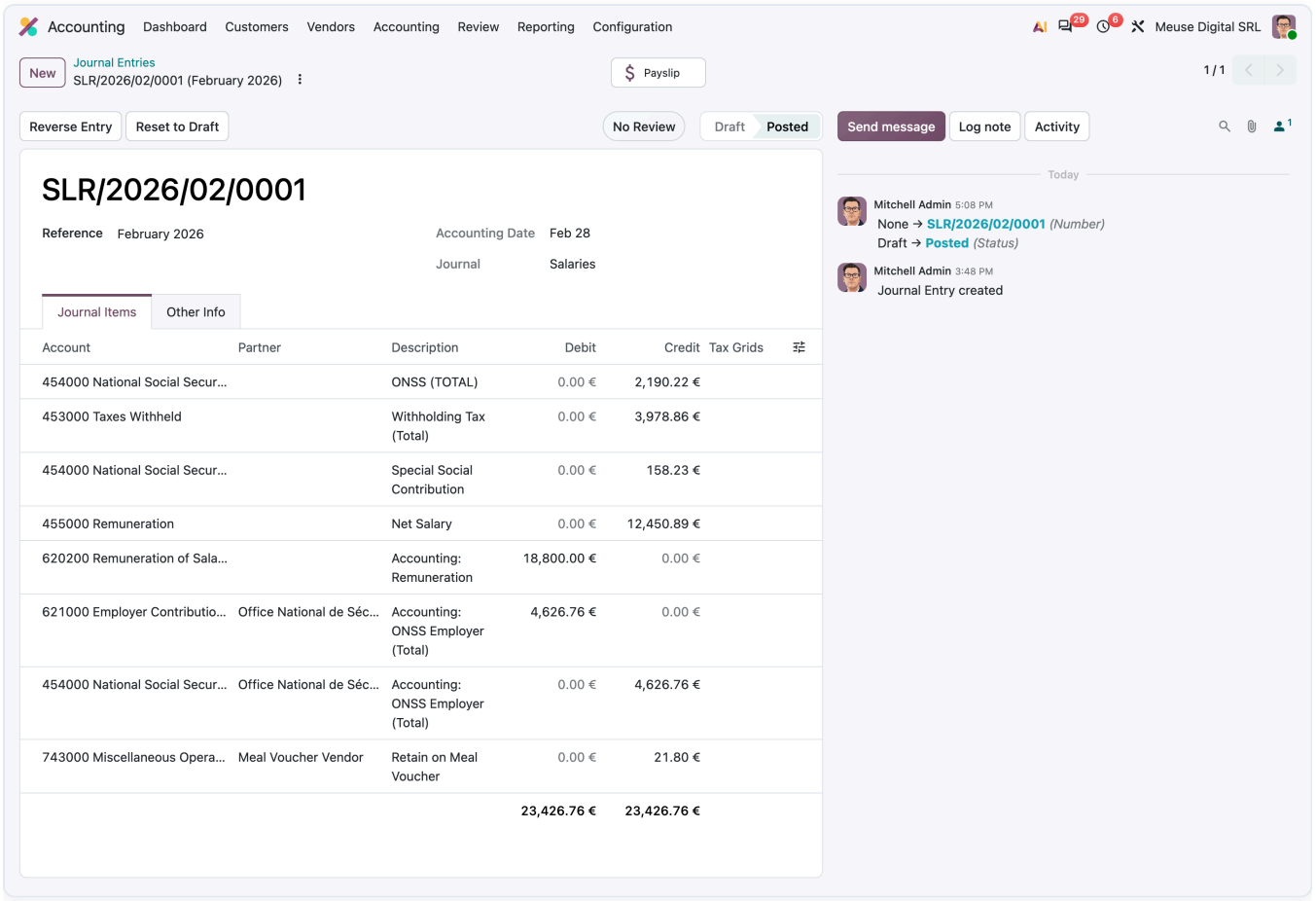
When the entry is created

The entry is created **the moment the run is validated**, in four steps:



Step 3 matters most: Odoo never refuses an unbalanced entry, it quietly balances it. The next section shows why that line is a warning sign.

Reading February's entry

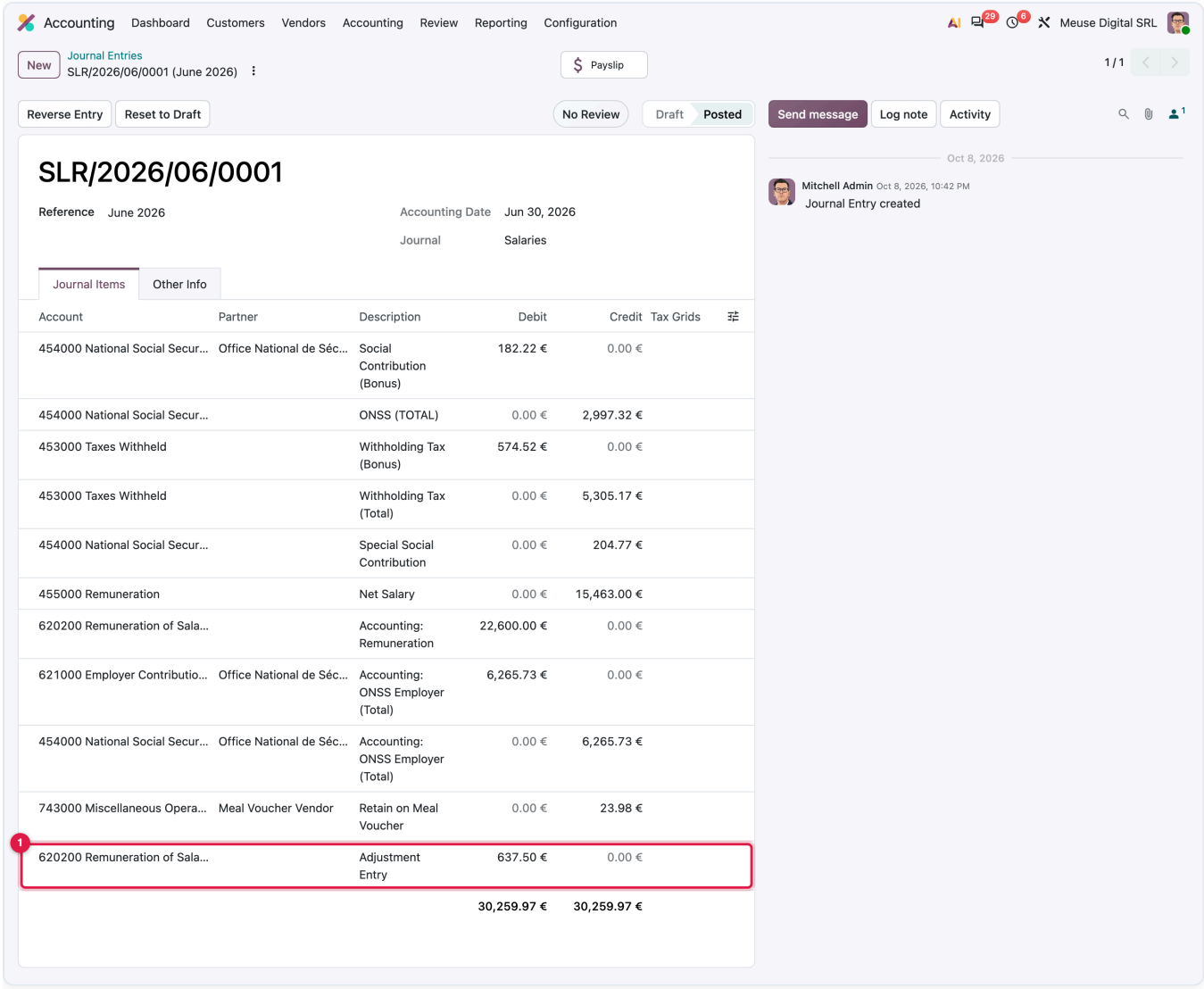


Accounting > Journal Entries > SLR/2026/02/0001, the entry of the whole February run (posted, dated 28 February). Debits and credits both total 23,426.76 €.

Account	Label	Debit (€)	Credit (€)
620200 Remuneration	Remuneration	18,800.00	
621000 Employer contributions	Employer social security	4,626.76	
454000 Social security office	Employee social security		2,190.22
454000 Social security office	Special social security contribution		158.23
454000 Social security office	Employer social security		4,626.76
453000 Taxes withheld	Withholding tax		3,978.86
455000 Remuneration	**Net salary**		**12,450.89**
743000 Miscellaneous income	Retain on meal voucher		21.80
Total		**23,426.76**	**23,426.76**

The company spends 18,800.00 € of salaries and 4,626.76 € of employer contributions, and owes it to the social security office (6,975.21 €), the tax office (3,978.86 €) and the employees (12,450.89 €). Employer contributions appear on both sides because their rule carries **two** accounts, an expense and a liability: that is how Odoo books costs that never touch net pay.

> **🔑 Key idea: the Adjustment Entry line is a smoke test**
>
> If a rule has no account, or one side where two are needed, debits and credits differ. Odoo does not refuse the entry: it adds an *Adjustment Entry* line on the journal's default account, and the books look right while being wrong. After the first month, open the entry: an *Adjustment Entry* means a missing or doubled account.



Entry SLR/2026/06/0001 of the June run, balanced at 30,259.97 € on each side. ① the Adjustment Entry line: 637.50 € debited on 620200, the amount the June payslip lines failed to book. Mission 16.4 asks you to explain it.

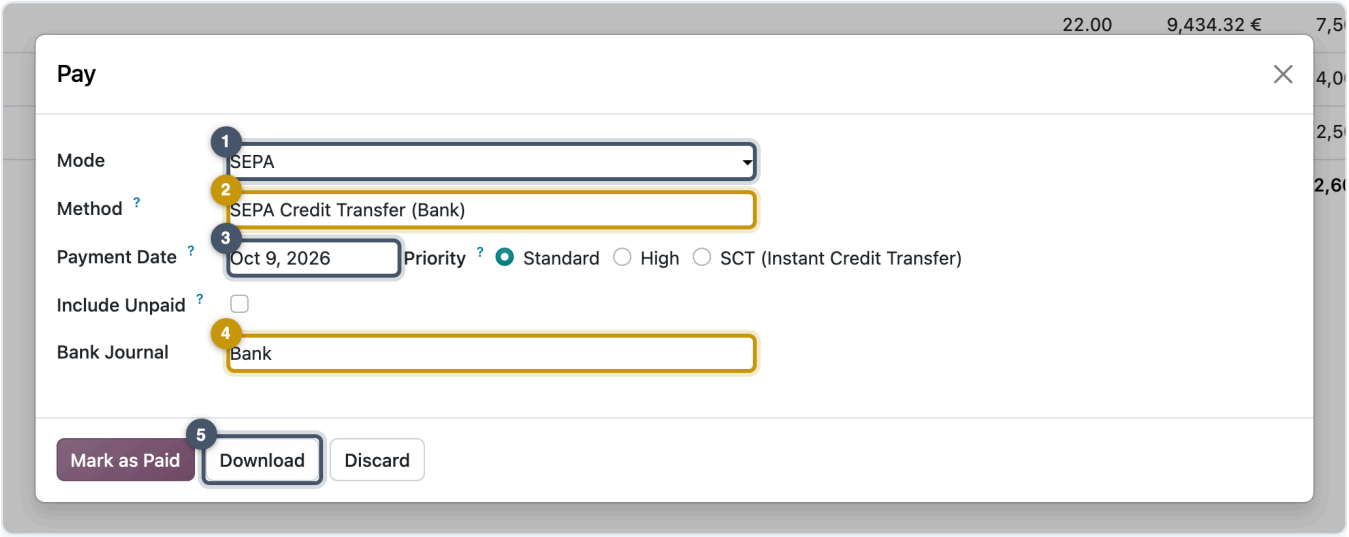
> **⚠️ Common pitfall: rules activated after the chart of accounts**
>
> Rules installed archived (Belgium, until the joint committee is chosen) get no accounts if the chart was loaded first, and the first entry shows a large *Adjustment Entry*. Reload the chart's payroll accounts (Chapter 22); do not type them rule by rule.

Corrections book too: the refund of Mission 15.2 posts the reverse of the original, the correction posts the corrected amounts. Cancelling a payslip deletes or reverses its entry.

Paying

Validated run net owed on 455000 → Payment wizard: CSV or SEPA file → Mark as Paid: payslips Paid, paid date set → Register a payment on the 455000 lines → reconciled: payslips Paid when nothing is left

The run's *Pay* button opens the payment wizard. With the SEPA module (Mission 12.3) it builds an ISO 20022 credit-transfer file from each payslip's net and the employee's trusted bank account, with the category purpose `SALA` (salary payment). An employee with several payslips in a run gets one summed transfer.



*The Pay wizard of a run, in SEPA mode. ① mode SEPA. ② payment method SEPA Credit Transfer (Bank). ③ the payment date, which becomes the requested execution date. ④ the bank journal that pays, with the company's IBAN. ⑤ **Download** builds the XML file; **Mark as Paid** sets the payslips to Paid.*

> ⚠️ **Common pitfall: Paid is not booked**
>
> *Mark as Paid* changes the payslips' state; it creates no payment and posts nothing. After paying February, account 455000 still owes 12,450.89 € until the bank statement is reconciled. The payslip state is a payroll status; 455000 is the accounting truth.

> 🌍 **Around the world**
>
> - **Euro countries**: SEPA credit transfer files.
> - **Switzerland**: the Swiss ISO 20022 variant.
> - **UAE, Saudi Arabia, Oman**: WPS files (the Wage Protection System that proves salaries were paid).
> - **United States**: NACHA files through batch payments.
> - **Everywhere**: the generic CSV export.

> 💡 **In practice: close the month in this order**
>
> 1. Validate the run. 2. Check the entry (no *Adjustment Entry*). 3. Post it. 4. Generate and send the bank file. 5. Mark as paid. 6. When the bank statement arrives, reconcile the salary transfers with 455000, and the social security and tax payments with 454000 and 453000.

Missions

🎯 Mission 16.1 · Read and post January ★

Situation. The accountant asks for January's payroll entries, including the correction of Mission 15.2.

Your goal. Find every January payroll entry, check that none has an adjustment line, and post them.

Check. Three posted entries in *Salaries* dated 31/01/2026: the run (**23,264.01 €**), the refund (**5,032.00 €**) and the correction (**4,799.77 €**). No *Adjustment Entry* line.

▼ **Hint**

A payslip's *Journal Entry* smart button opens its entry; the journal's list can be filtered by date.

🎯 Mission 16.2 · Pay February by SEPA ★★

Situation. MyVidangeCompany's bank account is BE77 0014 5678 9042 (BIC GEBABEBB). February's salaries leave by transfer.

Your goal. Produce the SEPA file of the February run and mark the run as paid.

Check. The file holds **4 transactions**, control sum **12,450.89 €**, category purpose **SALA**; the February run is **Paid**.

▼ **Hint**

The first attempt fails with a clear message: read it. The IBAN belongs on the *Bank* journal.

🎯 Mission 16.3 · Paid is not booked ★★

Situation. End of May: every run from January to May is validated, only February is paid. The accountant asks: "How much net pay do we still owe our employees?"

Your goal. Post the entries of February to May and read the balance of the net-pay account.

Check. February's entry totals **23,360.94 €**; account **455000** shows a credit balance of **68,910.13 €**: every net since January, February included, because no bank statement has been reconciled.

▼ **Hint**

Select several draft entries in the *Salaries* journal and post them at once; the general ledger gives the balance.

🎯 Mission 16.4 · The adjustment line of June ★★★

Situation. June's run (run it now, as in Chapter 15) pays the annual sectoral bonus (`SECTORIAL.BONUS`) to the CP 200 employees. The accountant finds an *Adjustment Entry* line in June's entry and asks why.

Your goal. Run, validate and post June, then explain the adjustment line from the payslip lines and the rules' accounts.

Check. An *Adjustment Entry* debit of **637.50 €** on 620200 = the sectoral bonuses (**1,394.24 €**, booked by no rule) minus the bonus social security (**182.22 €**) and withholding (**574.52 €**), booked a second time as debits on 454000 and 453000. Those liabilities are understated by 756.74 € in total.

▼ **Hint**

Show all on a June payslip, list the lines whose code contains `BONUS`, and open those rules' *Accounting* tab: are their amounts already in `ONSSTOTAL` and `PPTOTAL`?

Summary

- A validated payslip creates accounting only if its structure has a salary journal; the rules' accounts decide the lines.
- With *Anonymize Journal Entries* (Belgium's default), a run gives one entry per journal and month, dated at month end, in draft.
- February's entry: 18,800.00 € salaries + 4,626.76 € employer contributions = 23,426.76 €, owed to social security, the tax office and the employees.
- Rules with two accounts book employer costs without touching net; an *Adjustment Entry* line means a missing or doubled account.
- The *Pay* wizard produces the bank file and marks payslips paid; it does not reconcile anything. The net-pay account clears only when the bank statement is reconciled.

Reports, declarations and a leaver

Pivots for you, statutory returns for the authorities, Emma's departure, and a month in the life

AFTER THIS CHAPTER YOU CAN

- Use the payroll reports to check a pay run, knowing what their filters hide
- Explain how Odoo builds a declaration from validated payslips, and why categories matter
- Read the Belgian monthly withholding return, quarterly social security return, yearly tax certificates and individual accounts as examples of the generic patterns
- Process a leaver: departure record, exit payslips, and their place in the month
- Replay a full payroll month end to end, in the right order

A payslip is not the end of payroll. You check the run with **reports**; the authorities receive **declarations**. This chapter covers both, then follows Emma out of the company and closes Part III with a full month.

Why payroll reports to the authorities

An employer that runs payroll holds money that is not its own: the tax and the contributions it kept back from every payslip. The authorities need to know how much it kept, for whom, and when it pays it over. Employees need it too: their pension, unemployment and health rights, and their own tax return, depend on what the employer declared. So every country asks for reports. The names change from country to country; the four kinds below do not.

Hire and exit notices

Social security knows who is insured from the first day, and undeclared work becomes visible.

BELGIUM Dimona, before the first hour of work

ELSEWHERE Hong Kong IR56E and IR56F; Türkiye's hiring notice

Periodic tax return

The employer pays over the tax it withheld, and says how much.

BELGIUM form 274, every month

ELSEWHERE US form 941 (quarterly); Philippines 1601-C (monthly); India's quarterly tax-at-source statement

Periodic social security return

Contributions are paid, and each worker's pay and time is recorded for their future rights.

BELGIUM DmfA, every quarter

ELSEWHERE Philippines SSS file; Kenya's pension and health sheets; Switzerland's ELM transmission

Yearly certificate per employee

The employee and the tax office see the year's pay and tax, for the personal tax return.

BELGIUM form 281.10

ELSEWHERE US W-2; Philippines 2316; Hong Kong IR56B; Kenya P9

A few countries go further and receive payroll **as it happens**: Mexico wants one signed electronic payslip per payslip, Australia a report to the tax office at every pay run. The idea is the same; only the rhythm changes (Chapter 7).

Reports

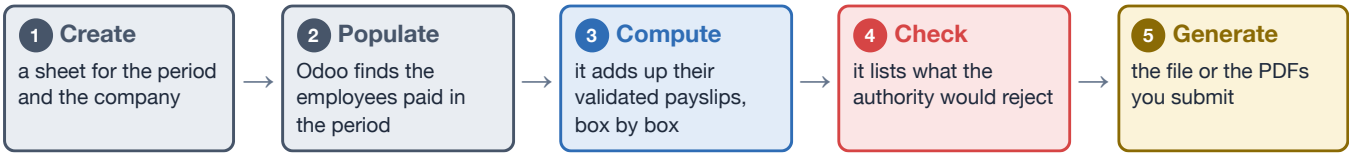
Payroll > *Reporting* > *Payslip* offers pivots and graphs on payslip lines (per rule, category, employee, period), worked days and inputs.

> ⚠️ **Common pitfall: the report filters on state**
>
> The payslip reports open filtered on validated and paid payslips, so a draft run is invisible. Remove the filter to preview the open month.

> 💡 **In practice: three cross-checks before you declare**
>
> 1. **Tax**: the month's withholding lines (`PPTOTAL`) = the credit on 453000 = the withholding return. February: 3,978.86 € in all three places.
> 2. **Social security**: the quarter's employee and employer contributions = the credits on 454000.
> 3. **Headcount**: every employee paid in the period appears in the declaration, and nobody else.

How Odoo builds a declaration

Every declaration in Odoo, whatever the country, is a **declaration sheet** for one period. You create it, and Odoo does the rest in a fixed order:



The framework provides this design once; each country adds its own boxes, file format and checks.

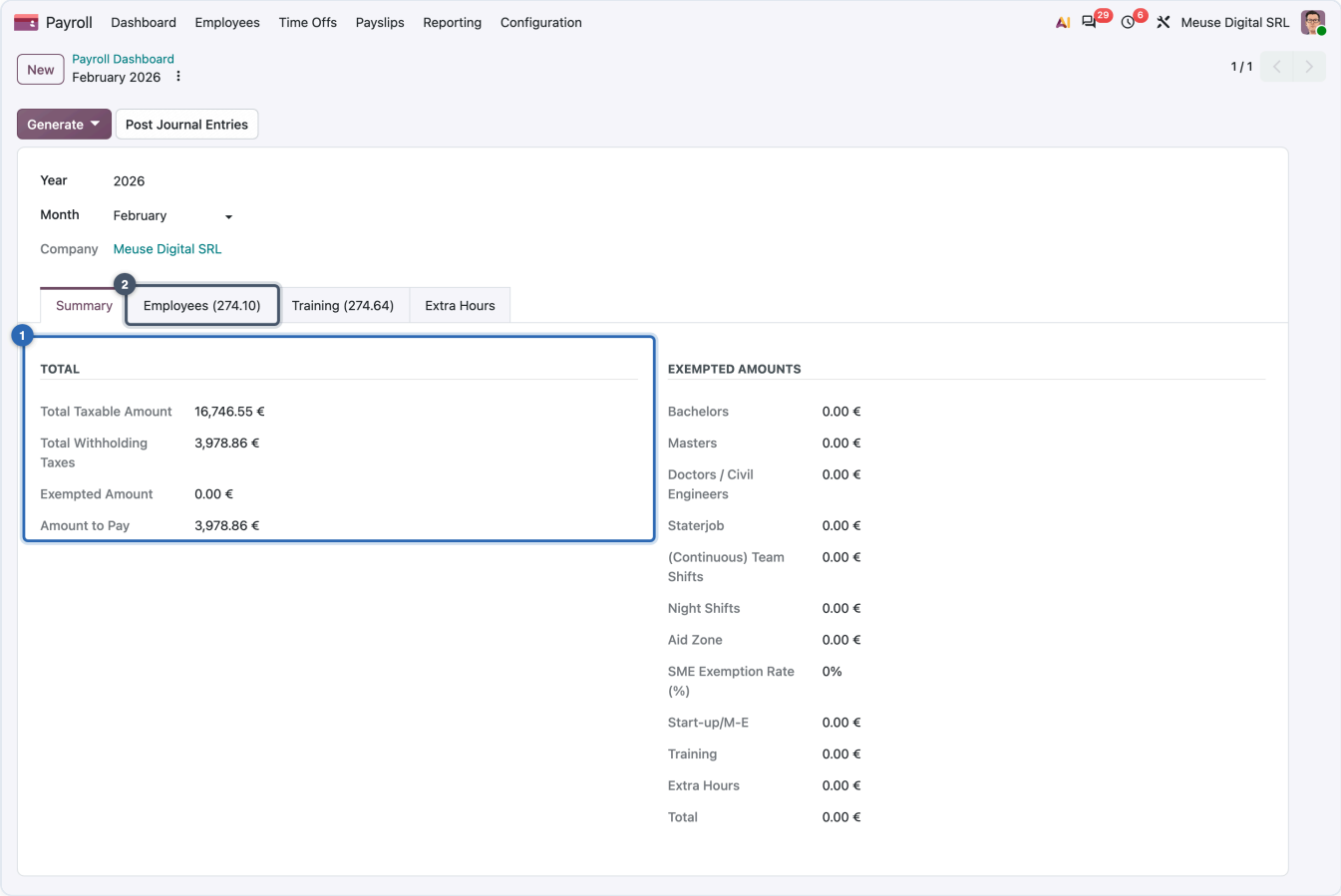
> 🔑 **Key idea: a form is a sum of categories**
>
> A declaration rarely looks at individual rules. It sums **salary rule categories** over the period: taxable base, withholding, contribution base. If a rule sits in the wrong category, the payslip can be right and the return wrong. Designing categories is designing the declarations (Chapters 20 and 22).

MyVidangeCompany's declarations

Declaration	Generic role	Frequency	Same role elsewhere
Form 274	the **withholding-tax return** to the tax office	month	US 941, Philippines 1601-C
DmfA	the **social security return**: pay and time per worker	quarter	Philippines SSS file, Kenya pension sheet
Form 281.10	the **tax certificate** per employee, to employee and tax office	year	US W-2, Philippines 2316, Hong Kong IR56B
Individual account	the **record of pay per employee** the employer must keep	year	Switzerland

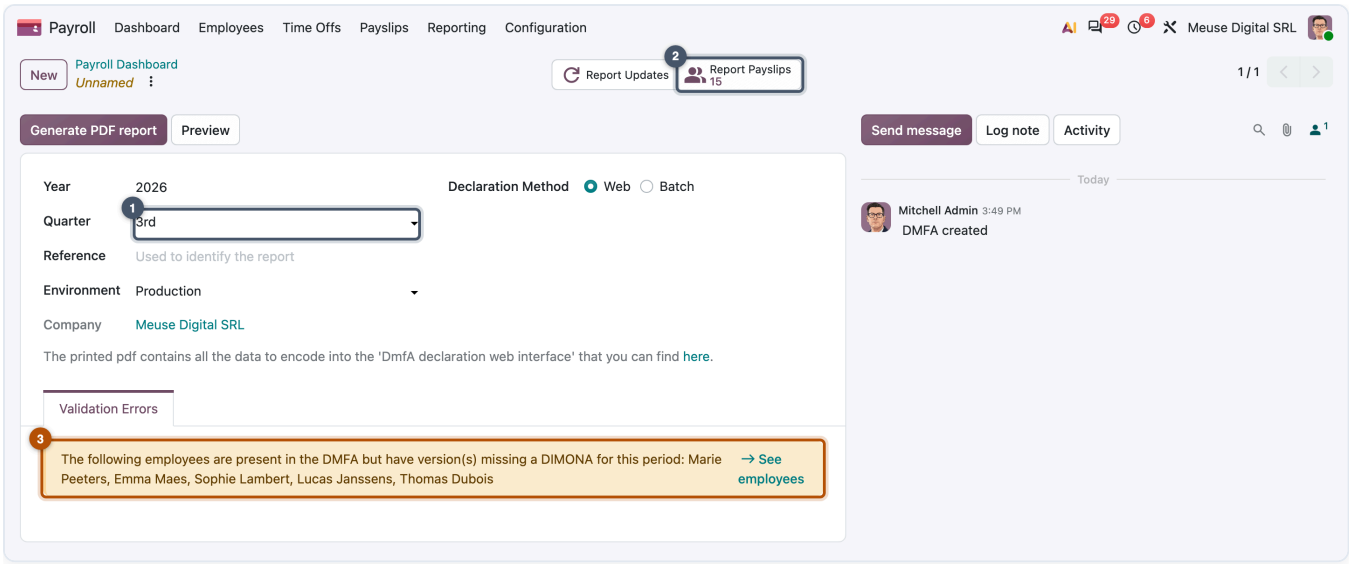
Declaration	Generic role	Frequency	Same role elsewhere
Dimona	the **hiring and exit declaration**, before the first hour	event	Hong Kong IR56E / IR56F

All but Dimona live under *Reporting* > *Belgium*; Dimona is sent from the employee form.

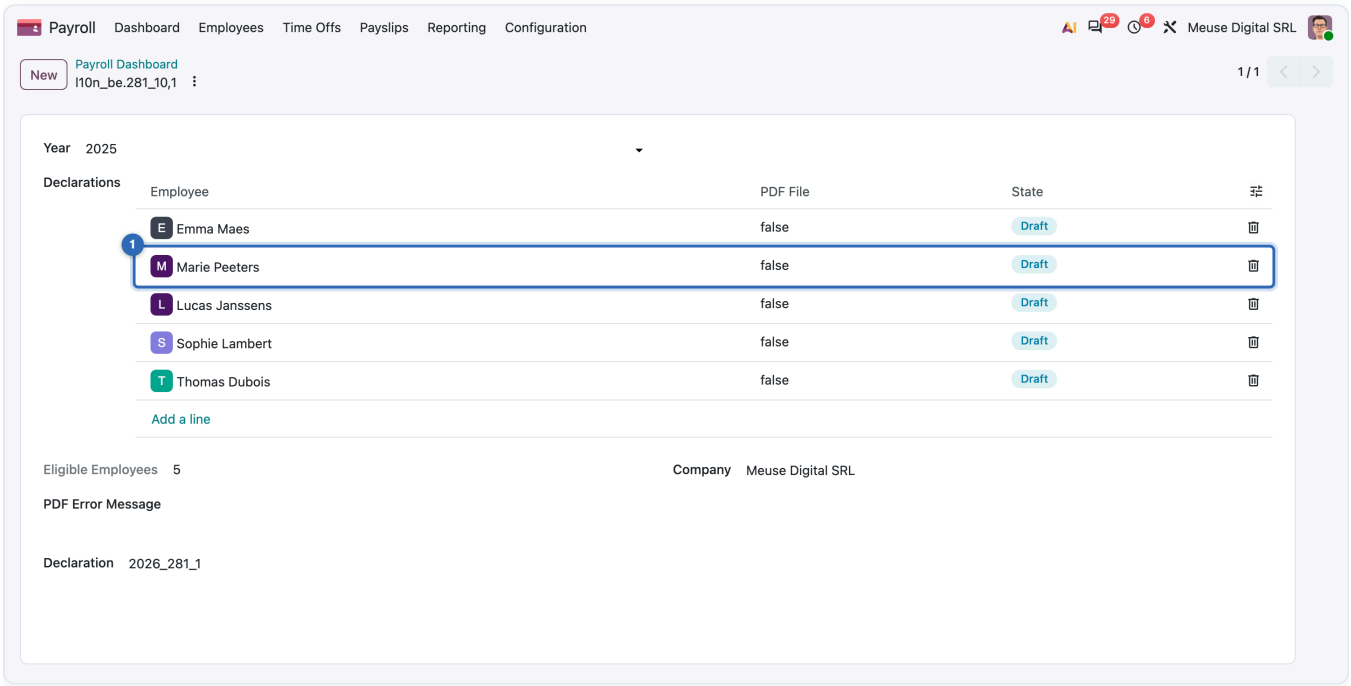


The withholding-tax return (form 274) of February 2026. ① the totals: taxable amount 16,746.55 €, withholding taxes 3,978.86 €, no exemption, 3,978.86 € to pay. ② the tab of per-employee lines. The right column lists the partial exemptions an employer may claim (none for MyVidangeCompany).

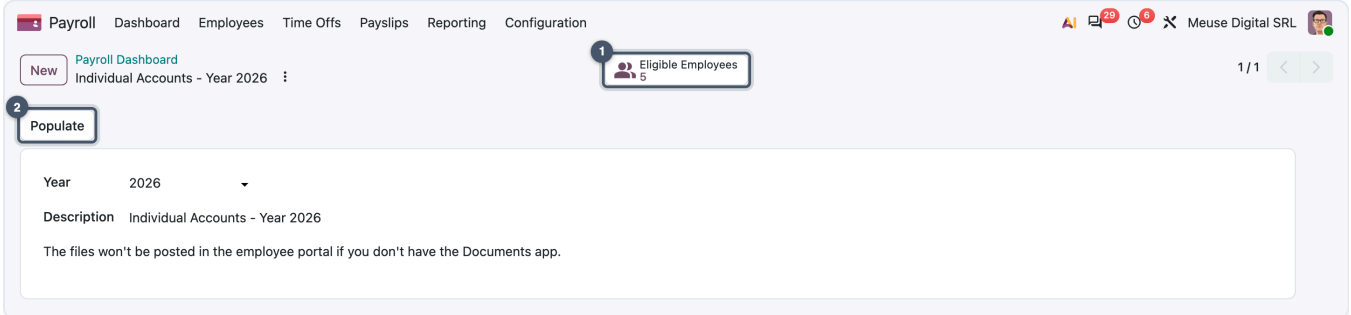
3,978.86 € is the sum of the four withholding lines (826.69 + 164.60 + 1,742.31 + 1,245.26) and the 453000 credit of February's entry: three documents, one number.



*The DmfA of the third quarter of 2026 on the full demo database. ① the quarter. ② **Report Payslips 15**: the validated payslips of July to September (5 employees × 3 months). ③ a validation error: the five employees have no Dimona for the period. Odoo catches it before the authority's rejection mail.*



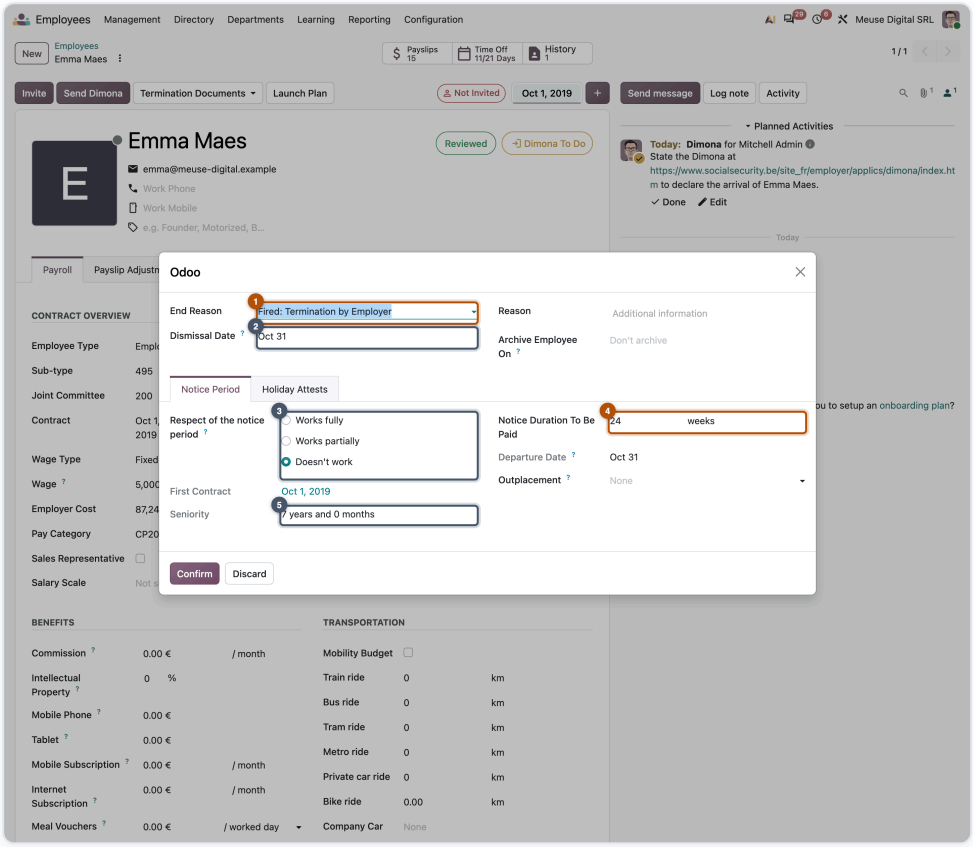
The 281.10 sheet of the 2026 tax-certificate declaration: one line per employee, each with its own PDF and state. ① Marie's 281.10. The sheet's own Year field shows 2025, its default, while the parent declaration is 2026: check it before generating.



*Individual accounts of 2026. ① 5 eligible employees, found from the year's payslips. ② **Populate** creates one line, then one PDF, per employee.*

A leaver: Emma's departure

Emma is dismissed on 31 October 2026 and does not work her notice: the company pays it as an indemnity. A leaver is an **event** that creates **off-cycle payslips**, each on its own structure, next to the last regular one.

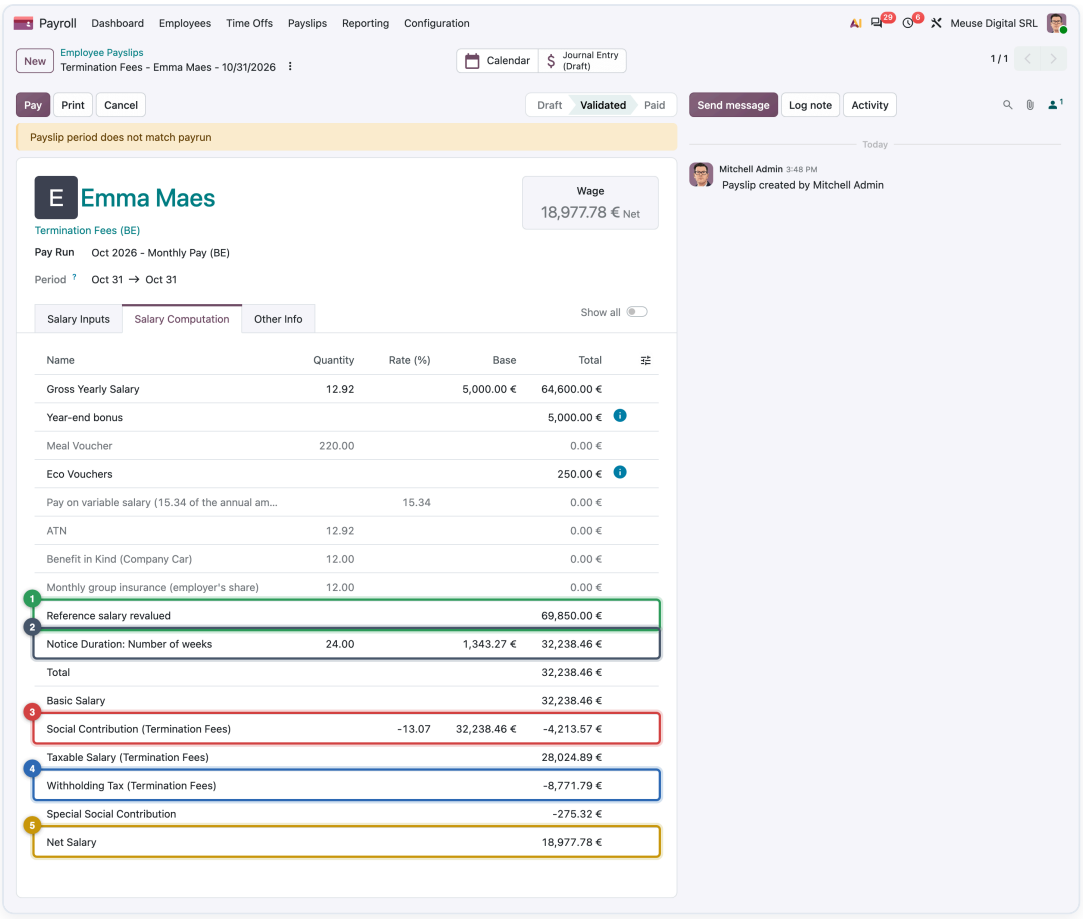


Emma's End of Collaboration dialog (employee form, gear menu), Notice Period tab. ① end reason Fired: Termination by Employer. ② dismissal date 31 October. ③ the notice is not worked. ④ Odoo computes 24 weeks to be paid. ⑤ from the first contract date, a seniority of 7 years and 0 months.

The dialog creates no payslip. When Emma's last regular payslip enters the October run, Odoo adds her exit payslips to it:

Emma's October payslips	Structure	Net (€)
Last month's salary	Monthly Pay (BE)	3,159.73
Indemnity in lieu of notice	Termination Fees (BE)	18,977.78
Exit holiday pay, current year	Termination Holidays N (BE)	3,204.29
Exit holiday pay, previous year	Termination Holidays N-1 (BE)	0.00
13th month, pro rata	13th Month (BE)	1,757.44

An indemnity and holiday pay are taxed at their own rates (Chapter 4): that is why they are separate payslips.



Emma's termination-fee payslip. ① the reference salary, 69,850.00 € a year (12.92 × 5,000 + the year-end bonus + eco-vouchers). ② 24 weeks at 1,343.27 € = 32,238.46 €. ③ employee social security on the indemnity. ④ withholding tax on the indemnity. ⑤ net 18,977.78 €.

In Belgium

The notice length is a dated parameter (Chapter 12 shows it): 24 weeks for 7 years. Exit holiday pay settles holidays earned but not taken (N-1 is zero: the demo has no 2025 payslips).

⚠ Common pitfall: process the departure before the run

Record the departure *before* you generate October's run. Otherwise the exit payslips land outside the run, or nowhere. To create them by hand, use the *Termination Documents* menu of the employee form: one button per exit payslip (termination fees, holiday attests, 13th month), each with its inputs filled from the departure.

A month in the life

Part III showed each piece on its own. A payroll officer runs them as one routine, every month, in the same order. The order is not a habit: each step needs the one before it to be finished.

Before the run
all month long

- **Record the events:** hires, raises, time off, departures (Ch. 13, 14).
- **Why first:** the payslips are computed from them.

→ **Pay run**
a few days before payday

- **Generate and compute** the run (Ch. 15).
- **Review** warnings, the change against last month, the reports.
- **Validate:** the amounts are now final.

→ **Book and pay**
payday

- **Post** the journal entry; check there is no adjustment line (Ch. 16).
- **Pay** the net with the bank file (Ch. 16).

→ **Declare**
by the legal deadlines

- **Monthly:** the withholding return and its payment.
- **Quarterly and yearly:** social security return, employee certificates.

Most payroll mistakes come from breaking that order: a sick day recorded after validation becomes a correction (Chapter 15); a run validated before the departure is recorded leaves the exit payslips out; a return filed before the payslips are validated declares the wrong amounts. The capstone mission below plays the whole routine on October 2026.

Missions

🎯 Mission 17.1 · February's withholding return ★

Situation. The monthly withholding return for February is due. Your accountant wants the amount to pay.

Your goal. Create the February 2026 return and check it against the payslips and accounting.

Check. Taxable amount **16,746.55 €**, withholding **3,978.86 €**, which is also the 453000 credit in February's entry.

▼ **Hint**

Reporting > *Belgium* > *274.XX Sheets* > *New*: year and month are enough.

🎯 Mission 17.2 · Summer: holiday pay and the third quarter ★★

Situation. In June Belgian employers pay a second, "double" holiday pay on its own structure. July to September are ordinary months, except Emma's holiday in August (Mission 14.4).

Your goal. Run and validate a run *Double Holiday Pay 2026* in June on *Double Holidays (BE)*, then July, August and September on *Monthly Pay (BE)*.

Check. The double holiday run has **5 payslips**, net **8,138.92 €** (Mélanie **1,864.04 €**, Jurgen **1,328.02 €**). Emma's August payslip shows **10 days** of paid time off worth **2,307.69 €**.

▼ **Hint**

Pick the structure in the run's form. Yannick's double holiday pay is zero: he has no previous year at MyVidangeCompany.

🎯 Mission 17.3 · Capstone: October, Emma's last month ★★★

Situation. Emma is dismissed on 31 October 2026 and will not work her notice. Everything else is a normal month.

Your goal. Record the departure, then run and validate October, post its entry, pay it by SEPA, and prepare October's withholding return.

Check.

Step	Value
Departure	notice **24 weeks**, seniority **7 years and 0 months**, end of notice **16/04/2027**
October run	**9 payslips** (5 regular, 4 exit), net **39,018.19 €**; Emma's termination fees **18,977.78 €**
SEPA file	**5 transactions**, control sum **39,018.19 €** (Emma's four payslips in one transfer of 27,099.24 €)
Return 274	taxable **57,907.30 €**, withholding **18,301.63 €**
Entry	total **93,305.69 €**, *Adjustment Entry* **16,140.65 €**: explain it as in Mission 16.4

▼ **Hint**

Departure first, then the run: *End of Collaboration* is under the gear of Emma's form; save with *Schedule*. If it offers *Apply* instead (you play after October 2026), first set *Archive Employee On* after today, or Emma is left out of the run.

Summary

- Payslip reports are pivots on lines, worked days and inputs; they filter on validated payslips by default.
- A declaration finds the employees, sums validated payslips by category, checks, then builds a file or PDFs.
- MyVidangeCompany files a monthly withholding return (274), a quarterly social security return (DmfA), yearly certificates (281.10), individual accounts, and hiring and exit declarations (Dimona).
- February's withholding, 3,978.86 €, matches the payslips, the 453000 credit and the return.
- A leaver is an event that creates off-cycle payslips on their own structures; record the departure before the run.
- Emma's exit: 24 weeks of notice worth 32,238.46 € gross, 18,977.78 € net, plus exit holiday pay and a pro-rata 13th month.
- A month is eight steps; the system computes, books and sums, people decide, check and send.

PART IV

How the payroll engine works

What a salary rule is, what it reads and does, how structures, categories and dated parameters fit together, and how to tell whether a rule is right, in any country.

The engine in one picture

What goes into a payslip, what comes out, and the short loop in between

AFTER THIS CHAPTER YOU CAN

- Say in one sentence what the payroll engine does
- Name what goes into a payslip and where its lines go afterwards
- Read the engine's core loop and explain why it is the same in every country
- Name the five records you work with, who owns each, and how often each changes
- Tell which records ship with a country module and which belong to the customer

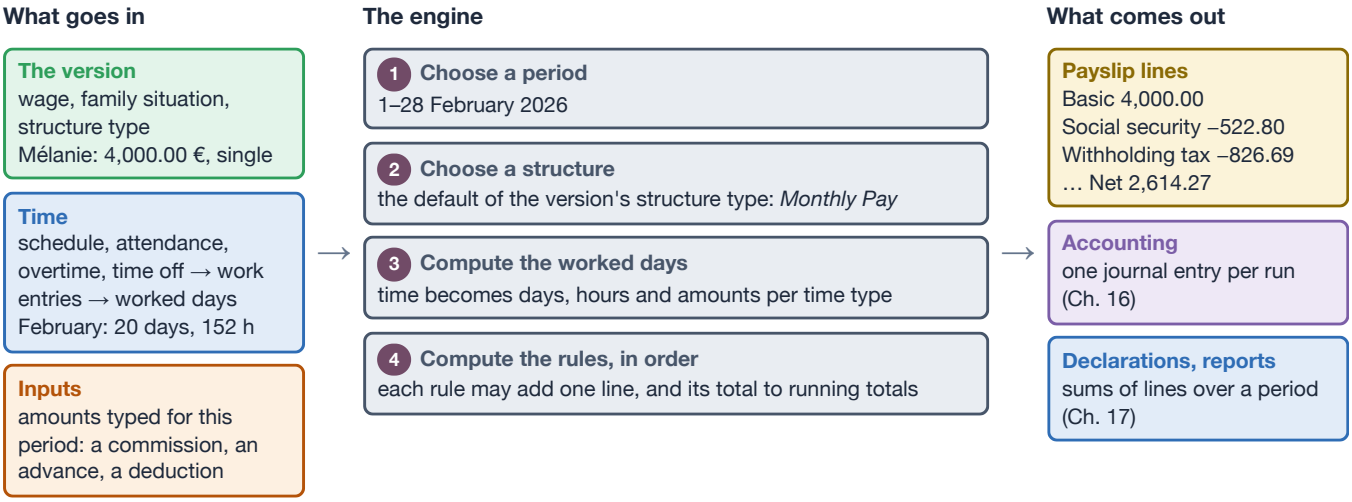
Mélanie's February payslip has 44 lines; Marie's, on the same structure, has 49 (her car, her meal vouchers). A Mexican weekly payslip looks nothing like either. Yet nobody wrote a "Belgian payslip program" or a "Mexican" one. One small engine read a list of rules and applied them, one after the other. This part of the book explains that engine at the level you need to create payroll records and to review them.

What the payroll engine is

The **payroll engine** is the part of Odoo that turns one employee's data for one period into **payslip lines**. It takes the rules of one structure and applies them, in order, to the employee's contract terms, time and inputs.

Three things follow from that definition:

- **The engine knows no country.** Everything a law says is in the rules and their parameters, which are data. The engine only runs them.
- **The engine decides nothing.** Whether a line appears, and how much it is, is decided by a rule. When a payslip is wrong, the cause is a rule or the data it read, almost never the engine.
- **It runs again on demand.** A draft payslip is recomputed whenever you click *Compute* or its data changes. A validated payslip is frozen (Chapter 15).



Read the figure from left to right. On the left, the three kinds of data a payslip needs: **who** (the employee's version, Chapter 13), **how much time** (Chapter 14) and **what else happened** this period (inputs). In the middle, the four steps of the engine; Part III showed the first three, this part explains the fourth. On the right, the lines, and everything that reads them later: the journal entry, the declarations, the reports. Nothing downstream recomputes anything; it reads the lines.

> 💡 **In practice**
>
> When a payslip looks wrong, follow the figure from the left: is the **version** right (wage, situation, structure type)? Are the **worked days** right? Are the **inputs** right? Only then open a **rule**. Most "rule bugs" are data in one of the first three.

Why a payroll engine at all?

Payroll law differs in every country and changes every year. If each country's payslip were a program, every new tax table would be a code change, and every customer who pays a sector bonus would need a developer. Odoo takes the other road: **rules are data**.

A salary rule is a record: a name, a code, a condition and a small formula, usually a few lines of Python. Country modules ship their rules in data files; a customer can create more from the screen. One engine evaluates all of them the same way, giving each rule a dictionary of what it may see: the version, the worked days, the totals so far (Chapter 19 lists them). Here is that engine, simplified to its core:

```
def compute_lines(payslip):
    seen = payslip._get_localdict()             # what a rule may read
    lines = []
    for rule in sorted(payslip.struct_id.rule_ids, key=lambda r: r.sequence):
        if not rule._satisfy_condition(seen):   # condition false: no line at all
            continue
        amount, quantity, rate = rule._compute_rule(seen)[:3]
        total = amount * quantity * rate / 100
        seen['result_rules'][rule.code] = {'total': total, 'amount': amount}
        rule.category_ids._sum_salary_rule_category(seen, total)   # running totals
        lines.append({'code': rule.code, 'amount': amount, 'quantity': quantity,
                      'rate': rate, 'total': total})
    return lines
```

For each rule, in order: test the condition; if it holds, compute an amount, a quantity and a rate; multiply them into a total; remember the result for later rules; add it to the rule's running totals (its categories, Chapter 20); keep one line. The real method, `_get_payslip_lines`, adds batches of payslips, year-to-date sums and a few hooks for countries, but **the whole engine fits on one page; the names may change, the loop does not.** That is why you can review any country's payroll once you can read its rules.

$$\text{line total} = \text{amount} \times \text{quantity} \times \text{rate} \div 100$$

Mélanie's social-security line: amount 4,000.00 € (her contribution base), quantity 1, rate –13.07 %, total –522.80 €.

> 🔑 **Key idea**
>
> The engine is small and stable; the rules are many and change every year. Everything in this part is about reading and placing rules, because that is where every country's law lives.

The five records you work with

Five kinds of record decide how a payslip is computed. Chapter 12 introduced four of them by following Mélanie's payslip; here they are side by side, with who owns them and how often they change.

Structure type

The kind of pay: who is paid how, and how often. Set on each employee's version; names the default structure.

OWNER	country module; the customer picks one per version
CHANGES	almost never
EXAMPLE	BE *CP200 BE*; MX *Employee*

Structure

A named list of rules for one kind of payslip (Chapter 19).

OWNER	country module; a customer may add one
CHANGES	when a new kind of payslip is needed
EXAMPLE	BE *Monthly Pay*, *13th Month*; MX *Christmas Bonus*

Salary rule

One line of the payslip: when it applies, how much, which totals it feeds (Chapter 19).

OWNER	country module; customers add their own, such as a company allowance
CHANGES	when the law's method changes
EXAMPLE	BE `ONSS`; MX `ISR`

Salary category

A named running total that rules add to and later rules read (Chapter 20).

OWNER	country module
CHANGES	when the law creates a new base
EXAMPLE	`GROSS`, `NET`; MX `SBC`

Rule parameter

A legal number or table with its dated values: rates, caps, tax scales (Chapter 21).

OWNER	country module; Odoo adds the new values, not the customer
CHANGES	every year, sometimes several times a year
EXAMPLE	BE professional-expenses cap: 5,930 in 2025, 6,070 in 2026; MX `l10n_mx_uma`

Two more long-lived concepts surround them: the employee's **version**, which the customer fills in (Chapter 13), and the **time types**, which the country module ships and which say how each kind of hour is paid (Chapter 14).

Notice the *Changes* lines: the records change at very different speeds. A parameter changes every year; a rule only when the method changes; a structure type almost never. Keeping each fact in the record that changes at its speed is what turns a new tax year into a data update instead of a rewrite.

> ⚠ **Common pitfall: fixing one employee in a shared rule**
>
> A rule is shared by every employee on its structures. Editing the `ONSS` rule because one employee's line looks wrong changes everyone's payslips, and stops the rule from receiving Odoo's fixes (Chapter 21). Look for the cause in that employee's version, worked days or inputs first.

Five design ideas

Everything in the next three chapters follows from five decisions.

1 · Rules are data

A rule is a record with a small formula, not a program. Countries ship rules as data; customers add their own from the screen; one engine runs them all.

WHERE
salary rules and structures Chapter 19

2 · Legal numbers live in dated parameters

A rule asks for "the cap" by its code and gets the value valid on the payslip's date. Next year's law is a new value, never an edit.

WHERE
rule parameters Chapters 11, 21

3 · Time is computed, not stored

Work entries are derived from the schedule and time off whenever a payslip needs them, so a late change of leave is never forgotten.

WHERE
work entries and worked days Chapter 14

4 · Totals connect the rules

Rules do not call each other. Each one adds to running totals (gross, taxable base, net), and later rules read the totals.

WHERE
salary categories Chapter 20

5 · Countries are layers on a shared framework

The engine, payslips, pay runs, accounting and declarations are shared. A country module adds rules, parameters, categories, fields and its filings on top, and never replaces the engine.

WHERE
country modules `l10n_xx_hr_payroll` Chapters 9, 22

🌍 Around the world

The same five records carry very different laws:

- **Mexico**: one structure type, two structures (regular pay and the Christmas bonus), frequencies from weekly to monthly.
- **Belgium**: about ten structures for one structure type, because holiday pay, the 13th month and termination are separate payslips.
- **India**: a second structure type for interns.
- **United States**: one structure holding the income-tax rule of every state; each rule's condition tests the employee's work address.
- **Netherlands**: a small module that is almost only data.

💡 Review checklist

Before reviewing a payroll database or a country module, sort its records by owner:

- **Ships with the country module and is updated by Odoo**: structure types, structures, rules, categories, parameters and their values, the payroll settings of time types, dashboard warnings.
- **Belongs to the customer**: employees and their versions, time off, inputs, company settings, and any rule or structure they created (Odoo marks those as created by a user).
- **A shipped record the customer edited** no longer receives Odoo's updates (Chapter 21). List them; each one needs a reason.
- A legal number typed in a customer record, or a customer-specific amount in a shipped rule, is in the wrong place.

Summary

- The payroll engine turns one employee's version, time and inputs for one period into payslip lines, by applying the rules of one structure in order.
- Accounting, declarations and reports read the lines; they never recompute.
- Rules are data: small formulas stored as records, shipped by country modules or created by users, run by one engine that fits on a page.
- Each line is `amount × quantity × rate ÷ 100`, and its total is added to the rule's categories for later rules.
- Five records decide a payslip: structure type, structure, salary rule, salary category, rule parameter. They change at different speeds, from almost never to every year.
- The country module owns the configuration; the customer owns versions, time and inputs; a shipped record the customer edited stops receiving updates.

Structures and rules

A structure is a list of rules for one kind of payslip; a rule is one line, with a condition, an amount and a place in the order

AFTER THIS CHAPTER YOU CAN

- Explain what a structure is, why a country has several, and how a payslip finds its own
- Read any salary rule: its code, condition, amount, categories and position
- Say what a rule can read, what it cannot, and what it is allowed to do
- Explain why the order of the rules carries meaning
- Recognise the three shapes of rule and review each one

Jurgen and Mélanie are paid on the same structure, *Monthly Pay*. Yet Jurgen's February payslip has a low-wage bonus line that Mélanie's lacks, and Marie's has a company-car line that neither of them has. Nobody added or removed those lines by hand: the structure holds every rule, and each rule decides for itself, payslip by payslip, whether it applies.

What a structure is

A **salary structure** (also *pay structure*) is a named list of salary rules for one kind of payslip. When a payslip is computed, the engine runs the rules of its structure, and only those.

Three things follow:

- **One structure per kind of payslip.** A regular month, a 13th month, holiday pay and a leaver's indemnity are taxed and declared differently, so each is a separate payslip on its own structure.
- **A rule can belong to several structures.** The social-security rule of a country is usually shared by its regular and bonus structures; it is one record, maintained once.
- **Structures are grouped by structure type.** The type (the kind of pay on the employee's version) lists its structures and names the **default** one, for the regular payslip.

Regular	Mélanie's version	→	its structure type: **CP200 BE**	→	the type's default: **Monthly Pay**
Special	a pay run or a payslip created on purpose	→	a structure chosen for it	→	**13th Month**, **Double Holidays**, **Termination Fees**…

Either way, a payslip has exactly **one** structure, and only that structure's rules run.

A regular pay run takes each employee's default structure from the version. For a payment outside the regular month, you create a run (or a single off-cycle payslip) and choose its structure (Chapter 15).

Country	Structures	Why so many, or so few
Belgium	ten: *Monthly Pay*, *Double Holidays*, *13th Month*, *Termination Fees*, two exit holiday-pay structures, *Advance Pay*, and three bonus plans	each has its own withholding method or declaration code
Mexico	two: *Regular Pay*, *Christmas Bonus*	the bonus has an exempt part and its own tax

Country	Structures	Why so many, or so few
Most other countries	one regular structure	bonuses are lines on the regular payslip

> ⚠️ **Common pitfall: a rule missing from one structure**
>
> A rule runs only on the structures it is attached to. A new allowance added to *Monthly Pay* but not to *Termination Fees* is simply absent from leavers' final pay, with no error. When you add or review a rule, check its **structures** list.

Anatomy of a salary rule

A **salary rule** computes at most one line of the payslip. Every rule, in every country, answers the same five questions. Here is the Belgian employee social-security rule, as you can open it under *Configuration* > *Salary* > *Rules*:

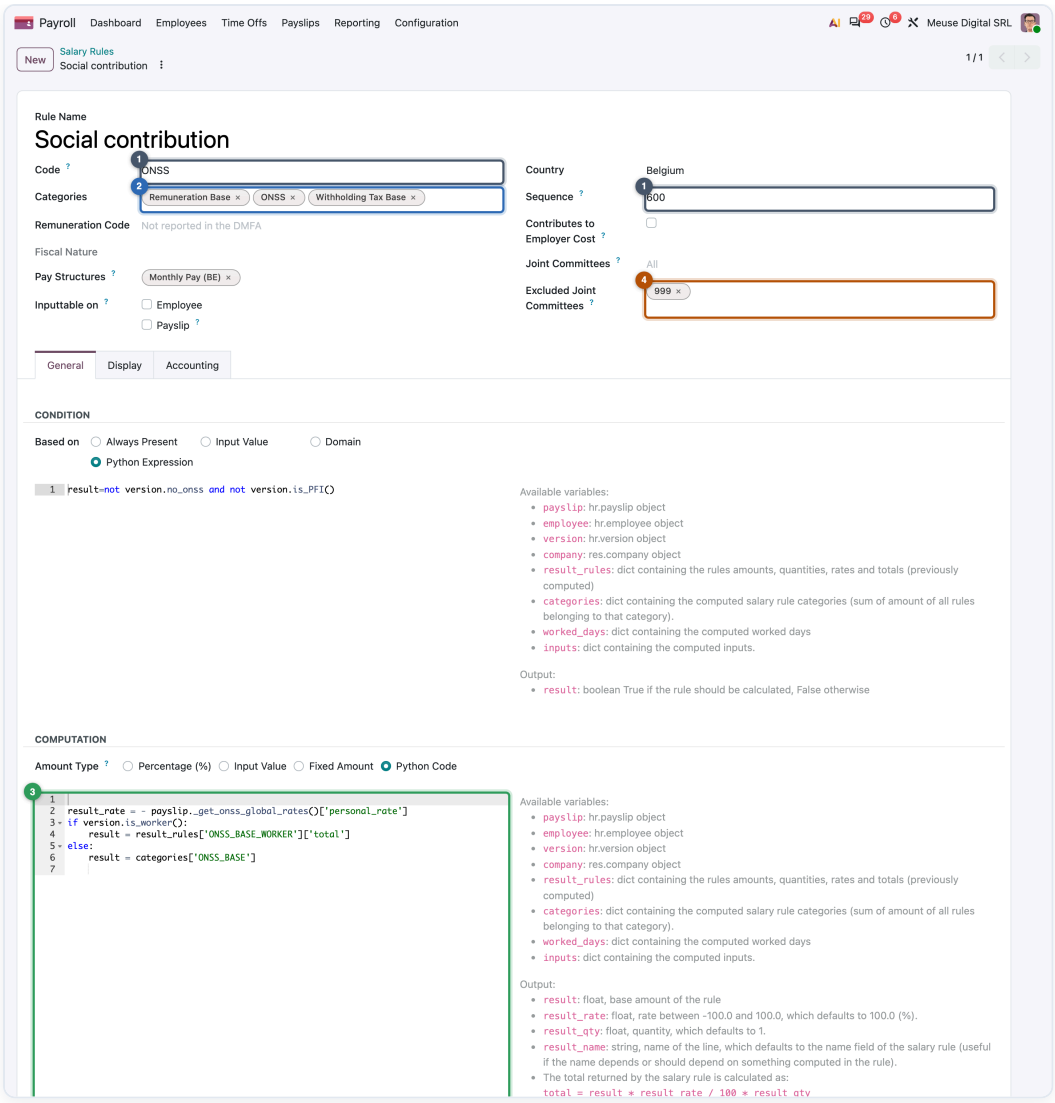
Social contribution `ONSS`

1 **Identity**	name and **code** The code is how other rules refer to its result (`result_rules['ONSS']`), and the code on the payslip line.
2 **When**	condition, Python: `result = not version.no_onss and not version.is_PFI()` Always present, except for the few employees exempt from social security. Other choices: always, an input is present, a domain.
3 **How much**	amount = the contribution base (a total, `ONSS_BASE`); rate = −13.07 %, read from dated rates Other choices: a fixed amount, a percentage of a total, the amount of an input.
4 **Where it goes**	categories: the running net, the taxable base, the social-security total; partner: the social security office; shown on the payslip: always Accounts sit on the rule too, when the country maps them.
5 **Order**	sequence 600 After the gross salary lines, before the taxable salary (`GROSS` , 1000) and the tax.

Mélanie, February 2026

amount	**4,000.00**
× quantity	**1**
× rate	**−13.07 %**
= total	**−522.80**

The line shows all four numbers, so anyone can check it without opening the rule.



The same rule in Odoo. ① its code and its sequence. ② its categories. ③ the amount code: the rate comes from a helper that reads the downloaded rates. ④ a Belgian addition: the sectors the rule does not apply to.

Two details of the card matter in review. The **rate** is not written in the rule: it comes from dated parameters (Chapter 21), so a new rate next year needs no rule change. And the rule feeds **three** totals at once, which is how one contribution lowers both the net and the taxable base (Chapter 20).

What a rule can read

A rule's formula runs with a dictionary of names, the same in every country. The list is short and stable:

What	Name in the rule	Example
The employee, the version, the company	`employee`, `version`, `company`	`version.wage`
The payslip and its period	`payslip`	`payslip.date_to`
Time worked this period	`worked_days`	`worked_days['WORK100'].number_of_days`
Amounts typed for this period	`inputs`	`inputs['COMMISSIONS'].amount`
Totals so far	`categories`	`categories['GROSS']`

What	Name in the rule	Example
Results of earlier rules	`result_rules`	`result_rules['SBC']['total']`
Legal values on the payslip's date	`payslip._rule_parameter(code)`	`payslip._rule_parameter('l10n_mx_uma')`

The form of every rule lists these names next to the code editor (the right side of the screenshot above). A country module can add a few of its own; the Belgian one adds the employee's earlier payslips of the year.

What a rule **cannot** read is a rule that comes later. At its turn, a later rule has not run yet, and its total, or a category only it feeds, reads as **0, without any error**. The same silence hides a misspelt category code. By contrast, `worked_days` and `inputs` only contain the codes present on this payslip: reading `inputs['BONUS']` when no bonus was typed stops the computation with an error. That is why input rules start by checking (`result = 'BONUS' in inputs`), or use the *Input Value* condition, which does it for them.

What a rule can do

A rule produces **one line or none**. Nothing else.

Condition true: the rule returns an amount, a quantity (default 1) and a rate (default 100 %), and may rename its line. The engine stores one line: amount × quantity × rate ÷ 100. A total of zero is still a line; the rule's *Visibility* setting decides whether it is printed.

Condition false: no line at all. Its code is absent from the payslip, its totals receive nothing, and later rules read 0 for it.

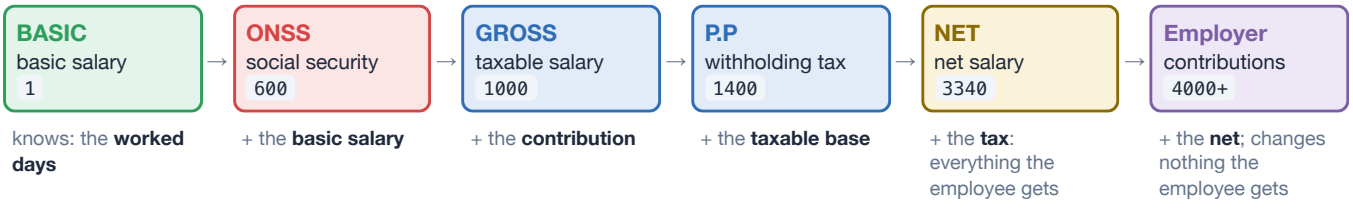
A rule does **not** change records. It does not write on the employee, create a leave or update a balance; those are actions of the payslip's validation or of other modules. Rule code runs in a restricted interpreter (no imports), but it can still reach the database through the records it reads, which is why only payroll managers may edit rules, and why a rule that writes anything is a review finding.

Worked example: Marie's meal vouchers

Marie received 20 meal vouchers in February. The rule *Retain on Meal Voucher* returns amount 1.09 € (the employee's share of one voucher: face value minus the employer's part, both on her version), quantity 20 (one voucher per day worked) and rate −100 %. Total: 1.09 × 20 × −100 % = **−21.80 €**. The payslip line reads "20 × 1.09", which the employee can check without asking anyone.

Order is meaning

Rules run in the order of their **sequence** number. There is no other dependency mechanism: a rule sees exactly what ran before it. So the order of a structure is not cosmetic; it is the law's order of operations.



The Belgian monthly structure, read as a timeline. Each rule can use what the rules on its left produced:

- **Contributions before tax.** In Belgium, as in many countries, the taxable base is the salary *after* the employee's social security, so the contribution must run first.
- **Net last** among the employee's lines, because it sums them all.
- **Employer contributions after the net.** They are a cost for the company, not a deduction from the employee; running them after the net makes it obvious they cannot change it. (Some countries run them earlier, and keep them out of the net by category instead: Chapter 20.)

The numbers themselves mean nothing: Mexico uses 1 to 210, Belgium 1 to over 6,000. Do not memorise them; read them relative to their neighbours, and leave gaps when you create rules so the next law fits in between.

> ⚠️ **Common pitfall: reading a total that is filled later**
>
> A rule at sequence 500 that reads `categories['GROSS']`, filled at 1000, gets 0 and computes a wrong line silently. When a line is unexpectedly zero, compare the sequence of the rule with the sequences of the rules that feed what it reads.

Three shapes of rule

Open any country's rules and you find three shapes.

1 · No code	2 · Input rule	3 · Formula
A fixed amount, or a percentage of a total, chosen in the form.	The amount is typed on the payslip or the employee for this period.	A few lines of Python reading totals, time and parameters.
Fits an employer's own simple rule.	Fits commissions, bonuses, one-off deductions.	Fits contributions, taxes, caps.
Anyone can read it.	The rule is also the input type.	Most rules of a country module.

No code. A customer who pays a company pension contribution of 3 % of the basic salary needs no developer: *Amount Type* "Percentage (%)", based on `categories['BASIC']`, 3 %, condition "Always Present", category "Deduction", attached to the regular structure. (An illustrative rule; country modules rarely use this shape, because their rates belong in dated parameters.)

Input rule. Mexico's `COMMISSIONS` has condition and amount both set to *Input Value*: the line exists only when someone types a commission on the payslip, and its amount is what was typed. It belongs to the category *Taxable Allowance*, so commissions are taxed, and it is shown only when not zero. There is no separate "input type" record: the rule **is** the input type, and its code is the key in `inputs`.

Formula reading a parameter. Mexico's additional sickness-and-maternity quota (employee part) is due on the part of the daily contribution base above three times the UMA, the official reference unit:

```
base_contribution_salary = result_rules['SBC']['total']
daily_uma = payslip._rule_parameter('l10n_mx_uma')['daily']
result = max(0, (base_contribution_salary - 3 * daily_uma))
result_qty = days_without_disabilities
result_rate = -0.4
```

It reads an earlier result (`SBC`, sequence 124, before this rule at 136), a dated parameter (the UMA) and sets amount, quantity and rate separately, so the line explains itself: *daily excess* × *days* × *–0.4 %*. A reviewer would still note one thing: the 0.4 % rate is written in the code while the UMA is a parameter. If the law changes the rate, this rule changes, and so would every old payslip recomputed with it.

🌍 **Around the world: switching rules on and off**

Conditions are how one structure serves very different employees:

- **United States**: one income-tax rule per state, each with `result = version.address_id.state_id.code == 'CA'` (or its own state).
- **India**: income tax is withheld only in the months the rule's condition selects.
- **Mexico**: commissions and bonuses appear only when an input is typed.
- **Belgium**: a filter in front of every condition skips the rules of other sectors (joint committees), so one structure serves them all.

💡 **Review checklist**

For each rule you create or review:

- **No legal number in the code.** Rates, caps and brackets come from a parameter (Chapter 21); a literal `0.4` or `25 * ` is a finding.
- **Everything it reads is computed earlier.** Each category and result it reads is fed by rules with a lower sequence.
- **It feeds the right totals.** A rule with no category, or a missing one, never reaches the gross, the taxable base or the net (Chapter 20).
- **It checks inputs and time codes exist** before reading them, or uses the *Input Value* condition.
- **It rounds where the law rounds**, and only there. The engine does not round inside a computation; Belgium's withholding rule, for instance, rounds the monthly tax down to the cent itself.
- **It is on every structure where it applies**, and changes no record.

Summary

- A structure is a named list of rules for one kind of payslip; a country has one per payslip that is taxed or declared differently.
- A payslip gets its structure from the version's structure type (the default), or from the pay run or off-cycle payslip that names one.
- A rule answers five questions: identity (name, code), when (condition), how much (amount × quantity × rate), where it goes (categories, accounts, visibility) and order (sequence).
- A rule reads the employee and version, the payslip, worked days, inputs, totals so far, earlier results and parameters; never a later rule, which reads as 0.
- A rule produces one line or none, and changes no record.
- Order is meaning: contributions before tax, net last, employer contributions after the net.
- Three shapes: no code, input rule, formula reading a parameter.

Categories: how rules talk to each other

Running totals that rules add to and later rules read: the gross, the bases, the net

AFTER THIS CHAPTER YOU CAN

- Define a salary category as a running total, and say why rules read totals rather than each other
- Walk the standard chain BASIC → ALW → GROSS → DED → NET, with employer contributions outside the net
- Explain how one rule feeds several totals, with Marie's company car
- Say who reads categories after the payslip is computed
- Check that a country's categories match its legal bases

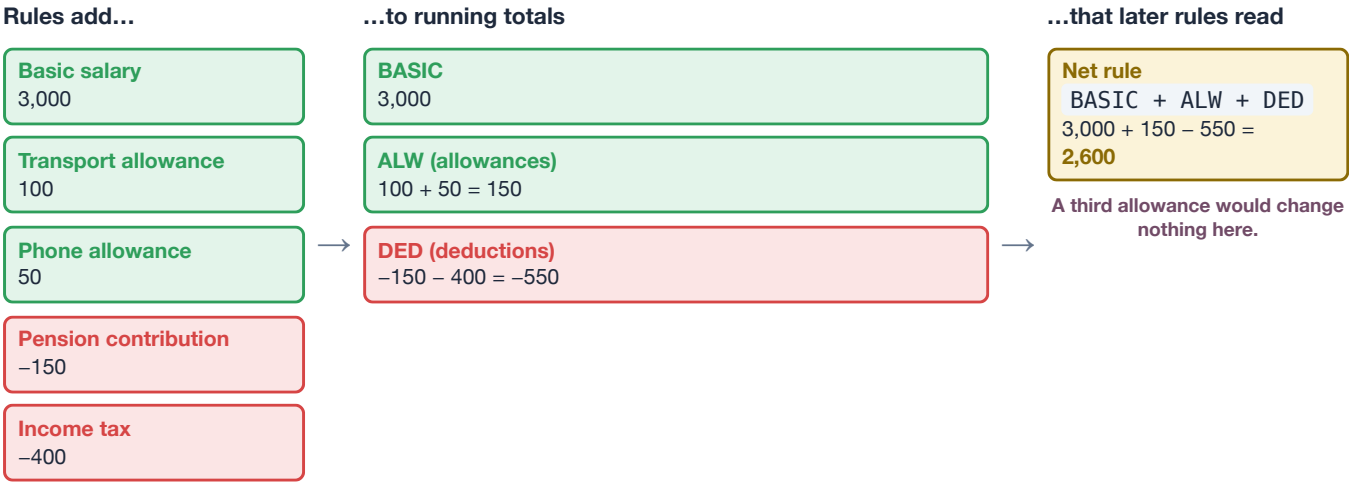
Belgium adds a new tax-free allowance. The social-security rule, the withholding rule and the net rule do not change: the new rule simply joins the right **totals**, and every rule that reads those totals sees it. That is the whole point of categories: rules never name each other; they meet in shared totals.

What a salary category is

A **salary category** (*salary rule category* in Odoo) is a named running total. Each rule adds its result to its categories; later rules read the total instead of naming every rule that fed it. Categories are how rules talk to each other.

Three things follow:

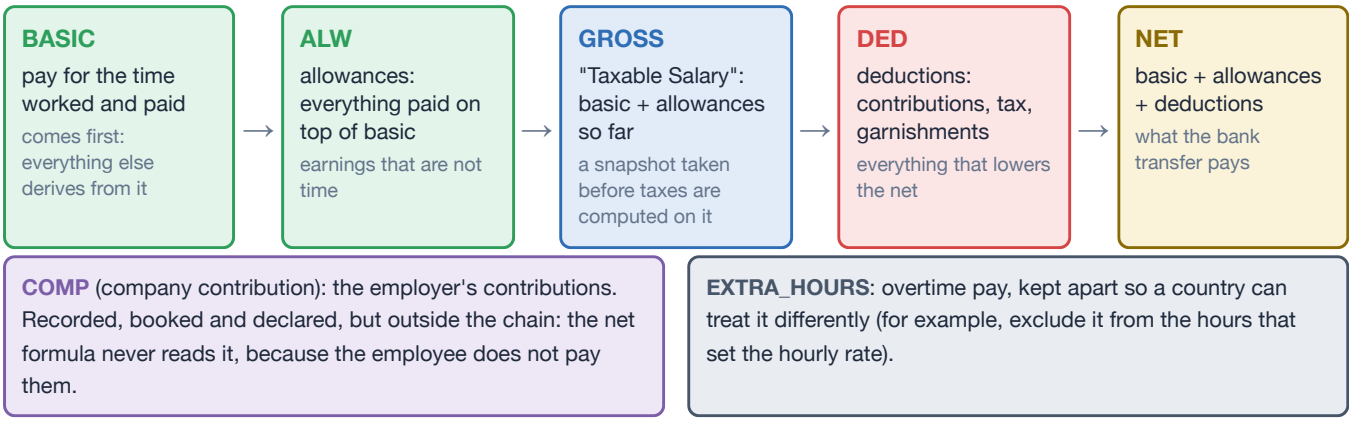
- **A new rule needs no change elsewhere.** Put a new allowance in the allowances total, and the gross and the net pick it up.
- **One rule can feed several totals.** A contribution can lower both the net and the taxable base.
- **A total is not a line.** It lives during the computation. When the law needs it on the payslip, a rule copies it into a line (that is what the `GROSS` rule does).



Illustrative numbers. The net rule names three totals, not five rules.

The standard chain

The payroll framework ships seven categories, and every country starts from them. Five form a chain from the basic salary to the net:



Two words decide where a rule lands: its **category** and its **position**.

> 🔑 **Key idea: category and position**
>
> - `GROSS` **is "taxable" by position.** It reads `BASIC + ALW` at the moment it runs. An expense reimbursement is in `ALW` too, but runs after `GROSS`: the net includes it, the taxable salary does not. An expense paid back is not salary.
> - **Garnishments come after taxes** because the law protects a part of the *net* (Chapter 6); the seizable amount is known only once taxes are withheld.
> - **Employer contributions stay out of the net by category, not by position.** The net reads `BASIC`, `ALW` and `DED`, never `COMP`, wherever the employer rules run.

Where the worked days go

Before any rule runs, each worked-days line adds its amount to the categories of its **time type** (Chapter 14). Attendance usually feeds `BASIC`; a paid leave type feeds what the country decides; unpaid leave is worth 0 and adds nothing. That is how time reaches the totals: the basic salary rule often just reads `categories['BASIC']`.

> ⚠️ **Common pitfall: a paid time type with no category**
>
> If a paid time type has no category, its amount is on the worked-days line but never reaches `BASIC`, so the gross and the net miss it. When a country adds a time type, check its categories. The opposite mistake also exists: a rule that both reads a total seeded by worked days and writes into it counts the amount twice.

One rule, several totals: Marie's company car

Belgian law needs three bases, not one: the **running net** (what will be paid), the **contribution base** (what social security is computed on) and the **taxable base**. So Belgium keeps one total per base, and each rule says which bases it belongs to.

Marie's company car is a **benefit in kind**: taxed as if it were salary, but never paid in cash (Chapter 2). Its rule, `ATN.CAR`, adds 136.77 € to the taxable base and nothing to the running net. The social-security rule adds its −980.25 € to both.

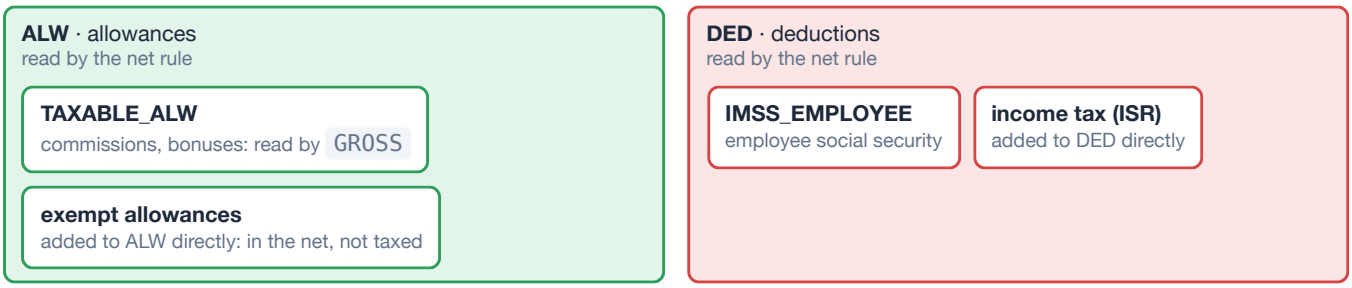
Marie, February 2026	Line total	Running net	Taxable base
Worked days (attendance)		7,500.00	7,500.00
ONSS social security, 13.07 %	−980.25	6,519.75	6,519.75
ATN.CAR company car	136.77	6,519.75	**6,656.52**
GROSS reads the taxable base	6,656.52		

The car changes the tax, not the cash: 7,500.00 − 980.25 + 136.77 = **6,656.52 €** of taxable salary. The withholding rule, the net rule and the declarations never mention the car; they read totals. A new benefit in kind next year will be one more rule in the taxable base.

> **Worked example: Mélanie's three bases**
>
> Mélanie has no benefit in kind. Her social security (−522.80 €) lowers the running net and the taxable base but not the contribution base, which stays at 4,000.00 €. So her `GROSS` line, the taxable salary, is **3,477.20 €**, not 4,000.00 €. After withholding (−826.69 €) and a special contribution (−36.24 €), the net rule reads the running net: **2,614.27 €**.

Parent categories roll up

A category can have a **parent**. When a rule adds to a category, the engine also adds to its parent, and to the parent's parent. A country can then make the generic chain more precise without rewriting it.



Mexico, as shipped. Its two formulas:

```
# GROSS, the taxable salary: only the taxable allowances
result = categories['BASIC'] + categories['TAXABLE_ALW']
# NET: unchanged from the standard; the children roll up into ALW and DED
result = categories['BASIC'] + categories['ALW'] + categories['DED']
```

The net formula is the generic one, yet it sees commissions and employee social security, because their categories roll up into `ALW` and `DED`. The taxable salary reads only the child, so exempt allowances stay out of it.

Who reads categories after the payslip

Categories exist during the computation; afterwards, each line remembers its rule, and the rule its categories. That is enough for everything downstream:

Who	Reads
Declarations and yearly certificates	the sum of a category over a period, across validated payslips: the taxable base of the quarter, the tax withheld in the year (Chapter 17)
Payroll reports	payslip lines grouped by rule or by category
The journal entry	each rule's accounts, not its categories (Chapter 16)
The payslip's gross and net fields	the lines with codes `BASIC`, `GROSS`, `NET`

> 💡 **In practice**
>
> If a declaration total is wrong while every payslip looks right, look for a rule in the wrong category. A declaration sums categories, so designing a country's categories is designing its declarations: start from the boxes of the forms.

How countries extend the chain

The rule of thumb: **one category per legal base**. Whatever amount the law computes something *on* (a contribution base, a taxable base, a pensionable salary) gets its own total, and each rule says which bases it belongs to. Countries keep the codes `BASIC`, `GROSS` and `NET`, because the payslip fields and reports read them, and add what their law needs between them.

> 🌍 **Around the world**
>
> - **Mexico** keeps the generic tree, adds child categories (`TAXABLE_ALW`, `IMSS_EMPLOYEE`) and a separate total for the social-security base, `SBC`, capped by its own rule.
> - **India** keeps the generic formulas: `GROSS = BASIC + ALW`, so there `GROSS` really is the gross salary.
> - **Belgium** uses a **running net**: one total (`REMUNERATION_BASE`) that every rule paying or withholding cash feeds, plus a contribution base and a taxable base. The net rule just reads the running net.
> - **United States** keeps pre-tax deductions in their own category, because they lower the taxable wages.

> ⚠️ **Common pitfall: GROSS does not mean the same everywhere**
>
> The `GROSS` category is labelled *Taxable Salary*. In India it is the gross; in Mexico, the taxable part of the earnings; in Belgium, the salary after employee social security (Mélanie: 3,477.20 €, not 4,000.00 €). Before comparing "gross" across countries, or building a report on it, open the `GROSS` rule and read its formula.

💡 **Review checklist**

- **Every legal base has a category**: list the bases in the law (contribution base, taxable base, pensionable pay…) and find each one.
- **Each rule feeds exactly the bases the law says**: for every earning, ask "is it subject to contributions? to tax? paid in cash?" and compare with its categories.
- **Is `GROSS` gross or taxable here?** Read the formula, and check the reports and declarations that use it agree.
- **Every paid time type has a category**, and no rule both reads and writes the same seeded total.
- **Employer contributions are outside the net** by category, whatever their position.

Summary

- A category is a named running total: rules add to it, later rules read it, and rules never name each other.
- The standard chain is `BASIC` → `ALW` → `GROSS` (taxable salary) → `DED` → `NET`; employer contributions (`COMP`) stay outside the net.
- Category decides which totals a rule changes; position decides which rules see it. A reimbursement after `GROSS` is paid but not taxed.
- Worked days add their amounts to their time types' categories before the first rule.
- One rule can feed several totals: Marie's car is in the taxable base, not in the running net.
- Parent categories roll up, so a country can refine the chain without changing the net formula.
- Declarations sum categories over a period; one category per legal base is the design rule.

Parameters, updates and safe extensions

Dated legal values, how Odoo's fixes reach every customer, what to do when data is not enough, and how to prove the result

AFTER THIS CHAPTER YOU CAN

- Explain a rule parameter and predict which value a payslip reads
- Decide where a value belongs: parameter, company setting, version field or rule
- Explain how country updates reach customers, and what happens to a record the customer edited
- Choose the lightest extension when data is not enough
- Prove a localization works with golden payslips frozen as tests

On 1 January the minimum wage, a tax scale and a contribution cap change. Nobody touches a rule. Odoo adds three dated values to the country's data, every customer database receives them within a week, and the first January payslip reads them, while a December payslip recomputed in February still reads December's law. This chapter shows how, and what to do when the law asks for more than a new number.

What a rule parameter is

A **rule parameter** is a code with a list of **dated values**: a rate, a cap, a tax scale, each valid from a date until the next value starts. A rule never contains the number itself; it asks for the parameter by its code, and the payslip gets the value valid on its own date. Next year's law is a **new value**, never an edit.

Three things follow:

- **History is kept.** Old values stay, so any past payslip recomputes with the law of its period (Chapter 11).
- **A value has a start date only.** It lasts until the next one; a temporary measure is two values, the measure and the return to normal.
- **A value can be any shape** the rule needs: a number, a list of brackets, a table per frequency. Keep the same shape every year, so the rule never needs to know the year.

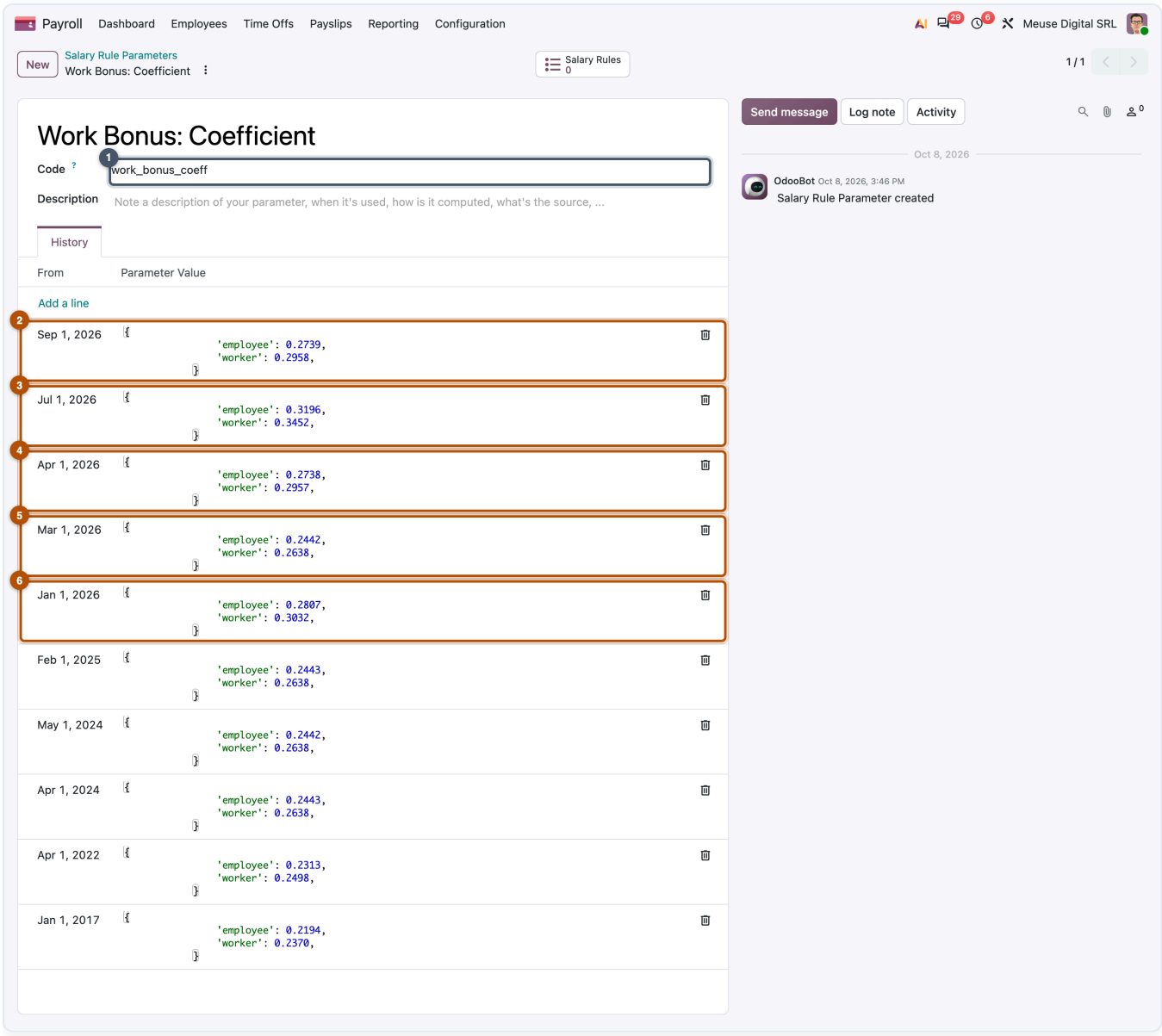


An illustrative parameter `minimum_wage` with five values. The June 2024 payslip asks for it:

```
payslip._rule_parameter('minimum_wage')     # 1300
```

The rule is simple: **the value with the latest start date on or before the payslip's end date**. For June 2024 that is the value of 1 January 2024. Note the 2023 value is lower than 2022's: values follow the law, not a trend, and nobody edits the old ones to "fix" them.

Belgium's cap on the flat-rate professional expenses works exactly like this: 5,930 € from 1 January 2025, 6,070 € from 1 January 2026. Mélanie's February 2026 payslip reads 6,070; a December 2025 payslip recomputed today still reads 5,930.

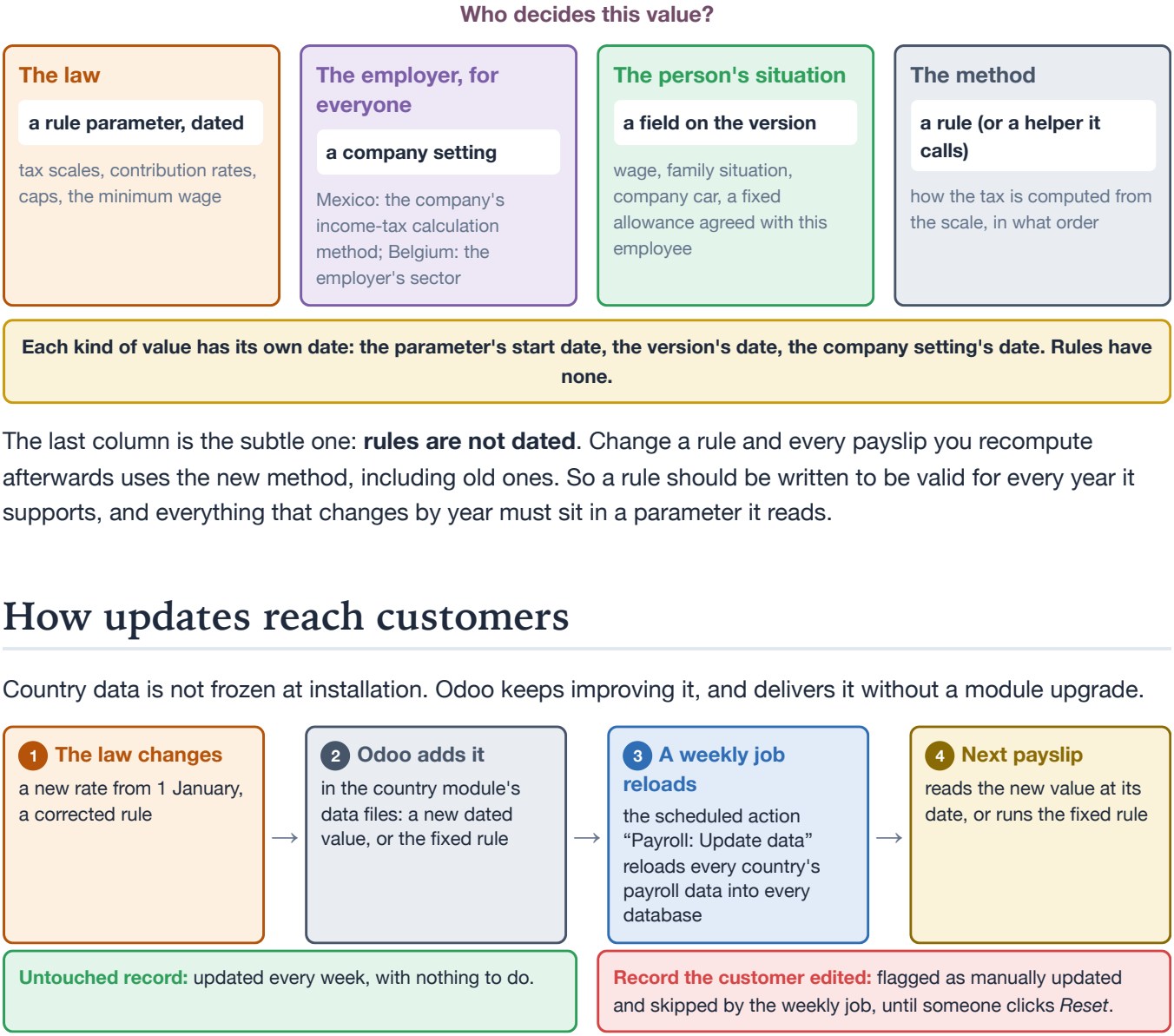


A Belgian parameter, Work Bonus: Coefficient. ① its code, which rules use to read it. ②–⑥ five values dated in 2026, newest first; older values stay below, so any past payslip recomputes with its own law.

> ⚠️ **Common pitfall: a period that straddles a date**
>
> One value applies to the whole payslip, read at its **end** date. A weekly payslip from 29 June to 5 July reads the July value for the whole week. When the law requires splitting, the rule must ask twice, with two reference dates, and prorate.

Where a value belongs

Not every number is a parameter. Ask who decides it, and for whom.

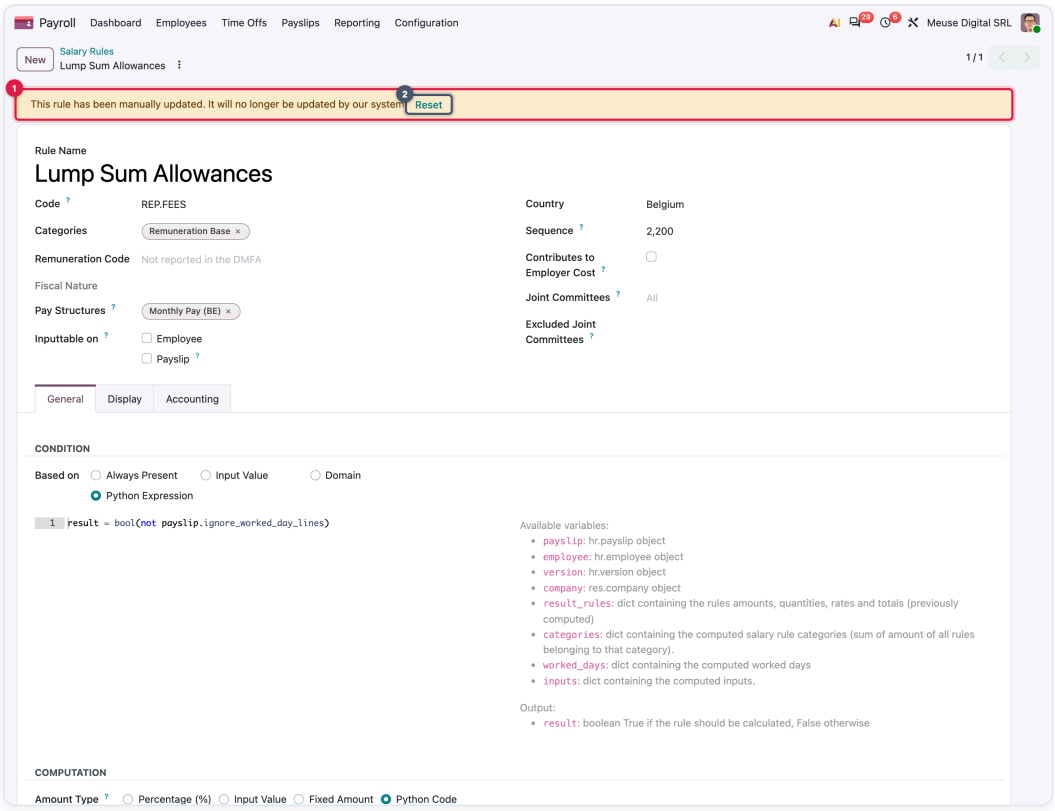


The last column is the subtle one: **rules are not dated**. Change a rule and every payslip you recompute afterwards uses the new method, including old ones. So a rule should be written to be valid for every year it supports, and everything that changes by year must sit in a parameter it reads.

How updates reach customers

Country data is not frozen at installation. Odoo keeps improving it, and delivers it without a module upgrade.

Why the protection? A customer may have a reason to change what a rule computes, and a weekly reload must not silently undo it. So when a user edits a shipped rule's computation (its code, condition, amount, categories, structures or sequence; not its label), or edits a shipped parameter, Odoo marks the record and stops updating it. Time types are protected the same way. Records the customer created from scratch are never touched by the update.



*A Belgian rule edited by a user. ① the banner: the rule will no longer be updated. ② **Reset** reloads the standard version, drops the manual changes and resumes the updates. On a parameter, Reset also removes the values users added.*

Worked example: a customer patches a cap

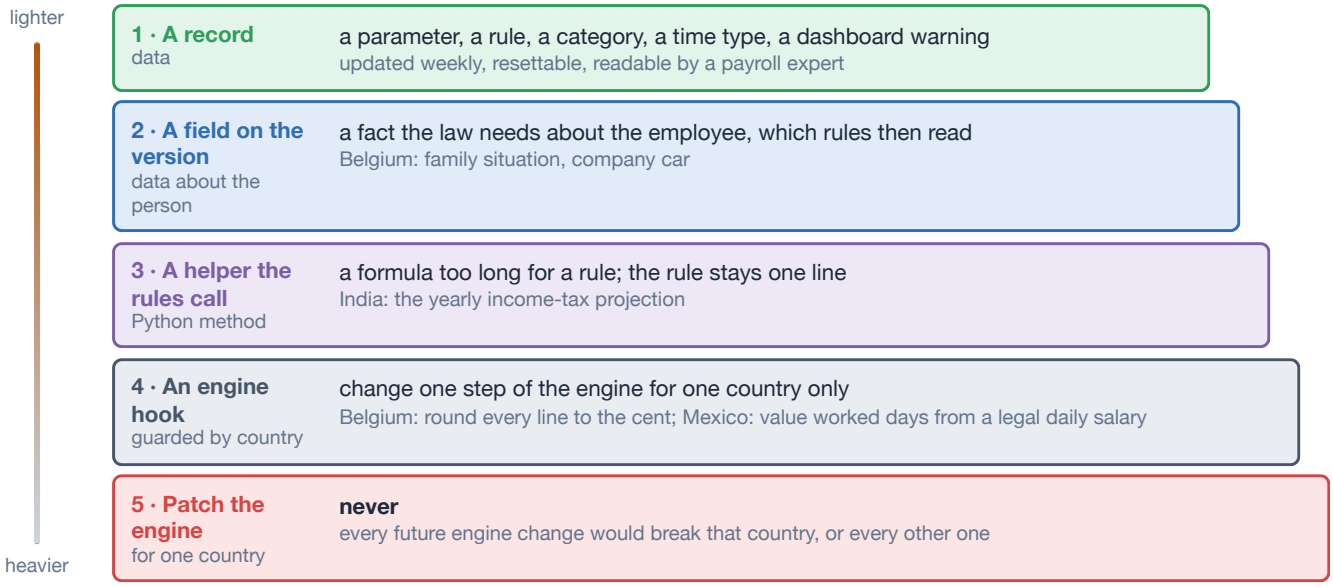
MyVidangeCompany's payroll officer thinks the 2026 professional-expenses cap is wrong and edits it "until Odoo fixes it". The parameter is now flagged. When Odoo ships the 2027 value, the weekly job **skips** this parameter: January 2027 payslips silently keep reading the 2026 cap. The right move was to report the value; once the record is flagged, **Reset** brings back Odoo's values and the updates.

In practice

In a review or a support case, search the rules and parameters flagged as manually updated. Each one is a place where the customer stopped receiving the law's changes. Ask why; if the reason is gone, reset it. If the customer needs a different computation for good, a rule of their own next to the standard one is easier to maintain than an edited standard rule.

When data is not enough

Most of a localization is data: parameters, rules, categories, time types, warnings. Sometimes the law needs more. Climb the ladder only as far as needed.



Each rung is harder to change and to review than the one above. A record can be changed by a payroll expert and is delivered by the weekly job; a helper or a hook needs a developer and a code release. Rung 4 always starts with a **country guard**, because one database can hold a Belgian and a Mexican company:

```
def _get_payslip_line_total(self, amount, quantity, rate, rule):
    total = super()._get_payslip_line_total(amount, quantity, rate, rule)
    if self.company_id.country_id.code != 'BE':
        return total
    return float_round(total, precision_rounding=0.01, rounding_method='HALF-UP')
```

Belgium's line rounding: other countries get the standard total untouched. Chapter 22 lists the few hooks a new localization actually needs.

> 🔑 **Key idea**
>
> Before writing Python, ask: "could a payroll officer need to change this next year?" If yes (a rate, a cap, a list of codes), it belongs in a record.

Proving it works

A payslip can be wrong by one cent for years before anyone notices. The proof is a set of **golden payslips** (Chapter 10): payslips computed independently of Odoo, from the law and the administration's examples, and frozen as automated tests. Each test sets up an employee, computes a payslip for a period, and compares every line with the expected amount:

```
def test_standard_month(self):
    payslip = self._generate_payslip(date(2026, 2, 1), date(2026, 2, 28))
    self._validate_payslip(payslip, {
        'BASIC': 4000.0, 'ONSS': -522.80, 'GROSS': 3477.20,
        'P.P': -826.69, 'NET': 2614.27,
    })
```

Illustrative, with Mélanie's February 2026 values. The tests run on every change to the module, so a fix for one case cannot silently break another.

Write a test…	Because	Case study
per situation the law distinguishes	each one takes a different path through the rules	Mélanie (standard), Jurgen (low wage), Marie (car, family)
per special time	absences, joiners and overtime hide bugs better than arithmetic	Yannick joins on 12 March; Jurgen's April
per extra structure	each kind of payslip has its own rules	Emma's exit payslips
on both sides of every date change	proves the new value starts on the right day, and the old one still applies before	Jurgen's low-wage threshold, March and April 2026
above every cap and in the top bracket	a cap that is never reached is never tested	Marie

Tests run on fixed dates, so a test of 2025 keeps checking the 2025 law forever. When you add next year's values, keep the old tests unchanged: if one fails, an old value was edited instead of a new one added. And break the module on purpose once (multiply a cap by ten): if no test fails, the suite has a hole.

⚠️ **Common pitfall: expected values copied from Odoo**

A test whose expected amounts were produced by the code it tests proves nothing about the law; it can only detect a change. The first expected value of each case must come from an independent calculation, with its source in a comment.

🌍 **Around the world**

- **Mexico** reads the UMA and the tax tables from dated parameters, with one table per pay frequency.
- **Belgium** downloads social-security rates every day into dated parameters, one per employer class and worker type.
- **Switzerland** imports tax tariffs from official files through a wizard.
- **United States** builds a parameter code per state for unemployment insurance rates, so a new state is a new parameter, not new code.

💡 **Review checklist**

- **New values are added, not edited.** Each yearly change is a new dated value; old values and old tests are untouched.
- **Every legal number is a parameter**, with a code prefixed by the country (`l10n_xx_…`): codes are shared by all countries.
- **Customer edits are known**: rules and parameters flagged as manually updated are listed and justified.
- **Code is guarded by country**, and sits on the lightest rung of the ladder that works.
- **A test per new case**: each new situation, structure or dated value comes with a golden payslip, on both sides of its date.

Summary

- A rule parameter is a code with dated values; a payslip reads the value with the latest start date on or before its end date. Next year's law is a new value, never an edit.
- Law goes in parameters, the employer's choices in company settings, the person's facts on the version, the method in rules. Rules are not dated.
- A weekly job reloads every country's payroll data, so new values and fixes arrive without an upgrade; a record the customer edited is flagged and skipped until Reset.
- When data is not enough, climb the ladder: record, version field, helper, country-guarded hook; never patch the engine for one country.
- Golden payslips frozen as tests are the proof: one per legal situation, per structure, on both sides of every date, above every cap.

PART V

Building a localization

The pieces every payroll localization ships, as a skeleton and code templates, and a review checklist for any country.

A template for any localization, and how to review one

The few pieces every country module needs, why each one exists, how it stays correct year after year, and a checklist to review it

AFTER THIS CHAPTER YOU CAN

- Say what a payroll localization is and name its five groups of pieces
- Write the data of a country module (structure, categories, rules, dated parameters) and explain why each field is written that way
- Add the Python a country needs (wiring hooks, version fields, guarded overrides) and only that
- Produce the outputs (declarations, accounting) and prove the module with golden-payslip tests
- Run the yearly upkeep, and review a localization with a graded checklist

A company in a new country wants Odoo Payroll. The engine is ready: Part IV showed that it runs any rules it is given. What is missing is the country: its rules, its rates, its forms. This chapter is the template for that country module, piece by piece. For each piece it says **why** it exists (what breaks without it), **what** the code does, and **when** you need it. It ends with a review checklist, because a payroll bug never crashes: it pays someone a little wrong every month until the year-end reconciliation finds it.

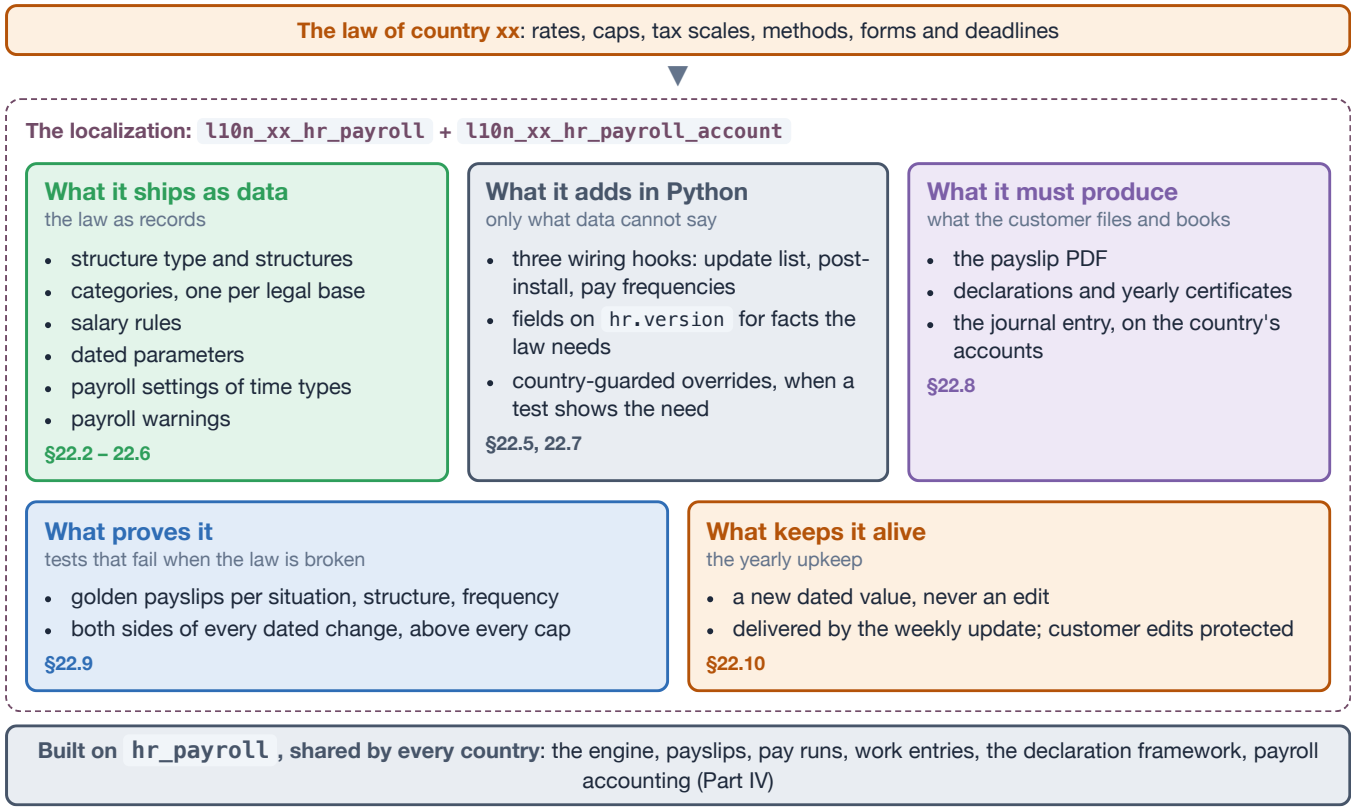
Every snippet uses the country code `xx` and follows a real module: the Dutch one (`l10n_nl_hr_payroll`, almost only data), the Mexican one (`l10n_mx_hr_payroll`, several engine hooks) and the Belgian one (`l10n_be_hr_payroll`, the largest).

22.1 What a localization module is

A **payroll localization** (also *country module*, *l10n module*) is the country layer of Chapter 9: one or two Odoo modules, `l10n_xx_hr_payroll` and `l10n_xx_hr_payroll_account`, that turn one country's payroll law into records and a little Python on top of the shared engine. The engine, payslips, pay runs, work entries and the declaration framework stay in `hr_payroll`; the localization brings only what the country knows.

Three things follow:

- **It is mostly data.** Rules, categories and dated parameters (Part IV) carry the law. Python is for what data cannot say, and stays small.
- **It must stay correct every year.** The law changes every January, sometimes more often; the module is built so that a change is a new record that reaches every customer within a week.
- **It must be provable.** Nobody can check forty rules by reading them; golden payslips frozen as tests do.



Read the figure from the top: the law comes in, the module turns it into five groups of pieces, and everything stands on the shared framework, which a localization never patches (Chapter 21). The three boxes in the upper row are what the customer *uses*; the two below are what keeps it *right*. Section 22.11 reviews a module along the same groups.

22.2 The skeleton: two modules and a manifest

A localization is two modules, not one, because payroll and accounting are separate purchases. Some customers run payroll in Odoo and send the totals to an outside accountant; and the accounts a rule posts to belong to the country's chart of accounts (`l10n_xx`), which the payroll module must not force on anyone.

l10n_xx_hr_payroll

The law. Depends only on `hr_payroll` and `hr_holidays` , so it installs in any database with payroll.

DATA/	categories, structure type, structures, dated parameters, rules, time-type settings
MODELS/	the wiring hooks, fields on the version, declarations, overrides
VIEWS/	the payslip PDF, version fields on the employee form, declaration screens
SECURITY/	access rights for any model the module adds

l10n_xx_hr_payroll_account

The bridge. Depends on `hr_payroll_account` , the chart `l10n_xx` and the module on the left; installs itself when both sides are present.

MODELS/	which account each rule posts to
DATA/	loads that mapping for existing companies; marks the net as the employee's debt
TESTS/	the golden payslips: the test base needs a company with a chart
OTHER	any e-filing that needs accounting (Mexico's electronic payslip)

The name is not cosmetic. *Reset* on a rule or a parameter reloads only the modules whose name starts with `l10n_<country code>` , and the structure's payslip-report field offers only reports whose name contains it. A module called `xx_payroll` installs fine, but *Reset* never restores its rules.

The manifest of the main module, following the Dutch one:

```
{
    'name': 'Xxland - Payroll',
    'countries': ['xx'],
    'category': 'Human Resources/Payroll',
    'depends': ['hr_payroll', 'hr_holidays'],                # ①
    'auto_install': ['hr_payroll'],                          # ②
    'data': [                                                # ③
        'data/hr_salary_rule_category_data.xml',
        'data/hr_payroll_structure_type_data.xml',
        'data/hr_work_entry_type_data.xml',
        'views/hr_payroll_report.xml',
        'data/hr_payroll_structure_data.xml',
        'data/hr_rule_parameters_data.xml',
        'data/hr_salary_rule_data.xml',
        'views/report_payslip_templates.xml',
        'views/hr_employee_views.xml',
    ],
    'demo': ['data/l10n_xx_hr_payroll_demo.xml'],
    'post_init_hook': '_l10n_xx_hr_payroll_post_install',    # ④
    'author': 'Odoo S.A.',
    'license': 'OEEL-1',
}
```

1. **No accounting, no chart.** `hr_holidays` is there because time off produces the work entries the payslip pays. Add a dependency here only if a rule or a model really needs it.
2. **Installs itself.** With `countries`, Odoo installs it with payroll only when a company of country xx exists. Without it, a customer must know the module exists.
3. **Load order.** Odoo loads files top to bottom, and a reference to a record not yet loaded fails the install. So: categories before the rules that use them, the structure type before the structure, the report action (`hr_payroll_report.xml`) before the structure that names it, the structure before its rules.
4. **Post-install hook.** Switches existing data to the country's time types (§22.5).

WHEN Always. The account module's manifest is the same shape: `depends` on the three modules above, `auto_install` on the main module and `hr_payroll_account` .

22.3 The pay structure and the categories

The first data file gives the payslips a home: a structure type for the employees' versions, and a structure listing the rules. These are the Dutch records, renamed:

```
<record id="structure_type_employee_xx" model="hr.payroll.structure.type">   <!-- ① -->
    <field name="name">Xxland: Employee</field>
    <field name="country_id" ref="base.xx"/>
</record>

<record id="hr_payroll_structure_xx_employee_salary" model="hr.payroll.structure">  <!-- ② -->
    <field name="name">Xxland: Regular Pay</field>
    <field name="code">XXMONTHLY</field>
    <field name="country_id" ref="base.xx"/>
    <field name="type_id" ref="structure_type_employee_xx"/>
    <field name="report_id" ref="action_report_payslip_xx"/>
    <field name="rule_ids" eval="[]"/>                                        <!-- ③ -->
</record>
<record id="structure_type_employee_xx" model="hr.payroll.structure.type">   <!-- ④ -->
    <field name="default_struct_id" ref="hr_payroll_structure_xx_employee_salary"/>
    <field name="default_work_entry_type_id" ref="hr_work_entry.xx_work_entry_type_attendance"/>
</record>
```

1. **The structure type** is what HR picks on each employee's version (Chapter 19). Its country hides it from companies of other countries.
2. **The structure** is the list of rules for the regular payslip, with the country's payslip PDF.
3. `rule_ids eval="[]"` : a new structure otherwise receives a copy of the framework's generic rules (basic, gross, net), which would then sit next to the country's own and duplicate their lines.
4. **A second write on the type.** The type must name its default structure, and the structure needs its type: neither can be created first with both links. So the type is created bare, then completed. It also names the time type used for normal work (Chapter 14).

WHEN Always. Add one structure per kind of payslip the law taxes or declares differently (a 13th month, a termination); a bonus paid without worked days sets `use_worked_day_lines` to `False` .

Then the **categories**, the running totals of Chapter 20. Design them from the declarations: each total a form asks for, and each base the law computes something on, gets a category.

```
<record id="hr_salary_rule_category_social_security_employee" model="hr.salary.rule.category">
    <field name="name">Employee Social Security Deductions</field>
    <field name="code">SOCIALSECURITYEMPLOYEE</field>
    <field name="parent_id" ref="hr_payroll.DED"/>                    <!-- ① -->
    <field name="country_id" ref="base.xx"/>
</record>
```

1. **Parent `DED` .** Everything added here also rolls up into the deductions, so the generic net formula sees it unchanged (Chapter 20). The declaration reads the child category for its "employee social security" box.

WHEN One per legal base or declaration box. Most countries keep the generic chain and add children (Netherlands, Mexico); a country where many items enter and leave the net at different points uses a running net instead (Belgium, Chapter 20).

Last, the **time types**. They are created for each country in the community module `hr_work_entry` , so attendance and time off work without payroll. The payroll module only says which total each one feeds:

```
<record id="hr_work_entry.xx_work_entry_type_attendance" model="hr.work.entry.type">
    <field name="category_ids" eval="[(6, 0, [ref('hr_payroll.BASIC')])]"/>
</record>
```

Without this line, the worked-days amount is on the payslip but never reaches `BASIC` , so the gross and the net miss it (the pitfall of Chapter 20). Write one such record for every paid time type of the country.

22.4 Rules

A rule is a record with five parts (Chapter 19). In a localization it is written in XML, and *how* it is written decides whether Odoo can keep it correct later. Here is the Dutch old-age contribution, simplified and renamed:

```
<record id="l10n_xx_old_age_insurance" model="hr.salary.rule">
    <field name="name">Old Age Insurance</field>
    <field name="code">XX_OLD_AGE</field>                                    <!-- ① -->
    <field name="sequence">105</field>                                       <!-- ② -->
    <!-- ③ -->
    <field name="category_ids" eval="[(6, 0,
[ref('hr_salary_rule_category_social_security_employee')])]"/>
    <field name="condition_select">none</field>
    <field name="amount_select">code</field>
    <field name="amount_python_compute">                                     <!-- ④ -->
rates = payslip._rule_parameter('l10n_xx_social_security_rates_employee')['OLD_AGE']
result_rate = -rates[0]
result = min(categories['GROSS'], rates[1] / 12)
    </field>
    <!-- ⑤ -->
    <field name="struct_ids" eval="[(6, 0, [ref('hr_payroll_structure_xx_employee_salary')])]"/>
    <field name="country_id" ref="base.xx"/>
</record>
```

1. **The code** is how later rules (`result_rules['XX_OLD_AGE']`), tests and declarations find this line. Choose it once; renaming it breaks all three.
2. **Sequence 105**: after `GROSS` (100), which the formula reads; a rule placed before it would read 0 without any error (Chapter 19). Leave gaps: next year's new contribution fits in at 107 without renumbering.
3. **`(6, 0, [...])` means "exactly this list".** The weekly update rewrites only what the XML says. Written as "add this category" (`(4, id)`), a category you later remove from the XML would stay in every customer database. Same reason to write the condition, the amount, the structures and the country every time, even when they look like defaults.
4. **The formula.** Line 1 reads the rate and the yearly cap from a dated parameter (§22.5). Line 2 sets the rate, negative because a deduction lowers the net. Line 3 sets the amount: the taxable salary, capped at a twelfth of the yearly cap. The payslip line then shows base × rate, which anyone can check. No number is written in the code: a literal can only change with a code release, and since rules are not dated, changing it also changes every old payslip you recompute (Chapter 21).
5. **Structures.** The rule runs only on these. A contribution due on the 13th month too must list that structure as well.

WHEN One record per line the law defines on a payslip.

Three habits keep rules reviewable:

- **A long formula goes to a helper.** Past about ten lines, move the computation to a method on `hr.payslip` and call it from the rule (`result = payslip._get_xx_income_tax()`); a method can be tested alone and read with an editor.
- **Never rely on leftovers.** All rules of a payslip share one dictionary, so a variable set in one rule is still there in the next. Read earlier results through `result_rules` or `categories` instead, and never name a variable like a helper (`date`).
- **Every total a rule reads is fed by an earlier rule.** A misspelt or empty category reads 0, silently.

22.5 Dated parameters and the three wiring hooks

The rule above asked for `l10n_xx_social_security_rates_employee`. Here it is, in the Dutch shape (the values are placeholders):

```xml
<record id="rule_parameter_social_security_rates_employee" model="hr.rule.parameter">
    <field name="name">Xxland: Employee Social Security Rates and Thresholds</field>
    <field name="code">l10n_xx_social_security_rates_employee</field>     <!-- ① -->
    <field name="country_id" ref="base.xx"/>
</record>
<!-- {code: [rate, yearly cap]} -->                                       <!-- ② -->
<record id="rule_parameter_social_security_rates_employee_2025" model="hr.rule.parameter.value">
    <field name="parameter_value">{
    'OLD_AGE': [17.9, 35129.00],
    'SURVIVOR': [0.1, 35129.00],
}</field>
    <field name="rule_parameter_id" ref="rule_parameter_social_security_rates_employee"/>
    <field name="date_from" eval="datetime(2025, 1, 1).date()"/>          <!-- ③ -->
</record>
```

1. **The parameter** is only a name and a code; rules ask for the code. Prefix it with `l10n_xx_` : one database holds every installed country's parameters, and two countries would otherwise collide on `social_security_rates` .
2. **The shape** (here, a rate and a cap per contribution) is chosen once and kept every year, so the rule never needs to know the year. The comment documents it for the next developer.
3. **The value's start date** is the date the law takes effect. It lasts until the next value; the record's id ends with the year, so next year is a new record (§22.10).

WHEN For every number, scale or table the law sets. How a payslip picks its value is in Chapter 21.

Records are not enough on their own. Three small hooks connect them to the rest of Odoo; every localization has all three.

Hook 1: the update list. It tells the weekly job which files to reload:

```python
class HrPayslip(models.Model):
    _inherit = 'hr.payslip'

    def _get_data_files_to_update(self):                                # ①
        # Note: file order should be maintained
        return super()._get_data_files_to_update() + [(
            'l10n_xx_hr_payroll', [                                      # ②
                'data/hr_salary_rule_category_data.xml',
                'data/hr_payroll_structure_type_data.xml',
                'data/hr_payroll_structure_data.xml',
                'data/hr_rule_parameters_data.xml',
                'data/hr_salary_rule_data.xml',
            ])]


class HrWorkEntryType(models.Model):
    _inherit = 'hr.work.entry.type'

    def _get_data_files_to_update(self):                                # ③
        return super()._get_data_files_to_update() + [(
            'l10n_xx_hr_payroll', ['data/hr_work_entry_type_data.xml'])]
```

1. **Why.** The scheduled action Payroll: Update data reloads, every week and in every database, exactly the files listed here. A file not in the list is loaded at installation and never again: a fixed rate in it reaches new customers only.
2. **Same order as the manifest**, for the same reason: a file can only refer to records loaded before it.
3. **Time types have their own list**, reloaded in a separate step, after the community time types are created, so the references in the file always resolve.

Hook 2: the post-install hook, in the module's `__init__.py` :

```python
def _l10n_xx_hr_payroll_post_install(env):
    env.ref('base.xx')._adapt_work_entry_types_to_country()
```

A database that already used Odoo before installing payroll has schedules, time off, allocations and structure types pointing at the *generic* time types. This call switches those of the country's companies to the country's own time types with the same code. Without it, existing employees keep generic types that carry none of the country's payroll settings.

Hook 3: the pay frequencies the law allows:

```
class HrPayrollStructureType(models.Model):
    _inherit = 'hr.payroll.structure.type'

    def _get_selection_schedule_pay(self):
        if self.env.company.country_code == 'XX':
            return [('monthly', self.env._('month'))]
        return super()._get_selection_schedule_pay()
```

The structure type's *Scheduled Pay* field offers only these choices, so nobody runs a weekly payroll where the law and the rules only know months. The test is on the current company's country; other countries fall through to `super()`. Mexico lists seven frequencies, including 10 and 14 days, and also overrides `_schedule_timedelta` on `hr.payslip` to tell the engine where such a period ends.

22.6 Payroll warnings

A payslip computed from incomplete data is wrong without any error: an employee with no sector gets no sector rules, an employee with no bank account cannot be paid, a missing national number gets the declaration rejected months later. A **payroll warning** catches that earlier. It is a data record (`hr.payroll.warning`) holding a check, either a domain or a few lines of Python. Every record that matches it appears on a card of the payroll dashboard, with a button to open the list; a warning can also **block** the validation of those employees' payslips.

The framework ships the generic ones (*Missing Bank Account*); a localization adds the checks its law needs. Belgium's meal-voucher check:

```
<record id="hr_payroll_warning_incorrect_configuration_meal_voucher" model="hr.payroll.warning">
    <field name="name">Incorrect Meal Voucher Configuration</field>                <!-- ① -->
    <field name="description">Its value cannot exceed €10 and must be greater than the employee's
share, which cannot be less than €1.09.</field>
    <field name="country_id" ref="base.be"/>                                       <!-- ② -->
    <field name="warning_color_class">warning</field>
    <field name="button_name">Review Employees</field>
    <field name="block_payslips">True</field>                                      <!-- ③ -->
    <field name="warning_type">domain</field>                                      <!-- ④ -->
    <field name="model_id" ref="hr.model_hr_employee"/>
    <field name="warning_domain">[('is_meal_voucher_valid', '=', False)]</field>
</record>
```

1. **Title and description**, as shown on the dashboard card. Say what is wrong and what the law requires, so the payroll officer can fix it without asking.
2. **Country.** Only companies of this country see it. The generic warnings leave it empty.
3. **Blocking.** Validating a payslip of a matching employee fails with the warning's name. Without it, the card only informs.
4. **The check**: a domain on a model; every matching record is listed. When a domain cannot express it, use `warning_type` `python` and an `evaluation_code` that sets `warning_records` (Belgium's invalid national numbers).

WHEN One per piece of data the rules or the declarations cannot do without. Make it **BLOCKING** only when every payslip of the employee would be wrong or unpayable, and the fix is quick (a voucher amount out of the legal range). Data that matters for a later filing (Belgium's *Employees without a Joint Committee*) stays a plain card: blocking it would stop the whole pay run for something that can be fixed before the declaration.

22.7 Python: fields on the version and guarded overrides

Python enters only when data cannot express the law. The two most common reasons are a **fact about the person** the standard version lacks, and an **engine step** that works differently in the country (the ladder of Chapter 21).

A fact about the person goes on the version. The Dutch 30 % ruling for skilled migrants, renamed:

```python
class HrVersion(models.Model):
    _inherit = 'hr.version'

    l10n_xx_30_percent = fields.Boolean(                                    # ①
        string="30% Exemption", groups="hr_payroll.group_hr_payroll_user", tracking=1)

    @api.model
    def _get_whitelist_fields_from_template(self):                          # ②
        whitelisted_fields = super()._get_whitelist_fields_from_template() or []
        if self.env.company.country_id.code == "XX":
            whitelisted_fields += ["l10n_xx_30_percent"]
        return whitelisted_fields


class HrEmployee(models.Model):
    _inherit = 'hr.employee'

    l10n_xx_30_percent = fields.Boolean(                                    # ③
        readonly=False, related="version_id.l10n_xx_30_percent", inherited=True,
        groups="hr_payroll.group_hr_payroll_user")
```

1. **On the version, not the employee**, because the fact can change and versions are dated (Chapter 13): a payslip of last year reads last year's situation. `groups` hides it from users without payroll rights; `tracking` logs every change for audits.
2. **Whitelist.** Contract templates copy only whitelisted fields into new versions; the country test keeps other countries' templates unchanged.
3. **Mirror on the employee**, so HR edits it on the employee form like any standard version field. The view shows it only when `country_code` is the country's.

WHEN Only when a rule needs a fact the version lacks. If a rule reads the field, also add it to `_get_fields_that_recompute_payslip`, or draft payslips keep the old amount until someone clicks *Compute*. A list with its own life (Mexico's housing-loan deductions) becomes its own model, with access rights and a multi-company rule.

An **engine override** changes one step for one country. Mexico values a worked-days line from a legal daily salary instead of the monthly wage:

```python
def _compute_amount(self):
    mx_worked_days = self.filtered(
        lambda wd: wd.payslip_id.struct_id.country_id.code == "MX")     # ①
    for worked_days in mx_worked_days:
        ...   # value the line from the Mexican daily salary           # ②
    return super(HrPayslipWorkedDays, self - mx_worked_days)._compute_amount()  # ③
```

1. **The country guard.** One database can hold a Mexican and a Belgian company. Keep only the records of your country, here through the payslip's structure.
2. **Your country's logic**, on those records only.
3. **Everything else goes to `super()`** : other countries get the standard behaviour, and other modules' overrides of the same method still run. Forget the guard and Belgian payslips are valued the Mexican way; forget `super()` and every other country's worked days stay at zero.

Which hook to override depends on what the law asks. These are the ones localizations use, all on `hr.payslip` unless noted:

THE LAW SAYS...	OVERRIDE
pay periods the core lacks (10 or 14 days)	→ `_schedule_timedelta` — Mexico: where a 10-day period ends
a day of work is worth a legal daily rate	→ `hr.payslip.worked_days._compute_amount` — Mexico: the daily salary; Belgium: an hourly rate
some rules apply only in some sectors	→ `hr.salary.rule._satisfy_condition` — Belgium: a filter in front of every condition
every line is rounded to the cent	→ `_get_payslip_line_total` — Belgium: half-up rounding (code in Chapter 21)
rules need the year's earlier payslips	→ `_get_common_localdict` — Belgium: loaded once for the whole batch, not once per payslip
an amount appears without anyone typing it	→ `_compute_input_line_ids` — Belgium: recovery of a salary advance

A new localization usually starts with none of them. Add one when a golden payslip cannot be matched with data, and never override `_get_payslip_lines`, the engine loop itself.

22.8 Declarations and the accounting bridge

A module the customer cannot file with is not finished. Most countries require at least a **yearly certificate per employee** (what was paid and withheld). The framework gives a mixin with the year, one line per employee, background PDF generation and the screens; a localization fills in four methods. Following the Kenyan certificate:

```python
class L10nXxAnnualCertificate(models.Model):
    _name = 'l10n_xx.annual.certificate'
    _inherit = ['hr.payroll.declaration.mixin']
    _description = 'Annual Salary Certificate'

    def _country_restriction(self):                                  # ①
        return 'XX'

    def action_generate_declarations(self):                          # ②
        for sheet in self:
            payslips = self.env['hr.payslip'].search([...])  # validated, this year
            sheet.line_ids = [(5, 0, 0)] + [(0, 0, {
                'employee_id': employee.id, 'res_model': self._name, 'res_id': sheet.id,
            }) for employee in payslips.employee_id]
        return super().action_generate_declarations()

    def _get_rendering_data(self, employees):                        # ③
        ...   # {employee: {...}} from payslips._get_line_values(['GROSS', 'XX_TAX'])

    def _get_pdf_report(self):                                       # ④
        return self.env.ref('l10n_xx_hr_payroll.action_report_annual_certificate')
```

1. **Only companies of the country** can open the screen.
2. **Who gets a certificate**: one line per employee with a validated payslip in the year. The mixin's `super()` refuses an empty declaration.
3. **The numbers**, read from payslip lines by code or category, never recomputed. If the certificate is wrong, the payslips are (Chapter 18).
4. **The PDF** printed for each employee.

WHEN For each per-employee form the law requires. Monthly returns and electronic files are their own models with an export (Chapter 17).

Two habits from the larger modules apply to every filing. **Map payslips to boxes with data**: Mexico stores the official concept code on each rule, so a new allowance needs a rule, not a change to the export. And **collect every validation error before raising**, so the payroll officer fixes ten missing tax numbers at once rather than one per try.

The **accounting bridge** says which account each rule posts to. The core finds a method named after the chart, `_configure_payroll_account_<chart code>`, and calls it whenever a company loads that chart. Following the Mexican one:

```python
def _configure_payroll_account_xx(self, companies):
    wages = 'xx_account_wages'                    # ① xml ids in the l10n_xx chart
    wages_payable = 'xx_account_wages_payable'
    rules_mapping = defaultdict(dict)
    basic = self.env.ref('l10n_xx_hr_payroll.l10n_xx_regular_pay_basic')
    rules_mapping[basic]['debit'] = wages         # ②
    net = self.env.ref('l10n_xx_hr_payroll.l10n_xx_regular_pay_net')
    rules_mapping[net]['credit'] = wages_payable
    self._configure_payroll_account(             # ③
        companies, "XX", account_refs=[wages, wages_payable],
        rules_mapping=rules_mapping, default_account=wages_payable)
```

1. **Accounts by xml id** of the country's chart, so the mapping works in every company whatever the account numbers.
2. **One debit or credit per rule**: the cost side of earnings, the liability side of deductions and of the net (Chapter 16).
3. **The core helper** sets the payroll journal on the country's structures and writes the accounts on each rule, company by company. An account it cannot find only logs a warning: the rule stays without an account and the entry gets an Adjustment Entry line.

The account module also ships two small data files, both `noupdate` because accounts are the company's choice and the weekly reload must not overwrite them:

```
<data noupdate="1">                                                    <!-- ① -->
    <function model="account.chart.template" name="_load_payroll_accounts">
        <value eval="[]"/>
        <value>xx</value>
        <value model="res.company" eval="obj().search([('chart_template', '=', 'xx')])"/>
    </function>
</data>

<odoo noupdate="1">                                                    <!-- ② -->
    <record id="l10n_xx_hr_payroll.hr_payroll_rule_xx_net" model="hr.salary.rule">
        <field name="employee_move_line" eval="True"/>
    </record>
</odoo>
```

1. **Companies that already have the chart** get the mapping at install. Companies created later get it when their chart loads.
2. **The net is the employee's debt.** Its journal item carries the employee as partner, so each salary payment is reconciled per person (Chapter 16).

> ⚠️ **Common pitfall: rules activated after the chart**
>
> A mapping that finds its rules with a `search` skips archived ones. Belgium installs many rules archived and activates them when the employer's sector is chosen; if the chart was loaded first, those rules have no accounts and the first entry shows a large *Adjustment Entry*. The fix is to run `_load_payroll_accounts` again for the company once the rules are active, not to type accounts rule by rule.

22.9 Tests

Chapter 21 explains golden payslips and which situations to cover. This is the test file that holds them, in the account module, following the Dutch one:

```
@tagged('post_install_l10n', 'post_install', '-at_install', 'payslips_validation')  # ①
class TestPayslipValidation(TestPayslipValidationCommon):

    @classmethod
    @TestPayslipValidationCommon.setup_country('xx')                    # ②
    def setUpClass(cls):
        super().setUpClass()
        cls._setup_common(                                              # ③
            country=cls.env.ref('base.xx'),
            structure=cls.env.ref('l10n_xx_hr_payroll.hr_payroll_structure_xx_employee_salary'),
            structure_type=cls.env.ref('l10n_xx_hr_payroll.structure_type_employee_xx'),
            version_fields={'wage': 5000},
        )

    def test_regular_payslip(self):
        payslip = self._generate_payslip(date(2025, 1, 1), date(2025, 1, 31))
        self._validate_worked_days(payslip, {'WORK100': (23.0, 184.0, 5000.0)})   # ④
        self._validate_payslip(payslip, {                               # ⑤
            'BASIC': 5000.0, 'GROSS': 5000.0,
            'XX_OLD_AGE': -524.01,    # capped: 35,129 / 12 × 17.9 %
            # ... every other line, from the independent calculation
        })
```

1. **Tags.** The test runs after every localization is installed, as in a real database with several countries, and can be run alone with the `payslips_validation` tag.
2. **A company of the country with its chart of accounts**. That is why the test lives in the account module.
3. **One employee, one version, one structure.** `version_fields` is the situation under test; other tests change it (`self.version.l10n_xx_30_percent = True`).
4. **Time first**: days, hours and amount per time type. If these are wrong, no rule can be right.
5. **Every line, with its source.** A line the dict does not list also fails the test, so a rule that suddenly applies is caught.

When a test fails, `_validate_payslip` prints a table with one message per code. Each points to a different place:

WRONG CALCULATION: the line exists, its total differs. Check the parameter value for the period, the rounding, the sequence.

MISSING LINE: a line the test does not expect appeared. A condition became true.

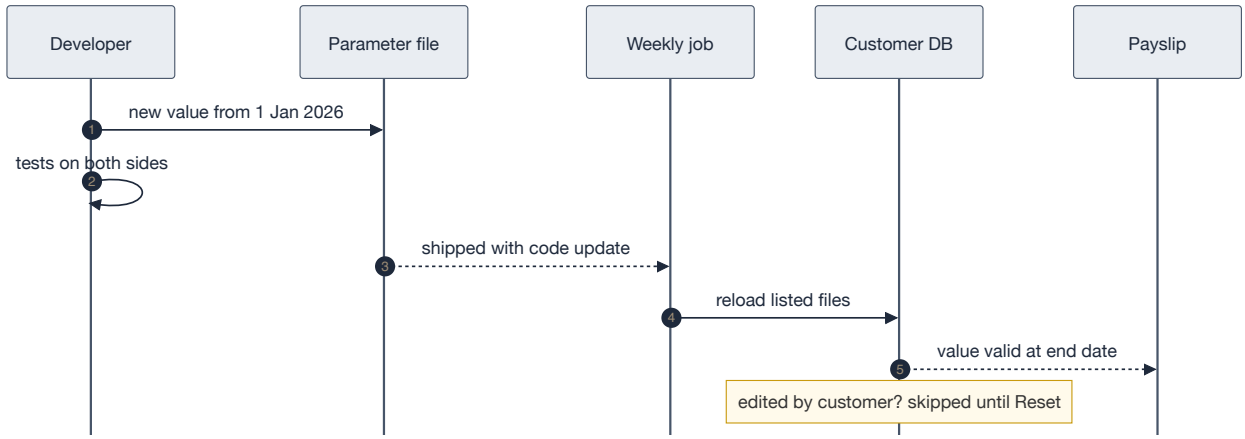
RULE NOT COMPUTED: an expected line is absent. Check the condition, the structures, an input code, an activation switch.

For payslips with dozens of lines, `_validate_payslip(payslip)` without a dict compares with a JSON snapshot in `tests/test_files/payslips/`. Review the snapshot's diff in the pull request as carefully as code: it *is* the expected result.

> 💡 **In practice: build in this order**
>
> Each step is a pull request that runs and can be reviewed alone. 1 · The country's time types in community. 2 · A skeleton that installs with no warning. 3 · Categories and parameters. 4 · Rules, with golden tests for a low and a high income. 5 · Version fields and helpers. 6 · Declarations. 7 · The accounting bridge: a pay run posts a balanced entry with no *Adjustment Entry* line.

22.10 The yearly upkeep

Every year the law publishes new values, usually in December for 1 January: a new tax scale, a higher cap, a new minimum wage. The design of Part IV turns that into a data job. Suppose country xx raises the old-age cap of §22.5 from 35,129 to 36,000 (placeholder values) on 1 January 2026:



Step by step:

1. **Add, never edit.** The developer adds a value next to the 2025 one. Editing the 2025 record instead would rewrite every 2025 payslip recomputed later, and break the 2025 tests.
2. **Test both sides of the date** (snippet below): December still reads the old cap, January the new one.
3. **Merged and shipped** in the supported versions. No module upgrade is needed on the customer side.
4. **The weekly job** Payroll: Update data reloads the listed files into every database: the new record is created, untouched records are refreshed.
5. **The next payslip** ending on or after 1 January reads 36,000; a December payslip recomputed later still reads 35,129.

! **Customer edits are protected** (the note in the diagram). If the customer edited this parameter, the job skips it and the new value never arrives, until someone clicks Reset (Chapter 21). When a customer reports that "the new rate did not come", look there first.

Steps 1 and 2 are the developer's whole job:

```
<record id="rule_parameter_social_security_rates_employee_2026" model="hr.rule.parameter.value">
    <field name="parameter_value">{
    'OLD_AGE': [17.9, 36000.00],
    'SURVIVOR': [0.1, 36000.00],
}</field>
    <field name="rule_parameter_id" ref="rule_parameter_social_security_rates_employee"/>
    <field name="date_from" eval="datetime(2026, 1, 1).date()"/>
</record>
```

```
def test_old_age_cap_both_sides(self):
    december = self._generate_payslip(date(2025, 12, 1), date(2025, 12, 31))
    self._validate_payslip(december, {'XX_OLD_AGE': -524.01}, skip_lines=True)  # 35,129 / 12 × 17.9 %
    january = self._generate_payslip(date(2026, 1, 1), date(2026, 1, 31))
    self._validate_payslip(january, {'XX_OLD_AGE': -537.00}, skip_lines=True)   # 36,000 / 12 × 17.9 %
```

Same shape, new id, new date: the rule does not change. The test uses a wage above both caps, otherwise the cap is never reached and the test passes even if the new value is wrong; `skip_lines=True` checks only the line the change affects. Belgium's professional-expenses cap is the same pattern in a real module: a `…_2025` record at 5,930 and a `…_2026` record at 6,070, side by side.

Not every change is a new value. Three kinds come back every year:

A new value

a rate, a cap, a tax scale, a minimum wage

YOU DO add a dated record with a new id; same shape

ARRIVES with the weekly job

TEST one payslip before the date, one after

A rule fix

the method was wrong, or the law changed the method

YOU DO edit the rule's XML; if old periods keep the old method, switch on a parameter, since rules are not dated

ARRIVES with the weekly job

TEST the golden payslip that showed the bug; old tests unchanged

A new filing format

the administration changes a file or a form

YOU DO change the export code, keyed on the period it applies to

ARRIVES with the code update; no data

TEST an exact-file or snapshot test

The upkeep also includes the check of Chapter 11: once a year, compare each value with its official source, line by line.

22.11 How to review a localization

Review in a fixed order, the order of the cards below: **law first**, because a perfectly coded wrong rate is still wrong; **screens and filings last**, but never skipped. Grade each finding:

BLOCKER wrong pay, a wrong filing, or a module that cannot be used. Request changes.

MAJOR wrong in a realistic case, or a maintenance trap. Fix now, or a ticket with an owner.

MINOR clarity, screens, style. A comment.

1 Law

- BLOCKER Every rate, cap and threshold cites an official source.
- BLOCKER Each `date_from` is the date the law takes effect.
- MAJOR A new value keeps last year's shape.

2 Data

- BLOCKER Module named `l10n_<cc>_…`; every rule, structure and parameter file in `_get_data_files_to_update`, in load order.
- MAJOR Every legal number is a dated `l10n_<cc>_` parameter, and every parameter is read by a rule.
- MAJOR New values are added; no past value is edited.
- MAJOR Each piece of data the rules or filings need has a payroll warning; blocking only where every payslip would be wrong.

3 Rule code

- BLOCKER Every category, code, input and field a rule reads exists and is filled earlier.
- BLOCKER Each rule is on every structure where the law applies it.
- MAJOR No legal number as a literal (`25 *`, `0.4`); rounding only where the law rounds.
- MAJOR Categories and structures written as `(6, 0, […])`.
- MINOR Sequences leave gaps; long formulas sit in helpers.

4 Engine and overrides

- BLOCKER Every override has a country guard and calls `super()`.
- MAJOR No `search` per payslip; batch data in `_get_common_localdict`.
- MAJOR A version field a rule reads is in `_get_fields_that_recompute_payslip`.

5 Tests

- BLOCKER Expected values come from an independent calculation, with the source in a comment.
- MAJOR Both sides of each dated change; one income above every cap and in the top bracket.
- MAJOR Every structure and every frequency has a payslip test.

6 Security

- BLOCKER Salary fields carry `groups="hr_payroll.group_hr_payroll_user"`.
- BLOCKER Every new model has access rights and a multi-company rule.
- MAJOR No rule writes on a record.

7 Screens

- MAJOR Country fields show only for the country (`invisible="country_code != 'XX'"`).
- MINOR Labels and help in plain English; the payslip PDF readable by an employee.

8 Filings and accounting

- BLOCKER A test pay run posts a balanced entry with no *Adjustment Entry* line.
- BLOCKER Declarations read payslip lines, through codes or a data mapping; they never recompute.
- MAJOR Filing checks collect every error before raising.

Common bugs we found

Reading Odoo's own payroll modules for this book turned up the same four patterns again and again. Look for them first.

A total nobody feeds

A rule reads a category that no rule adds to. It reads 0, with no error.

SEEN AS	commissions never enter a pension base
CARD	3 · Rule code

A literal next to its parameter

The number is typed in the rule while a parameter for it exists and is never read.

SEEN AS	Mexico's contribution cap: `25 *` in the rule, and a parameter worth 25 that no rule reads
CARD	2 · Data, 3 · Rule code

A bracket right only for low incomes

The formula is wrong above the second band, and every test is below it.

SEEN AS	`min(gross - low, high)` instead of `min(gross, high) - low`
CARD	5 · Tests

A lookup that silently finds nothing

A missing record only logs a warning, and the code goes on without it.

SEEN AS	account ids absent from the chart: rules without accounts
CARD	8 · Filings and accounting

⚠️ **Common pitfall: the green test run**

Tests run as an administrator, so a missing access right never shows; and most of them create one company. Before approving, open a payslip in a database with several localizations installed, logged in as a payroll user of the new country's company only.

🧱 **Worked example: reviewing a one-line change to a contribution cap**

A pull request changes Mexico's `SBC` rule from `25 * payslip._rule_parameter('l10n_mx_uma')['daily']` to read the multiplier from `l10n_mx_imss_limit_with_uma`. Card 1: is the source of the 25 cited? Card 2: a parameter nobody read becomes used, good. Card 5: is there a payslip with a daily salary **above** 25 times the reference unit? If not, ask for one: a suite of ordinary wages would stay green even if the cap were deleted.

Summary

- A payroll localization is the country layer: `l10n_xx_hr_payroll` for the law, `l10n_xx_hr_payroll_account` for the accounts and the tests, built on the shared `hr_payroll`. The `l10n_<cc>` prefix makes *Reset* work.
- Most of it is data: a structure type and structures, categories designed from the declarations, rules that write every field and contain no legal number, and dated parameters with a stable shape. Payroll warnings catch missing data on the dashboard, and block validation only when every payslip would be wrong.
- Three hooks wire it in: the list of files the weekly job reloads, the post-install switch to the country's time types, and the allowed pay frequencies.
- Python is for facts about the person (fields on the version, mirrored on the employee) and for engine steps the law changes, always behind a country guard and with `super()`.
- The module must produce the payslip PDF, the declarations (read from payslip lines) and the journal entry, through a rule-to-account mapping.
- Golden-payslip tests prove it. The yearly upkeep is a new dated record with a test on each side of the date; the weekly job delivers it, except to records the customer edited.
- Review along eight cards (law, data, rule code, engine, tests, security, screens, filings), grade each finding blocker, major or minor, and look first for the four known bug patterns.